

Practical No. 1 - Introduction to JPA – Entity Class and Persisting a Single Entity

Objective:

To understand the requirements of a JPA entity class (no-arg constructor, @Entity, @Id, @GeneratedValue), configure a relational datasource in application.properties, and persist/retrieve an entity using Spring Data JPA's CrudRepository.

Class Name(s) with Attributes:

Employee (@Entity)

- empId : int (@Id @GeneratedValue)
- empName : String
- department : String
- salary : double

EmployeeRepository (interface extends CrudRepository<Employee, Integer>)

- Inherited: save(), findAll(), findById(), deleteById()

EmployeeRestController (@RestController)

- employeeRepository : EmployeeRepository

Objective: Create an Employee REST API using Spring Boot, Spring Data JPA and MySQL.

Project Structure

```
src
├── main
│   ├── java
│   │   ├── com.first.first
│   │   │   ├── FirstApplication.java
│   │   │   ├── controller
│   │   │   │   ├── EmployeeRestController.java
│   │   │   ├── entity
│   │   │   │   ├── Employee.java
│   │   │   ├── repository
│   │   │   │   ├── EmployeeRepository.java
│   └── resources
│       └── application.properties
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: employee_db

Table: employee (created automatically by Hibernate)

Important Files

Employee.java(ENTITY model class)

```
package com.first.first.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;

    private String department;

    private double salary;

    public Employee() {
    }

    public Employee(Integer empId, String empName, String department,
double salary) {
        this.empId = empId;
        this.empName = empName;
        this.department = department;
        this.salary = salary;
    }

    public Integer getEmpId() {
        return empId;
    }

    public void setEmpId(Integer empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public String getDepartment() {
```

```
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

EmployeeRepository.java(Repository file)

```
package com.first.first.repository;

import org.springframework.data.repository.CrudRepository;
import com.first.first.entity.Employee;

public interface EmployeeRepository extends CrudRepository<Employee,
Integer> {

}
```

EmployeeController.java(Controller file)

```
package com.first.first.controller;

import com.first.first.entity.Employee;
```

```
import com.first.first.repository.EmployeeRepository;
import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.Optional;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository employeeRepository;

    @PostMapping
    public Employee addEmployee(@RequestBody Employee
employee) {
        return employeeRepository.save(employee);
    }

    @GetMapping
    public Iterable<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    @GetMapping("/{id}")
    public Optional<Employee> getEmployee(@PathVariable
Integer id) {
        return employeeRepository.findById(id);
    }

    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable Integer
id,
                                @RequestBody Employee employee)
    {
        employee.setEmpId(id);
        return employeeRepository.save(employee);
    }
}
```

```
@DeleteMapping("/{id}")
public String deleteEmployee(@PathVariable Integer id) {
    employeeRepository.deleteById(id);
    return "Employee Deleted Successfully";
}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=first
spring.datasource.url=jdbc:mysql://localhost:3306/employee_db
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

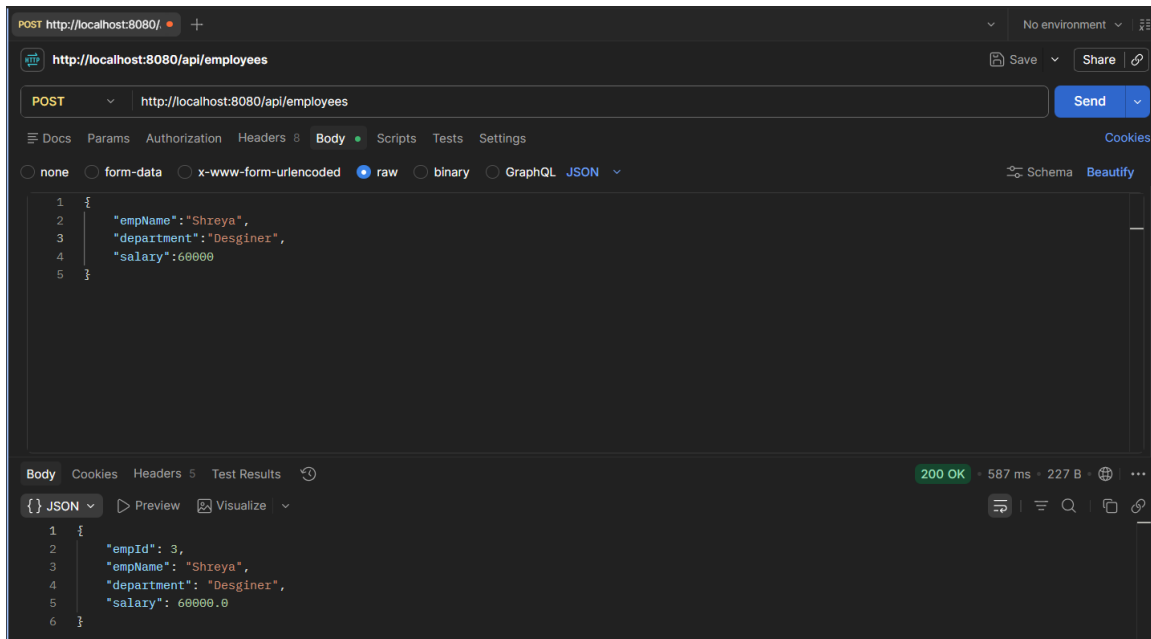
spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

6.1 POST API – Add Employee :

The POST API inserts a new employee into the database. The employee ID is generated automatically by MySQL.

POST <http://localhost:8080/api/employees>

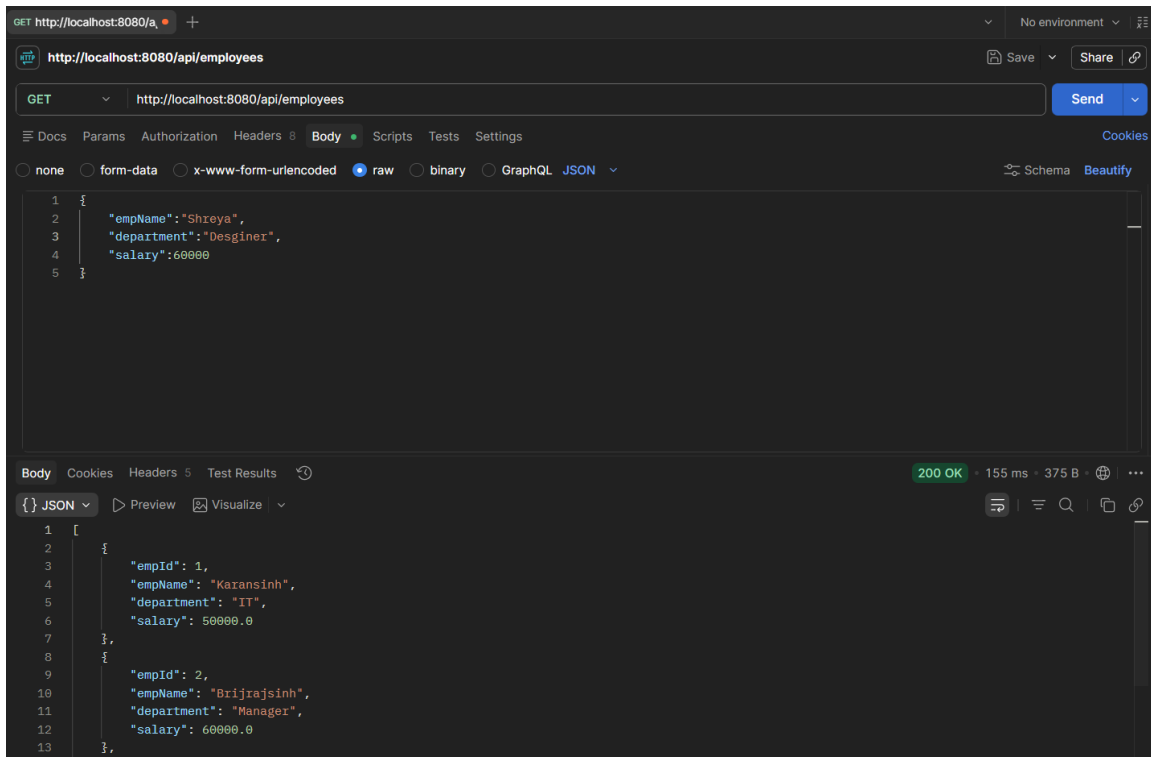


GET API (All Employees) :

6.2 GET API – Retrieve All Employees

The GET API retrieves all employee records stored in the database.

GET <http://localhost:8080/api/employees>

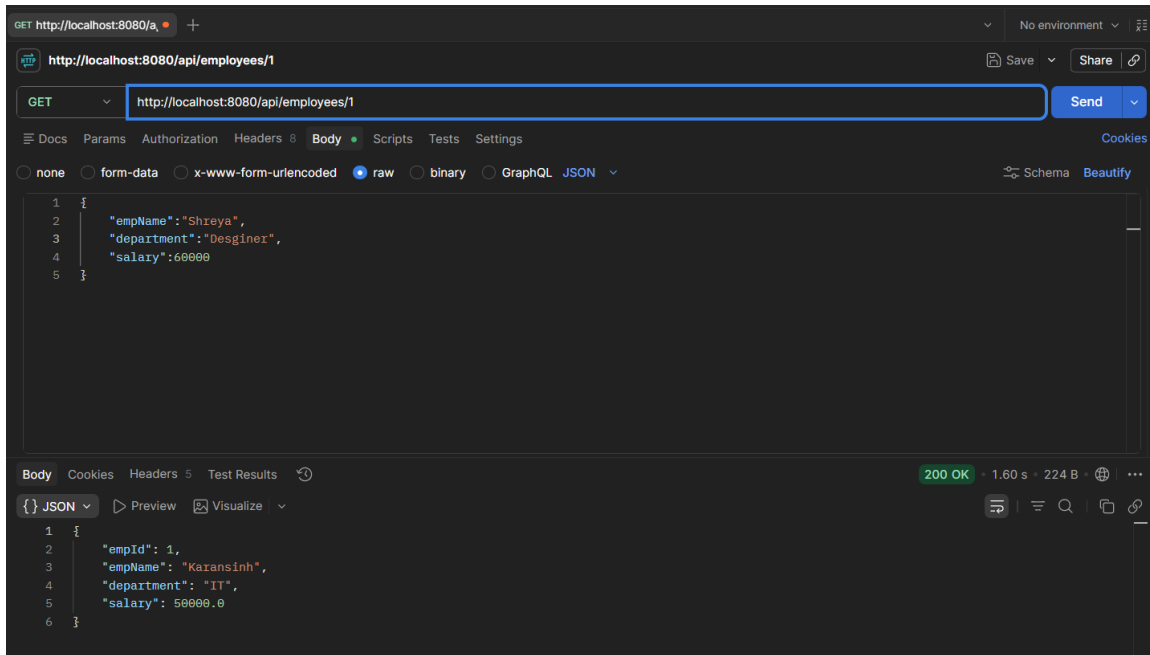


3. GET BY ID

6.3 GET API – Retrieve Employee by ID

Retrieves a specific employee using the employee ID.

GET `http://localhost:8080/api/employees/1`



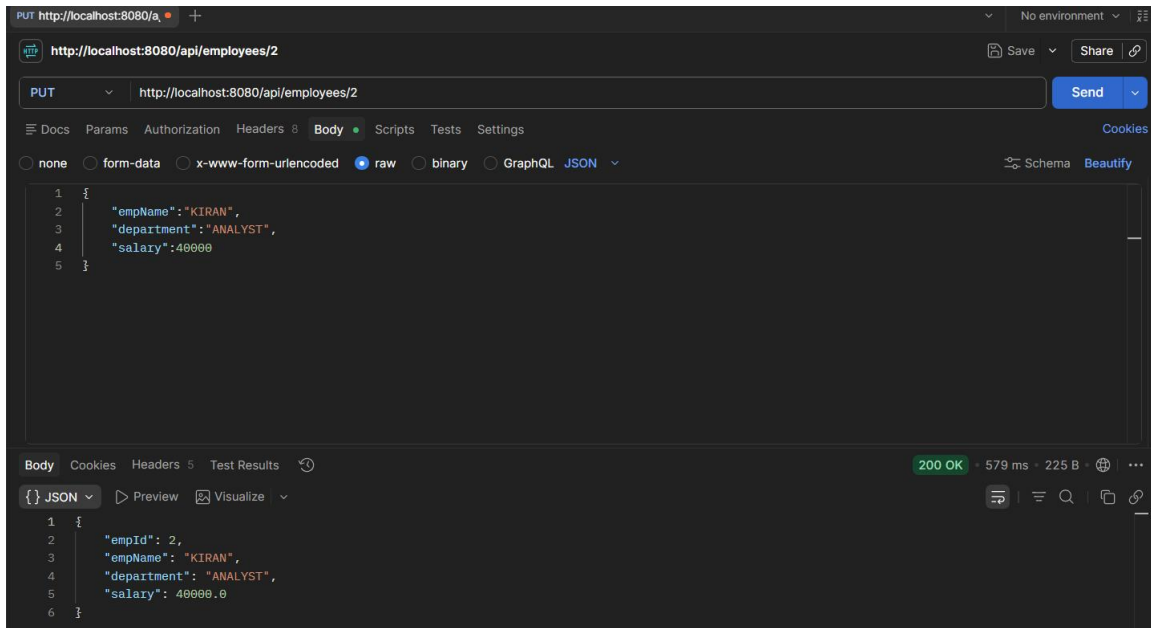
4. PUT {ID}

6.4 PUT API – UPDATE Employee

Updates the employee information using the employee ID.

PUT `http://localhost:8080/api/employees/2`

SEE AND VIEW IN LIST OF ALL EMPLOYEE'S AT ID 2 IT WAS BRIJRAJSINH AND MANAGER NOW IT UPDATE'S TO KIRAN AND ANALYST



5. DELETE {ID}

6.5 DELETE API – DELETE Employee

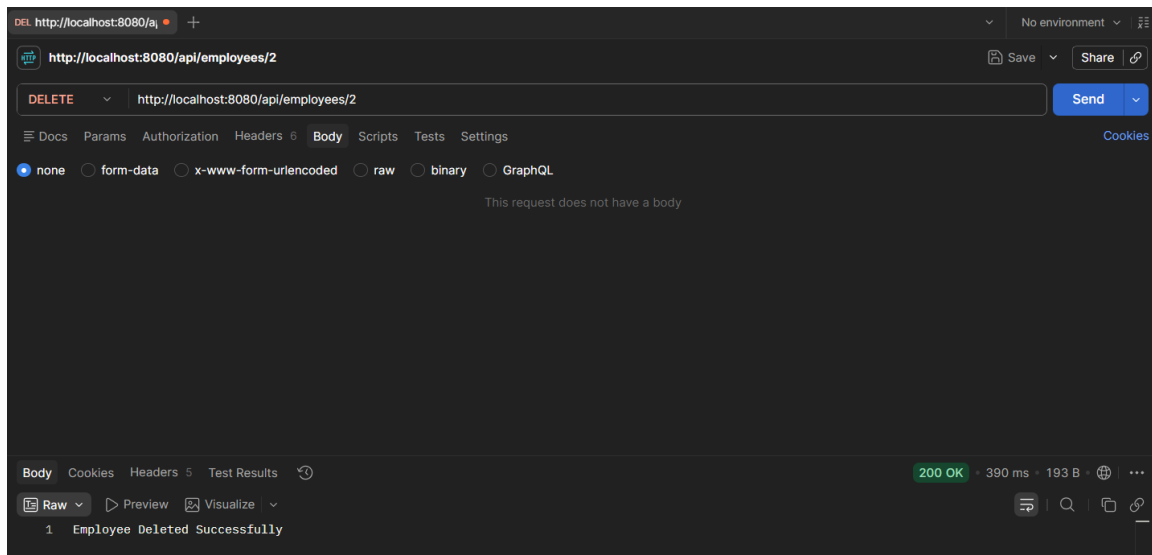
Deletes the employee record from the database.

DELETE http://localhost:8080/api/employees/2

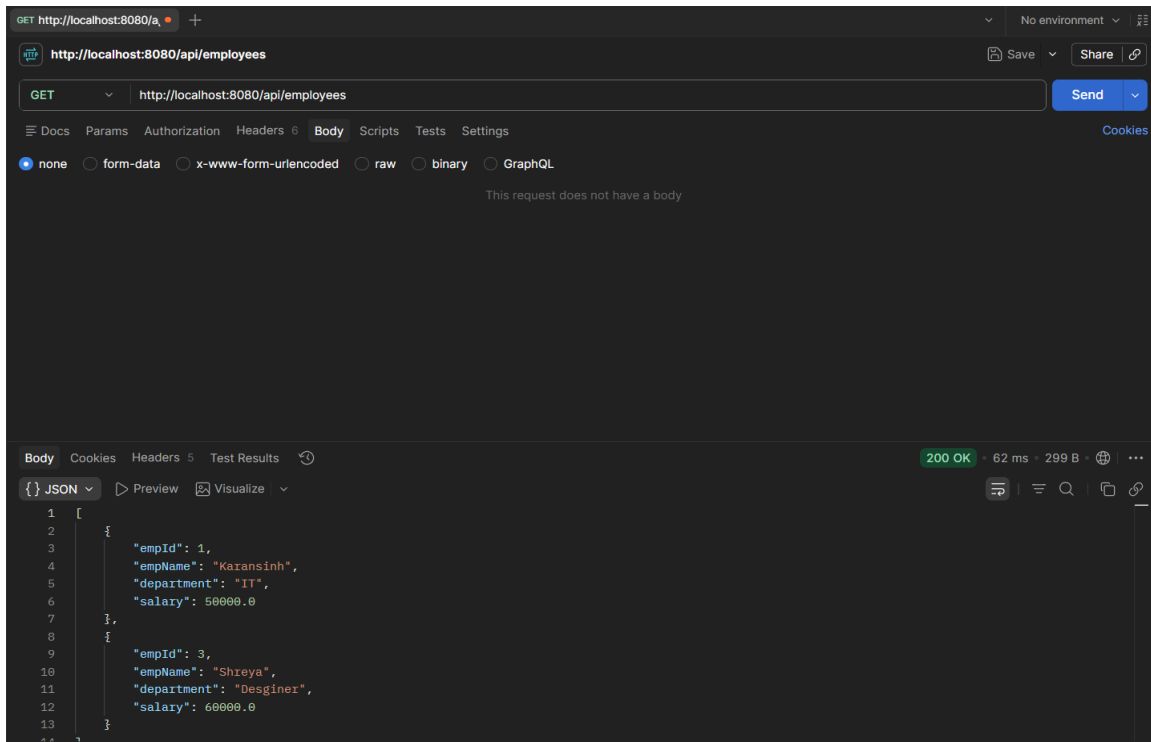
SEE THE ID WITH 2 WILL BE DELETED FROM LIST

NAME: SOLANKI KARANSINH

ENROLLMENT: 25CI2110110



NOW THIS TO CHECK WHETHER DELETED OR NOT :

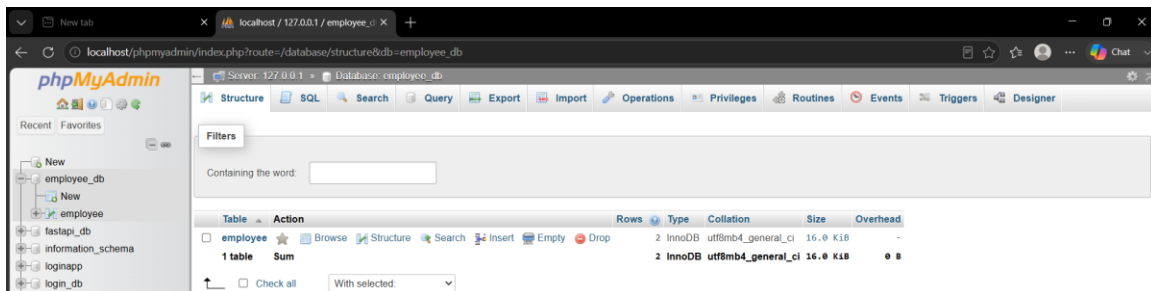


REST APIs

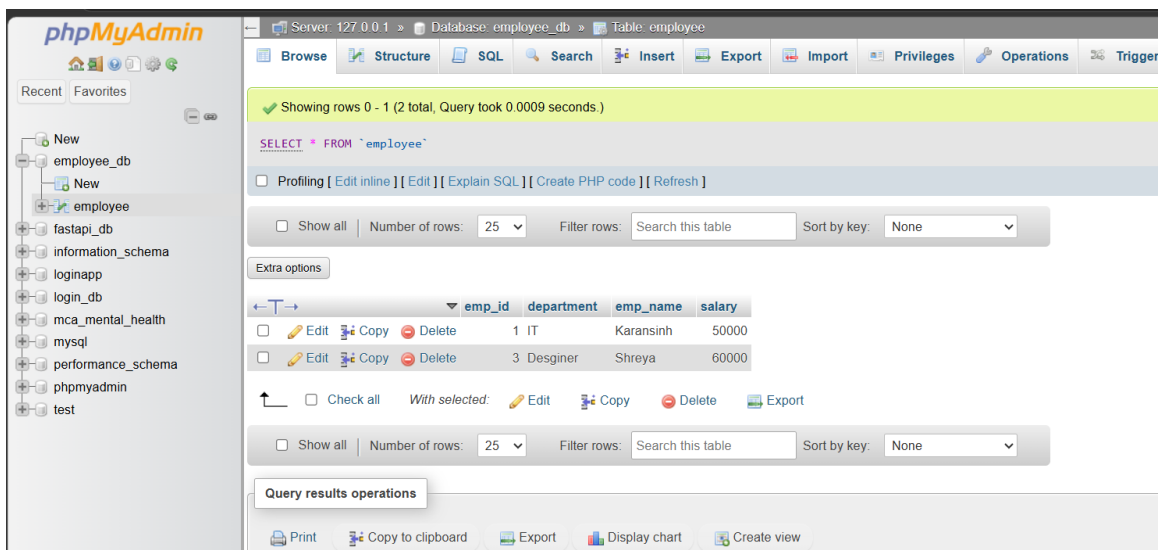
Method	URL	Purpose
POST	<code>/api/employees</code>	Add employee
GET	<code>/api/employees</code>	Get all employees
GET	<code>/api/employees/{id}</code>	Get employee by ID
PUT	<code>/api/employees/{id}</code>	Update employee
DELETE	<code>/api/employees/{id}</code>	Delete employee

Database Output

TABLE'S AND DATA STORING SUCCEFULLY



WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS



Practical No. 2 - Spring Data Repository **Hierarchy – CrudRepository vs** **PagingAndSortingRepository vs** **JpaRepository**

Objective:

To compare the three levels of the Spring Data repository hierarchy covered in this chapter — CrudRepository (basic CRUD), PagingAndSortingRepository (adds paging/ sorting), and JpaRepository (adds JPA-specific batch operations) — by rewriting the same repository at each level and observing the additional methods available.

Class Name(s) with Attributes:

Product (@Entity)

- productId : int (@Id @GeneratedValue)
- productName : String
- category : String
- price : double
- quantity : int

ProductRepository (progressively extends CrudRepository → PagingAndSortingRepository → JpaRepository)

ProductRestController (@RestController)

- productRepository : ProductRepository

Create a Product REST API using Spring Boot and demonstrate the Repository Hierarchy by implementing **CrudRepository**, **PagingAndSortingRepository**, and **JpaRepository** with CRUD operations, sorting, pagination, and JPA-specific methods.

Tested some api's using that pagingand sorting repository suppose you have 5 data and you want sort accouding to price or anything you can do is also if you want like that I want show the 2-4 data and don't want to show the first and last you can achieve it with the paging technique so it's so helpful to create this

Project Structure:

```
src
├── main
│   ├── java
│   │   ├── com.second.second
│   │   │   ├── SecondApplication.java
│   │   │   ├── controller
│   │   │   │   ├── ProductRestController.java
│   │   │   ├── entity
│   │   │   │   ├── Product.java
│   │   │   ├── repository
│   │   │   │   ├── ProductRepository.java
│   └── resources
│       └── application.properties
```

Dependencies

- Spring Web

- Spring Data JPA

- MySQL Driver

Database

Database Name: product_db

Table: product (created automatically by Hibernate)

Important Files

Product.java(ENTITY model class)

```
package com.second.second.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "product")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer productId;

    private String productName;

    private String category;

    private double price;
```

```
private int quantity;

public Product() {
}

public Integer getProductId() {
    return productId;
}

public void setProductId(Integer productId) {
    this.productId = productId;
}

public String getProductName() {
    return productName;
}

public void setProductName(String productName) {
    this.productName = productName;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
    this.price = price;
}

public int getQuantity() {
    return quantity;
}

public void setQuantity(int quantity) {
    this.quantity = quantity;
}
}
```

ProductRepository.java(Repository file)

```
package com.second.second.repository;

import com.second.second.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Integer>
{
}
}
```

ProductController.java(Controller file for REST apis)

```
package com.second.second.controller;

import com.second.second.entity.Product;
import com.second.second.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.*;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    @Autowired
    private ProductRepository repository;

    // Add a new product
    @PostMapping
```

```
public Product addProduct(@RequestBody Product product) {
    return repository.save(product);
}

// Add multiple products using saveAll()
@PostMapping("/batch")
public List<Product> addProducts(@RequestBody List<Product>
products) {
    return repository.saveAll(products);
}

// Retrieve all products
@GetMapping
public List<Product> getAllProducts() {
    return repository.findAll();
}

// Retrieve a product by ID
@GetMapping("/{id}")
public Product getProduct(@PathVariable Integer id) {
    return repository.findById(id).orElse(null);
}

// Update product details
@PutMapping("/{id}")
public Product updateProduct(@PathVariable Integer id,
    @RequestBody Product product) {

    product.setProductId(id);
    return repository.save(product);
}

// Delete a product by ID
@DeleteMapping("/{id}")
public String deleteProduct(@PathVariable Integer id) {

    repository.deleteById(id);
    return "Product Deleted Successfully";
}

// Sort products by any field
@GetMapping("/sorted")
public List<Product> sortProducts(@RequestParam String sortBy) {

    return repository.findAll(Sort.by(sortBy));
}

// Display products page by page
@GetMapping("/paged")
public Page<Product> getProductsPage(@RequestParam int page,
```

```
@RequestParam int size) {  
  
    Pageable pageable = PageRequest.of(page, size);  
    return repository.findAll(pageable);  
}  
  
    // Flush pending changes to the database  
    @PostMapping("/flush")  
    public String flushData() {  
  
        repository.flush();  
        return "Data Flushed Successfully";  
    }  
  
}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=second  
  
spring.datasource.url=jdbc:mysql://localhost:3306/product_db  
  
spring.datasource.username=root  
  
spring.datasource.password=  
  
spring.jpa.hibernate.ddl-auto=update  
  
spring.jpa.show-sql=true
```

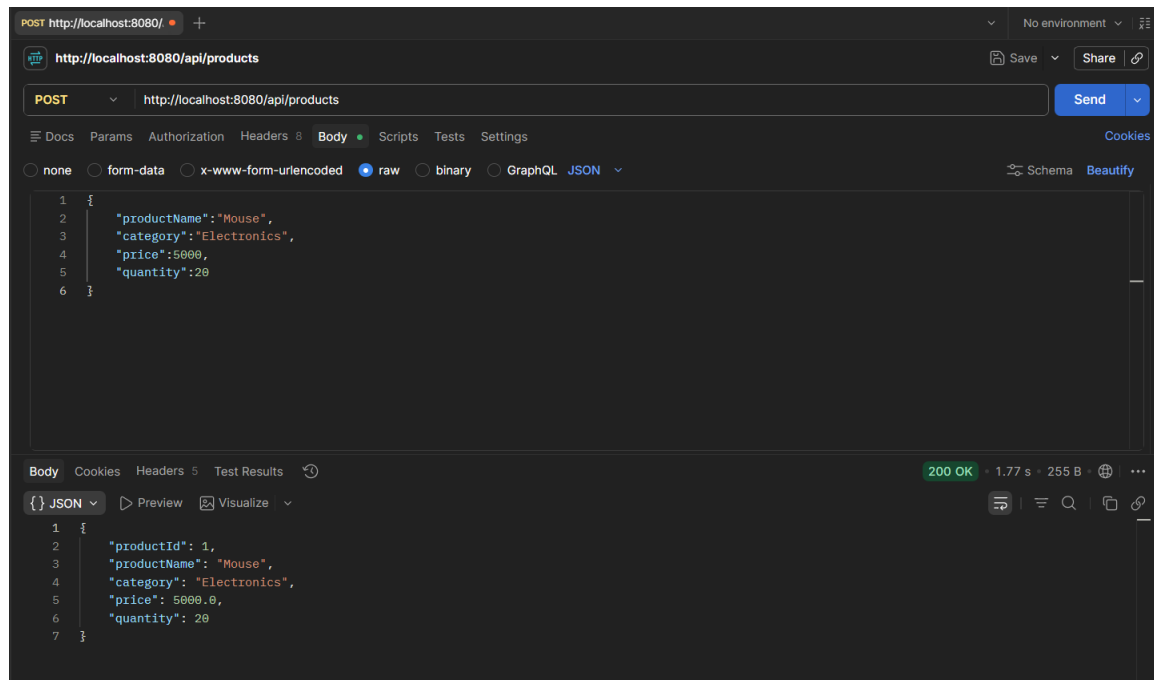
API'S TESTING USING POSTMAN

1. POST API – Add Product:

The POST API is used to insert a new product into the MySQL database. The product details are sent in the request body as JSON, and Spring Boot stores the record using the save() method.

POST

<http://localhost:8080/api/products>



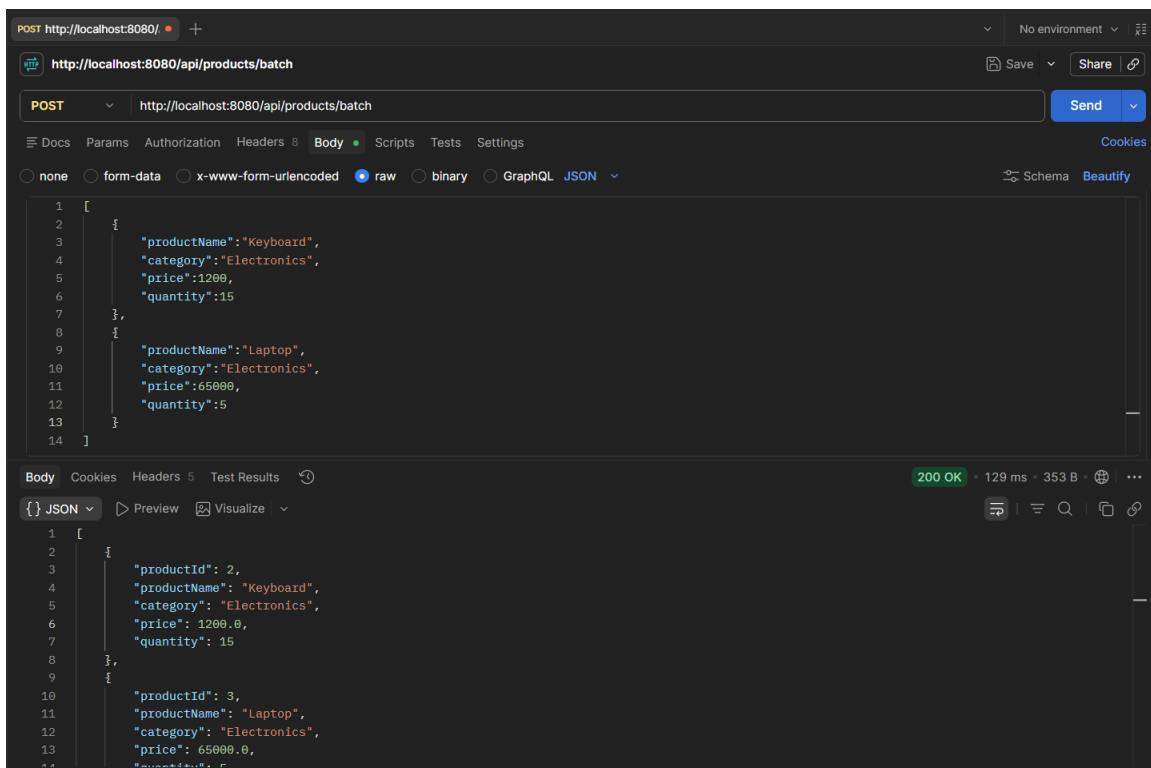
SEE THE PRODUCT IS ADDED SUCCESSFULLY INTO THE DATABASE.

2. POST API – Batch Insert Products:

The POST Batch API is used to insert multiple product records into the database at a single time. It uses the `saveAll()` method provided by **JpaRepository**.

POST:

<http://localhost:8080/api/products/batch>



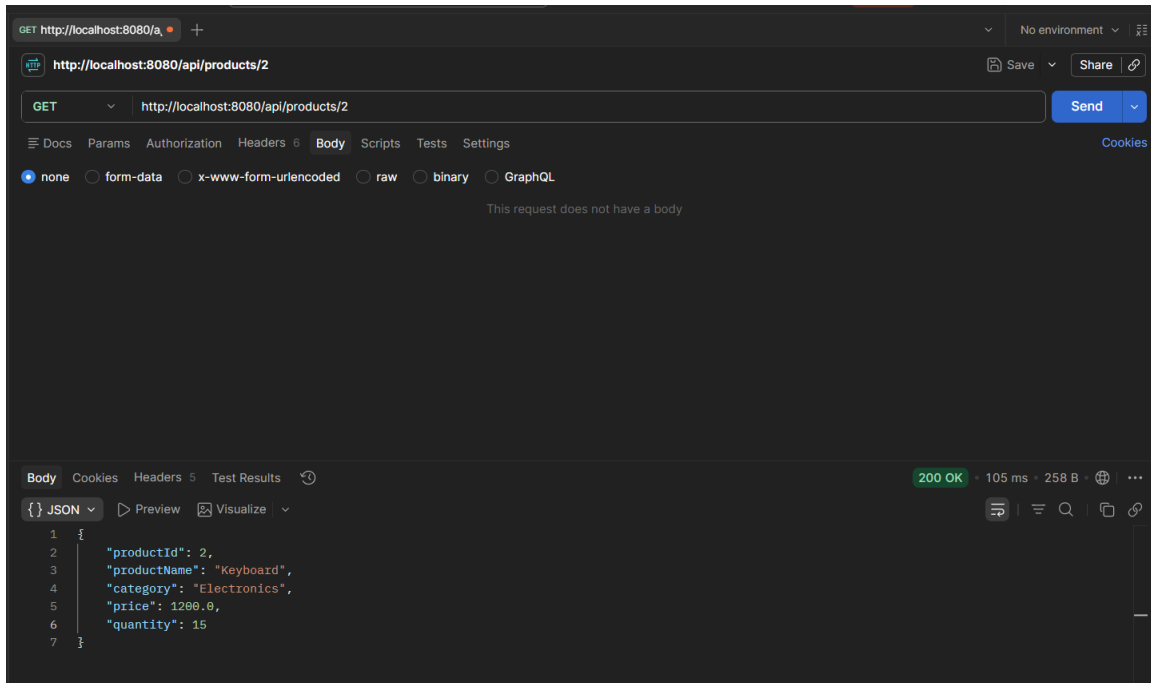
SEE THE MULTIPLE PRODUCT'S ARE ADDED SUCCESSFULLY INTO THE DATABASE.

3. GET API – Retrieve Product by ID:

This API retrieves the details of a single product based on the Product ID passed in the URL using the `findById()` method.

GET :

<http://localhost:8080/api/products/2>



SEE THE PRODUCT DETAILS ARE DISPLAYED SUCCESSFULLY USING THE PRODUCT ID.

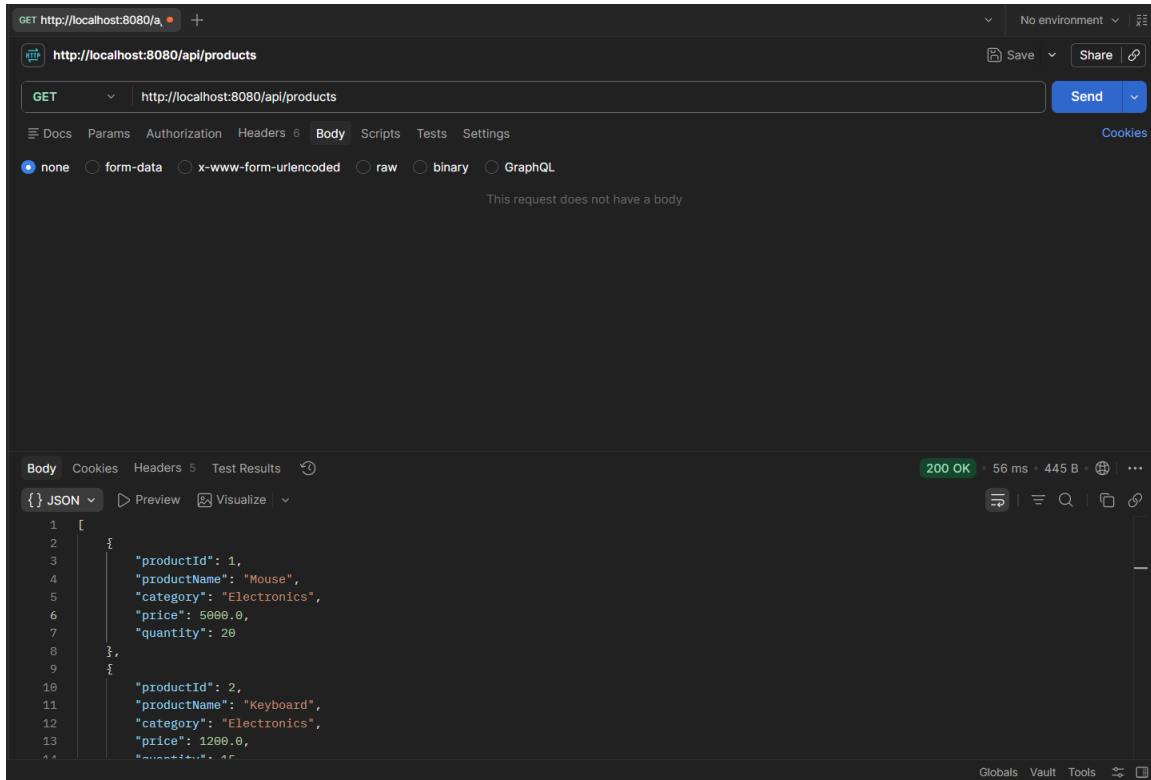
4. GET API – Retrieve All Products:

The GET API retrieves all product records available in the database. It returns the complete list of products in JSON format.

Using `findAll()` method

GET :

<http://localhost:8080/api/products>



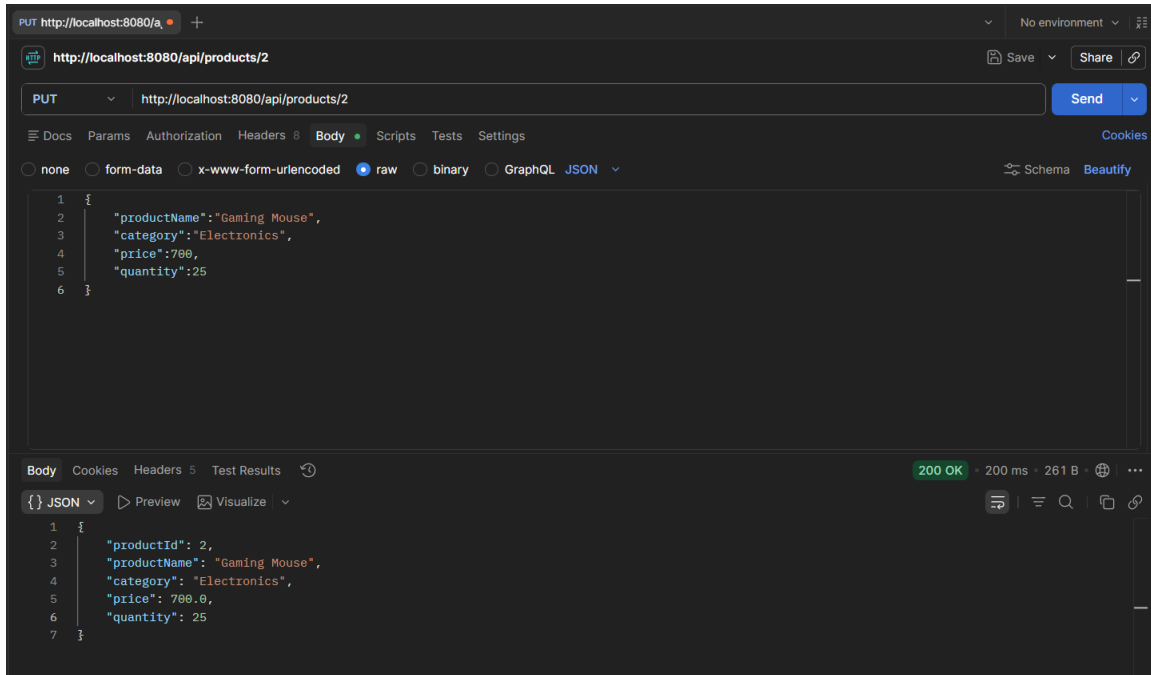
SEE ALL PRODUCT RECORDS ARE DISPLAYED SUCCESSFULLY.

5. PUT API – Update Product:

The PUT API is used to modify the details of an existing product. The Product ID is specified in the URL, while the updated values are provided in the request body.

Just give the id which you want to update give json data and data will be updated

PUT: <http://localhost:8080/api/products/2>



You can see before this the id 2 had the keyboard with different price now it's updated

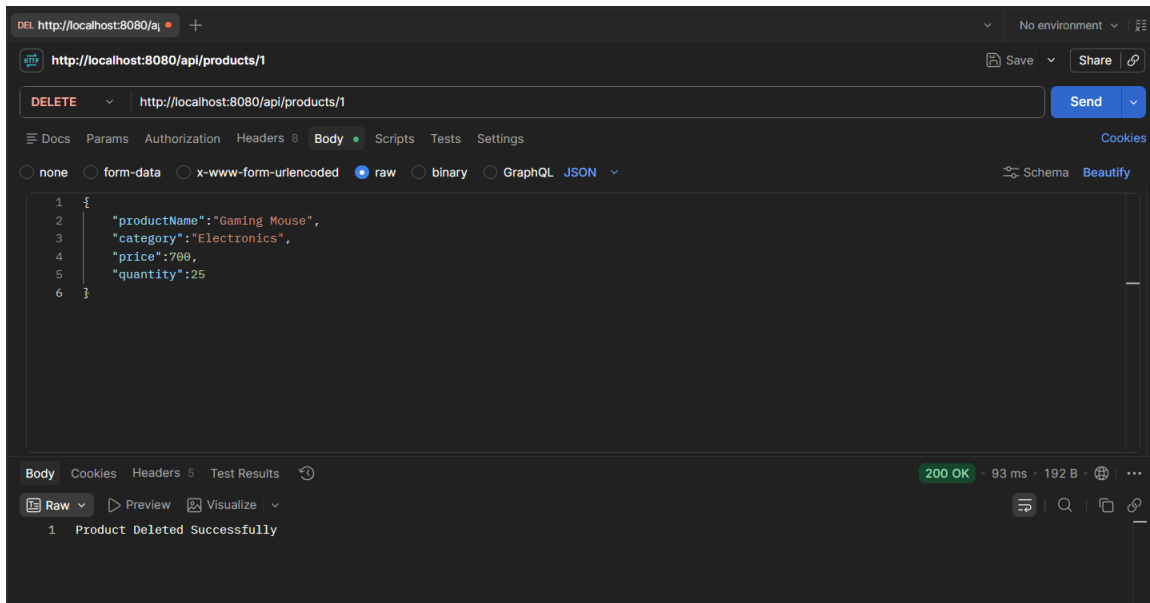
6. DELETE API – Delete Product:

The DELETE API removes the selected product from the database using its Product ID.

Just give the id which you want to delete in postman and it will be gone from database...!

Delete :

<http://localhost:8080/api/products/2>



Ignore that body with json data I forgot to remove It it's just none the body and json data only used in in post and put

The data is deleted from database with id 1

7. GET API – Sort Products(IMP MAIN TOPIC) :

The Sorting API retrieves all products in sorted order based on the field specified in the sortBy request parameter, such as price or productName.

Here I have given the two examples of sorting

1st sorting by price (lowest will come first)

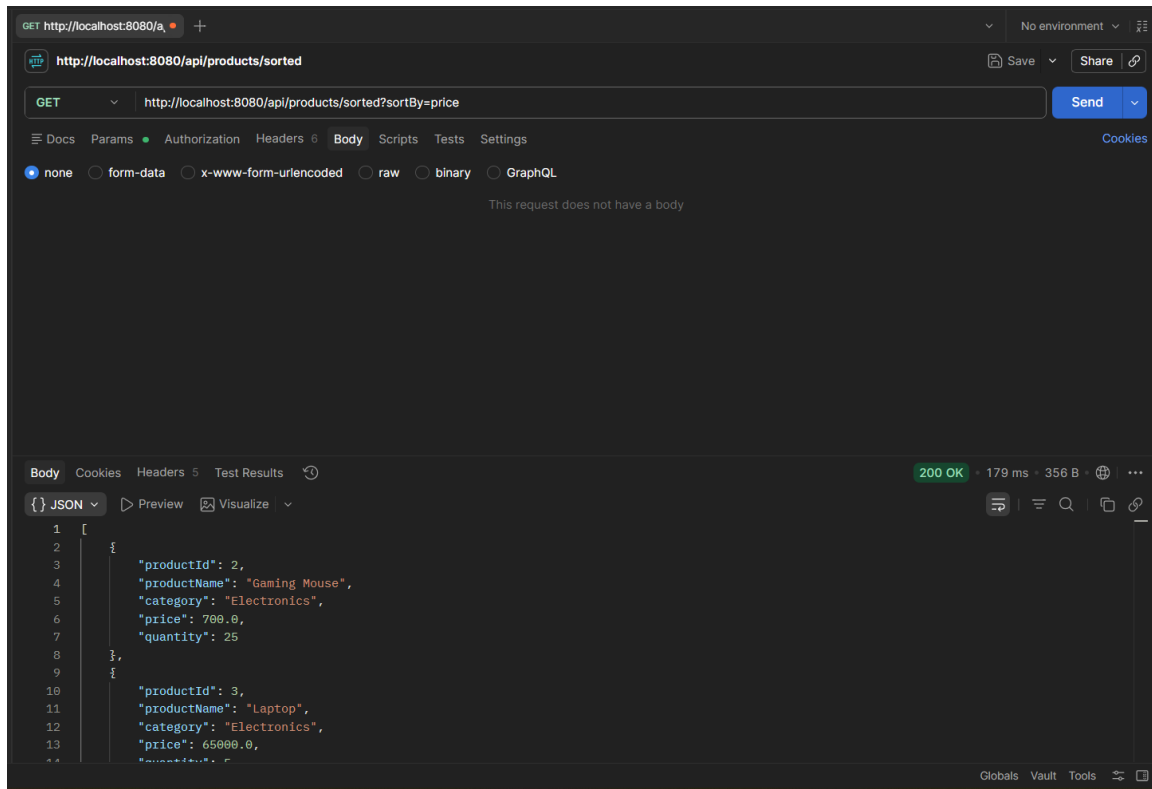
2nd sorting by quantity (lesser quantity come first)

NAME: SOLANKI KARANSINH

ENROLLMENT: 25CI2110110

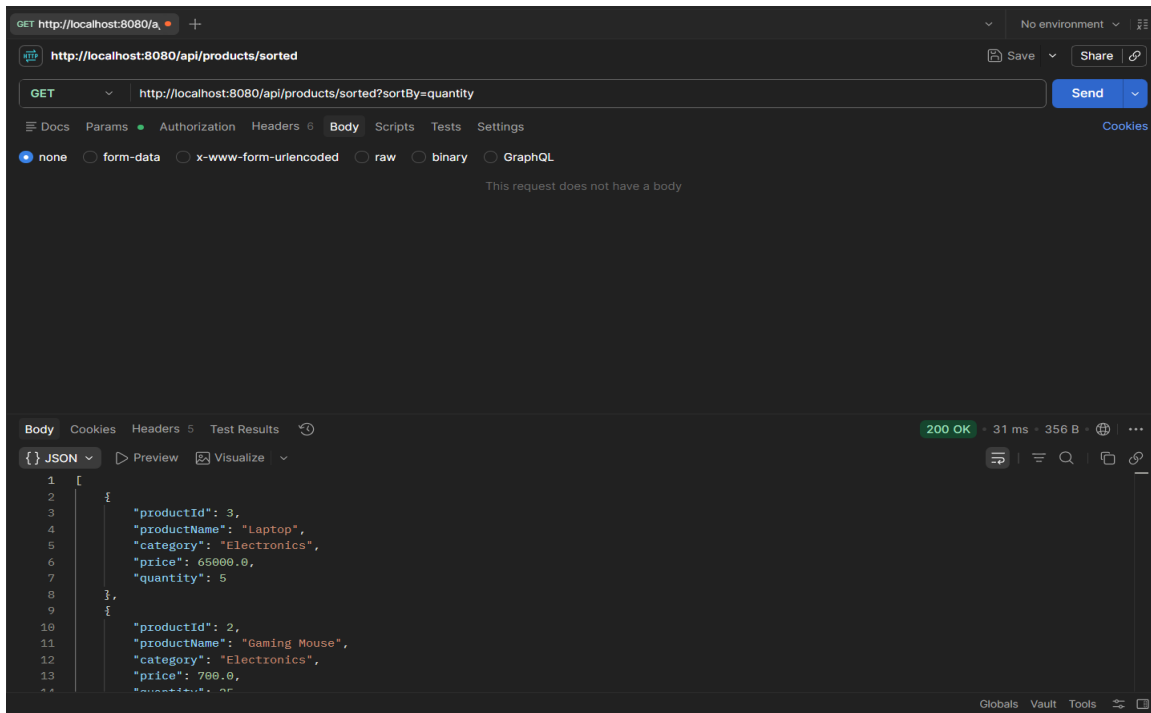
GET:

<http://localhost:8080/api/products/sorted?sortBy=price>



SEE ALL PRODUCTS ARE DISPLAYED IN SORTED ORDER ACCORDING TO PRICE.

2ND WITH QUANTITY

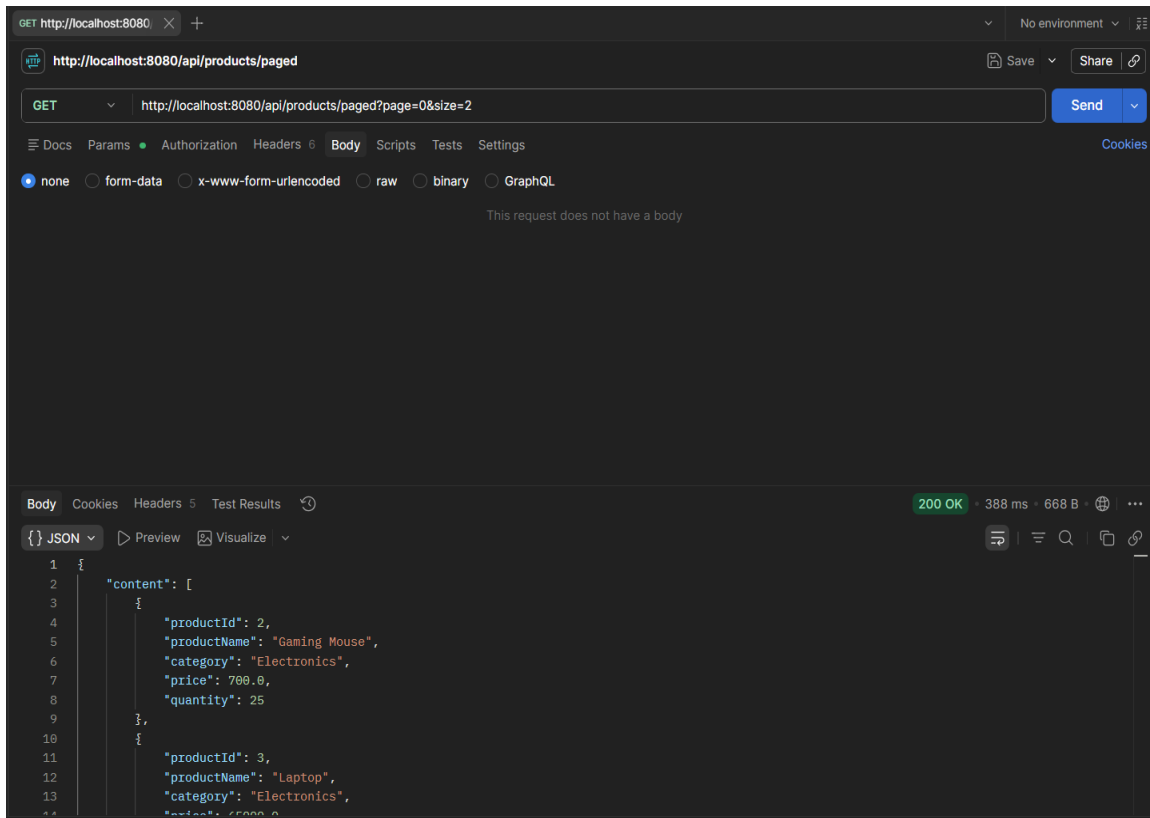


8. GET API – Pagination (IMP MAIN TOPIC) :

The Pagination API retrieves product records page by page. The page parameter specifies the page number, and the size parameter specifies how many records are returned per page.

GET :

<http://localhost:8080/api/products/paged?page=0&size=2>



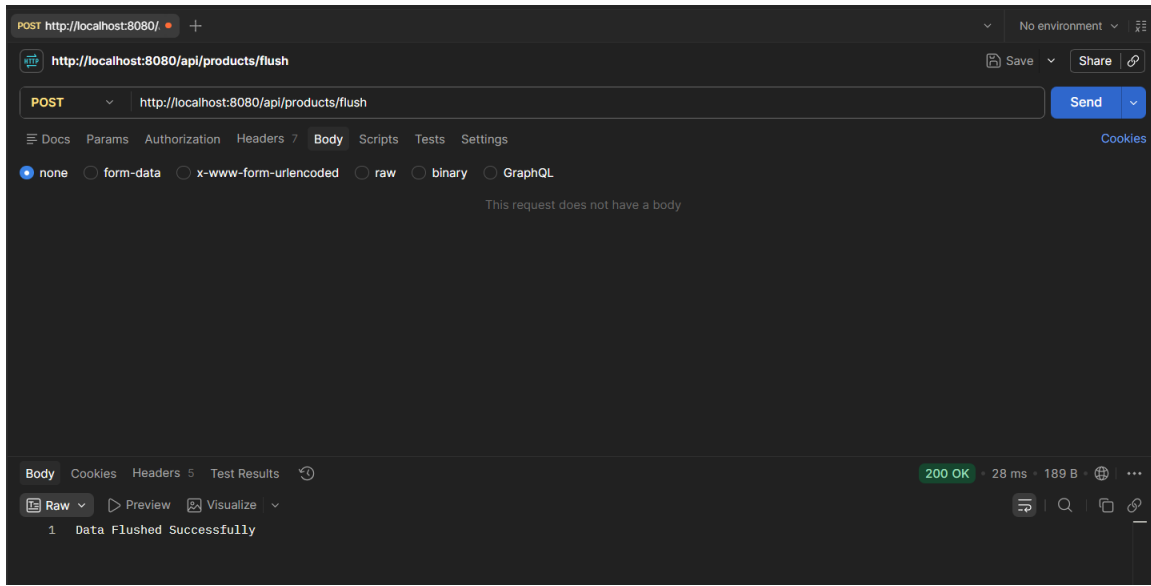
SEE ONLY THE SPECIFIED NUMBER OF PRODUCT RECORDS ARE DISPLAYED ON THE PAGE.

9. POST API – Flush Data (IMP MAIN TOPIC) :

The Flush API immediately synchronizes all pending changes from the persistence context to the MySQL database using the `flush()` method.

POST :

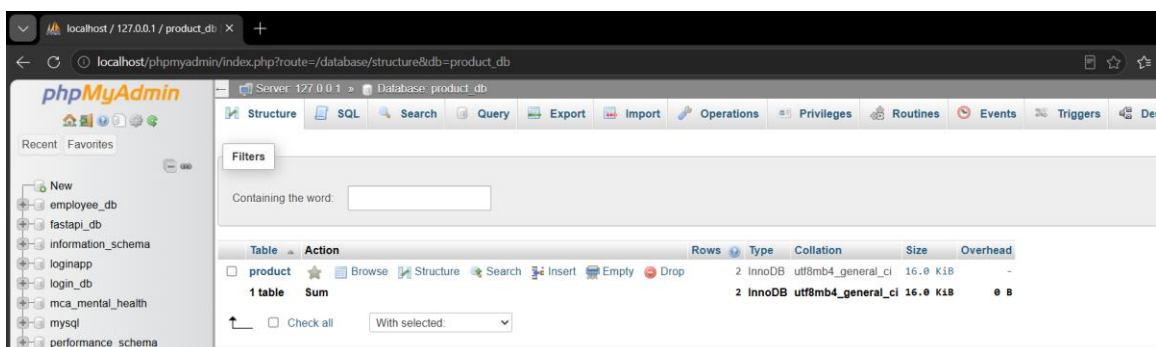
<http://localhost:8080/api/products/flush>



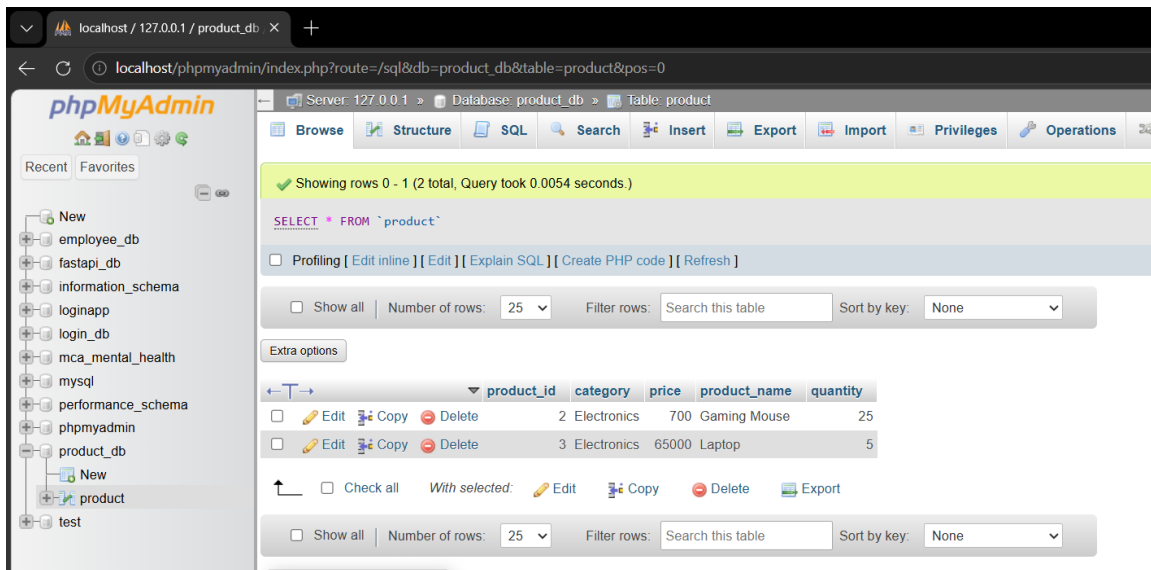
SEE ALL PENDING CHANGES ARE WRITTEN SUCCESSFULLY TO THE DATABASE.

Database Output

TABLE'S AND DATA STORING SUCCEFULLY



WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS



Practical No. 3: Custom Query Methods – findBy, countBy and @Query (JPQL)

Objective:

To derive query methods automatically from method names (query derivation) and to write custom JPQL queries using @Query for requirements that cannot be expressed by naming conventions alone, as covered in the Spring Data JPA query-methods section of this chapter.

Class Name(s) with Attributes:

Employee (reuse from Practical 1)

- empId, empName, department, salary

EmployeeRepository (additional derived & custom methods)

- List<Employee> findByDepartment(String department)
- long countByDepartment(String department)
- List<Employee> findBySalaryGreaterThan(double salary)
- @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")

findBySalaryRange(min,max)

folder structure:

```
src
├── main
│   ├── java
│   │   ├── com.third.third
│   │   │   ├── ThirdApplication.java
│   │   │   ├── controller
│   │   │   │   ├── EmployeeRestController.java
│   │   │   ├── entity
│   │   │   │   ├── Employee.java
│   │   │   ├── repository
│   │   │   │   ├── EmployeeRepository.java
│   └── resources
│       └── application.properties
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: employee_db

Table: employee (created automatically by Hibernate)

Important Files

Employee.java(ENTITY model class)

```
package com.first.first.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;

    private String department;

    private double salary;

    public Employee() {
    }

    public Employee(Integer empId, String empName, String department, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.department = department;
        this.salary = salary;
    }

    public Integer getEmpId() {
        return empId;
    }

    public void setEmpId(Integer empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }
}
```

```
}

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

EmployeeRepository.java(Repository file)

```
package com.third.third.repository;

import com.third.third.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

    List<Employee> findByDepartment(String department);

    long countByDepartment(String department);

    List<Employee> findBySalaryGreaterThan(double salary);

    @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
    List<Employee> findBySalaryRange(@Param("min") double min,
                                     @Param("max") double max);
}
```

EmployeeController.java(Controller file)

```
package com.third.third.controller;

import com.third.third.entity.Employee;
import com.third.third.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository repository;

    @PostMapping
    public Employee addEmployee(@RequestBody Employee employee) {
        return repository.save(employee);
    }

    @GetMapping("/department/{department}")
    public List<Employee> getDepartment(@PathVariable String department) {
        return repository.findByDepartment(department);
    }

    @GetMapping("/count/{department}")
    public long countDepartment(@PathVariable String department) {
        return repository.countByDepartment(department);
    }

    @GetMapping("/salary-above/{salary}")
    public List<Employee> salaryAbove(@PathVariable double salary) {
        return repository.findBySalaryGreaterThan(salary);
    }

    @GetMapping("/salary-range")
    public List<Employee> salaryRange(@RequestParam double min,
                                     @RequestParam double max) {

        return repository.findBySalaryRange(min, max);
    }

    @PutMapping("/{id}")
```

```
public Employee updateEmployee(@PathVariable Integer id,
                               @RequestBody Employee employee) {

    employee.setEmpId(id);
    return repository.save(employee);
}

@DeleteMapping("/{id}")
public String deleteEmployee(@PathVariable Integer id) {

    repository.deleteById(id);
    return "Employee Deleted Successfully";
}
}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=first
spring.datasource.url=jdbc:mysql://localhost:3306/employee_db
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

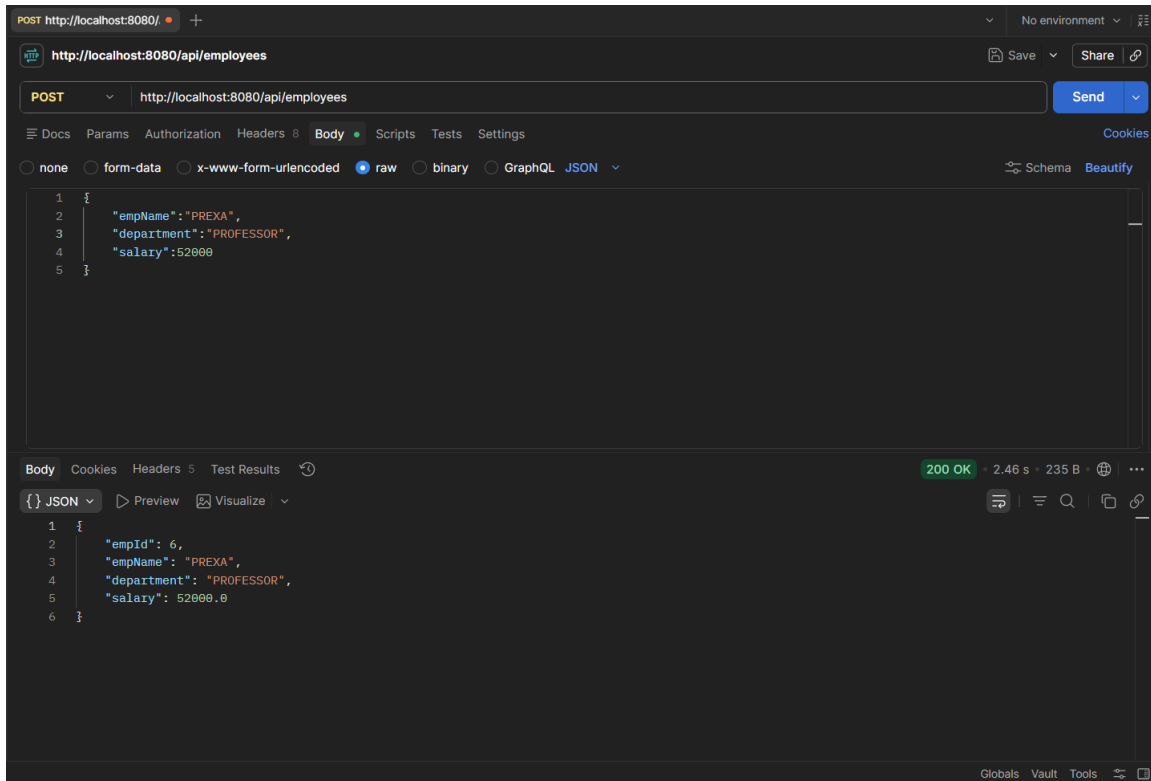
spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

1. POST API – Add Employee :

The POST API inserts a new employee into the database. The employee ID is generated automatically by MySQL.

POST <http://localhost:8080/api/employees>

SEE THE EMPLOYEE IS ADDED SUCCESSFULLY INTO THE DATABASE.

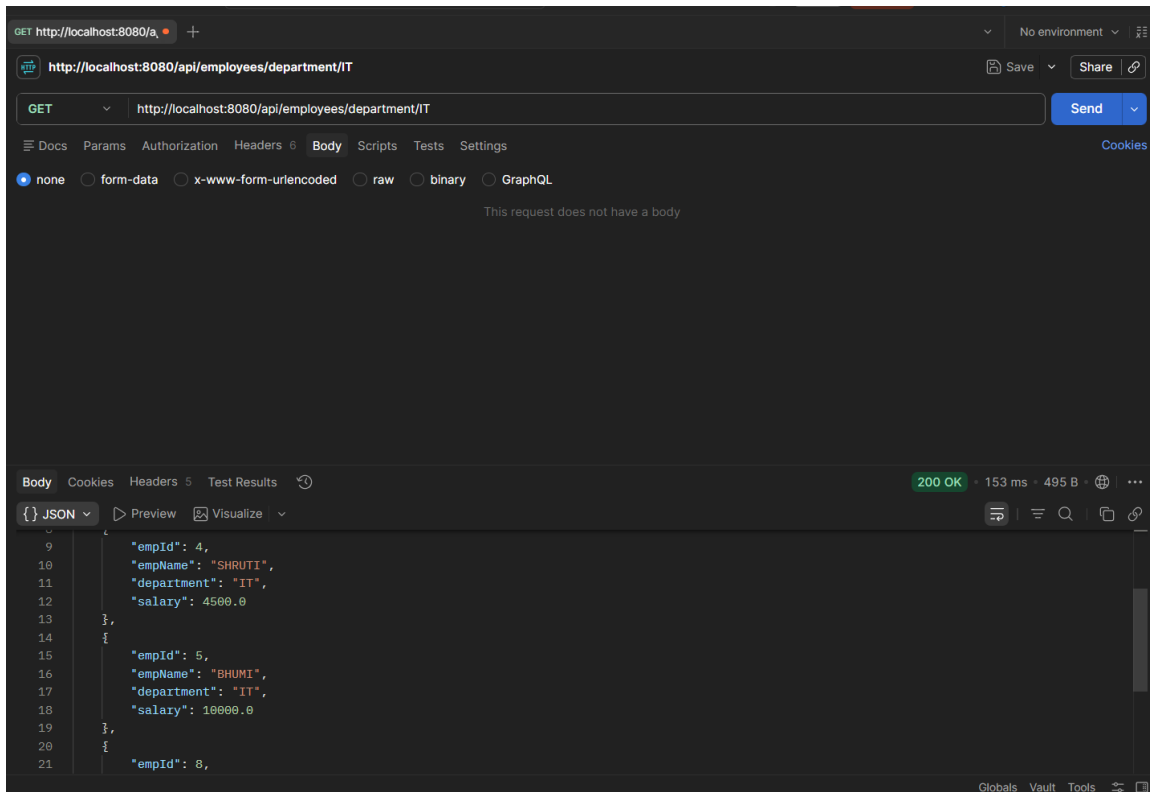
2. GET API – Find Employees by Department:

This API retrieves all employees belonging to a particular department. It uses the derived query method `findByDepartment()` provided by Spring Data JPA.

GET :

<http://localhost:8080/api/employees/department/IT>

all employees working in it field will be listed in postman



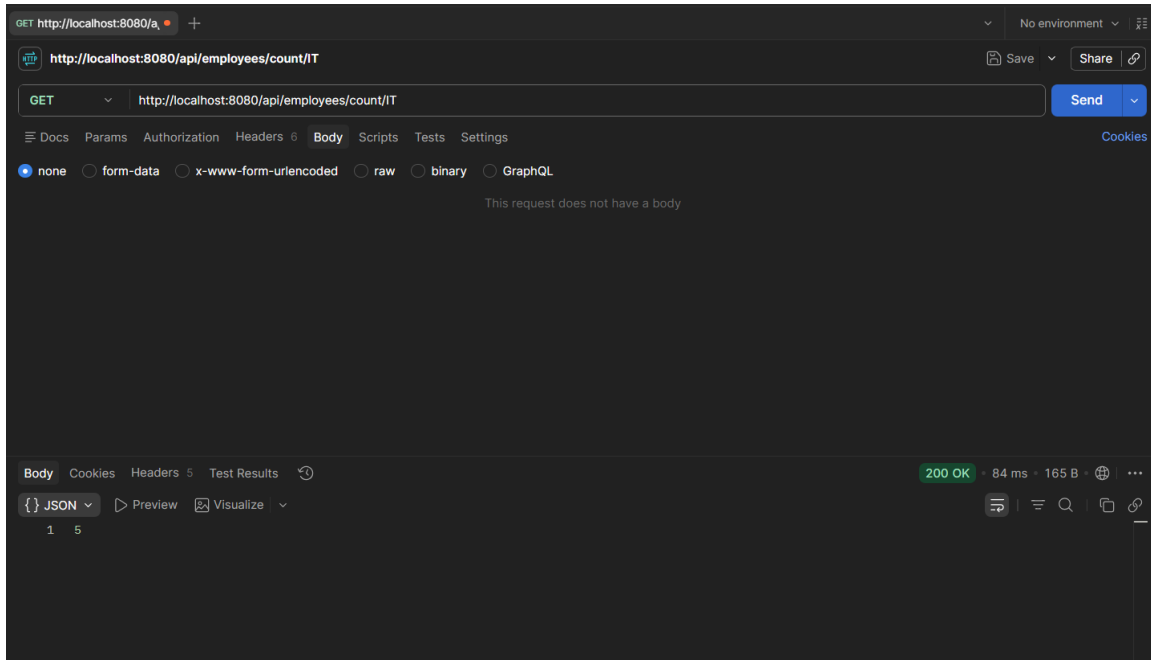
SEE ALL EMPLOYEES BELONGING TO THE IT DEPARTMENT ARE DISPLAYED SUCCESSFULLY.

3. GET API – Count Employees by Department:

This API retrieves all employees belonging to a particular department. It uses the derived query method `findByDepartment()` provided by Spring Data JPA.

GET :

<http://localhost:8080/api/employees/count/IT>



SEE THE TOTAL NUMBER(5) OF EMPLOYEES IN THE IT DEPARTMENT IS DISPLAYED SUCCESSFULLY.

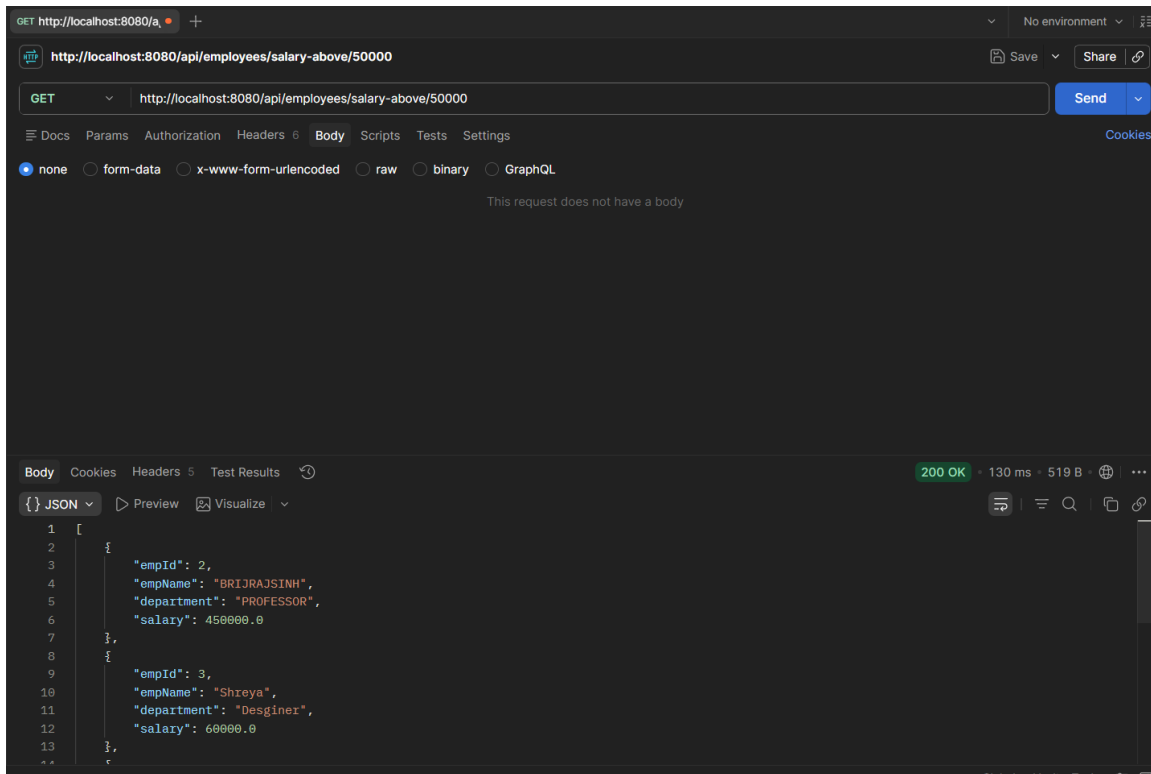
4. GET API – Find Employees with Salary Greater Than:

This API retrieves all employees whose salary is greater than the specified value. It uses the `findBySalaryGreaterThan()` derived query method.

GET :

<http://localhost:8080/api/employees/salary-above/50000>

the all employee who have more than 50k salary will be displayed



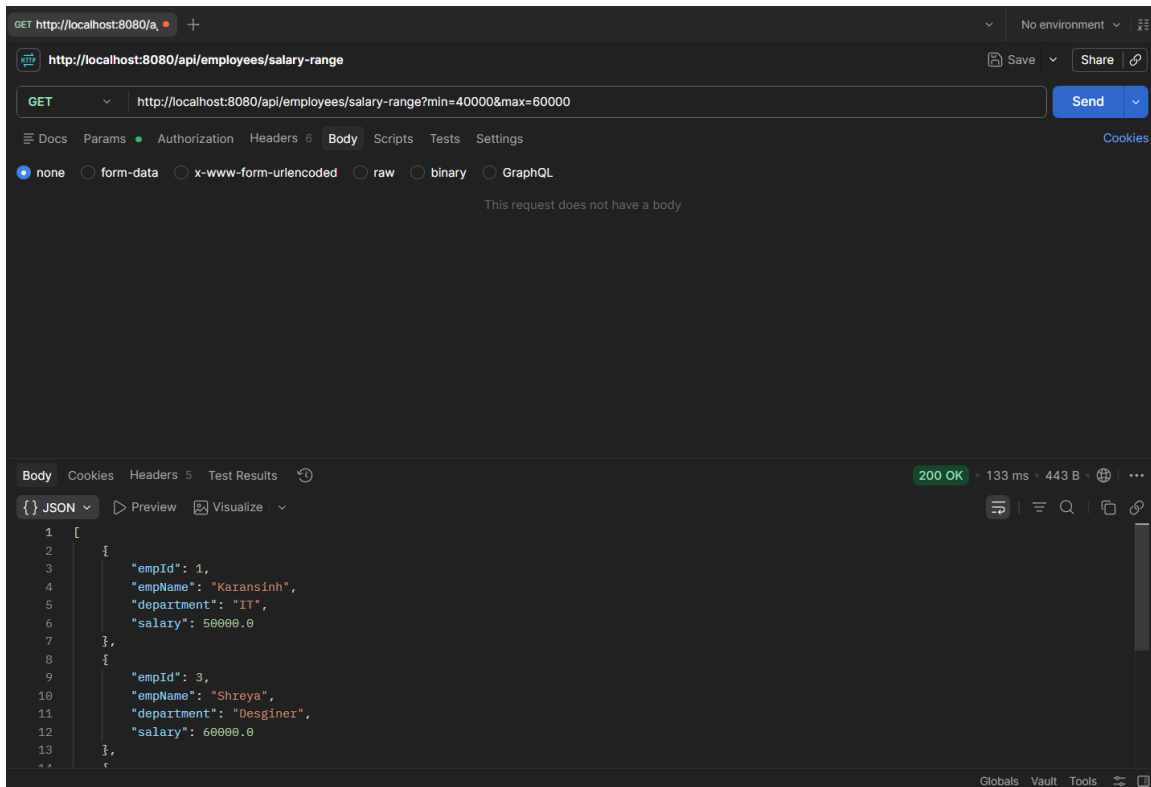
SEE ALL EMPLOYEES HAVING SALARY GREATER THAN ₹40,000 ARE DISPLAYED SUCCESSFULLY.

5. GET API – Find Employees by Salary Range (JPQL Query):

This API retrieves employees whose salary lies between the specified minimum and maximum values. It uses a custom JPQL query with the `@Query` annotation.

GET :

<http://localhost:8080/api/employees/salary-range?min=40000&max=60000>



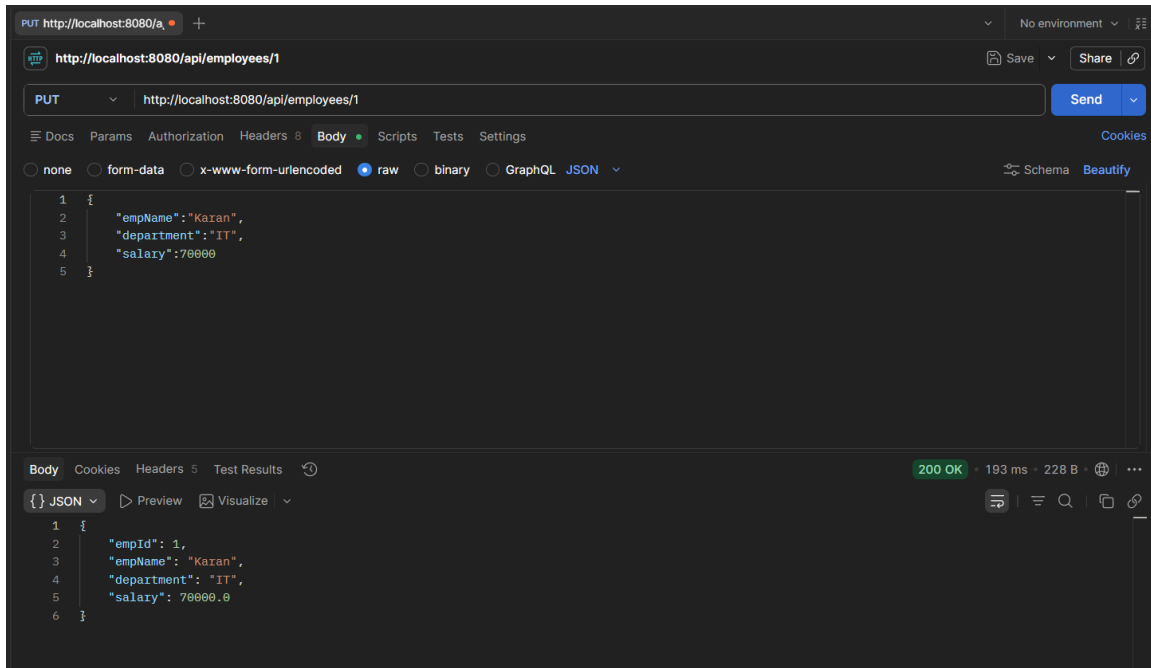
SEE ALL EMPLOYEES HAVING SALARY BETWEEN ₹40,000 AND ₹60,000 ARE DISPLAYED SUCCESSFULLY.

6. PUT API – PUT API – Update Employee

The PUT API is used to update an existing employee's information. The Employee ID is passed in the URL, while the updated details are sent in the request body.

PUT :

<http://localhost:8080/api/employees/1>



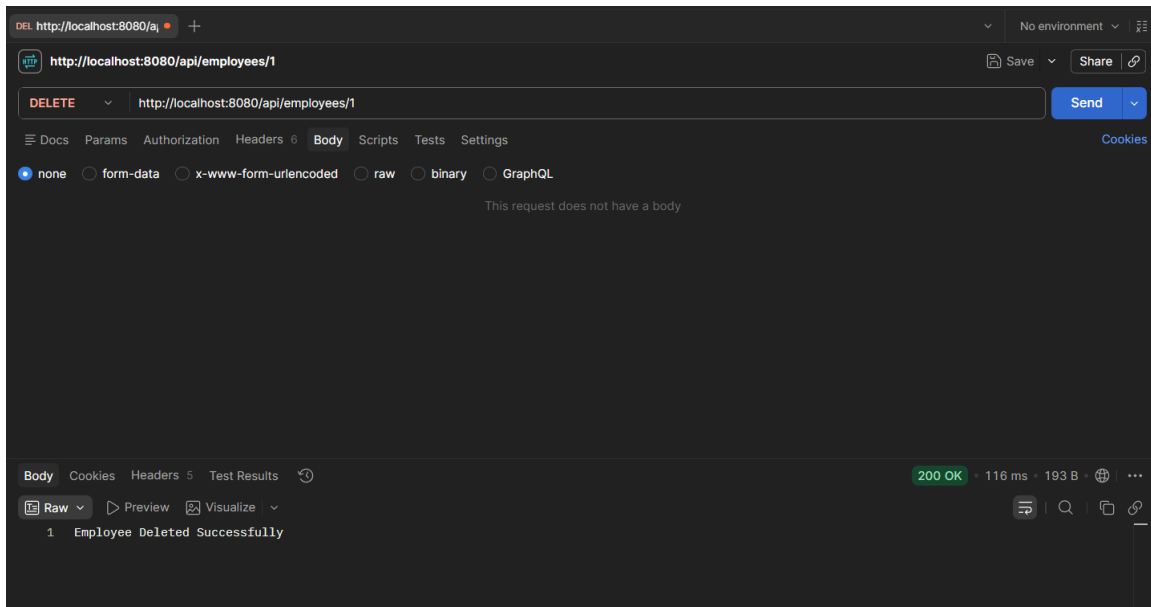
SEE THE EMPLOYEE DETAILS ARE UPDATED SUCCESSFULLY IN THE DATABASE.

6. DELETE API – Delete Employee

The DELETE API removes an employee record from the database using the Employee ID. It uses the `deleteById()` method of `JpaRepository`.

DELETE :

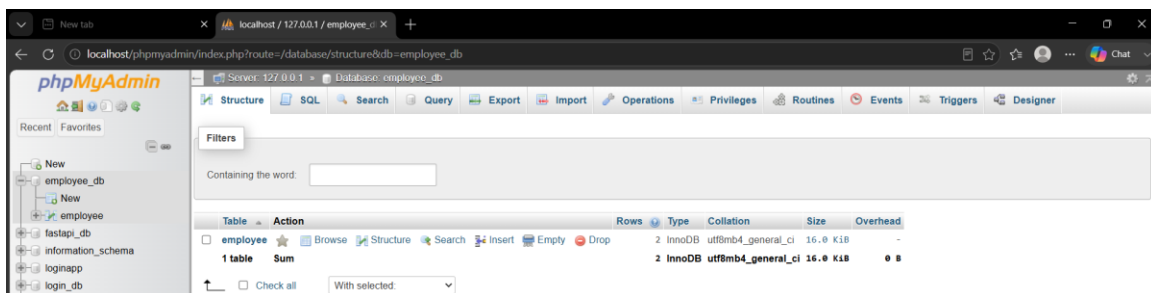
<http://localhost:8080/api/employees/1>



Database Output

TABLE'S AND DATA STORING SUCCEFULLY

NOTE : THIS TABLE AND DATABASE WAS ALREDY CREATED IN FIRST PRACTICAL



WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS

	emp_id	department	emp_name	salary
☐ Edit Copy Delete	2	PROFESSOR	BRIJRAJSINH	450000
☐ Edit Copy Delete	3	Desginer	Shreya	60000
☐ Edit Copy Delete	4	IT	SHRUTI	4500
☐ Edit Copy Delete	5	IT	BHUMI	10000
☐ Edit Copy Delete	6	PROFESSOR	PREXA	52000
☐ Edit Copy Delete	7	HR	Anita	45000
☐ Edit Copy Delete	8	IT	Rahul	65000
☐ Edit Copy Delete	9	Finance	Priya	38000
☐ Edit Copy Delete	10	IT	Karan	70000

Practical No. 4: One-to-One Mapping – Three Worked Examples

Objective:

To implement the One-to-One relationship using `@OneToOne`, `@JoinColumn` (owning side) and `mappedBy` (inverse side), with cascading so that saving a parent entity also persists its associated entity. Three independent one-to-one example pairs are implemented to reinforce the pattern across different domains; full CRUD is built and tested for the primary example (User–UserProfile).

Class Name(s) with Attributes:

User (@Entity — Example 1, owning side)

- `userId` : `int` (@Id @GeneratedValue)
- `username` : `String`

- email : String
- userProfile : UserProfile (@OneToOne, cascade=ALL, @JoinColumn(name="profile_id"))

UserProfile (@Entity — Example 1, inverse side)

- profileId : int (@Id @GeneratedValue)
- address : String
- phone : String
- dob : LocalDate
- user : User (@OneToOne(mappedBy="userProfile"))

Employee (@Entity — Example 2, owning side)

- empId : int (@Id @GeneratedValue)
- empName : String
- department : String
- passport : Passport (@OneToOne, cascade=ALL, @JoinColumn(name="passport_id"))

Passport (@Entity — Example 2, inverse side)

- passportId : int (@Id @GeneratedValue)
- passportNumber : String
- issuingCountry : String
- expiryDate : LocalDate
- employee : Employee (@OneToOne(mappedBy="passport"))

Country (@Entity — Example 3, owning side)

- countryId : int (@Id @GeneratedValue)
- countryName : String
- capital : Capital (@OneToOne, cascade=ALL, @JoinColumn(name="capital_id"))

Capital (@Entity — Example 3, inverse side)

- capitalId : int (@Id @GeneratedValue)

NAME: SOLANKI KARANSINH

ENROLLMENT: 25CI2110110

- cityName : String
- population : long
- country : Country (@OneToOne(mappedBy="capital"))

IN THIS PRACTICAL WE HAVE TO PERFORM 3 EXAMPLES

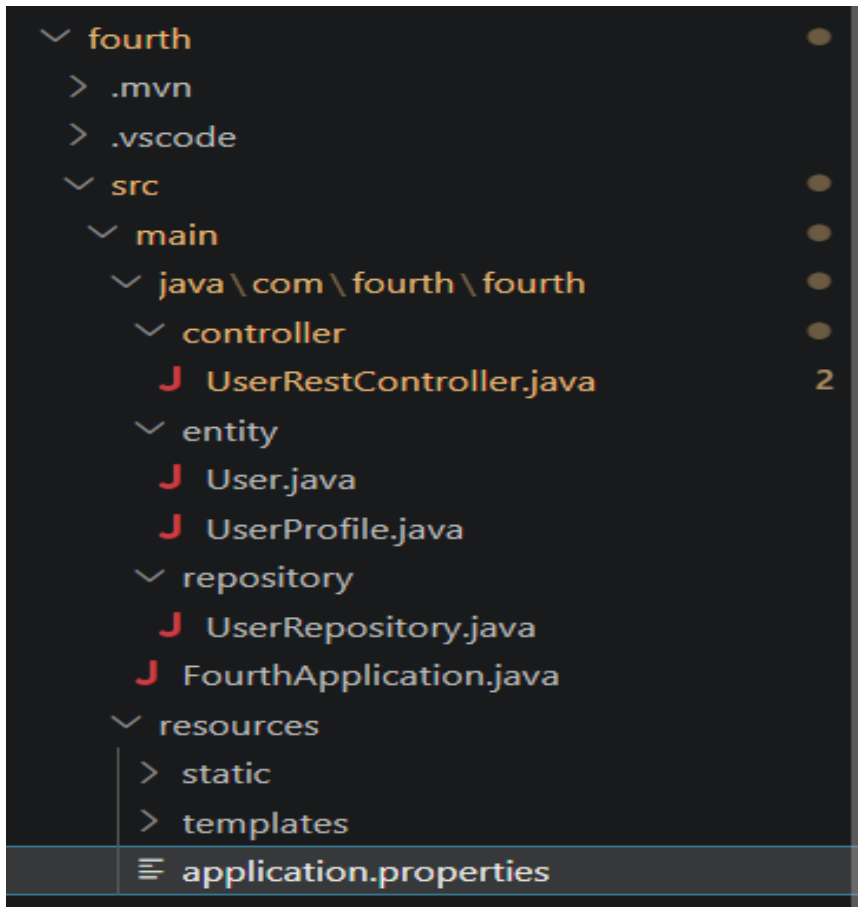
SO WE WILL GET WHOLE IDEA ABOUT HOW MAPPING ONE TO ONE WORKS
AND HOW TO JOIN THEM

THREE EXAMPLES ARE

- 1.USER ,USERPROFILE
- 2.EMPLOYEE ,PASSPORT
- 3.COUNTRY ,CAPITAL

1. USER EXAMPLE

- 1.user table(profileid as foreign key)
- 2.userprofile table(profileid as foreign key)



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: user

Table: 1.User (created automatically by Hibernate)

2.UserProfile (created automatically by Hibernate)

Important Files

User.java(ENTITY model class)

```
package com.fourth.fourth.entity;

import com.fasterxml.jackson.annotation.JsonManagedReference;
import jakarta.persistence.*;

@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer userId;

    private String username;

    private String email;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id")
    @JsonManagedReference
    private UserProfile userProfile;

    public User() {
    }

    public Integer getUserId() {
        return userId;
    }

    public void setUserId(Integer userId) {
        this.userId = userId;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
}
```

```
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public UserProfile getUserProfile() {  
    return userProfile;  
}  
  
public void setUserProfile(UserProfile userProfile) {  
    this.userProfile = userProfile;  
}  
}
```

UserProfile.java(ENTITY model class)

```
package com.fourth.fourth.entity;  
  
import com.fasterxml.jackson.annotation.JsonBackReference;  
import jakarta.persistence.*;  
  
import java.time.LocalDate;  
  
@Entity  
public class UserProfile {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Integer profileId;  
  
    private String address;  
  
    private String phone;  
  
    private LocalDate dob;  
  
    @OneToOne(mappedBy = "userProfile")  
    @JsonBackReference  
    private User user;  
}
```

```
public UserProfile() {  
}  
  
public Integer getProfileId() {  
    return profileId;  
}  
  
public void setProfileId(Integer profileId) {  
    this.profileId = profileId;  
}  
  
public String getAddress() {  
    return address;  
}  
  
public void setAddress(String address) {  
    this.address = address;  
}  
  
public String getPhone() {  
    return phone;  
}  
  
public void setPhone(String phone) {  
    this.phone = phone;  
}  
  
public LocalDate getDob() {  
    return dob;  
}  
  
public void setDob(LocalDate dob) {  
    this.dob = dob;  
}  
  
public User getUser() {  
    return user;  
}  
  
public void setUser(User user) {  
    this.user = user;  
}  
}
```

UserRepository.java(Repository file)

```
package com.fourth.fourth.repository;

import com.fourth.fourth.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Integer> {

}
```

UserController.java(Controller file)

```
package com.fourth.fourth.controller;

import com.fourth.fourth.entity.User;
import com.fourth.fourth.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserRestController {

    @Autowired
    private UserRepository repository;

    @PostMapping
    public User saveUser(@RequestBody User user) {
        return repository.save(user);
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Integer id) {
        return repository.findById(id).orElse(null);
    }

    @PutMapping("/{id}")
    public User updateUser(@PathVariable Integer id,
                           @RequestBody User user) {

        user.setUserId(id);
        return repository.save(user);
    }
}
```

```
}  
  
@DeleteMapping("/{id}")  
public String deleteUser(@PathVariable Integer id) {  
  
    repository.deleteById(id);  
    return "User Deleted Successfully";  
}  
  
}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=fourth  
spring.datasource.url=jdbc:mysql://localhost:3306/user  
spring.datasource.username=root  
spring.datasource.password=  
  
spring.jpa.hibernate.ddl-auto=update  
  
spring.jpa.show-sql=true  
  
spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

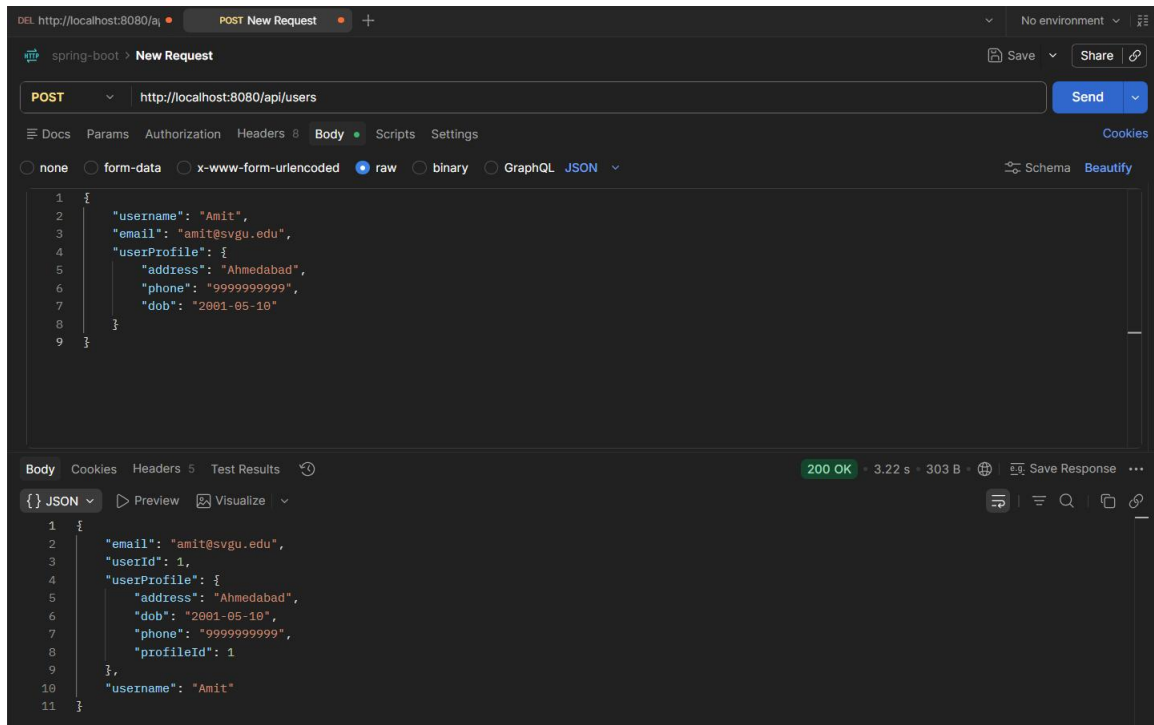
1. POST API – Add User with User Profile:

The POST API is used to insert a new **User** along with its associated **UserProfile** into the MySQL database. Because cascade = CascadeType.ALL is used, both the User and UserProfile are saved automatically with a single request.

POST:

<http://localhost:8080/api/users>

the value's given in raw(json) will be added to the both table



SEE THE USER AND ITS USER PROFILE ARE ADDED SUCCESSFULLY INTO THE DATABASE.

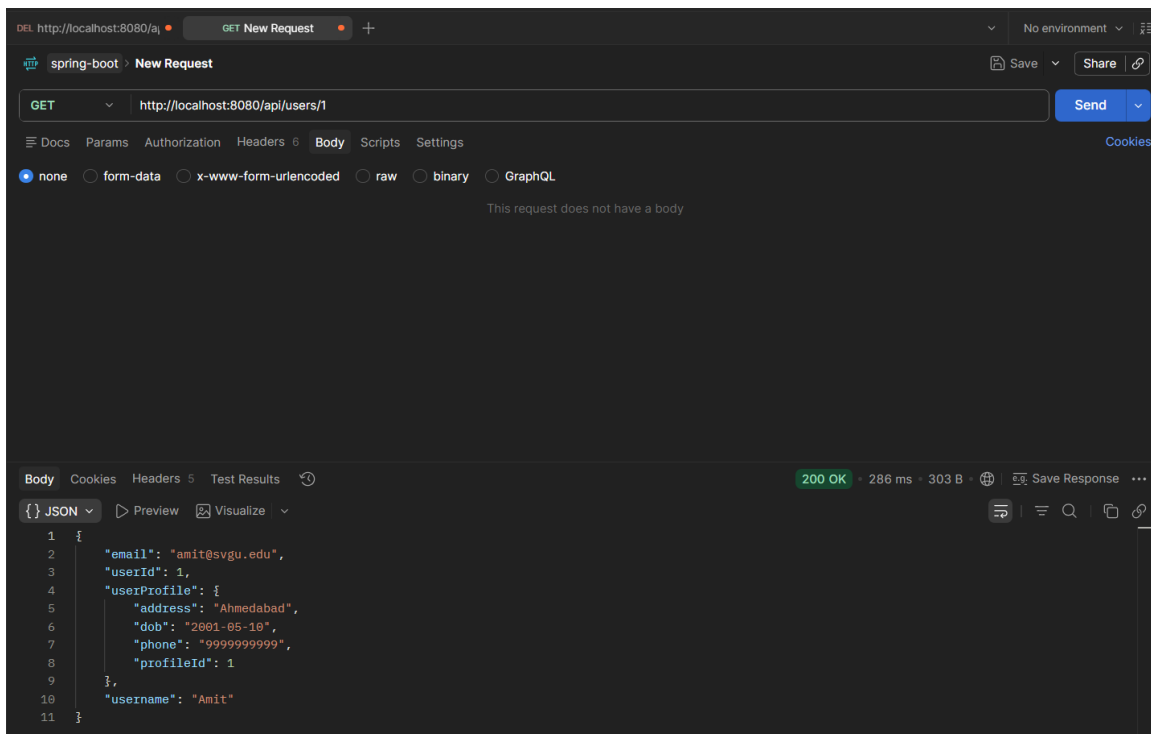
2. GET API – Retrieve User by ID:

This API retrieves a single user's information based on the User ID provided in the URL. It also returns the corresponding UserProfile associated with that user.

GET:

<http://localhost:8080/api/users/1>

the detail's of id 1 from both table will be shown



3. PUT API – Update User

NAME: SOLANKI KARANSINH

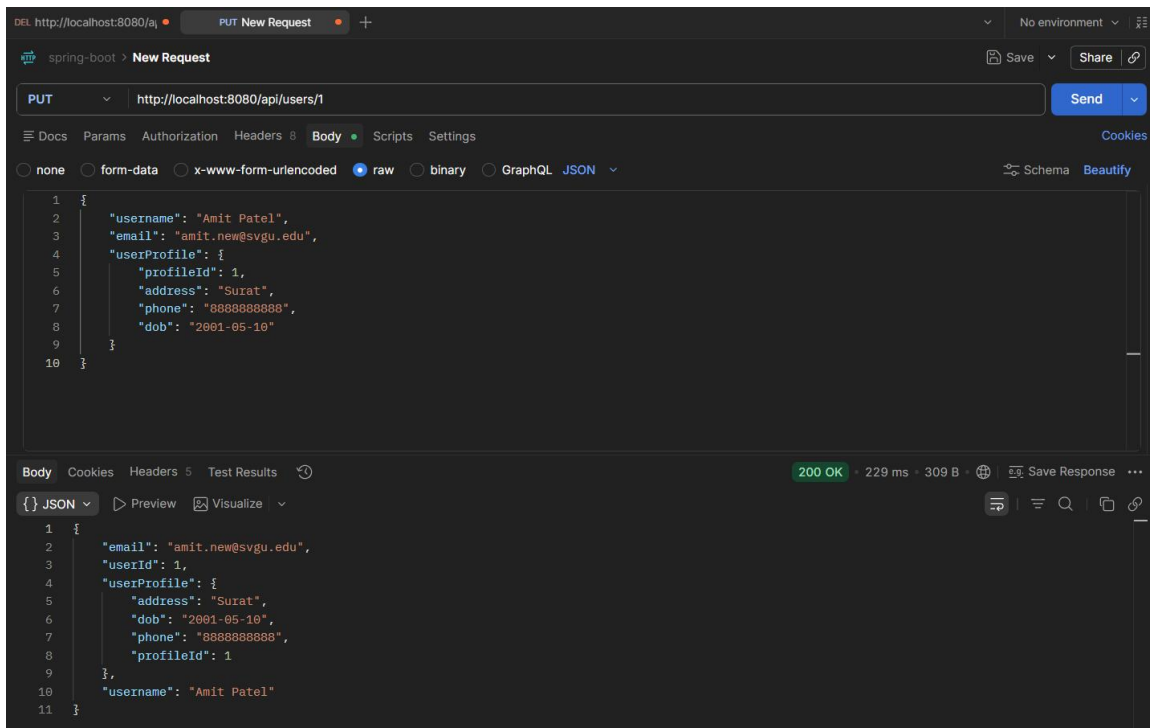
ENROLLMENT: 25CI2110110

The PUT API is used to update an existing user's information as well as the associated UserProfile details. The User ID is passed in the URL, and the updated data is sent in the request body

PUT:

<http://localhost:8080/api/users/1>

just add the details you want to update and send the data of id 1 from both table will be updated



4. DELETE API – Delete User

NAME: SOLANKI KARANSINH

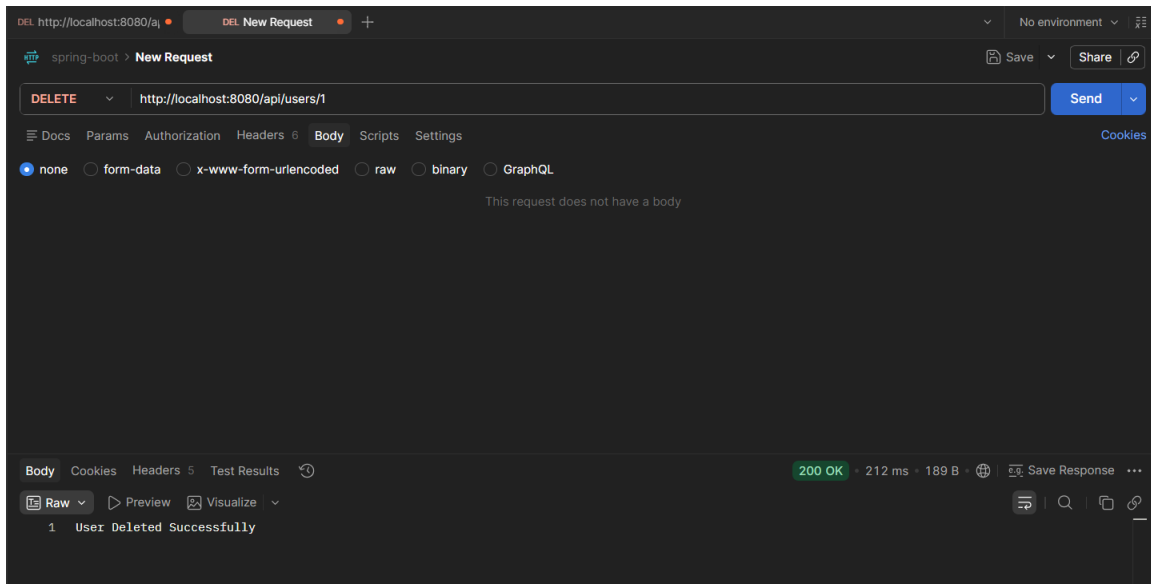
ENROLLMENT: 25CI2110110

The DELETE API removes a user from the database based on the User ID. Since CascadeType.ALL is enabled, the associated UserProfile is also deleted automatically.

Delete:

<http://localhost:8080/api/users/1>

id 1 will be removed from both table's



SECOND EXAMPLE FROM HERE

2.Employee Passport

entity

| — Employee.java

| — Passport.java

repository

| — EmployeeRepository.java

controller

| — EmployeeRestController.java

Other folder's same as upper

Database

Database Name: user

Table: 1.Employee (created automatically by Hibernate)

2.Passport (created automatically by Hibernate)

Important Files

Employee.java(ENTITY model class)

```
package com.fourth.fourth.entity;

import com.fasterxml.jackson.annotation.JsonManagedReference;
import jakarta.persistence.*;

@Entity
public class Employee {
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Integer empId;

private String empName;

private String department;

@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "passport_id")
@JsonManagedReference
private Passport passport;

public Employee() {
}

public Integer getEmpId() {
    return empId;
}

public void setEmpId(Integer empId) {
    this.empId = empId;
}

public String getEmpName() {
    return empName;
}

public void setEmpName(String empName) {
    this.empName = empName;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public Passport getPassport() {
    return passport;
}

public void setPassport(Passport passport) {
    this.passport = passport;
}
}
```

Passport.java(ENTITY model class)

```
package com.fourth.fourth.entity;

import com.fasterxml.jackson.annotation.JsonBackReference;
import jakarta.persistence.*;

@Entity
public class Passport {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer passportId;

    private String passportNumber;

    private String issuingCountry;

    private String expiryDate;

    @OneToOne(mappedBy = "passport")
    @JsonBackReference
    private Employee employee;

    public Passport() {
    }

    public Integer getPassportId() {
        return passportId;
    }

    public void setPassportId(Integer passportId) {
        this.passportId = passportId;
    }

    public String getPassportNumber() {
        return passportNumber;
    }

    public void setPassportNumber(String passportNumber) {
        this.passportNumber = passportNumber;
    }

    public String getIssuingCountry() {
        return issuingCountry;
    }
}
```

```
public void setIssuingCountry(String issuingCountry) {
    this.issuingCountry = issuingCountry;
}

public String getExpiryDate() {
    return expiryDate;
}

public void setExpiryDate(String expiryDate) {
    this.expiryDate = expiryDate;
}

public Employee getEmployee() {
    return employee;
}

public void setEmployee(Employee employee) {
    this.employee = employee;
}
}
```

EmployeeController.java(Controller file)

```
package com.fourth.fourth.controller;

import com.fourth.fourth.entity.Employee;
import com.fourth.fourth.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/employees-passport")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository repository;

    @PostMapping
    public Employee saveEmployee(@RequestBody Employee employee) {
        return repository.save(employee);
    }

    @GetMapping
    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }

    @GetMapping("/{id}")
```

```
public Employee getEmpl(@PathVariable Integer id) {  
    return repository.findById(id).orElse(null);  
}  
  
@PutMapping("/{id}")  
public Employee updateEmployee(@PathVariable Integer id,  
    @RequestBody Employee employee) {  
  
    employee.setEmpId(id);  
    return repository.save(employee);  
}  
  
@DeleteMapping("/{id}")  
public String deleteEmployee(@PathVariable Integer id) {  
  
    repository.deleteById(id);  
    return "Employee Deleted Successfully";  
}  
}
```

EmployeeRepository.java(Repository file)

```
package com.fourth.fourth.repository;  
  
import com.fourth.fourth.entity.User;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface UserRepository extends JpaRepository<User, Integer> {  
  
}  
}
```

API'S TESTING USING POSTMAN

1. POST API – Add Employee with Passport:

The POST API is used to insert a new **User** along with its associated **UserProfile** into the MySQL database. Because cascade =

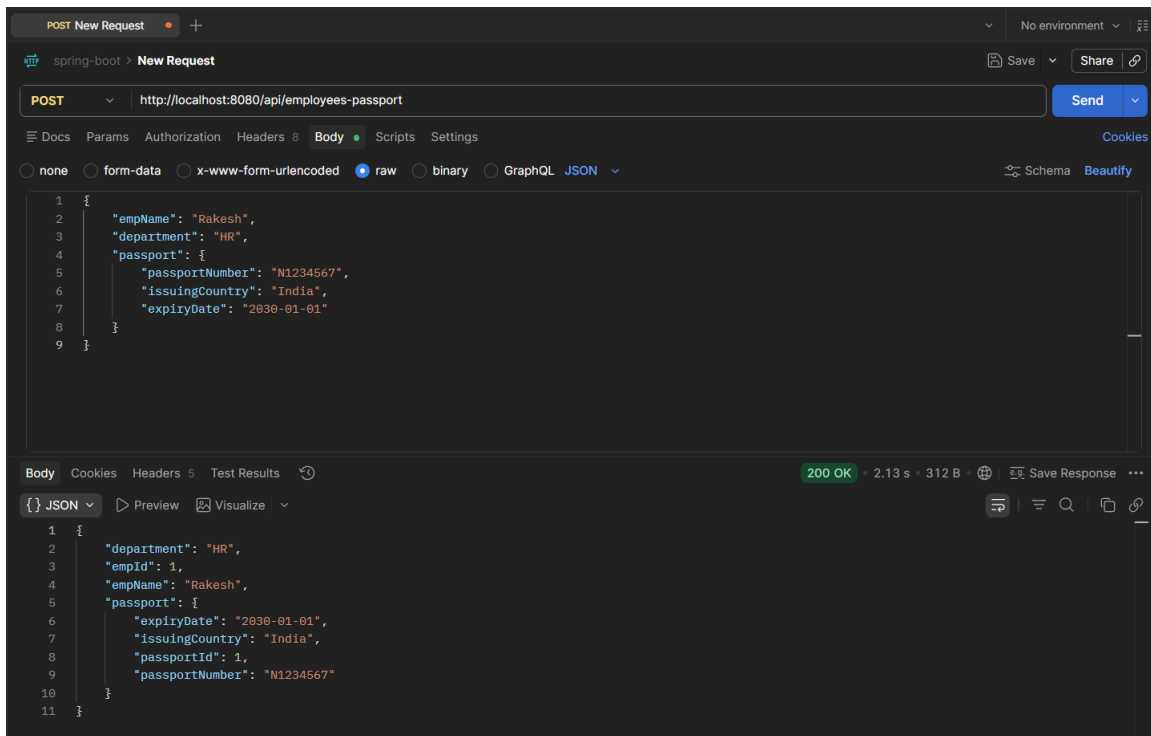
NAME: SOLANKI KARANSINH

ENROLLMENT: 25CI2110110

CascadeType.ALL is used, both the User and UserProfile are saved automatically with a single request.

POST:

<http://localhost:8080/api/employees-passport>



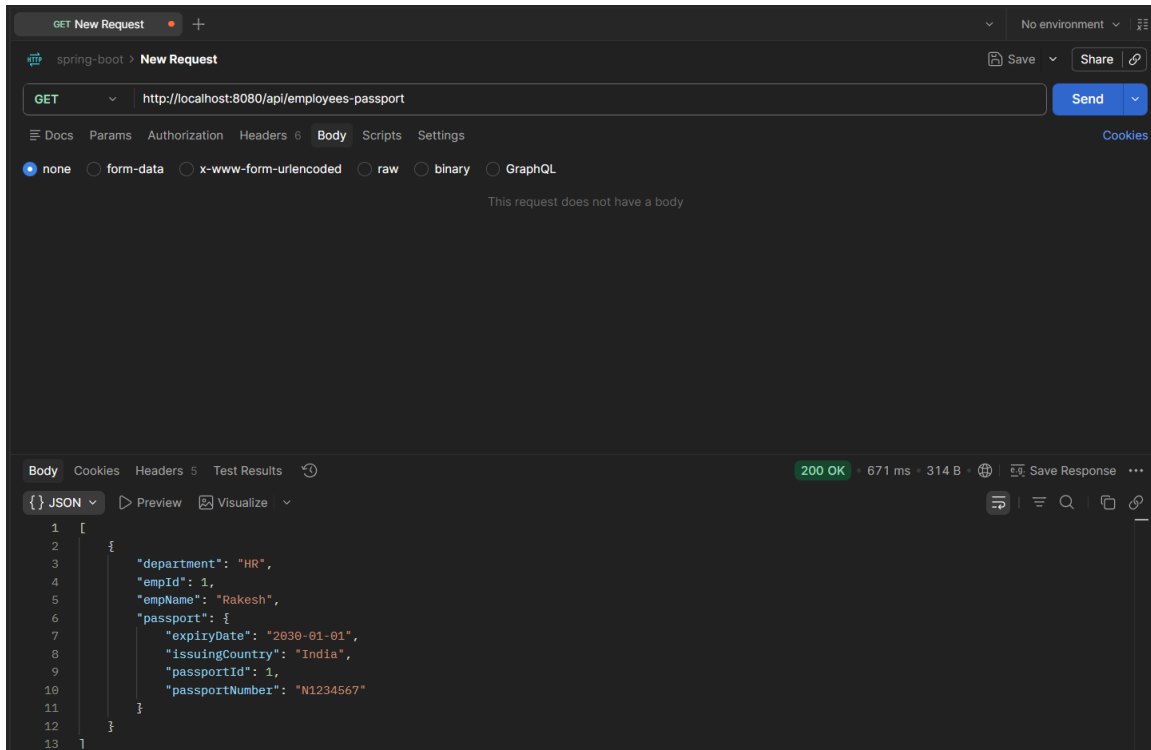
SEE THE EMPLOYEE AND ITS PASSPORT ARE ADDED SUCCESSFULLY INTO THE DATABASE.

2. GET API – Retrieve All Employees:

The GET API retrieves all Employee records available in the database along with their associated Passport details. The response is returned in JSON format.

GET:

<http://localhost:8080/api/employees-passport>



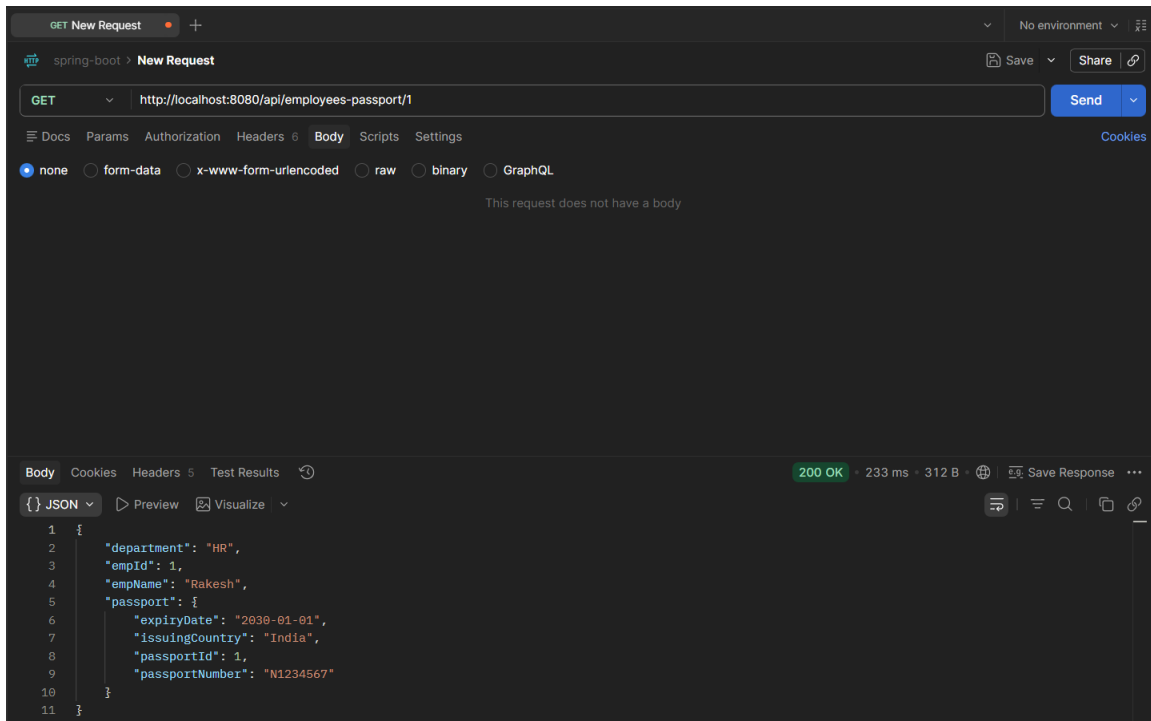
SEE ALL EMPLOYEE RECORDS ALONG WITH THEIR PASSPORT DETAILS ARE DISPLAYED SUCCESSFULLY.

3. GET API – Retrieve Employee by ID:

This API retrieves the details of a single Employee based on the Employee ID passed in the URL. It also returns the Passport associated with that Employee.

GET:

<http://localhost:8080/api/employees-passport/1>



THIRD EXAMPLE FROM HERE

3.Country ,Capital

entity

| — Country.java

| — Capital.java

repository

| — CountryRepository.java

controller

| — CountryRestController.java

Other folder's same as upper

Database

Database Name: user

Table: 1.Country (created automatically by Hibernate)

2.Capital (created automatically by Hibernate)

Important Files

Country.java(ENTITY model class)

```
package com.fourth.fourth.entity;

import com.fasterxml.jackson.annotation.JsonManagedReference;
import jakarta.persistence.*;

@Entity
public class Country {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer countryId;

    private String countryName;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "capital_id")
    @JsonManagedReference
    private Capital capital;

    public Country() {
    }

    public Integer getCountryId() {
        return countryId;
    }
}
```

```
}

    public void setCountryId(Integer countryId) {
        this.countryId = countryId;
    }

    public String getCountryName() {
        return countryName;
    }

    public void setCountryName(String countryName) {
        this.countryName = countryName;
    }

    public Capital getCapital() {
        return capital;
    }

    public void setCapital(Capital capital) {
        this.capital = capital;
    }
}
```

Capital.java(ENTITY model class)

```
package com.fourth.fourth.entity;

import com.fasterxml.jackson.annotation.JsonBackReference;
import jakarta.persistence.*;

@Entity
public class Capital {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer capitalId;

    private String cityName;

    private Long population;

    @OneToOne(mappedBy = "capital")
    @JsonBackReference
    private Country country;
}
```

```
public Capital() {}

public Integer getCapitalId() {
    return capitalId;
}

public void setCapitalId(Integer capitalId) {
    this.capitalId = capitalId;
}

public String getCityName() {
    return cityName;
}

public void setCityName(String cityName) {
    this.cityName = cityName;
}

public Long getPopulation() {
    return population;
}

public void setPopulation(Long population) {
    this.population = population;
}

public Country getCountry() {
    return country;
}

public void setCountry(Country country) {
    this.country = country;
}
}
```

CountryController.java(Controller file)

```
package com.fourth.fourth.controller;

import com.fourth.fourth.entity.Country;
import com.fourth.fourth.repository.CountryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
```

```
@RestController  
@RequestMapping("/api/countries")  
public class CountryRestController {  
  
    @Autowired  
    private CountryRepository repository;  
  
    @PostMapping  
    public Country addCountry(@RequestBody Country country) {  
        return repository.save(country);  
    }  
  
    @GetMapping  
    public List<Country> getAllCountries() {  
        return repository.findAll();  
    }  
  
    @GetMapping("/{id}")  
    public Country getCountry(@PathVariable Integer id) {  
        return repository.findById(id).orElse(null);  
    }  
}
```

CountryRepository.java(Repository file)

```
package com.fourth.fourth.repository;  
  
import com.fourth.fourth.entity.User;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface CountryRepository extends JpaRepository<User,  
Integer> {  
  
    }
```

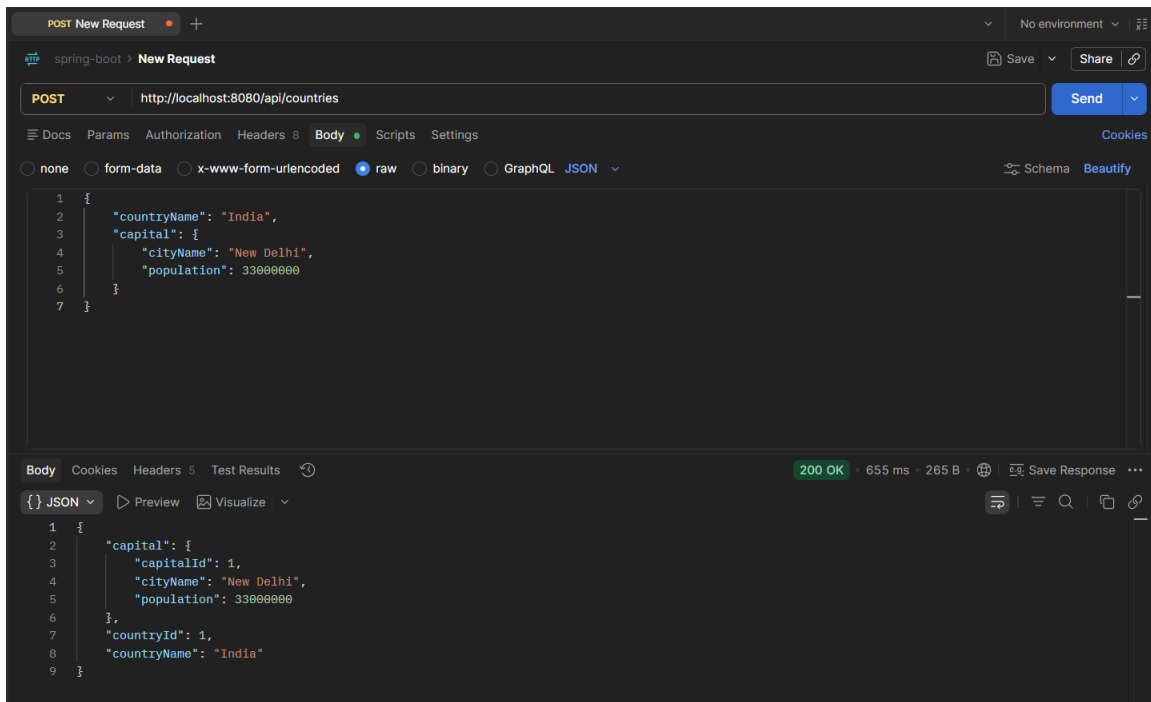
API'S TESTING USING POSTMAN

1. POST API – Add Country with Capital:

The POST API is used to insert a new **Country** along with its associated **Capital** into the MySQL database. Since CascadeType.ALL is used, the Capital record is automatically saved when the Country is saved.

POST:

<http://localhost:8080/api/countries>



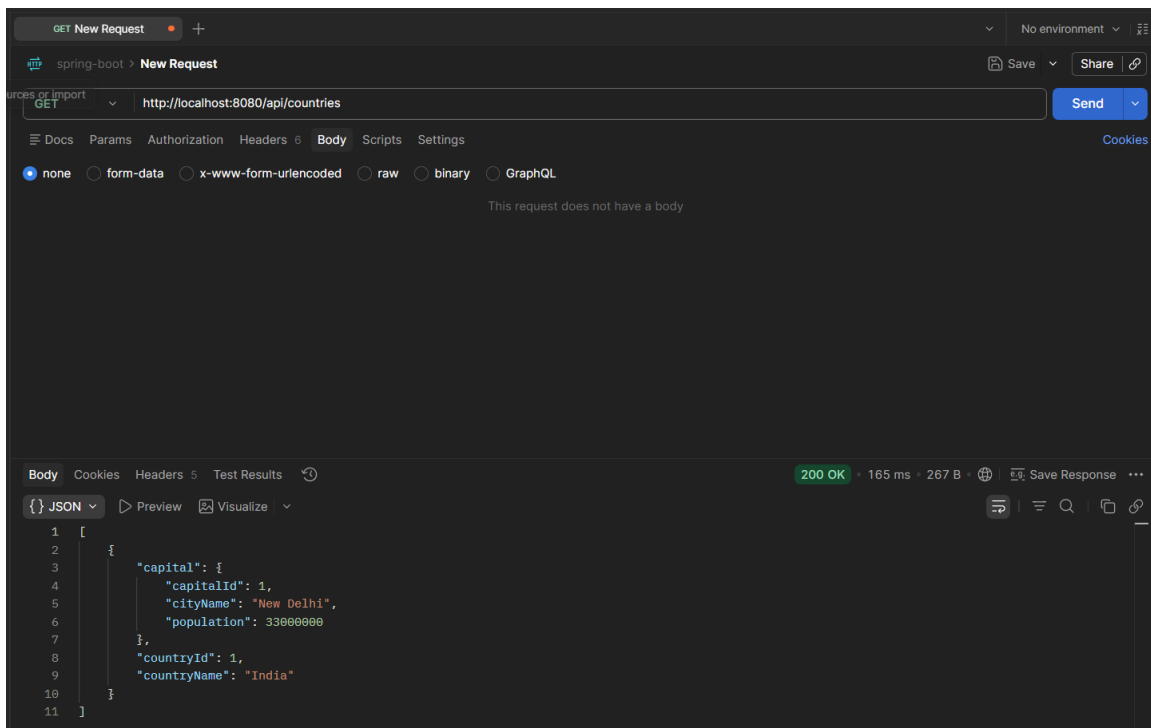
SEE THE COUNTRY AND ITS CAPITAL ARE ADDED SUCCESSFULLY INTO THE DATABASE.

2. GET API – Retrieve All Countries:

The GET API retrieves all Country records available in the database along with their associated Passport details. The response is returned in JSON format.

GET:

<http://localhost:8080/api/countries>



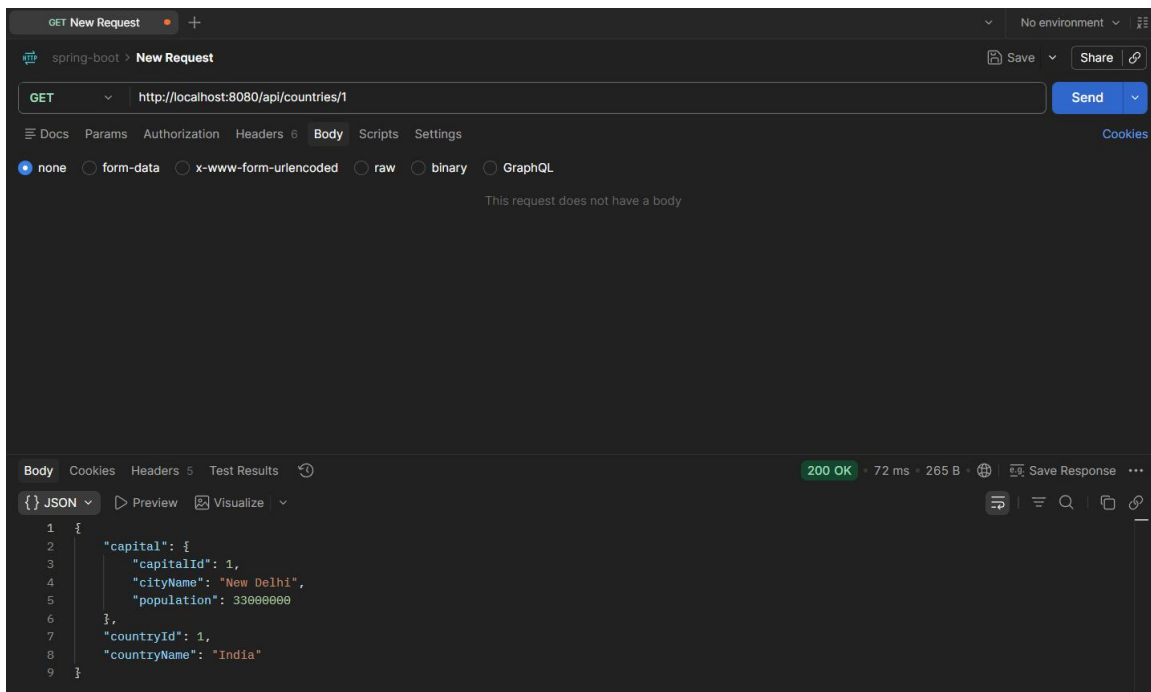
SEE ALL COUNTRY RECORDS ALONG WITH THEIR CAPITAL DETAILS ARE DISPLAYED SUCCESSFULLY.

3. GET API – Retrieve Country by ID:

This API retrieves the details of a single Country based on the Country ID passed in the URL. It also returns the Capital associated with that Country.

GET:

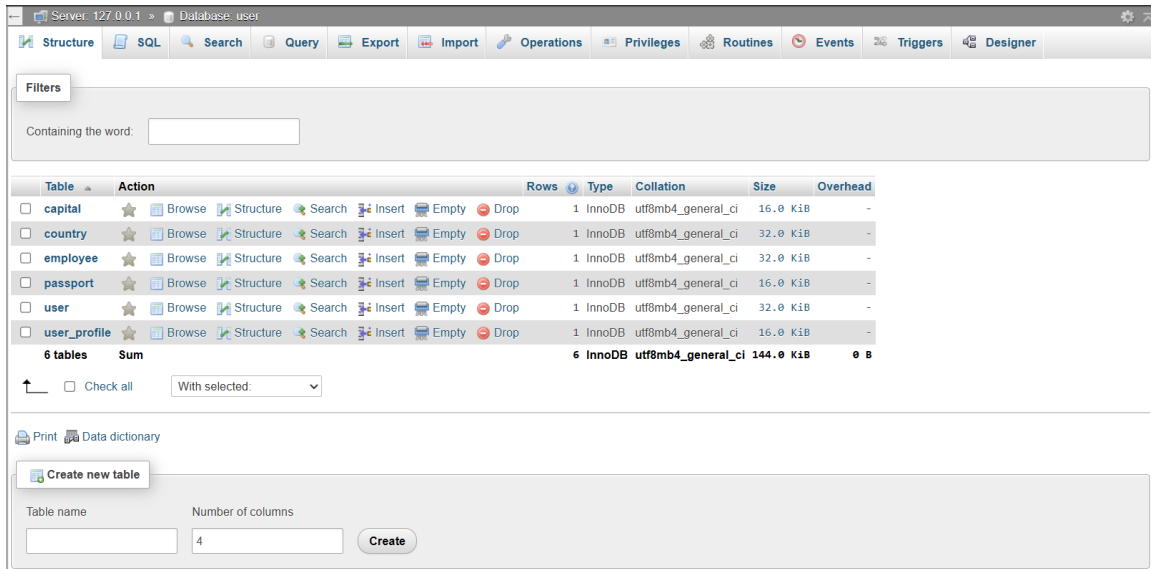
<http://localhost:8080/api/countries/1>



SEE THE COUNTRY DETAILS ALONG WITH THE CAPITAL INFORMATION ARE DISPLAYED SUCCESSFULLY.

Database Output

TABLE'S AND DATA STORING SUCCEFULLY



Server: 127.0.0.1 » Database: user

Structure | SQL | Search | Query | Export | Import | Operations | Privileges | Routines | Events | Triggers | Designer

Filters

Containing the word:

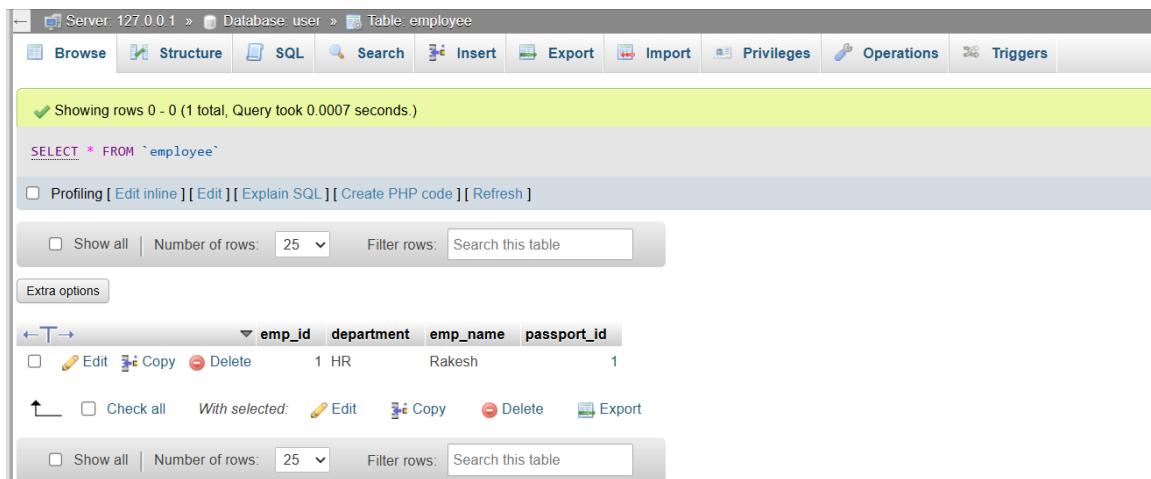
Table	Action	Rows	Type	Collation	Size	Overhead
☐ capital		1	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ country		1	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ employee		1	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ passport		1	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ user		1	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ user_profile		1	InnoDB	utf8mb4_general_ci	16.0 KiB	-
6 tables	Sum	6	InnoDB	utf8mb4_general_ci	144.0 KiB	0 B

☐ Check all With selected:

Print Data dictionary

Create new table

Table name: Number of columns:



Server: 127.0.0.1 » Database: user » Table: employee

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Triggers

Showing rows 0 - 0 (1 total, Query took 0.0007 seconds.)

`SELECT * FROM `employee``

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	emp_id	department	emp_name	passport_id
☐ Edit	1	HR	Rakesh	1

☐ Check all With selected:

☐ Show all | Number of rows: 25 | Filter rows:

Practical No. 5 One-to-Many / Many-to-One Mapping – Department and Employee

Objective:

To implement a bidirectional One-to-Many (Department → Employees) and Many-to-One (Employee → Department) relationship using mappedBy on the parent side and @JoinColumn on the child side, and to persist a parent with a list of children in one request.

Class Name(s) with Attributes:

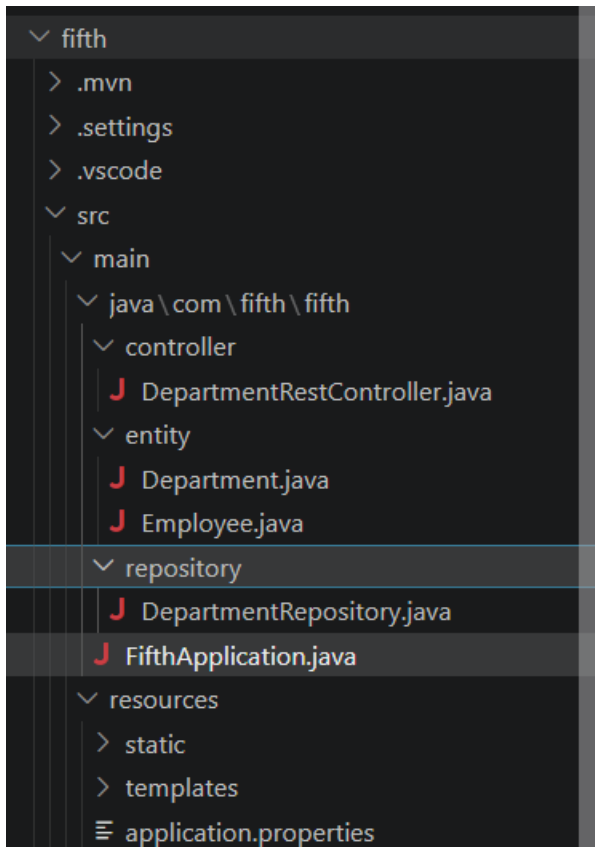
Department (@Entity — parent, One side)

- deptId : int (@Id @GeneratedValue)
- deptName : String
- employees : List<Employee> (@OneToMany(mappedBy="department", cascade=ALL))

Employee (@Entity — child, Many side)

- empId : int (@Id @GeneratedValue)
- empName : String
- salary : double
- department : Department (@ManyToOne @JoinColumn(name="dept_id"))

Showing many to one and one to many relations



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: employee_db

Table: 1.Department(created automatically by Hibernate)
2.Employee (created automatically by Hibernate)

Important Files

Department.java(ENTITY model class)

```
package com.fifth.fifth.entity;

import java.util.ArrayList;
import java.util.List;

import jakarta.persistence.*;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer deptId;

    private String deptName;

    @OneToMany(mappedBy = "department", cascade =
    CascadeType.ALL)
    private List<Employee> employees = new ArrayList<>();

    public Department() {
    }

    public Integer getDeptId() {
        return deptId;
    }

    public void setDeptId(Integer deptId) {
        this.deptId = deptId;
    }

    public String getDeptName() {
        return deptName;
    }
}
```

```
public void setDeptName(String deptName) {
    this.deptName = deptName;
}

public List<Employee> getEmployees() {
    return employees;
}

public void setEmployees(List<Employee> employees) {
    this.employees = employees;
}
}
```

Employee.java(ENTITY model class)

```
package com.fifth.fifth.entity;

import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;

    private double salary;

    @ManyToOne
    @JoinColumn(name = "dept_id")
    private Department department;

    public Employee() {
    }

    public Integer getEmpId() {
        return empId;
    }

    public void setEmpId(Integer empId) {
        this.empId = empId;
    }
}
```

```
public String getEmpName() {  
    return empName;  
}  
  
public void setEmpName(String empName) {  
    this.empName = empName;  
}  
  
public double getSalary() {  
    return salary;  
}  
  
public void setSalary(double salary) {  
    this.salary = salary;  
}  
  
public Department getDepartment() {  
    return department;  
}  
  
public void setDepartment(Department department) {  
    this.department = department;  
}  
}
```

DepartmentRestController.java(Controller file)

```
package com.fifth.fifth.controller;  
  
import com.fifth.fifth.entity.Department;  
import com.fifth.fifth.entity.Employee;  
import com.fifth.fifth.repository.DepartmentRepository;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.*;  
  
import java.util.List;  
  
@RestController  
@RequestMapping("/api/departments")  
public class DepartmentRestController {  
  
    @Autowired  
    private DepartmentRepository repository;  
  
    // Add Department with Employees
```

```
@PostMapping
public Department addDepartment(@RequestBody Department
department) {

    for (Employee employee : department.getEmployees()) {
        employee.setDepartment(department);
    }

    return repository.save(department);
}

// Get Department by ID
@GetMapping("/{id}")
public Department getDepartment(@PathVariable Integer id) {

    return repository.findById(id).orElse(null);
}

// Get Employees of a Department
@GetMapping("/{id}/employees")
public List<Employee> getEmployees(@PathVariable Integer id) {

    Department department = repository.findById(id).orElse(null);

    if (department == null) {
        return null;
    }

    return department.getEmployees();
}

// Update Employee of Department
@PutMapping("/{deptId}/employees/{empId}")
public Department updateEmployee(@PathVariable Integer deptId,
    @PathVariable Integer empId,
    @RequestBody Employee updatedEmployee) {

    Department department = repository.findById(deptId).orElse(null);

    if (department == null) {
        return null;
    }

    for (Employee employee : department.getEmployees()) {

        if (employee.getEmpId().equals(empId)) {

            employee.setEmpName(updatedEmployee.getEmpName());
            employee.setSalary(updatedEmployee.getSalary());
            break;
        }
    }
}
```

```
    }  
    }  
  
    return repository.save(department);  
}  
  
// Delete Employee from Department  
@DeleteMapping("/{deptId}/employees/{empId}")  
public String deleteEmployee(@PathVariable Integer deptId,  
                             @PathVariable Integer empId) {  
  
    Department department = repository.findById(deptId).orElse(null);  
  
    if (department == null) {  
        return "Department Not Found";  
    }  
  
    department.getEmployees().removeIf(employee ->  
employee.getEmpId().equals(empId));  
  
    repository.save(department);  
  
    return "Employee Deleted Successfully";  
}  
  
// Delete Department  
@DeleteMapping("/{id}")  
public String deleteDepartment(@PathVariable Integer id) {  
  
    repository.deleteById(id);  
  
    return "Department Deleted Successfully";  
}  
}
```

EmployeeRepository.java(Repository file)

```
package com.fifth.fifth.repository;  
  
import com.fifth.fifth.entity.Department;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface DepartmentRepository extends  
    JpaRepository<Department, Integer> {  
  
}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=fifth  
spring.datasource.url=jdbc:mysql://localhost:3306/employee_db  
spring.datasource.username=root  
spring.datasource.password=  
  
spring.jpa.hibernate.ddl-auto=update  
  
spring.jpa.show-sql=true  
  
spring.jpa.properties.hibernate.format_sql=true
```

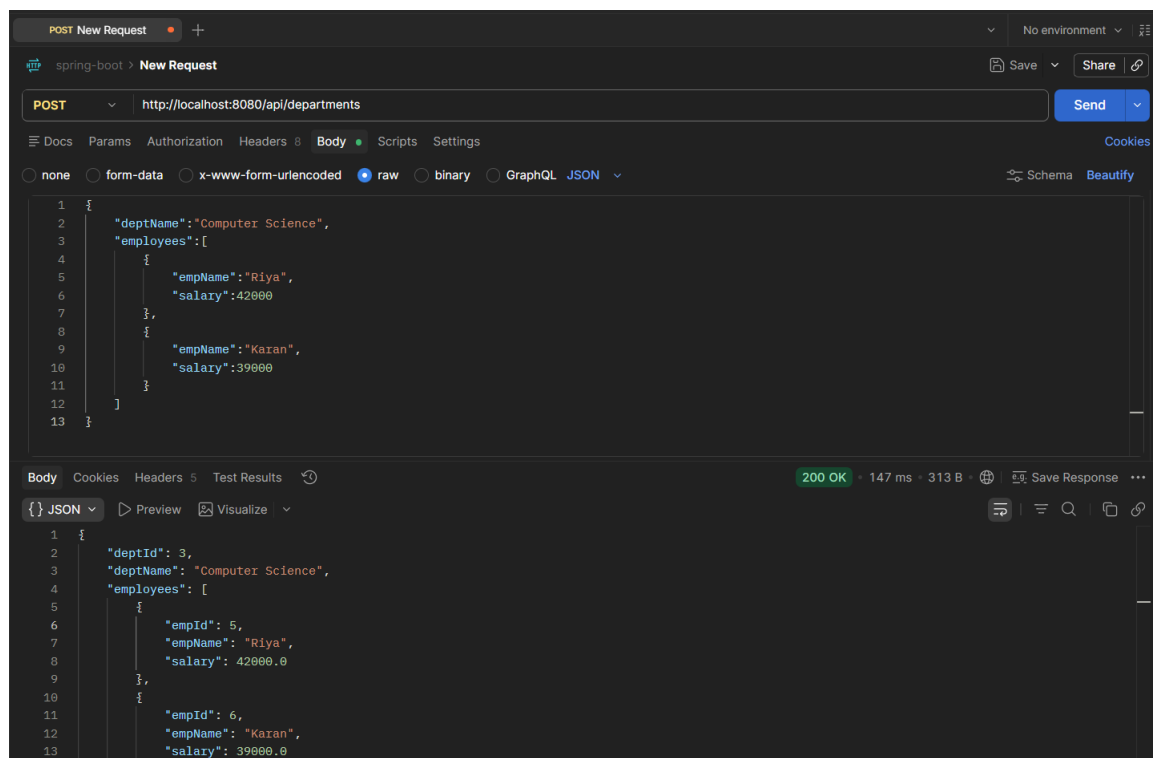
API'S TESTING USING POSTMAN

1. POST API – Add Department with Employees:

The POST API is used to insert a new **Department** along with multiple **Employee** records into the MySQL database. Since CascadeType.ALL is used, all employee records are automatically saved when the department is saved.

POST:

<http://localhost:8080/api/departments>



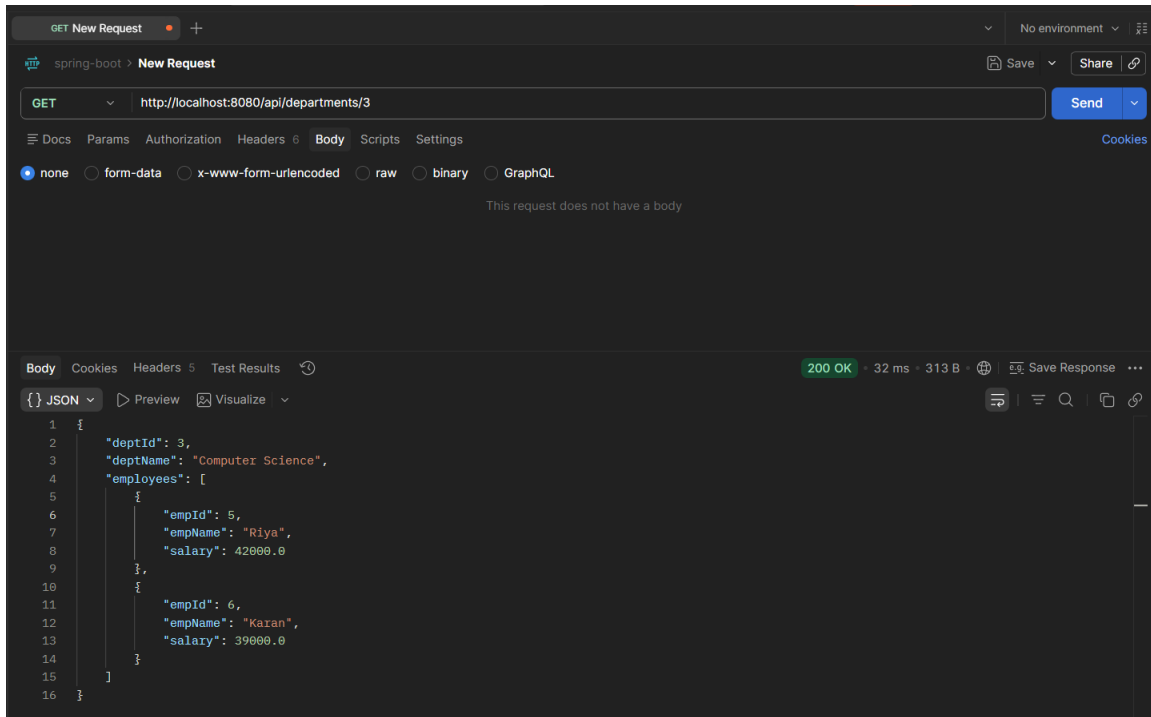
SEE THE DEPARTMENT AND ITS EMPLOYEES ARE ADDED SUCCESSFULLY INTO THE DATABASE.

2. GET API – Retrieve Department by ID:

This API retrieves the details of a single **Department** based on the Department ID passed in the URL. It also returns all employees associated with that department.

GET:

<http://localhost:8080/api/departments/3>



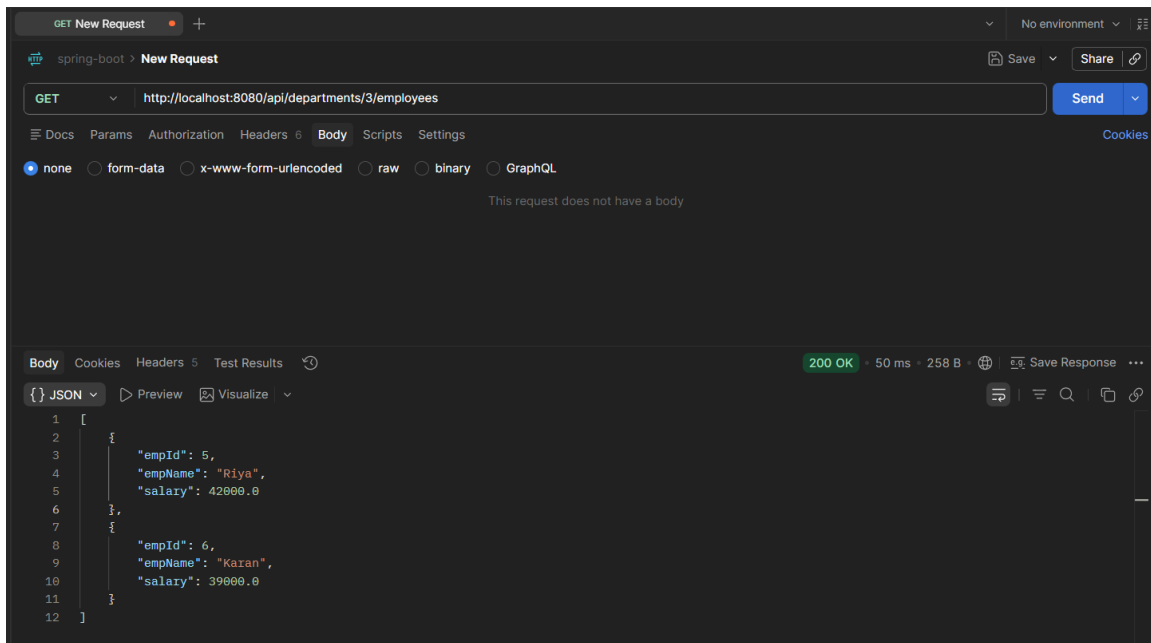
SEE THE DEPARTMENT DETAILS ALONG WITH ITS EMPLOYEE RECORDS ARE DISPLAYED SUCCESSFULLY.

3. GET API – Retrieve Employees of a Department:

This API retrieves all **Employee** records belonging to a specific Department using the Department ID provided in the URL.

GET:

<http://localhost:8080/api/departments/3/employees>



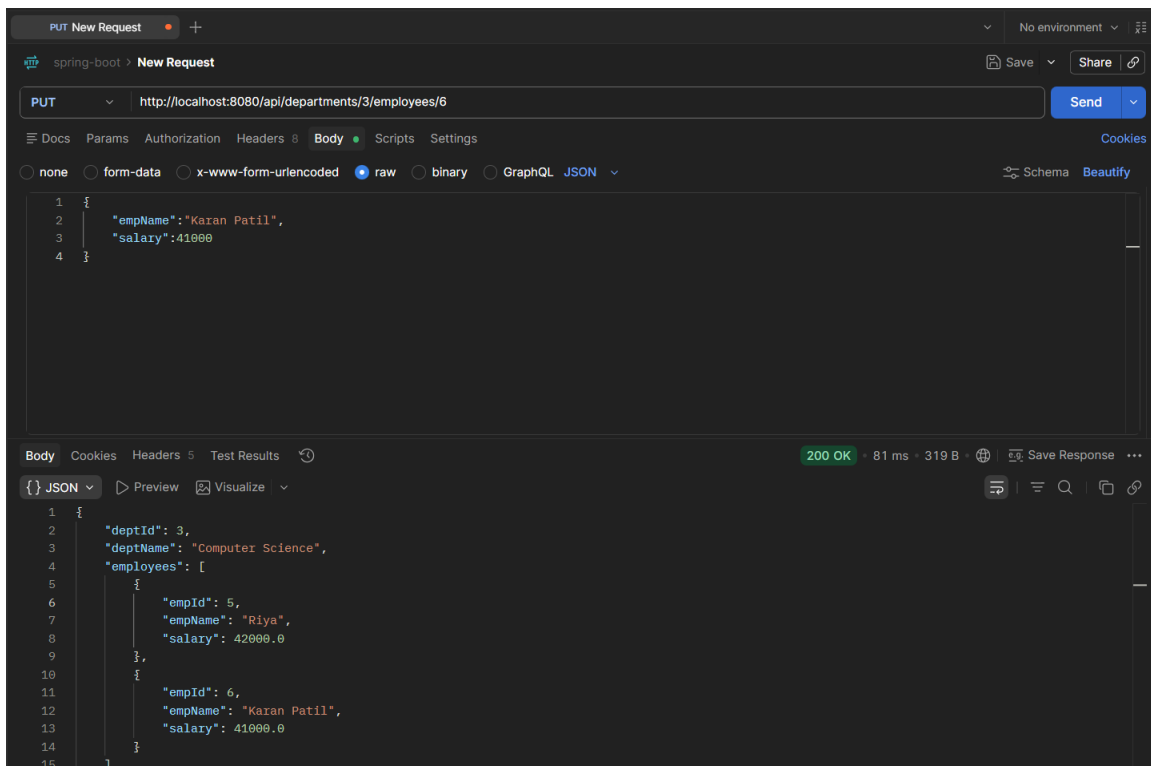
SEE ALL EMPLOYEES OF THE SELECTED DEPARTMENT ARE DISPLAYED SUCCESSFULLY.

4. PUT API – Update Employee Details:

The PUT API is used to update the details of an existing Employee within a specific Department. The Department ID and Employee ID are passed in the URL, while the updated employee information is provided in the request body.

PUT:

<http://localhost:8080/api/departments/1/employees/2>



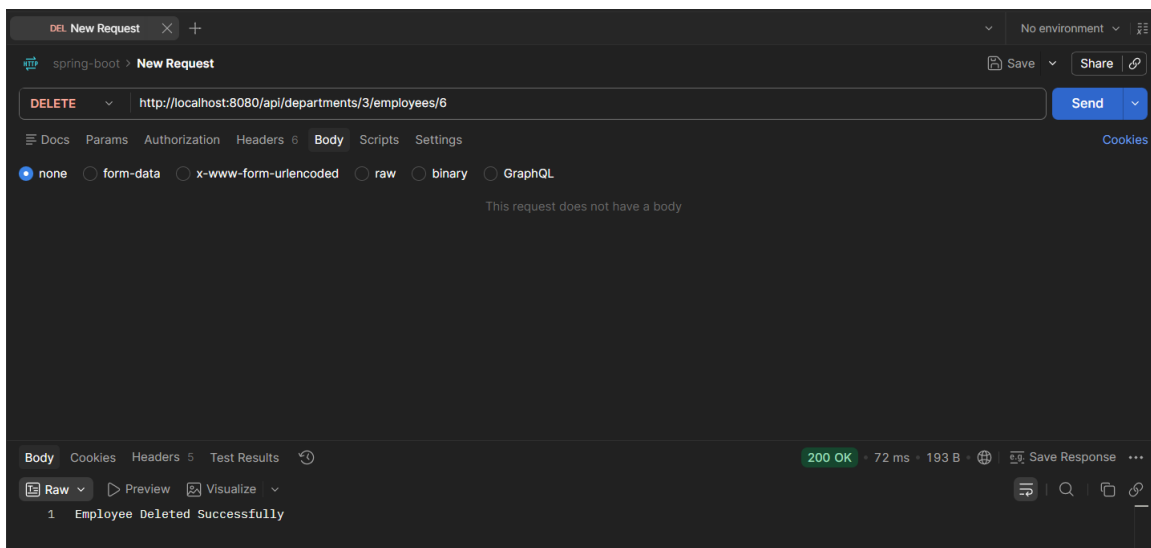
SEE THE EMPLOYEE DETAILS ARE UPDATED SUCCESSFULLY IN THE DATABASE

5. DELETE API – Delete Employee from Department:

The DELETE API removes a specific Employee from the selected Department using the Department ID and Employee ID provided in the URL.

DELETE:

<http://localhost:8080/api/departments/3/employees/6>



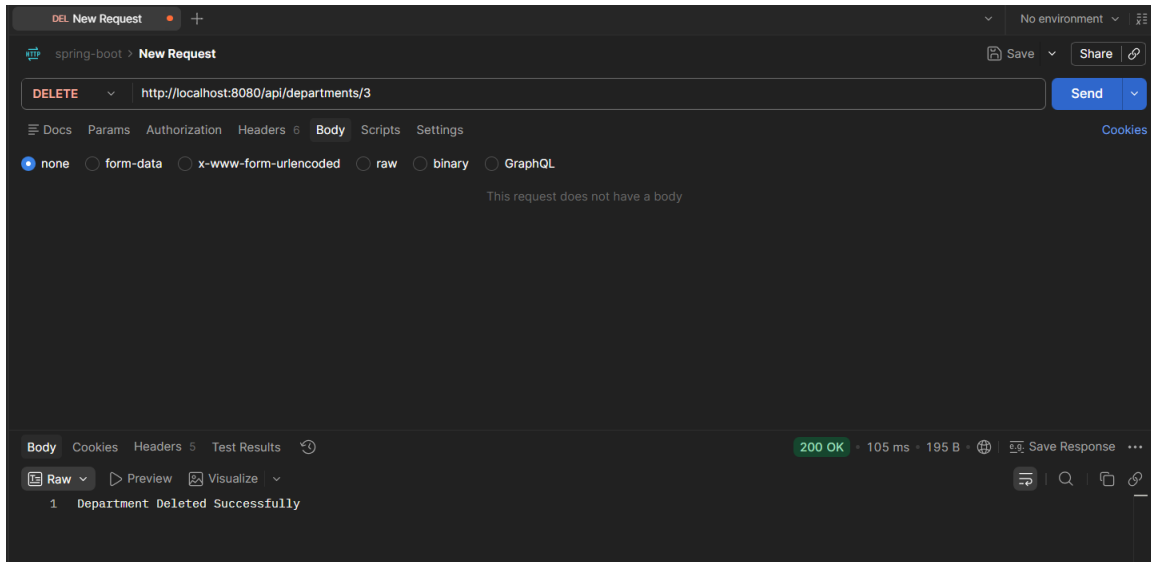
SEE THE EMPLOYEE IS DELETED SUCCESSFULLY FROM THE DEPARTMENT.

6. DELETE API – Delete Department:

The DELETE API removes the Department from the database using the Department ID. Since CascadeType.ALL is enabled, all employees associated with that department are also deleted automatically.

DELETE:

<http://localhost:8080/api/departments/3>

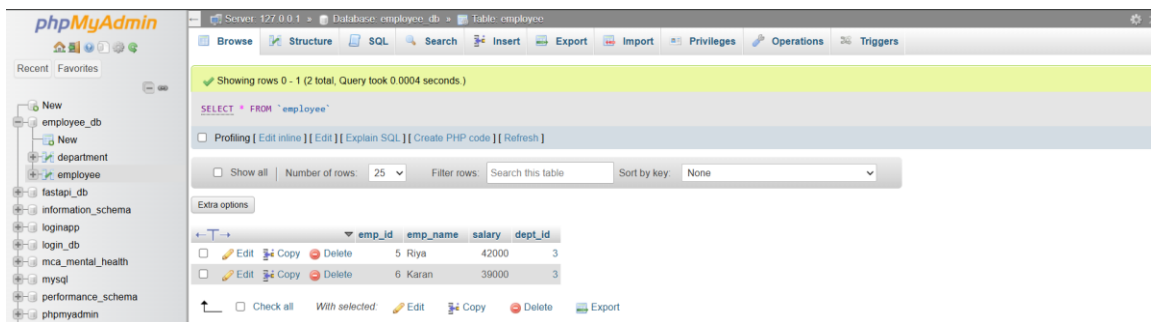


SEE THE DEPARTMENT AND ALL ITS EMPLOYEES ARE DELETED SUCCESSFULLY FROM THE DATABASE.

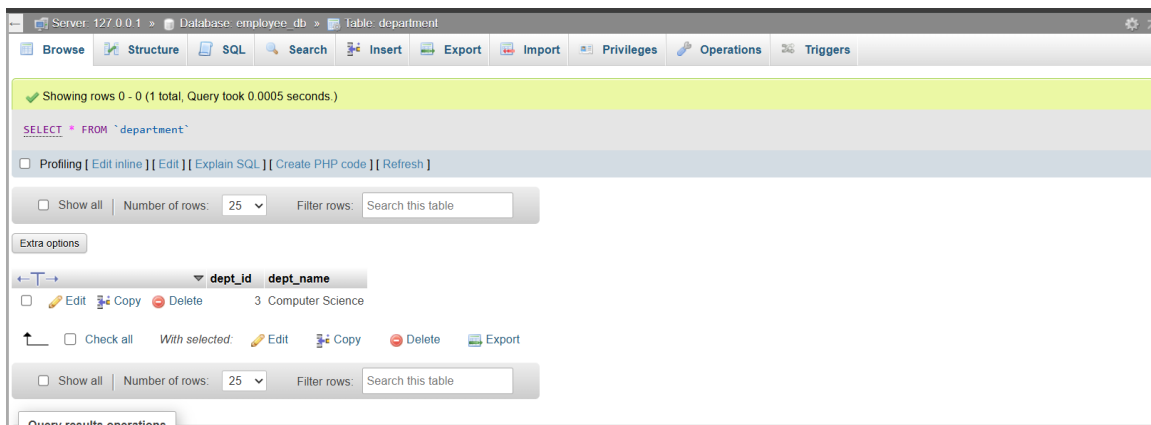
Database Output

TABLE'S AND DATA STORING SUCCEFULLY

1. Employee table



2. Department table



-----End of practical(5)-----

Practical No. 6 : Many-to-Many Mapping – Student and Course

Objective:

To implement a Many-to-Many relationship between Student and Course using @ManyToMany and a join table via @JoinTable, and to avoid infinite JSON recursion using @JsonManagedReference / @JsonBackReference.

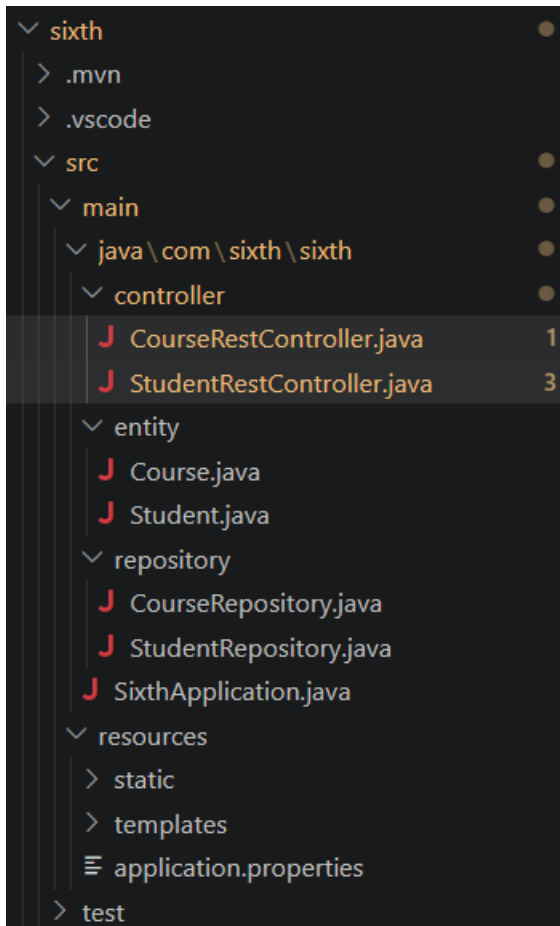
Class Name(s) with Attributes:

Student (@Entity)

- studentId : int (@Id @GeneratedValue)
- studentName : String
- courses : Set<Course> (@ManyToMany, @JoinTable(name="student_course"))

Course (@Entity)

- courseId : int (@Id @GeneratedValue)
- courseName : String
- students : Set<Student> (@ManyToMany(mappedBy="courses"))



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: employee_db

Table: 1.Student(created automatically by Hibernate)

2.Course (created automatically by Hibernate)

3.course_details(created automatically by Hibernate)

Important Files

Student.java(ENTITY model class)

```
package com.sixth.sixth.entity;

import java.util.HashSet;
import java.util.Set;

import com.fasterxml.jackson.annotation.JsonManagedReference;

import jakarta.persistence.*;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer studentId;

    private String studentName;

    @ManyToMany
    @JoinTable(name = "student_course", joinColumns =
    @JoinColumn(name = "student_id"), inverseJoinColumns =
    @JoinColumn(name = "course_id"))
    @JsonManagedReference
    private Set<Course> courses = new HashSet<>();

    public Student() {
    }

    public Integer getStudentId() {
        return studentId;
    }

    public void setStudentId(Integer studentId) {
        this.studentId = studentId;
    }

    public String getStudentName() {
```

```
        return studentName;
    }

    public void setStudentName(String studentName) {
        this.studentName = studentName;
    }

    public Set<Course> getCourses() {
        return courses;
    }

    public void setCourses(Set<Course> courses) {
        this.courses = courses;
    }
}
```

Course.java(ENTITY model class)

```
package com.sixth.sixth.entity;

import java.util.HashSet;
import java.util.Set;

import com.fasterxml.jackson.annotation.JsonBackReference;

import jakarta.persistence.*;

@Entity
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer courseId;

    private String courseName;

    @ManyToMany(mappedBy = "courses")
    @JsonBackReference
    private Set<Student> students = new HashSet<>();

    public Course() {
    }
}
```

```
public Integer getCourseId() {
    return courseId;
}

public void setCourseId(Integer courseId) {
    this.courseId = courseId;
}

public String getCourseName() {
    return courseName;
}

public void setCourseName(String courseName) {
    this.courseName = courseName;
}

public Set<Student> getStudents() {
    return students;
}

public void setStudents(Set<Student> students) {
    this.students = students;
}
}
```

StudentRestController.java(Controller file)

```
package com.sixth.sixth.controller;

import java.util.Set;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.sixth.sixth.entity.Course;
import com.sixth.sixth.entity.Student;
import com.sixth.sixth.repository.CourseRepository;
import com.sixth.sixth.repository.StudentRepository;

@RestController
@RequestMapping("/api/students")
public class StudentRestController {

    @Autowired
```

```
private StudentRepository studentRepository;

@Autowired
private CourseRepository courseRepository;

// Add Student
@PostMapping
public Student addStudent(@RequestBody Student student) {
    return studentRepository.save(student);
}

// Enroll Student into Course
@PostMapping("/{id}/enroll/{courseId}")
public Student enrollStudent(@PathVariable Integer id,
                             @PathVariable Integer courseId) {

    Student student = studentRepository.findById(id).orElse(null);

    Course course = courseRepository.findById(courseId).orElse(null);

    if (student == null || course == null)
        return null;

    student.getCourses().add(course);

    return studentRepository.save(student);
}

// Get Student
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentRepository.findById(id).orElse(null);
}

// Remove Course
@DeleteMapping("/{id}/unenroll/{courseId}")
public String unenrollStudent(@PathVariable Integer id,
                              @PathVariable Integer courseId) {

    Student student = studentRepository.findById(id).orElse(null);

    Course course = courseRepository.findById(courseId).orElse(null);

    if (student == null || course == null)
        return "Not Found";

    student.getCourses().remove(course);

    studentRepository.save(student);
}
```

```
        return "Course Removed Successfully";  
    }  
  
}
```

CourseRestController.java(Controller file)

```
package com.sixth.sixth.controller;  
  
import java.util.Set;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.*;  
  
import com.sixth.sixth.entity.Course;  
import com.sixth.sixth.entity.Student;  
import com.sixth.sixth.repository.CourseRepository;  
  
@RestController  
@RequestMapping("/api/courses")  
public class CourseRestController {  
  
    @Autowired  
    private CourseRepository repository;  
  
    // Add Course  
    @PostMapping  
    public Course addCourse(@RequestBody Course course) {  
        return repository.save(course);  
    }  
  
    // Get Students of Course  
    @GetMapping("/{id}/students")  
    public Set<Student> getStudents(@PathVariable Integer id) {  
  
        Course course = repository.findById(id).orElse(null);  
  
        if (course == null)  
            return null;  
  
        return course.getStudents();  
    }  
}
```

```
// Update Course
@PutMapping("/{id}")
public Course updateCourse(@PathVariable Integer id,
    @RequestBody Course course) {

    course.setCourseId(id);

    return repository.save(course);
}

// Delete Course
@DeleteMapping("/{id}")
public String deleteCourse(@PathVariable Integer id) {

    repository.deleteById(id);

    return "Course Deleted Successfully";
}
}
```

StudentRepository.java(Repository file)

```
package com.sixth.sixth.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.sixth.sixth.entity.Student;

public interface StudentRepository extends JpaRepository<Student,
Integer> {

}
```

CourseRepository.java(Repository file)

```
package com.sixth.sixth.repository;

import org.springframework.data.jpa.repository.JpaRepository;
```

```
import com.sixth.sixth.entity.Student;

public interface StudentRepository extends JpaRepository<Student,
Integer> {

}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=sixth
spring.datasource.url=jdbc:mysql://localhost:3306/sixth
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

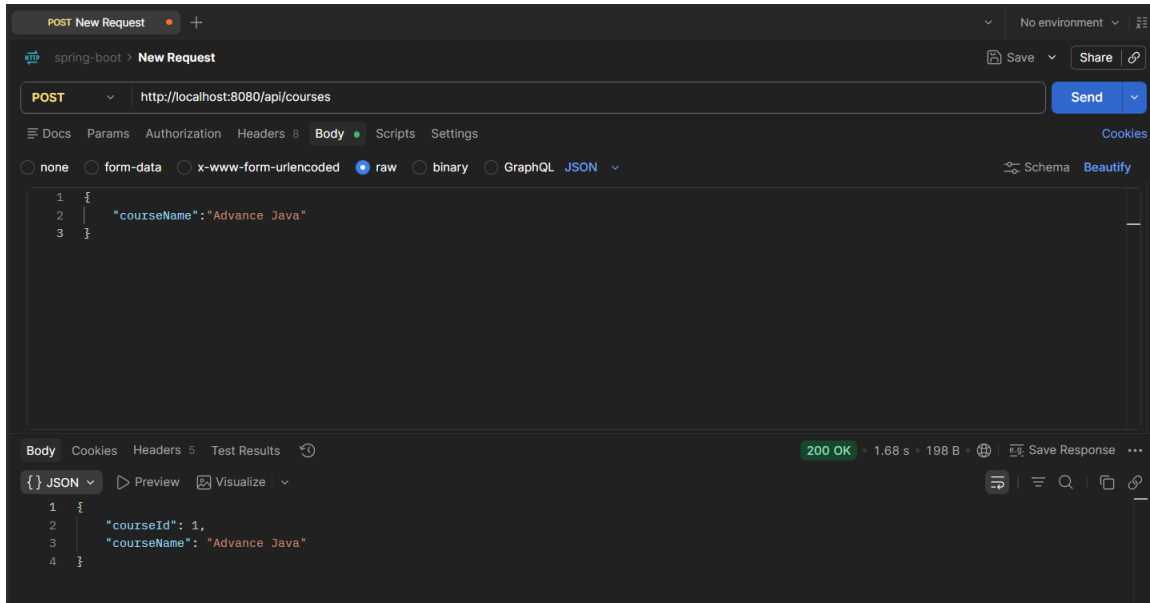
spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

1. POST API – Add Course:

The POST API is used to create a new course in the database. The course details are sent in JSON format, and Spring Boot stores the record using the save() method of CourseRepository.

<http://localhost:8080/api/courses>

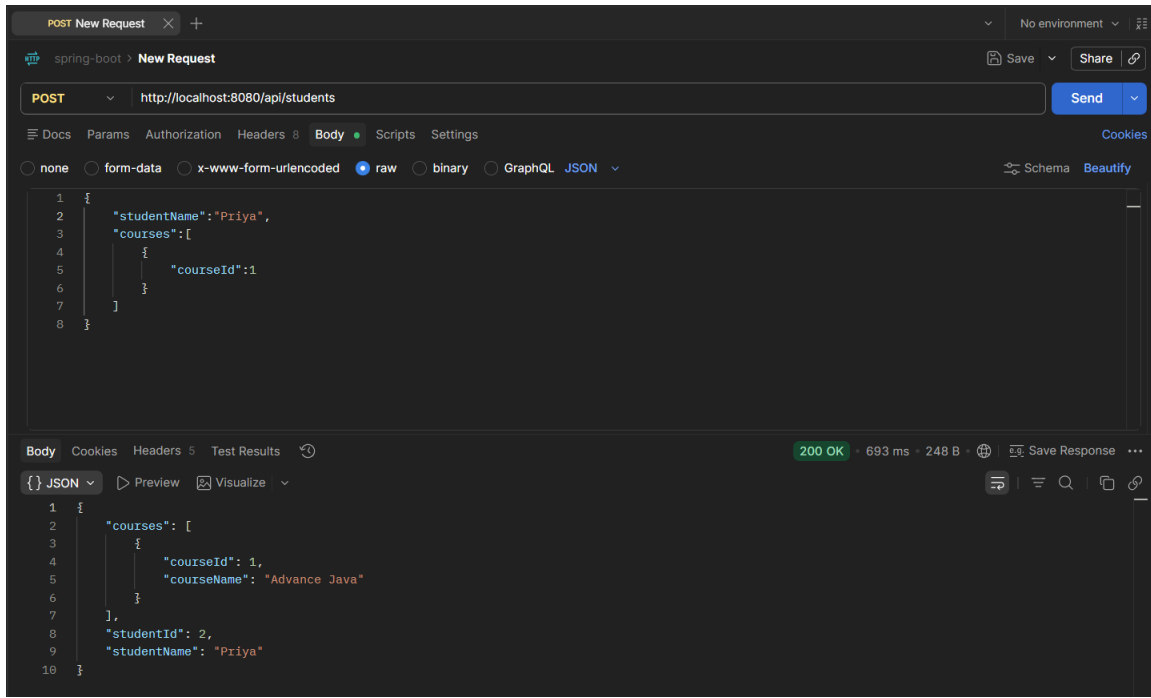


SEE THE COURSE IS ADDED SUCCESSFULLY INTO THE DATABASE.

2. POST API – Add Student:

The POST API is used to create a new student and enroll the student into one or more existing courses. The student details along with course IDs are sent in the request body, and the relationship is stored in the automatically generated join table (student_course)

<http://localhost:8080/api/students>

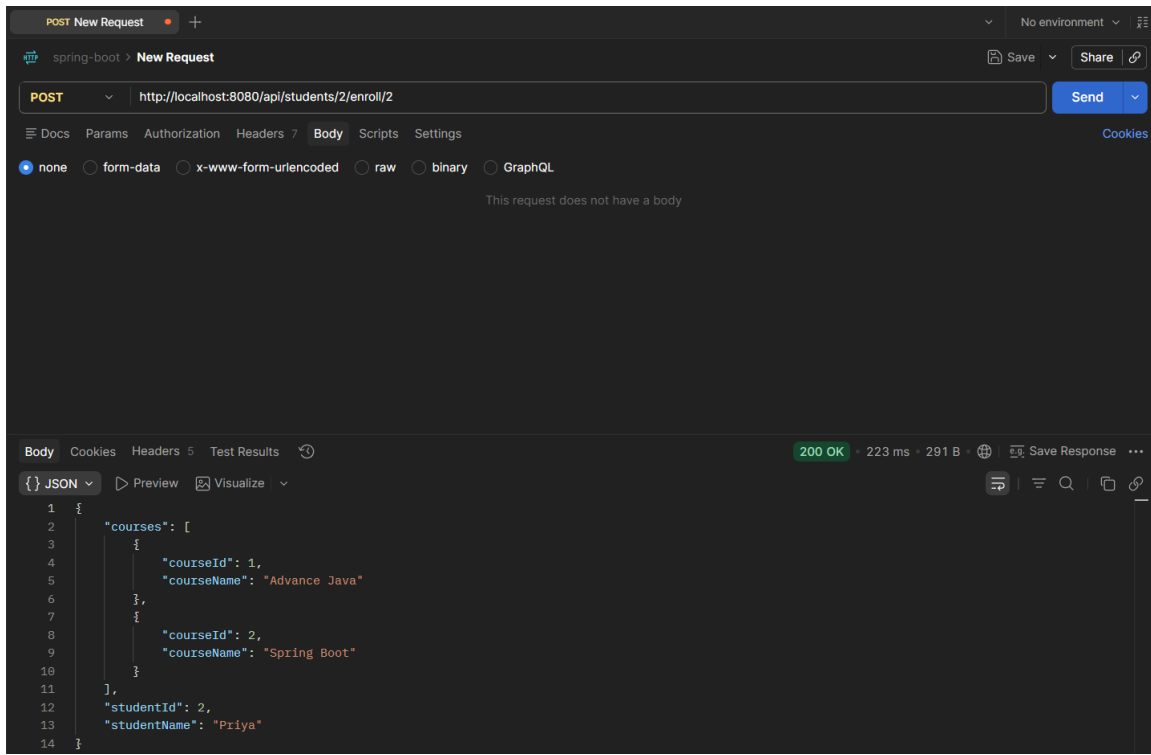


SEE THE STUDENT IS CREATED SUCCESSFULLY AND LINKED WITH THE SELECTED COURSE.

3. POST API – Enroll Student into Course:

This API is used to enroll an existing student into another existing course. It creates a new relationship between the student and the course by inserting a record into the join table.

<http://localhost:8080/api/students/2/enroll/2>

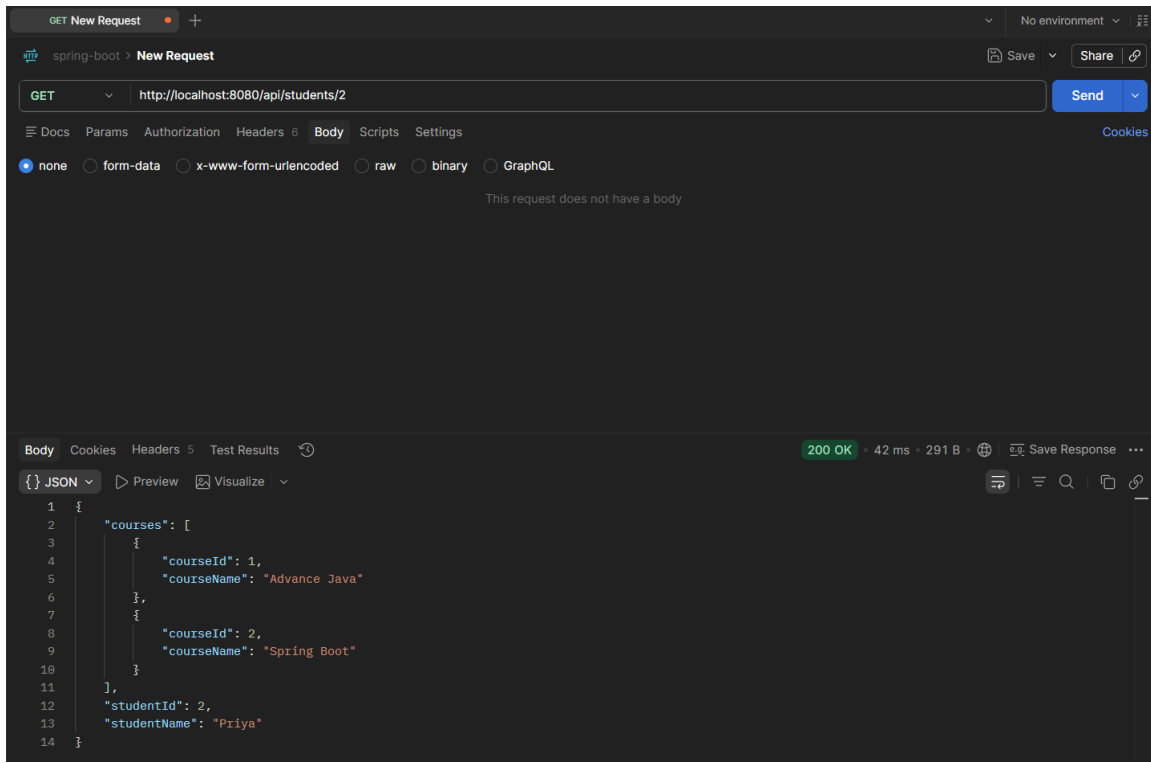


SEE THE STUDENT IS SUCCESSFULLY ENROLLED INTO THE SELECTED COURSE.

4. GET API – Retrieve Student by ID:

The GET API retrieves the details of a specific student along with all the courses in which the student is enrolled

<http://localhost:8080/api/students/2>

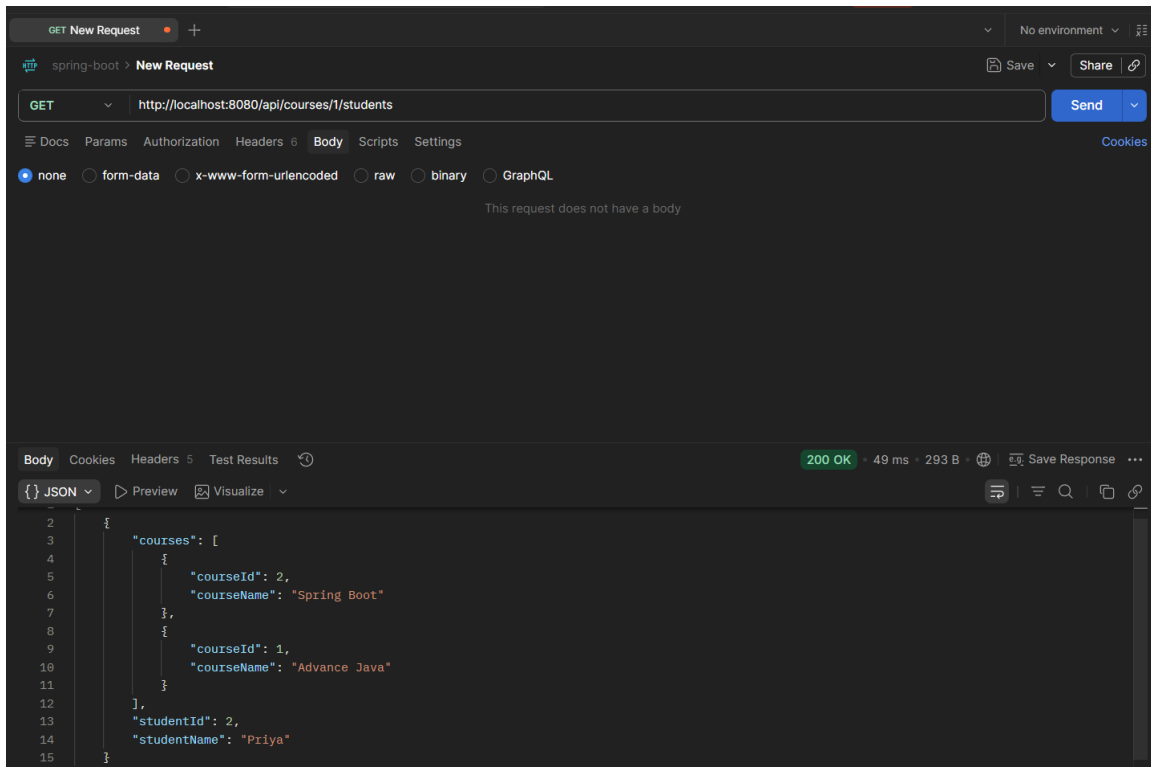


SEE THE STUDENT DETAILS ALONG WITH ENROLLED COURSES ARE DISPLAYED SUCCESSFULLY.

5. GET API – Retrieve Students of a Course:

This API retrieves all students enrolled in a particular course. It fetches the course using its ID and returns the list of associated students.

<http://localhost:8080/api/courses/1/students>

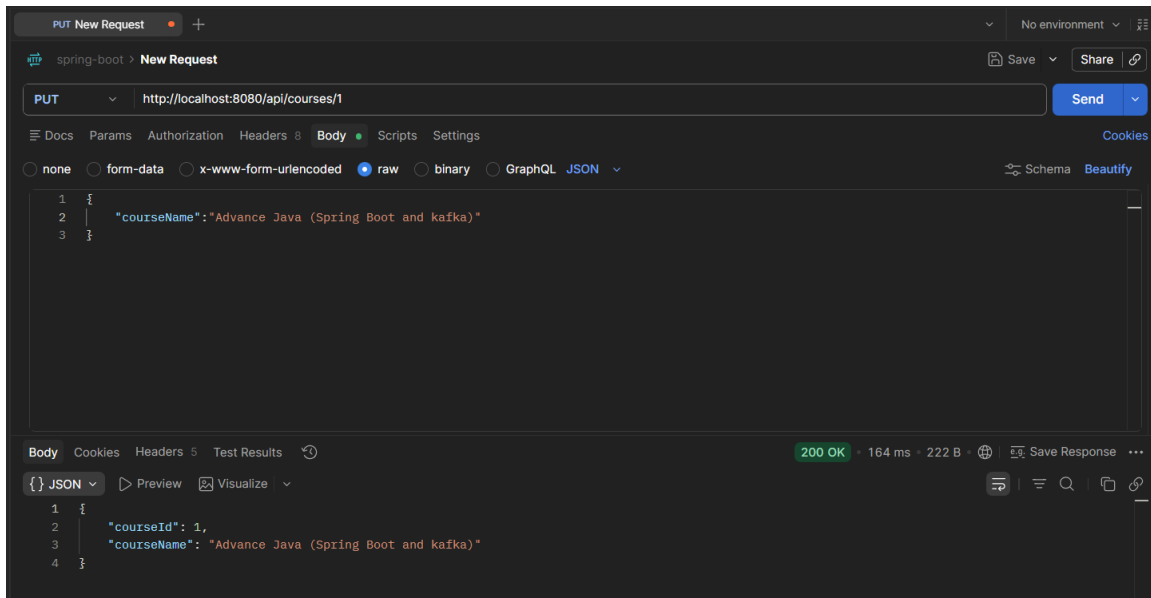


SEE ALL STUDENTS ENROLLED IN THE SELECTED COURSE ARE DISPLAYED SUCCESSFULLY.

6. PUT API – Update Course:

The PUT API is used to update the details of an existing course. The updated information is sent in the request body, and the record is updated in the database using the `save()` method..

<http://localhost:8080/api/courses/2>

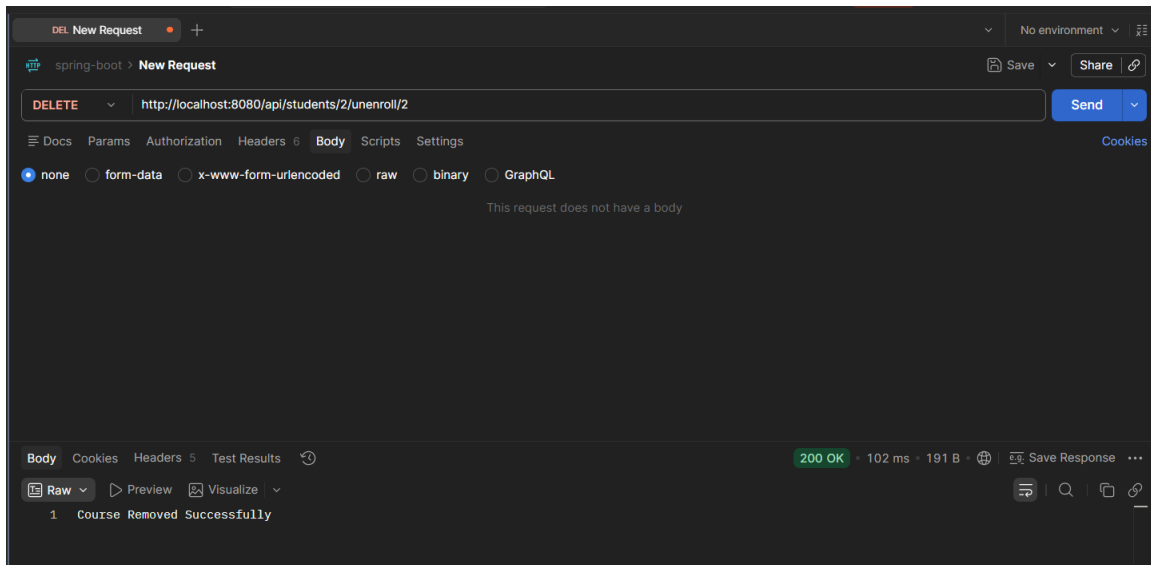


SEE THE COURSE DETAILS ARE UPDATED SUCCESSFULLY.

7. DELETE API – Unenroll Student from Course:

This DELETE API removes the relationship between a student and a course. Only the entry from the student_course join table is deleted, while both the student and the course records remain in the database

<http://localhost:8080/api/students/2/unenroll/2>

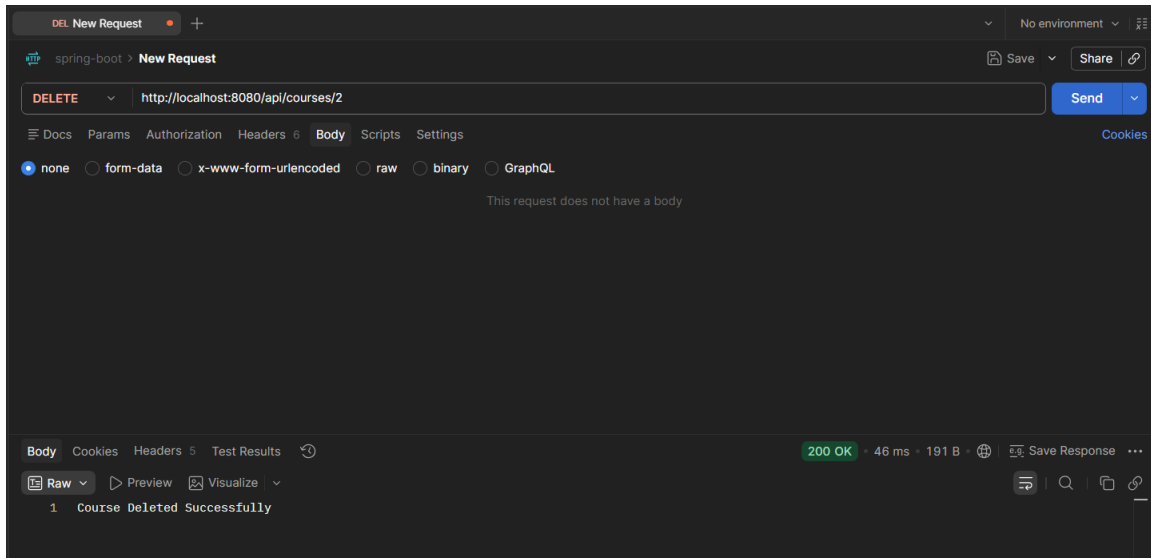


SEE THE STUDENT IS SUCCESSFULLY UNENROLLED FROM THE COURSE.

8. DELETE API – Delete Course:

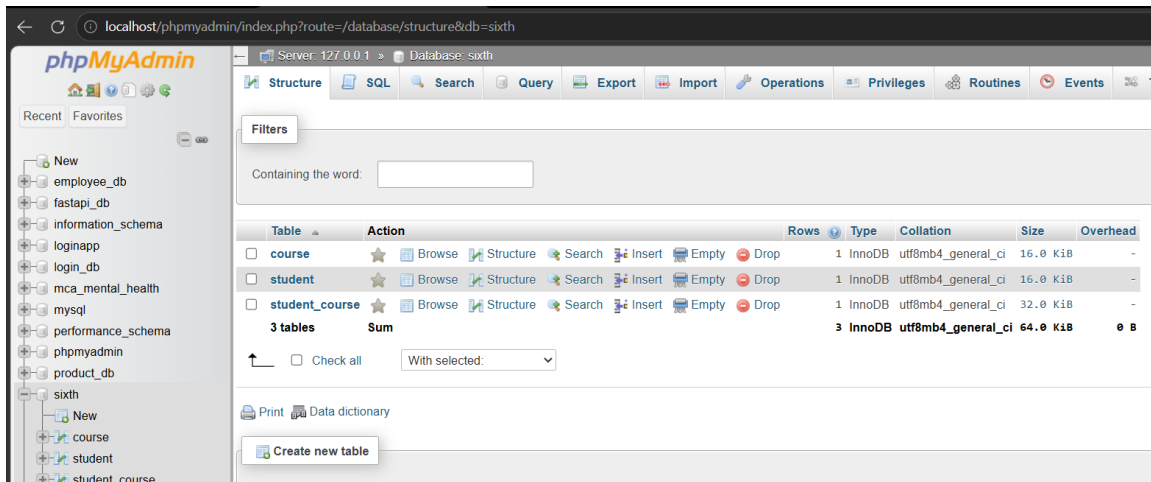
The DELETE API is used to remove a course from the database. The corresponding entries in the join table are also removed automatically, while student records remain unaffected.

<http://localhost:8080/api/courses/2>



SEE THE COURSE IS DELETED SUCCESSFULLY FROM THE DATABASE

Database : and all 3 tables created and data also added successfully



-----end-of-practical(6)-----

Practical No. 7: Transaction Management **with @Transactional – Bank Fund** **Transfer**

Objective:

To understand declarative transaction management in Spring Data JPA, and to ensure atomicity across multiple repository operations (debit + credit) using @Transactional, including automatic rollback when a runtime exception occurs.

Class Name(s) with Attributes:

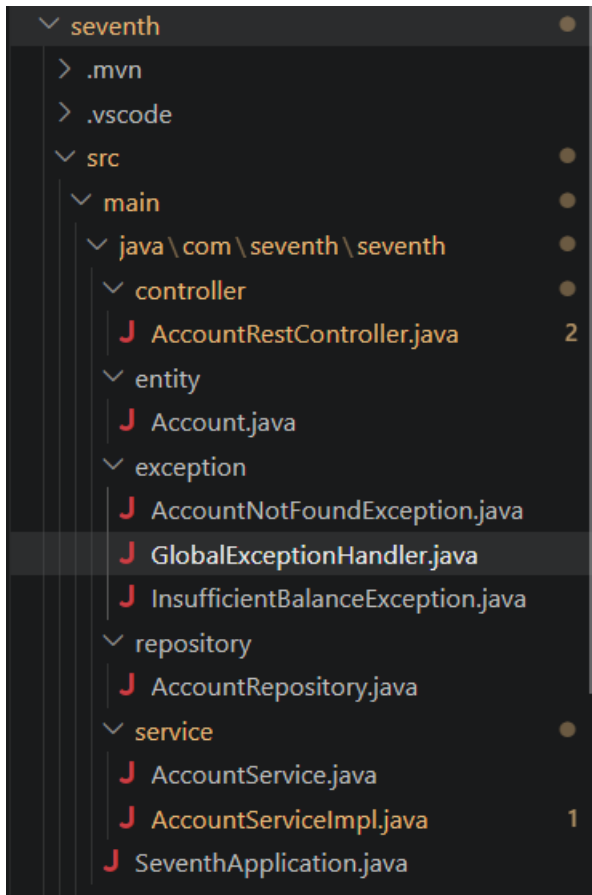
Account (@Entity)

- accountId : int (@Id @GeneratedValue)
- accountHolder : String
- balance : double

InsufficientBalanceException (custom RuntimeException)

AccountService / AccountServiceImpl (transferFunds is @Transactional)

- accountRepository : AccountRepository



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: seventh

Table: bank(created automatically by Hibernate)

Important Files

Account.java(ENTITY model class)

```
package com.seventh.seventh.entity;

import jakarta.persistence.*;

@Entity
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer accountId;

    private String accountHolder;

    private double balance;

    public Account() {
    }

    public Integer getAccountId() {
        return accountId;
    }

    public void setAccountId(Integer accountId) {
        this.accountId = accountId;
    }

    public String getAccountHolder() {
        return accountHolder;
    }

    public void setAccountHolder(String accountHolder) {
        this.accountHolder = accountHolder;
    }

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }
}
```

AccountRepository.java(Repository file spring data jpa)

```
package com.seventh.seventh.repository;

import org.springframework.data.jpa.repository.JpaRepository;
```

```
import com.seventh.seventh.entity.Account;

public interface AccountRepository extends JpaRepository<Account,
Integer> {

}
```

CUSTOM EXCEPTION FILE'S AND GLOBALEXCEPTION

- 1.AccountNotFoundException.java(if user entered account which didn't exist while transferring money)
- 2.InsufficientBalanceException.java(if user want to transfer insufficient amount)
- 3.GlobalExceptionHandler.java(to run this both exception we have to make it global so we also don't need to use try catch)

AccountNotFoundException.java(Custom Exception)

```
package com.seventh.seventh.exception;

public class AccountNotFoundException extends RuntimeException {

    public AccountNotFoundException(String message) {
        super(message);
    }

}
```

InsufficientBalanceException.java(Custom Exception)

```
package com.seventh.seventh.exception;
```

```
public class InsufficientBalanceException extends RuntimeException {  
    public InsufficientBalanceException(String msg)  
    {  
        super(msg);  
    }  
}
```

GlobalExceptionHandler.java(Exception Handler)

This will used to call the custom exception without getting 500 error

```
package com.seventh.seventh.exception;  
  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.ExceptionHandler;  
import org.springframework.web.bind.annotation.RestControllerAdvice;  
  
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(InsufficientBalanceException.class)  
    public ResponseEntity<String> handleInsufficientBalance(  
        InsufficientBalanceException ex) {  
  
        return new ResponseEntity<>(  
            ex.getMessage(),  
            HttpStatus.CONFLICT); // 409  
        }  
  
    @ExceptionHandler(AccountNotFoundException.class)  
    public ResponseEntity<String> handleAccountNotFound(  
        AccountNotFoundException ex) {  
  
        return new ResponseEntity<>(  
            ex.getMessage(),  
            HttpStatus.NOT_FOUND); // 404  
        }  
    }  
}
```

SERVICE LAYER FILE'S WHICH INCLUDE THE BUSINESS LOGIC IN OUR EXAMPLE TO TRANSFER THE FUND IS LOGIC

- 1.AccountService.java(interface which will implement in main file)
- 2.AccountServiceImpl.java(override the interface and main logic in this file)

AccountService.java(Service layer file)

```
package com.seventh.seventh.service;

public interface AccountService {

    void transferFunds(int fromId,int toId,double amount);

}
```

AccountServiceImpl.java(Service layer file)

```
package com.seventh.seventh.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.seventh.seventh.entity.Account;
import com.seventh.seventh.exception.AccountNotFoundException;
import com.seventh.seventh.exception.InsufficientBalanceException;
import com.seventh.seventh.repository.AccountRepository;

@Service
public class AccountServiceImpl implements AccountService {

    @Autowired
    private AccountRepository repository;

    @Override
    @Transactional
    public void transferFunds(int fromId,
```

```
        int toId,  
        double amount) {  
  
        Account fromAccount = repository.findById(fromId).orElse(null);  
  
        Account toAccount = repository.findById(toId).orElse(null);  
  
        if (fromAccount == null || toAccount == null) {  
            throw new AccountNotFoundException("Account Not Found");  
        }  
  
        if (fromAccount.getBalance() < amount) {  
            throw new InsufficientBalanceException("Insufficient Balance");  
        }  
  
        fromAccount.setBalance(  
            fromAccount.getBalance() - amount);  
  
        toAccount.setBalance(  
            toAccount.getBalance() + amount);  
  
        repository.save(fromAccount);  
  
        repository.save(toAccount);  
    }  
}
```

AccountRestController.java(main controller file)

```
package com.seventh.seventh.controller;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.*;  
import java.util.List;  
import com.seventh.seventh.entity.Account;  
import com.seventh.seventh.repository.AccountRepository;  
import com.seventh.seventh.service.AccountService;  
  
@RestController  
@RequestMapping("/api/accounts")  
public class AccountRestController {  
  
    @Autowired
```

```
private AccountRepository repository;

@Autowired
private AccountService service;

// Create
@PostMapping
public Account addAccount(@RequestBody Account account) {

    return repository.save(account);
}

// Get Account
@GetMapping("/{id}")
public Account getAccount(@PathVariable Integer id) {

    return repository.findById(id).orElse(null);
}

@GetMapping
public List<Account> getAll()
{
    return repository.findAll();
}

// Transfer Money
@PostMapping("/transfer")
public String transferMoney(
    @RequestParam Integer fromId,
    @RequestParam Integer toId,
    @RequestParam double amount) {

    service.transferFunds(fromId,
        toId,
        amount);

    return "Amount Transferred Successfully";
}

// Update
@PutMapping("/{id}")
public Account updateAccount(
    @PathVariable Integer id,
    @RequestBody Account account) {

    Account existing = repository.findById(id).orElse(null);

    if (existing == null)
        return null;
}
```

```
existing.setAccountHolder(
    account.getAccountHolder());

return repository.save(existing);
}

// Delete
@DeleteMapping("/{id}")
public String deleteAccount(
    @PathVariable Integer id) {

    Account account = repository.findById(id).orElse(null);

    if (account == null)
        return "Account Not Found";

    if (account.getBalance() != 0)
        return "Account Cannot Be Deleted. Balance Must Be Zero.";

    repository.delete(account);

    return "Account Deleted Successfully";
}
}
```

WHAT'S NEW IN THIS PRACTICAL

1.SERVICE LAYER FOR BUSINESS LOGIN IF WE WANT TO TRANSFER THE MONEY IT WOULD NEED TO IMPLEMENT THE LOGIC SO IT SHOULD BE COVERD IN SERVICE LAYER

2.@Transactional ANNOTATION SUPPOSE WE ARE TRANSFERING MONEY OF 5000 FROM A TO B IT'S DEBITED FROM A AND NOT CREDITED IN B SOMETHING WRONG HAPPENED IN MIDDLE SO TRANSACTIONAL WILL HANDLE THIS AND ROLLBACK A S MONEY.

1. POST API – Transfer Funds:

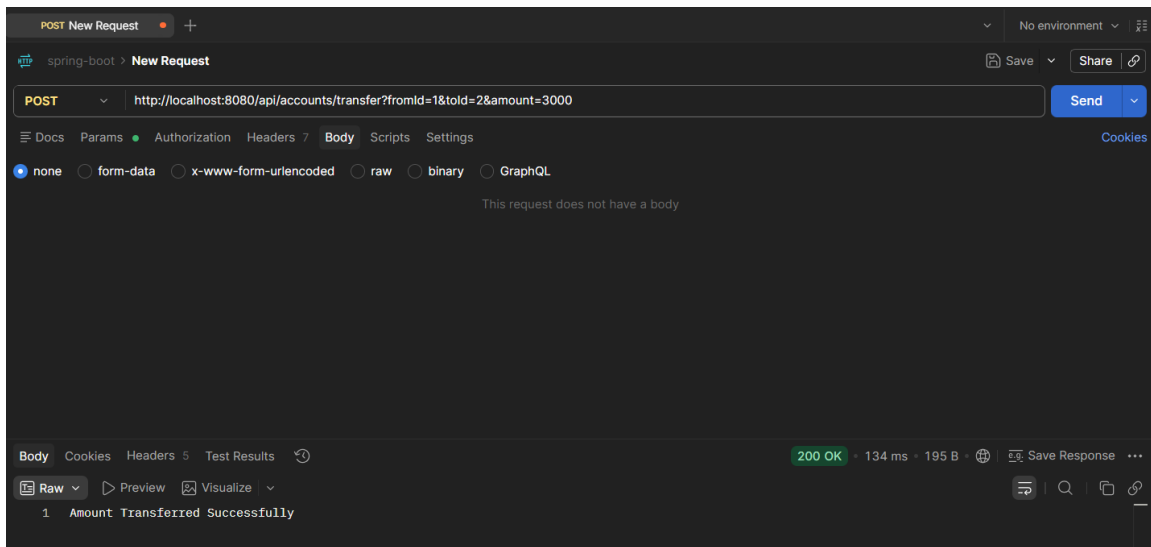
NAME: SOLANKI KARANSINH

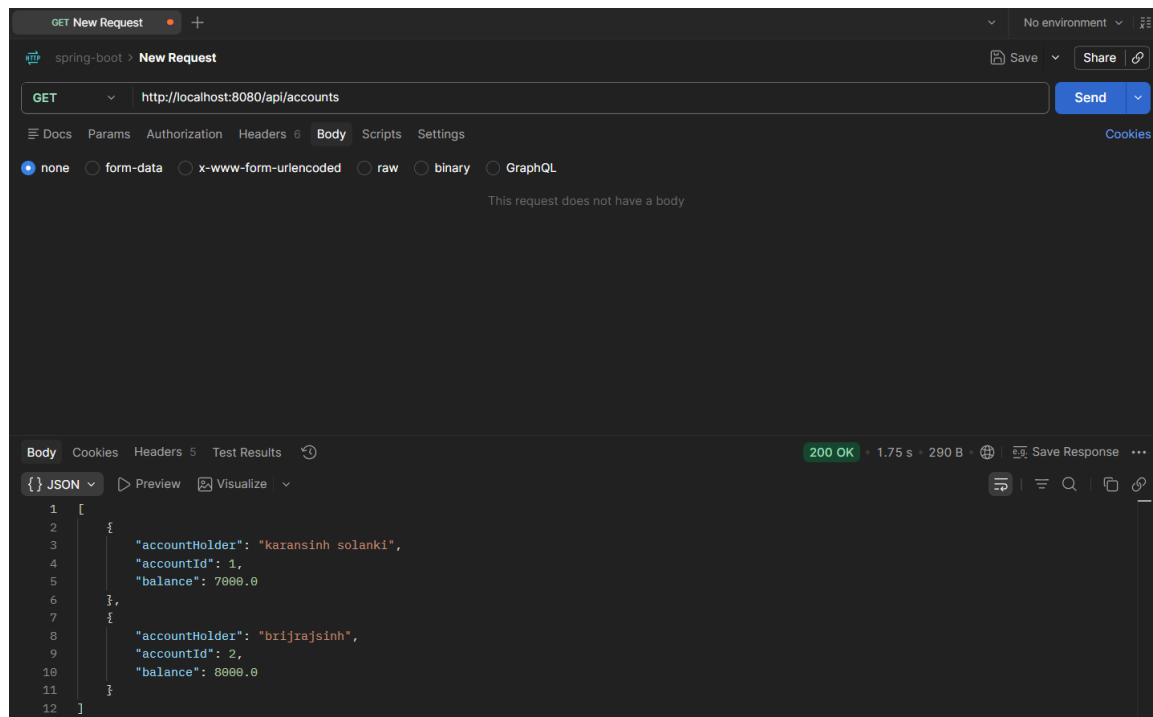
ENROLLMENT: 25CI2110110

The POST API is used to transfer money from one account to another. The sender account ID (fromId), receiver account ID (toId), and transfer amount (amount) are passed as request parameters. The service validates both accounts, checks the sender's balance, deducts the amount from the sender, credits the receiver, and saves both accounts within a single transaction using the @Transactional annotation. If the transfer is successful, the changes are permanently stored in the database.

(KARAN AND BRIJRAJSINH HAVE 10,000 AND 5000 MONEY RESPECTIVELY)

<http://localhost:8080/api/accounts/transfer?fromId=1&toId=2&amount=5000>





MONEY DEBITED 3000 FROM KARAN AND 3000 ADDED TO THE BRIJRAJSINH

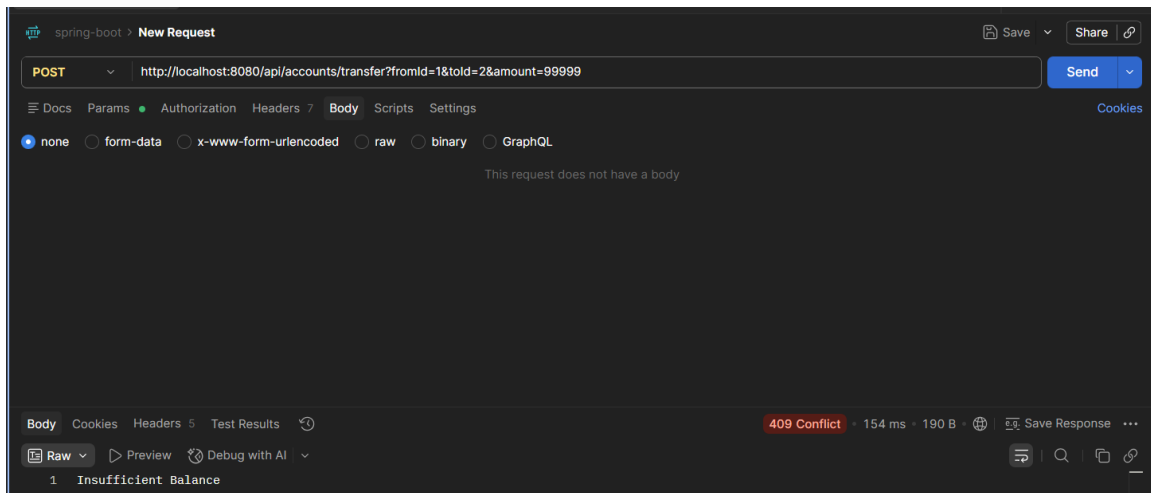
2. POST API – Transfer Funds (Insufficient Balance):

This API demonstrates transaction rollback using Spring Boot's `@Transactional` feature. If the sender's account balance is less than the requested transfer amount, the service throws an `InsufficientBalanceException`. The `GlobalExceptionHandler` catches the exception and returns an appropriate error response. Since the transaction is rolled back automatically, no changes are made to either account, ensuring data consistency.

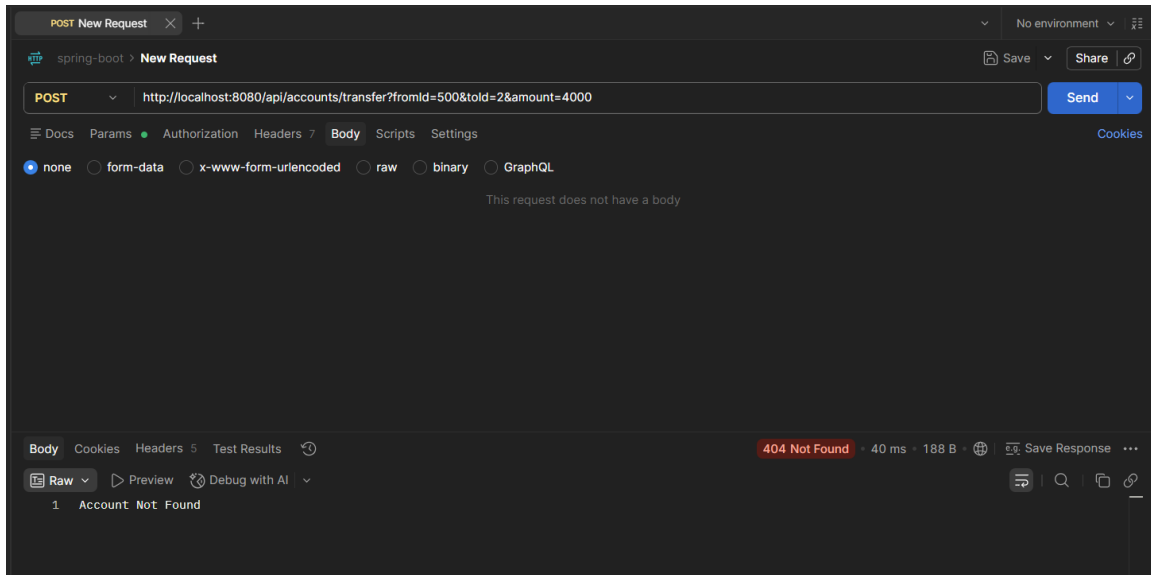
1.INSUFFICIENT BALANCE WITH 404 WHEN YOU ENTER 9999999 WHICH IS NOT AVAILABLE.

2.ACCOUNT NOT FOUND WHEN YOU ENTER ACCOUNT ID WHICH DIDN'T EXIST IN THE TABLE.

1. INSUFFICIENT BALANCE EXCEPTION HADNLED



2.ACCOUNT NOT FOUND EXCEPTION HANDLED

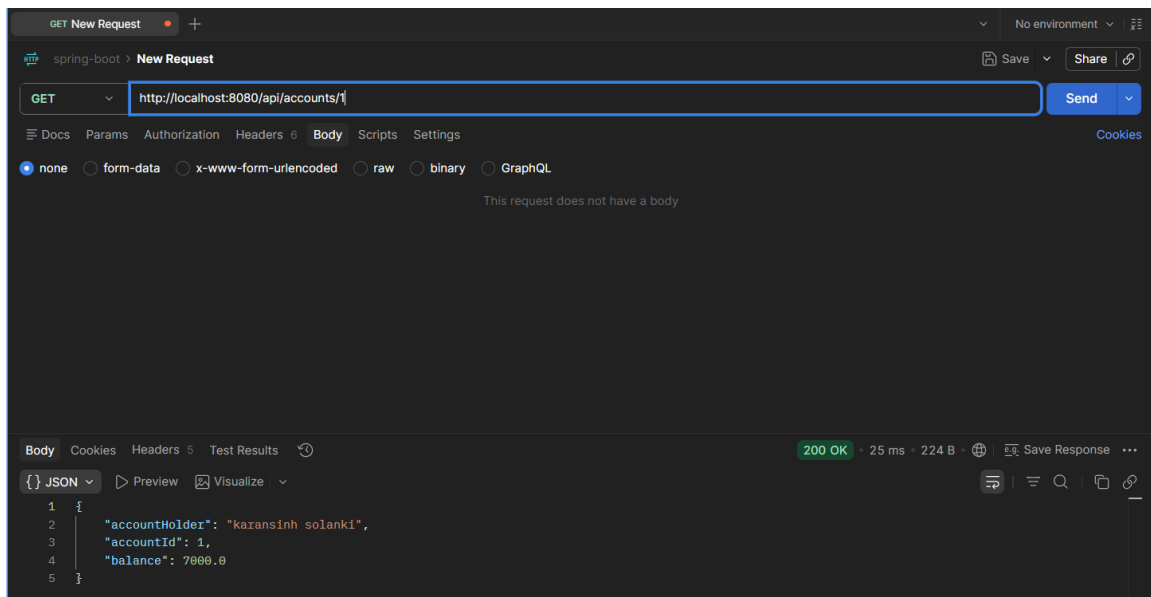


SEE THE APPLICATION RETURNS "INSUFFICIENT BALANCE" AND BOTH ACCOUNT BALANCES REMAIN UNCHANGED ALSO ACCOUNT NOT FOUND.

3. GET API – Get Account by ID:

The GET API is used to retrieve the details of a specific account using its unique account ID. The application searches the database using the `findById()` method and returns the account information in JSON format.

<http://localhost:8080/api/accounts/1>

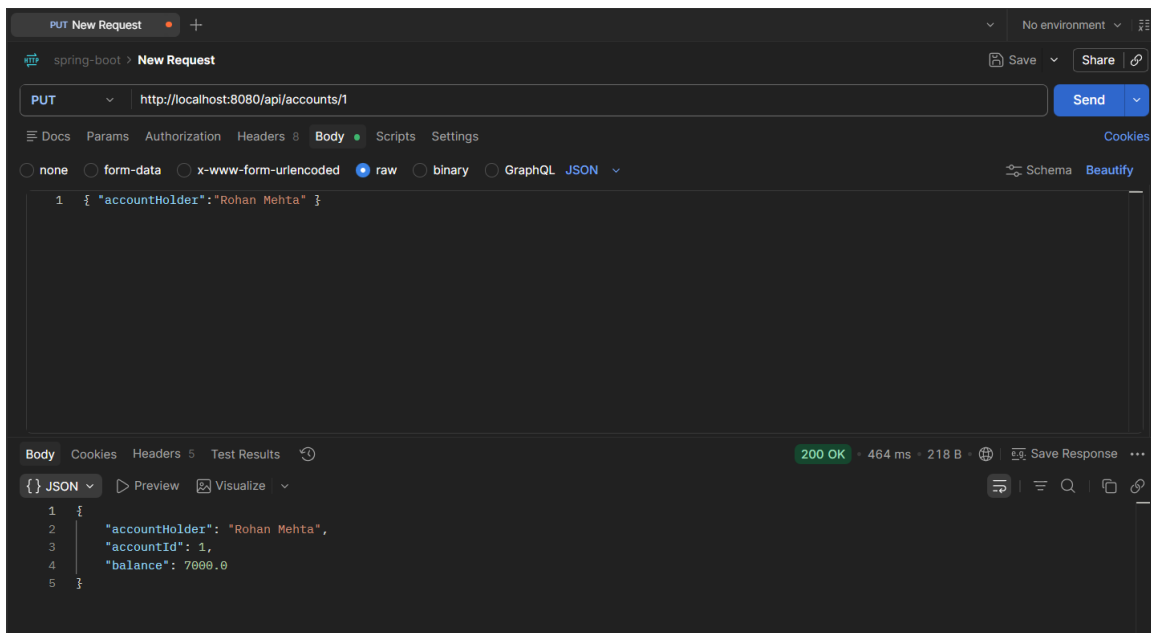


SEE THE ACCOUNT DETAILS ARE UPDATED SUCCESSFULLY IN THE DATABASE.

4. PUT API – Update Account:

The PUT API is used to update the details of an existing account. The account ID is provided in the URL, while the updated account information is sent in the request body. Spring Boot updates the existing account record and saves the modified details in the database.

<http://localhost:8080/api/accounts/1>



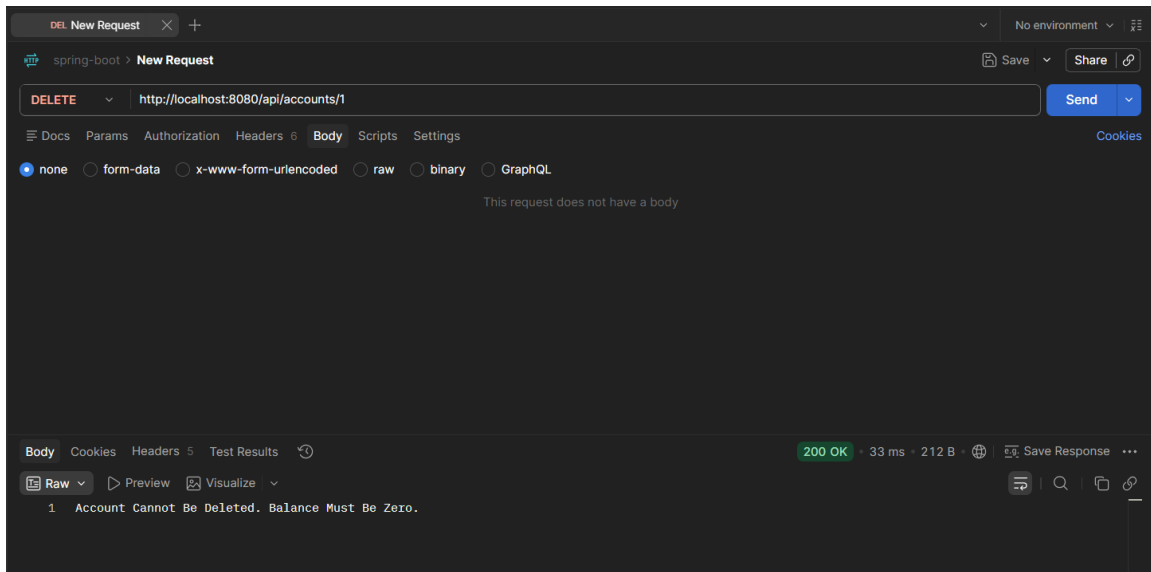
SEE THE ACCOUNT DETAILS ARE UPDATED SUCCESSFULLY IN THE DATABASE.

5. DELETE API – Delete Account:

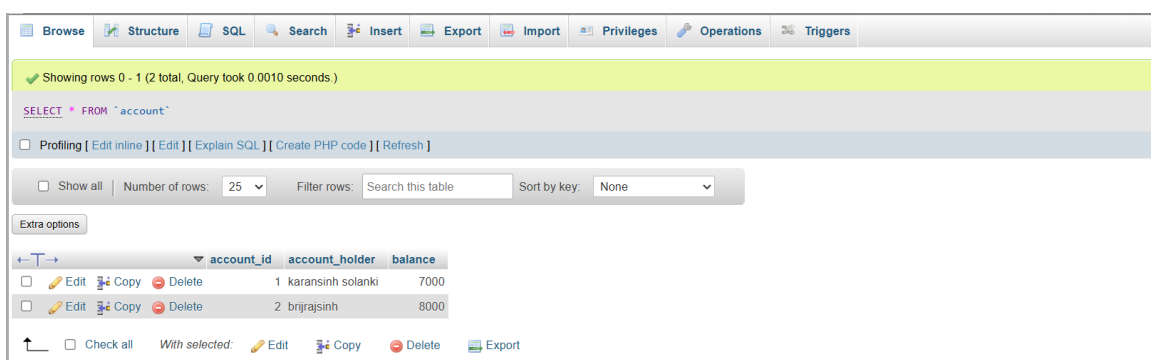
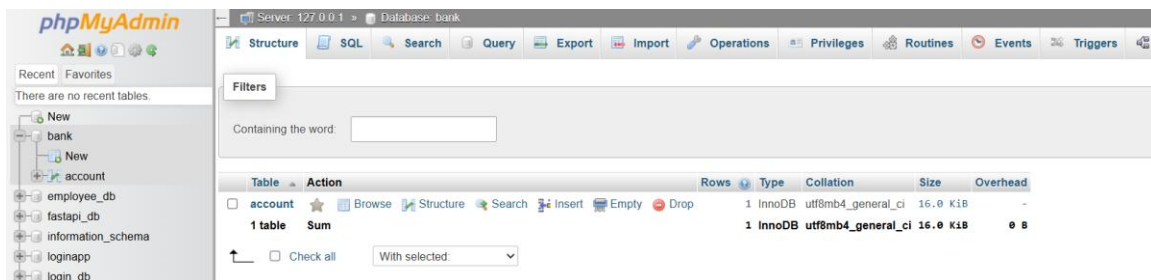
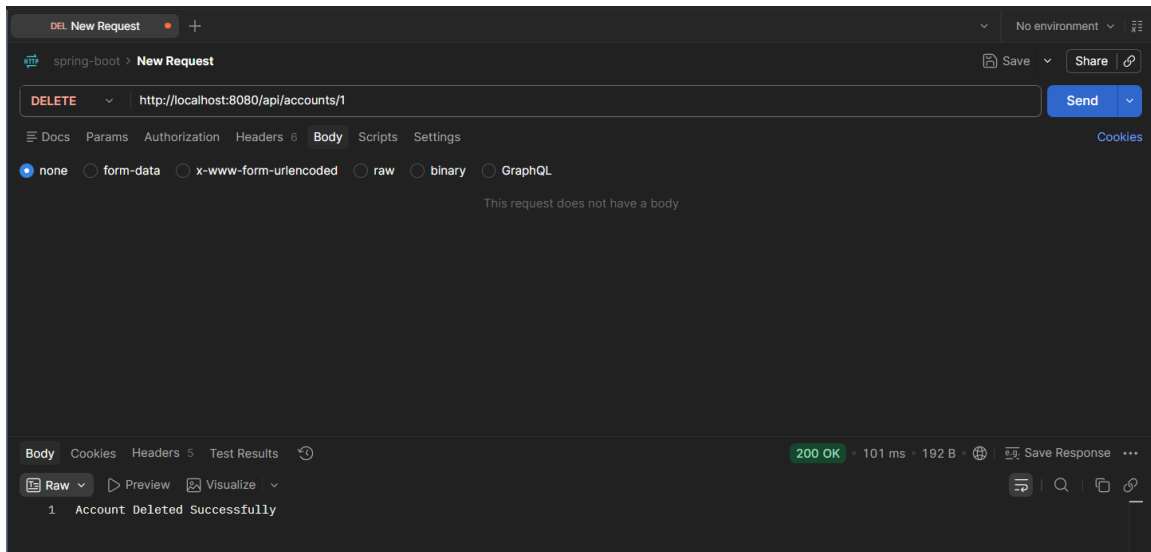
The DELETE API is used to remove an account from the database using its account ID. Before deletion, the application verifies that the account balance is zero. If the balance is zero, the account is deleted successfully. Otherwise, the application prevents deletion and returns an appropriate error message.

DELETE :

<http://localhost:8080/api/accounts/1>



IF THE ACCOUNT BALANCE IS NOT 0 YOU CAN'T DELETE THAT ACCOUNT SO FIRST MAKE IT 0 THEN YOU CAN DELETE



-----END-OF-PRACTICAL(7)-----

Practical No. 8 : Pagination and Sorting **with Spring Data JPA**

Objective:

To implement server-side pagination and sorting for a large result set using Spring Data's Pageable/PageRequest abstraction (PagingAndSortingRepository capability), exposed through query parameters on a REST endpoint.

Class Name(s) with Attributes:

Product (reuse from Practical 2)

- productId, productName, category, price, quantity

ProductRestController (extended)

- Method: `getProducts(page,size,sortBy) : Page<Product>`

Project Structure:

```
src
├── main
│   ├── java
│   │   ├── com.eight.eight
│   │   │   ├── SecondApplication.java
│   │   │   ├── controller
│   │   │   │   ├── ProductRestController.java
│   │   │   ├── entity
│   │   │   │   ├── Product.java
│   │   │   ├── repository
│   │   │   │   ├── ProductRepository.java
│   └── resources
│       └── application.properties
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: product_db

Table: product (created automatically by Hibernate)

Important Files

1. **Product.java(SAME AS 2ND PRACTICAL)**
2. **ProductRepository.java(SAME AS 2ND PRACTICAL)**

ProductController.java(Controller file for REST apis)

```
package com.eight.eight.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;

import com.eighth.eighth.entity.Product;
```

```
import com.eighth.eighth.repository.ProductRepository;

import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    @Autowired
    private ProductRepository repository;

    // Add Product
    @PostMapping
    public Product addProduct(@RequestBody Product product) {
        return repository.save(product);
    }

    // Update Product
    @PutMapping("/{id}")
    public Product updateProduct(@PathVariable Integer id,
        @RequestBody Product product) {

        product.setProductId(id);

        return repository.save(product);
    }

    // Delete Product
    @DeleteMapping("/{id}")
    public String deleteProduct(@PathVariable Integer id) {

        repository.deleteById(id);

        return "Product Deleted Successfully";
    }

    // Pagination and Sorting
    @GetMapping("/page")
    public Page<Product> getProducts(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size,
        @RequestParam(defaultValue = "price") String sortBy) {

        Pageable pageable = PageRequest.of(
            page,
            size,
            Sort.by(sortBy));

        return repository.findAll(pageable);
    }
}
```

```
}
```

What is difference from this and second one

This practical extends the Product CRUD application by implementing **Pagination** and **Sorting** using Spring Data JPA. Unlike Practical 2, where all product records are retrieved at once using `findAll()`, this practical fetches products **page by page**, reducing memory usage and improving performance. It also allows products to be sorted dynamically by fields such as **price**, **productName**, or **category**. These features are especially useful when working with large datasets in real-world applications.

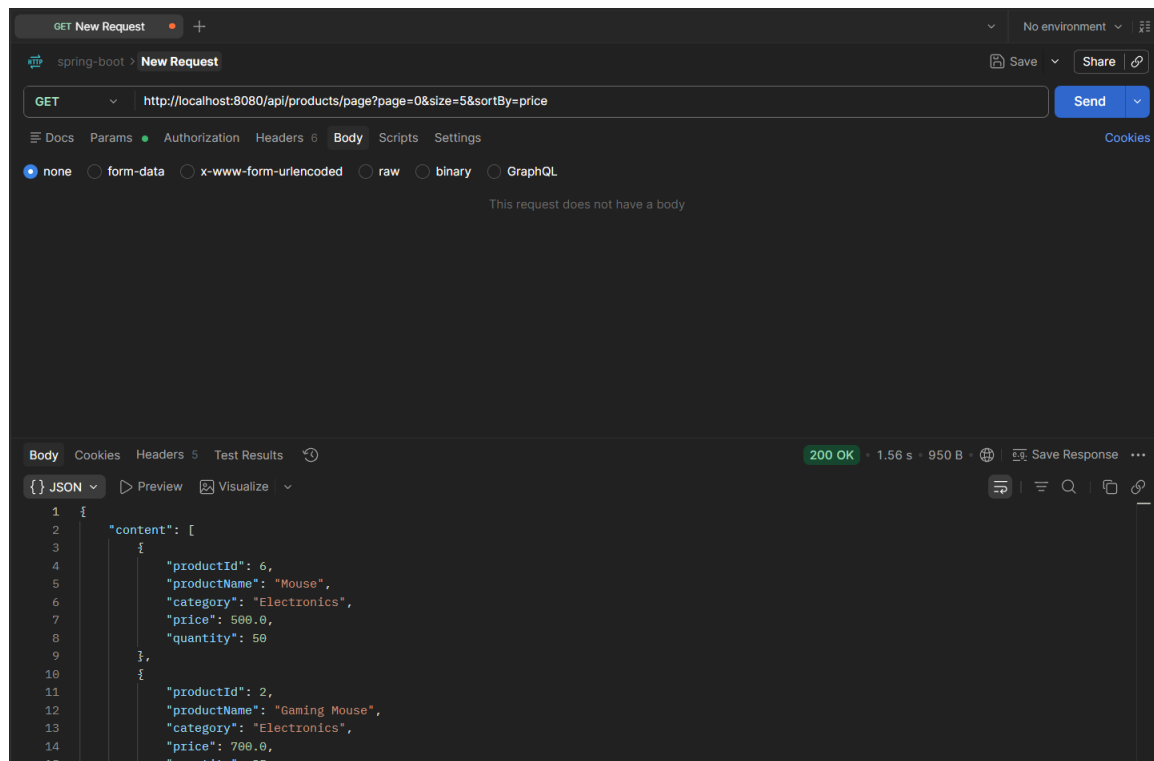
API'S TESTING USING POSTMAN

1. GET API – Display First Page of Products Sorted by Price:

The GET API is used to retrieve the first page of product records from the database. The API fetches only **5 products** from the first page and sorts them in **ascending order of price** using Spring Data JPA Pagination and Sorting. This improves application performance by loading only the required records instead of fetching all products.

GET:

<http://localhost:8080/api/products/page?page=0&size=5&sortBy=price>



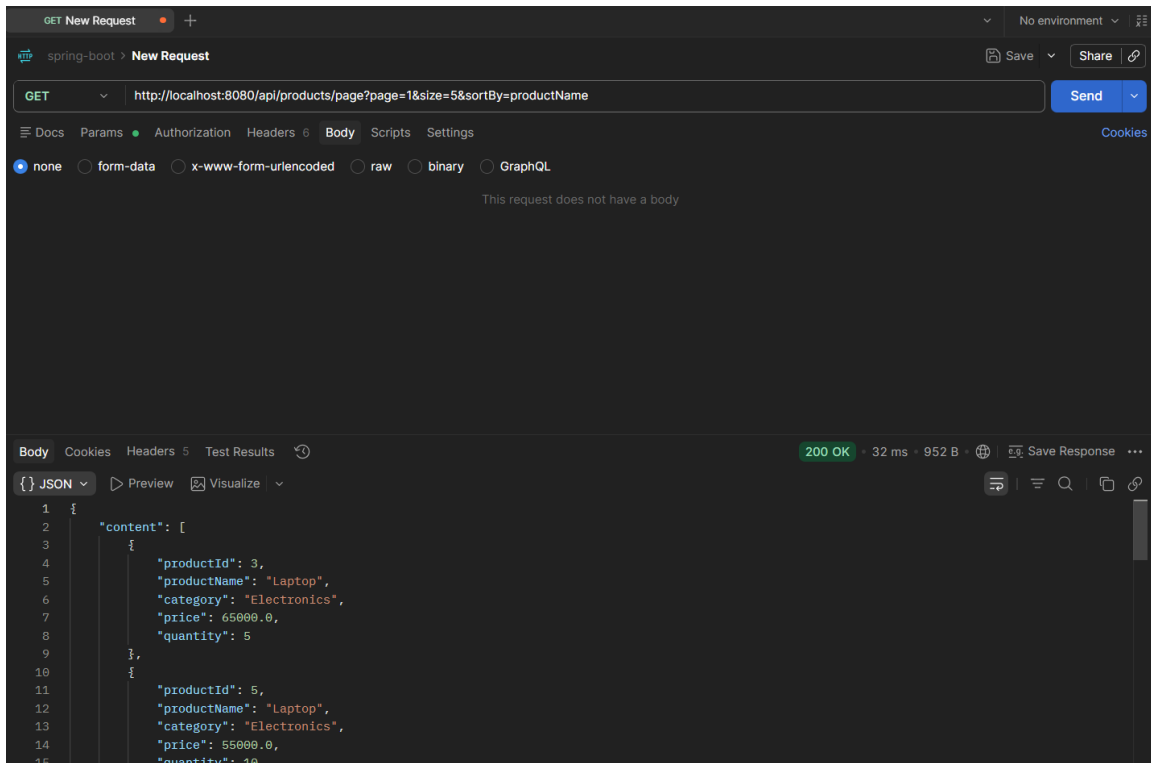
SEE THE FIRST PAGE OF PRODUCTS IS DISPLAYED SUCCESSFULLY IN ASCENDING ORDER OF PRICE.

2.GET API – Display Second Page of Products Sorted by Price:

The GET API retrieves products sorted alphabetically according to the **productName** field. The API displays the first page with five products while maintaining alphabetical order.

GET:

<http://localhost:8080/api/products/page?page=1&size=5&sortBy=productName>



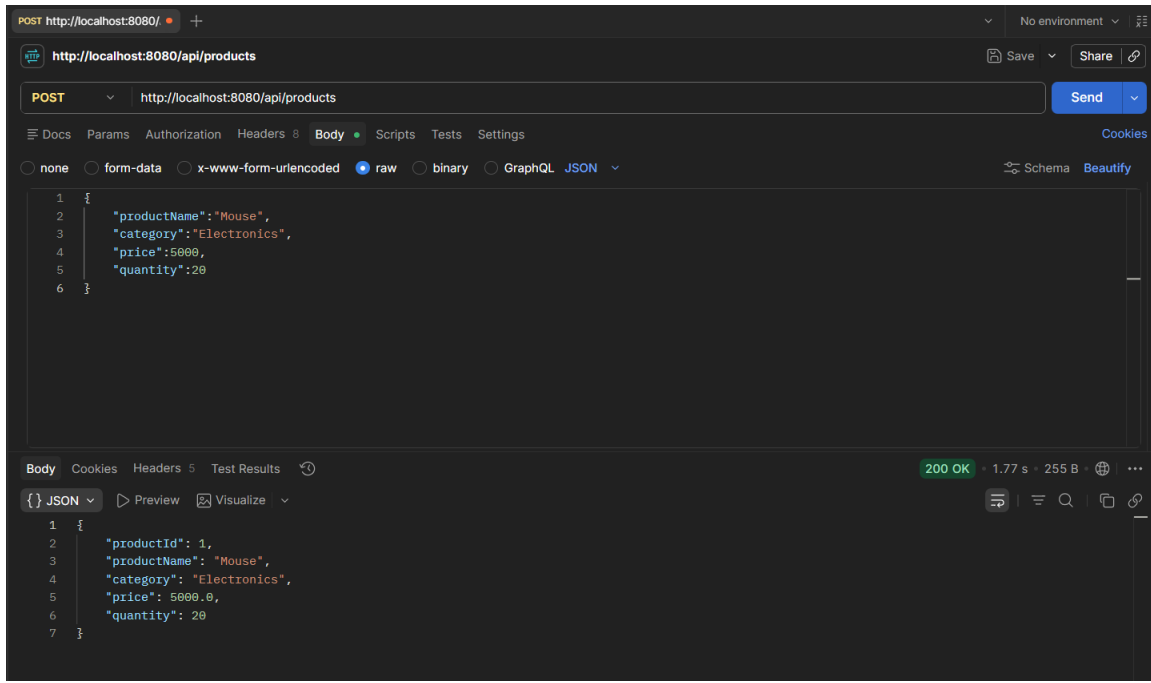
SEE THE PRODUCTS ARE DISPLAYED IN ALPHABETICAL ORDER OF PRODUCT NAME.

3. POST API – Add Product:

The POST API is used to insert a new product into the MySQL database. The product details are sent in the request body as JSON, and Spring Boot stores the record using the `save()` method.

POST

<http://localhost:8080/api/products>



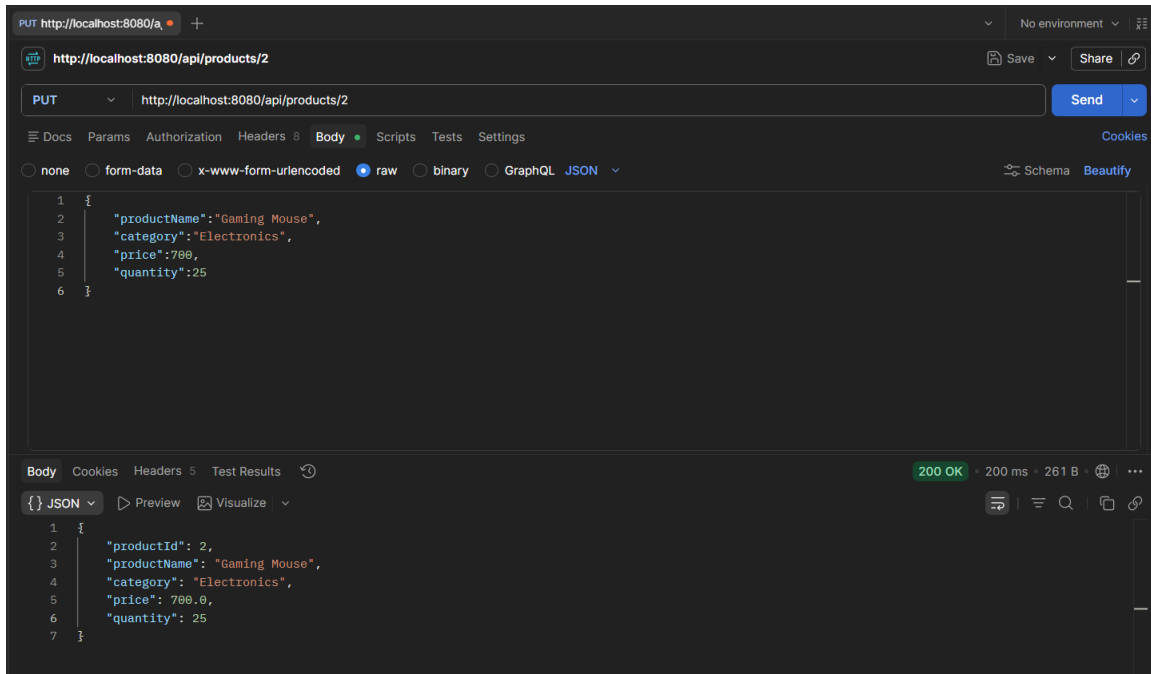
SEE THE PRODUCT IS ADDED SUCCESSFULLY INTO THE DATABASE.

5. PUT API – Update Product:

The PUT API is used to modify the details of an existing product. The Product ID is specified in the URL, while the updated values are provided in the request body.

Just give the id which you want to update give json data and data will be updated

PUT: <http://localhost:8080/api/products/2>



You can see before this the id 2 had the keyboard with different price now it's updated

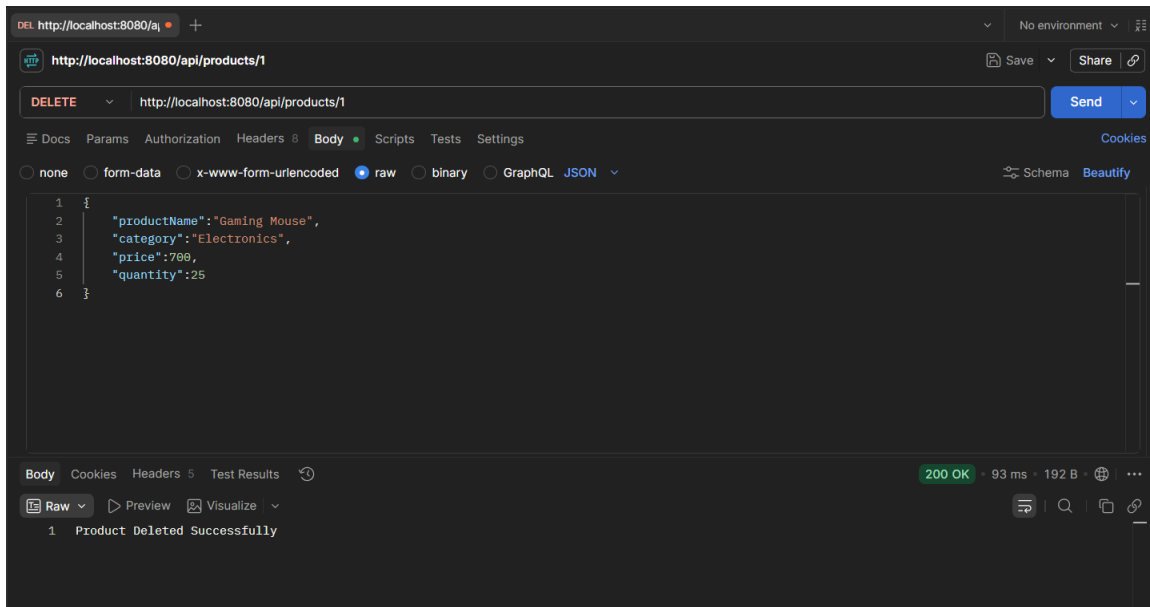
6. DELETE API – Delete Product:

The DELETE API removes the selected product from the database using its Product ID.

Just give the id which you want to delete in postman and it will be gone from database...!

Delete :

<http://localhost:8080/api/products/2>



Ignore that body with json data I forgot to remove It it's just none the body and json data only used in in post and put

The data is deleted from database with id 1

-----END-OF-PRACTICAL(8)-----

-----CHAPTER:5-ENDS-HERE-----

Chapter 6 — Building RESTful Microservices

Practical No. 9 : Building a RESTful Microservice using @RestController and ResponseEntity

Objective:

To design and build a RESTful microservice endpoint using `@RestController`, mapping HTTP verbs to CRUD operations, and using `ResponseEntity` to control the HTTP status code and headers returned to the client — the core REST-building technique introduced in this chapter.

Class Name(s) with Attributes:

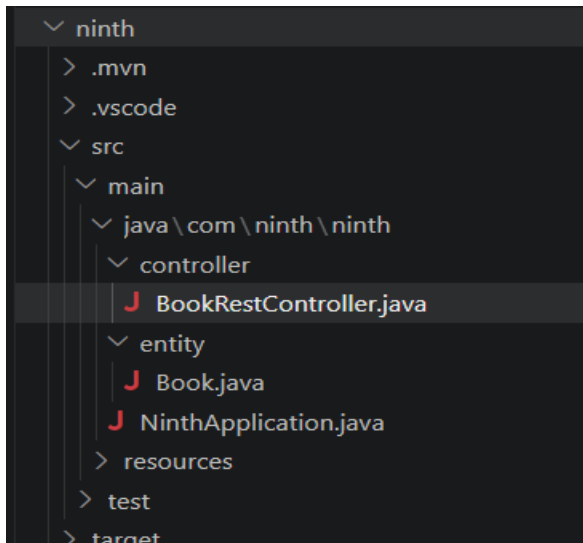
Book (POJO)

- id : int
- title : String
- author : String
- price : double

BookRestController (@RestController @RequestMapping("/api/books"))

- Methods return ResponseEntity / ResponseEntity<List>

Project Structure:



Dependencies

- Spring Web

Important Files

Book.java(POJO CLASS AND MODEL ENTITY)

```
package com.ninth.ninth.entity;

public class Book {

    private int id;
    private String title;
    private String author;
    private double price;

    public Book() {
    }

    public Book(int id, String title, String author, double price) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public double getPrice() {
        return price;
    }
}
```

```
}  
  
    public void setPrice(double price) {  
        this.price = price;  
    }  
}
```

BookRestController.java(controller file for restapi)

```
package com.ninth.ninth.controller;  
  
import java.util.ArrayList;  
import java.util.List;  
  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
import com.ninth.ninth.entity.Book;  
  
@RestController  
@RequestMapping("/api/books")  
public class BookRestController {  
  
    private List<Book> books = new ArrayList<>();  
  
    // Add Book  
    @PostMapping  
    public ResponseEntity<Book> addBook(@RequestBody Book book) {  
  
        books.add(book);  
  
        return ResponseEntity  
            .status(HttpStatus.CREATED)  
            .body(book);  
    }  
  
    // Get All Books  
    @GetMapping  
    public ResponseEntity<List<Book>> getAllBooks() {  
  
        return ResponseEntity.ok(books);  
    }  
  
    // Get Book by ID
```

```
@GetMapping("/{id}")
public ResponseEntity<Book> getBook(@PathVariable int id) {

    for (Book book : books) {

        if (book.getId() == id) {

            return ResponseEntity.ok(book);
        }
    }

    return ResponseEntity.notFound().build();
}

// Update Book
@PutMapping("/{id}")
public ResponseEntity<Book> updateBook(@PathVariable int id,
    @RequestBody Book updatedBook) {

    for (Book book : books) {

        if (book.getId() == id) {

            book.setTitle(updatedBook.getTitle());
            book.setAuthor(updatedBook.getAuthor());
            book.setPrice(updatedBook.getPrice());

            return ResponseEntity.ok(book);
        }
    }

    return ResponseEntity.notFound().build();
}

// Delete Book
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteBook(@PathVariable int id) {

    for (Book book : books) {

        if (book.getId() == id) {

            books.remove(book);

            return ResponseEntity.noContent().build();
        }
    }

    return ResponseEntity.notFound().build();
}
```

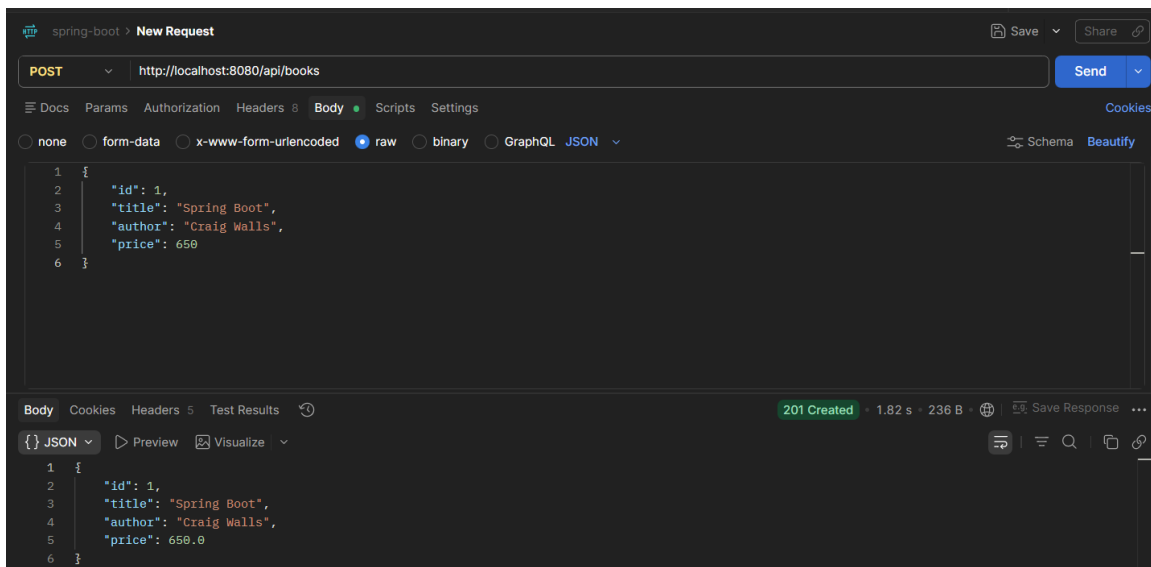
```
}
```

API'S TESTING USING POSTMAN

9.1 POST API – Add Book

The POST API is used to add a new book to the application. The book details are sent in JSON format, and the controller stores the book in an in-memory ArrayList. The API returns the added book along with the 201 Created HTTP status using ResponseEntity.

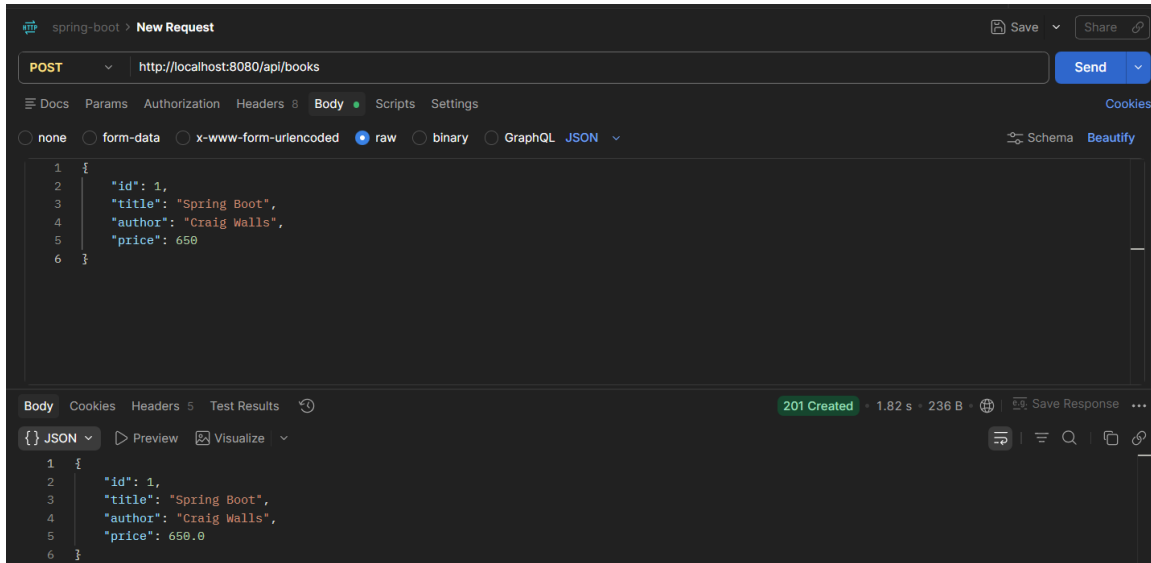
<http://localhost:8080/api/books>



9.2 GET API – Display All Books

The GET API is used to retrieve all books stored in the application. The controller returns the complete list of books from the in-memory ArrayList with the **200 OK** HTTP status..

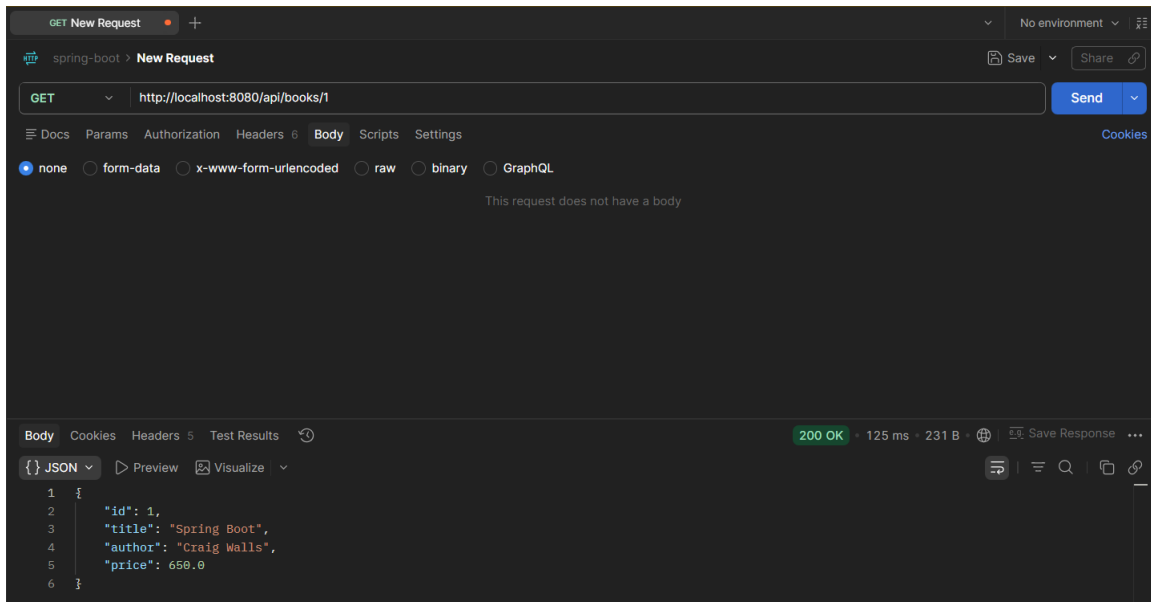
<http://localhost:8080/api/books>



9.3 GET API – Display Book by ID

The GET API is used to retrieve a specific book by its ID. If the book exists, the controller returns the book details with the **200 OK** status. If the book is not found, it returns **404 Not Found**.

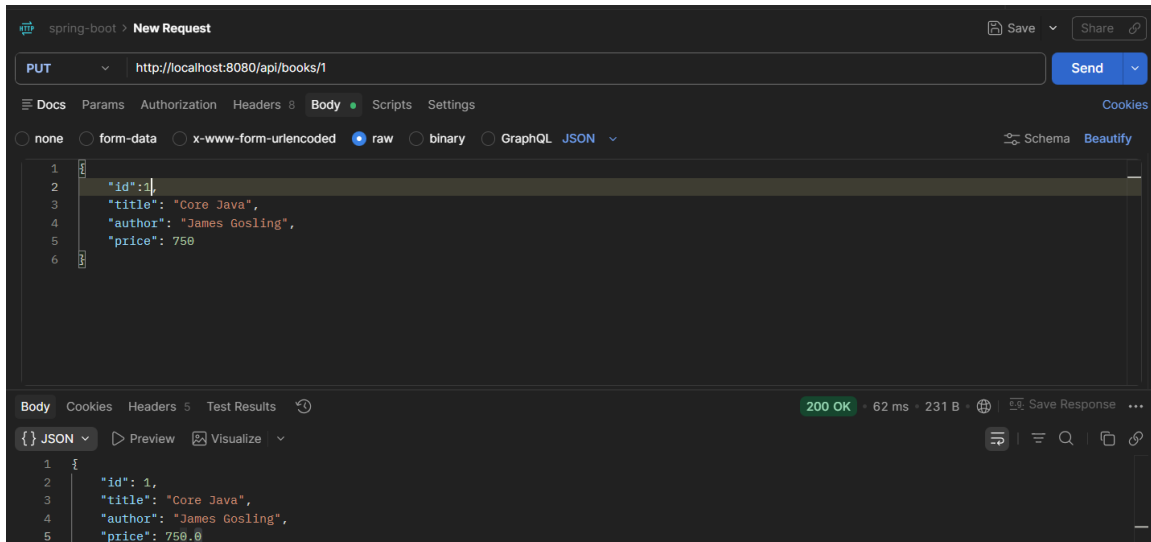
<http://localhost:8080/api/books/1>



9.4 PUT API – Update Book

The PUT API is used to update the details of an existing book. The controller searches for the book using its ID and updates its title, author, and price. After successful update, the modified book is returned with the **200 OK** status

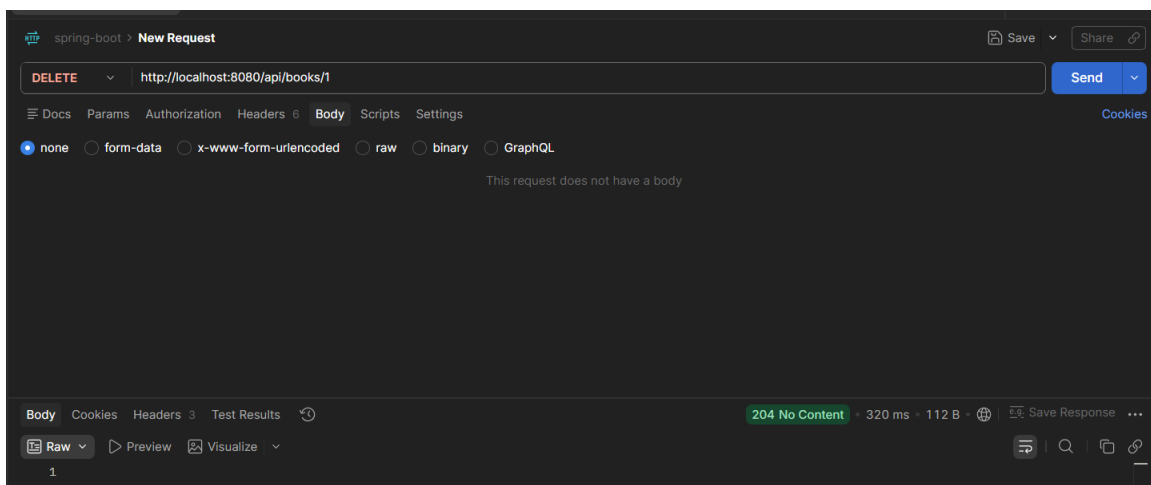
<http://localhost:8080/api/books/1>



9.5 DELETE API – Delete Book

The DELETE API is used to remove a book from the application using its ID. If the book exists, it is deleted from the in-memory list, and the controller returns the **204 No Content** HTTP status. If the book is not found, it returns **404 Not Found**.

<http://localhost:8080/api/books/1>



-----*****-----

Practical No. 10: Layered Microservice Architecture

– RestController + Service + JPA Repository

Objective:

To structure a RESTful microservice into the standard Controller–Service–Repository layers recommended in this chapter, keeping business logic out of the controller and persistence access out of the service.

Class Name(s) with Attributes:

Product (@Entity, reused)

- productId, productName, category, price, quantity

ProductService (interface)

- CRUD method signatures

ProductServiceImpl (@Service)

- productRepository : ProductRepository

ProductRestController (@RestController)

- productService : ProductService



Dependencies

- Spring Web

Mysql driver

jpa

Important Files

Product.java(MODEL ENTITY)

```
package com.tenth.tenth.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "product")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer productId;

    private String productName;

    private String category;

    private double price;

    private int quantity;

    public Product() {
    }

    public Integer getProductId() {
        return productId;
    }

    public void setProductId(Integer productId) {
        this.productId = productId;
    }

    public String getProductName() {
```

```
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}
```

ProductRepository.java

```
package com.tenth.tenth.repository;

import com.tenth.tenth.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product,
Integer> {

}
```

ProductService.java(SERVICE LAYER FOR BUSINESS)

```
package com.tenth.tenth.service;

import java.util.List;
import com.tenth.tenth.entity.Product;

public interface ProductService {

    Product addProduct(Product product);

    List<Product> getAllProducts();

    Product getProductById(Integer id);

    Product updateProduct(Integer id, Product product);

    void deleteProduct(Integer id);
}
```

ProductServiceImpl.java(SERVICE LAYER)

```
package com.tenth.tenth.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.tenth.tenth.entity.Product;
import com.tenth.tenth.repository.ProductRepository;

@Service
public class ProductServiceImpl implements ProductService {

    @Autowired
    private ProductRepository repository;
```

```
@Override
public Product addProduct(Product product) {

    return repository.save(product);
}

@Override
public List<Product> getAllProducts() {

    return repository.findAll();
}

@Override
public Product getProductById(Integer id) {

    return repository.findById(id).orElse(null);
}

@Override
public Product updateProduct(Integer id, Product product) {

    product.setProductId(id);

    return repository.save(product);
}

@Override
public void deleteProduct(Integer id) {

    repository.deleteById(id);
}
}
```

ProductRestController.java(Controller File)

```
package com.tenth.tenth.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.tenth.tenth.entity.Product;
import com.tenth.tenth.service.ProductService;
```

```
@RestController  
@RequestMapping("/api/products")  
public class ProductRestController {  
  
    @Autowired  
    private ProductService service;  
  
    // Add Product  
    @PostMapping  
    public Product addProduct(@RequestBody Product product) {  
  
        return service.addProduct(product);  
    }  
  
    // Get All Products  
    @GetMapping  
    public List<Product> getAllProducts() {  
  
        return service.getAllProducts();  
    }  
  
    // Get Product By Id  
    @GetMapping("/{id}")  
    public Product getProduct(@PathVariable Integer id) {  
  
        return service.getProductById(id);  
    }  
  
    // Update Product  
    @PutMapping("/{id}")  
    public Product updateProduct(@PathVariable Integer id,  
        @RequestBody Product product) {  
  
        return service.updateProduct(id, product);  
    }  
  
    // Delete Product  
    @DeleteMapping("/{id}")  
    public String deleteProduct(@PathVariable Integer id) {  
  
        service.deleteProduct(id);  
  
        return "Product Deleted Successfully";  
    }  
}
```

Application.properties

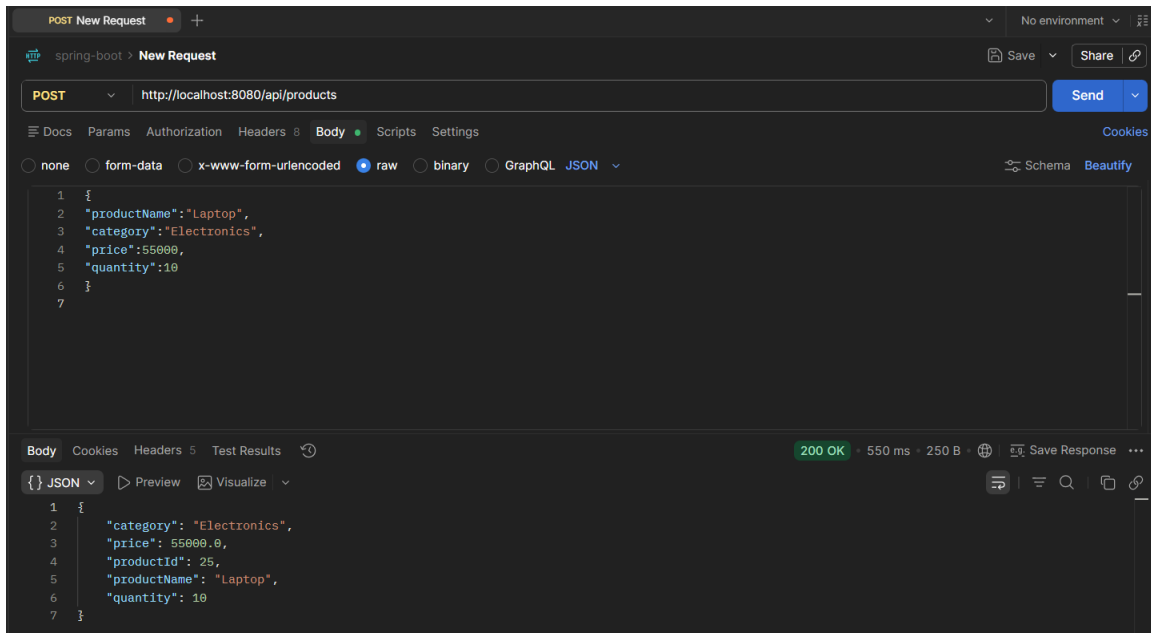
```
spring.application.name=tenth  
spring.datasource.url=jdbc:mysql://localhost:3306/product_db  
spring.datasource.username=root  
spring.datasource.password=  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

10.1 POST API – Add Product

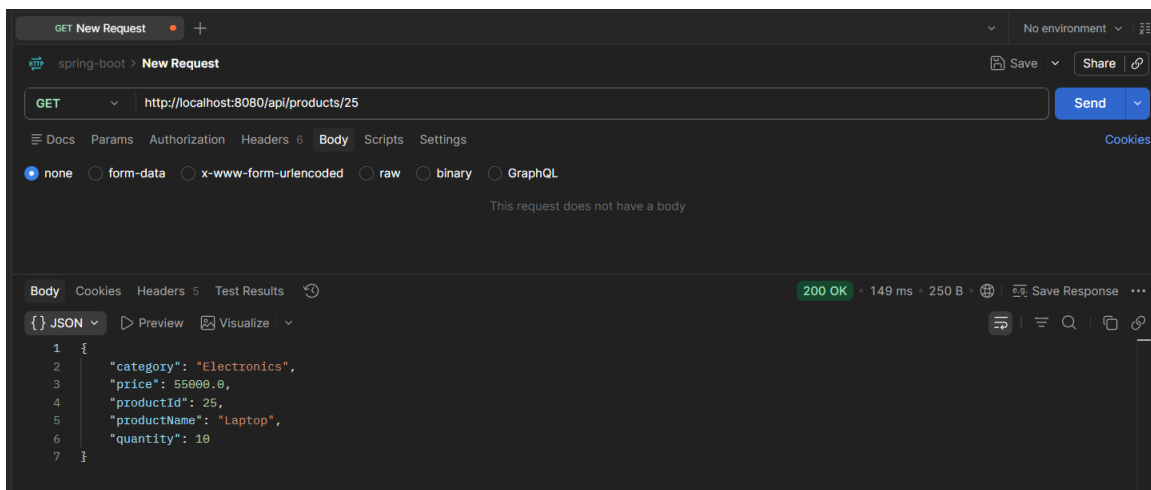
The POST API is used to add a new product to the database. The product details are sent in JSON format, and the request is handled by the ProductRestController. The controller forwards the request to the ProductService, which performs the business logic and stores the product using the ProductRepository.

<http://localhost:8080/api/books>



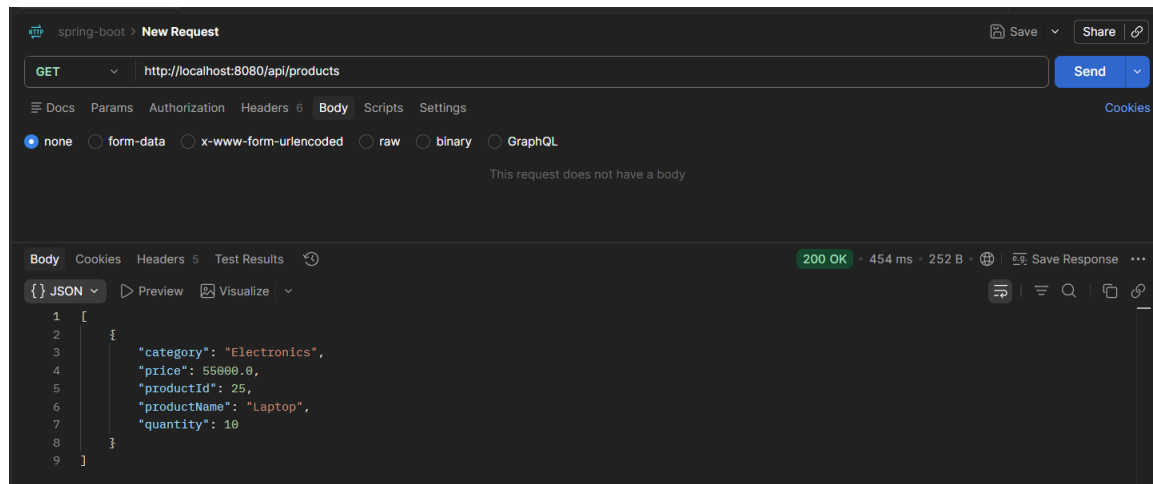
10.2 GET API – Display Product by ID

The GET API is used to retrieve a specific product using its ID. The controller receives the product ID, the service searches the database through the repository, and the corresponding product details are returned.



10.3 GET API – Display All Products

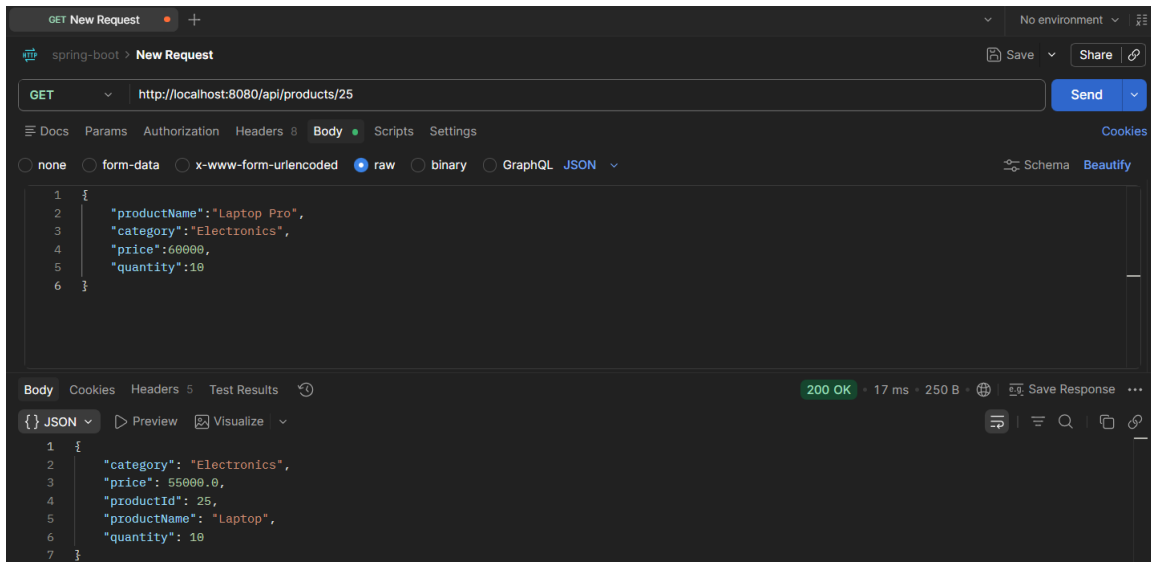
The GET API is used to retrieve all products stored in the database. The request is received by the controller, passed to the service layer, and the service fetches all product records using the repository. The complete product list is returned with **HTTP Status 200 OK**



10.4 PUT API – Update Product

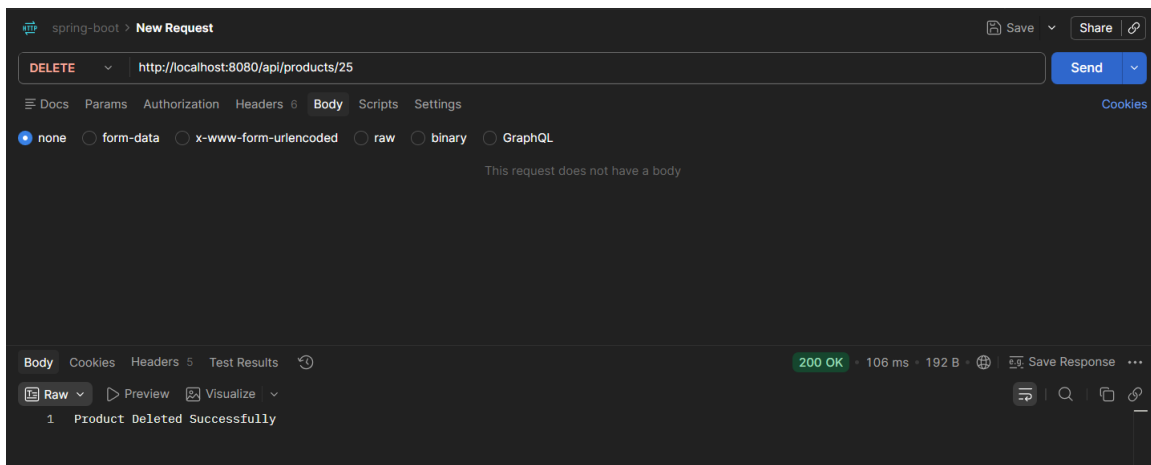
The PUT API is used to update an existing product. The product ID is provided in the URL, while the updated product details are sent in the request body. The service layer updates the product and saves the changes in the database.

<http://localhost:8080/api/products/1>



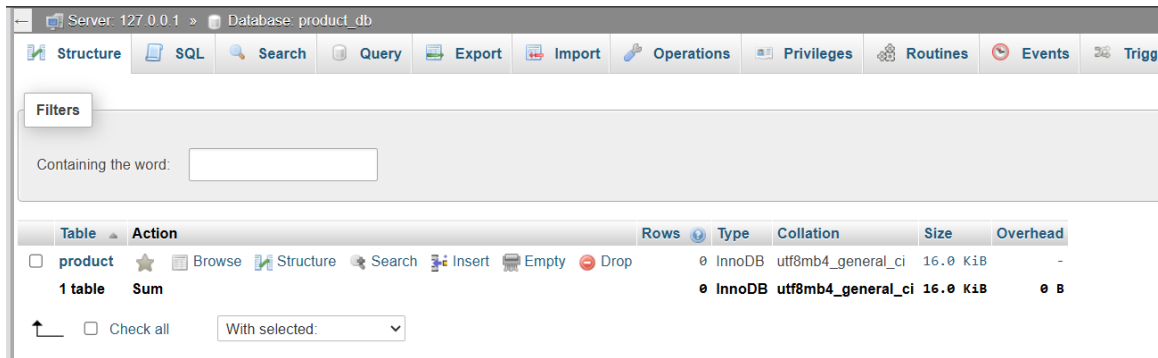
10.5 DELETE API – Delete Product

The DELETE API is used to remove a product from the database using its ID. The controller forwards the delete request to the service layer, which deletes the product through the repository. After successful deletion, a confirmation message is returned.

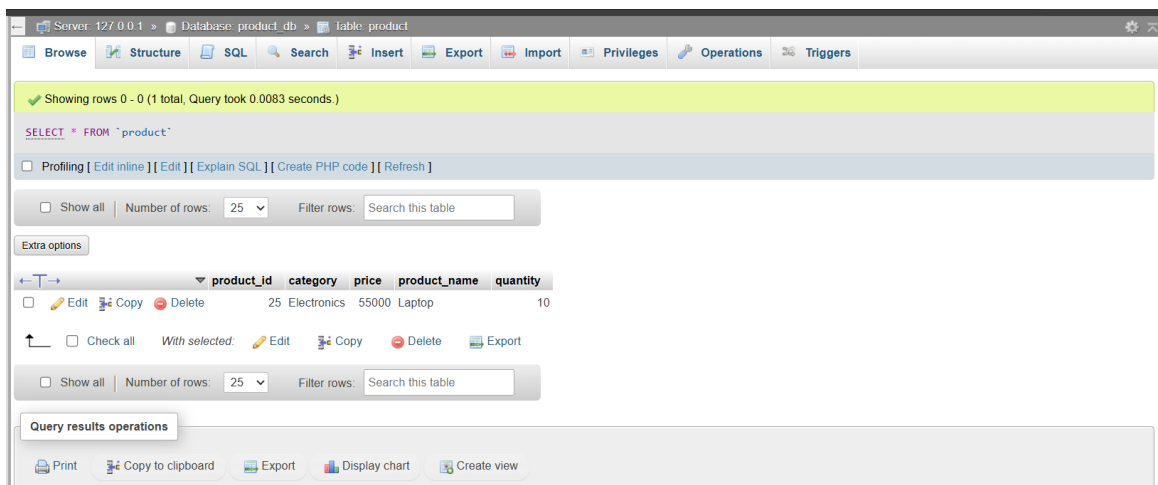


Database Output

TABLE'S AND DATA STORING SUCCEFULLY



WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS



Practical No. 11: Handling Request Data – @PathVariable, @RequestParam, @RequestBody

Objective:

To correctly use the three main ways a Spring REST controller receives client input — `@PathVariable` for identifying a resource, `@RequestParam` for optional query filters, and `@RequestBody` for JSON payloads — as covered in the request-handling section of this chapter.

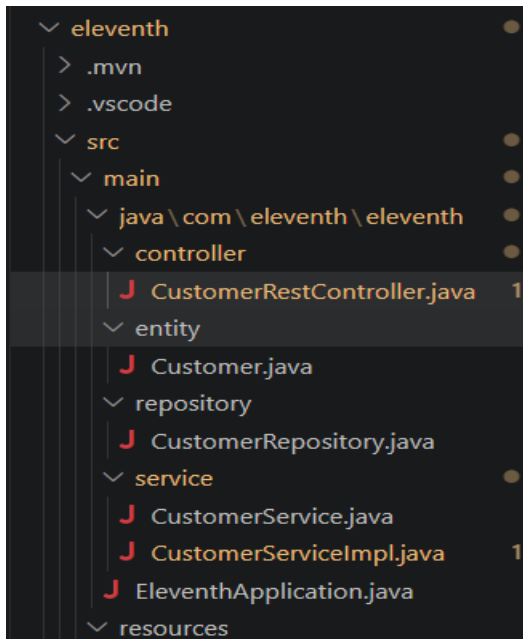
Class Name(s) with Attributes:

Customer (@Entity)

- `customerId : int (@Id @GeneratedValue)`
- `name : String`
- `city : String`
- `age : int`

CustomerRestController (@RestController)

- Method: `getById(@PathVariable id)`
- Method: `search(@RequestParam city, @RequestParam(required=false) minAge)`
- Method: `create(@RequestBody CustomerDTO)`



Dependencies

- Spring Web

Mysql driver

jpa

Important Files

Customer.java(MODEL ENTITY)

```
package com.eleventh.eleventh.entity;  
  
import jakarta.persistence.*;  
  
@Entity  
public class Customer {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Integer customerId;  
  
    private String name;
```

```
private String city;
private int age;

public Customer() {
}

public Integer getCustomerId() {
    return customerId;
}

public void setCustomerId(Integer customerId) {
    this.customerId = customerId;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getCity() {
    return city;
}

public void setCity(String city) {
    this.city = city;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}
}
```

CustomerRepository.java

```
package com.eleventh.eleventh.repository;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import com.eleventh.eleventh.entity.Customer;
```

```
public interface CustomerRepository extends JpaRepository<Customer, Integer> {  
  
    List<Customer> findByCityAndAgeGreaterThanOrEqualTo(String city, int age);  
  
}
```

CustomerService.java(SERVICE LAYER FOR BUSINESS)

```
package com.eleventh.eleventh.service;  
  
import java.util.List;  
  
import com.eleventh.eleventh.entity.Customer;  
  
public interface CustomerService {  
  
    Customer create(Customer customer);  
  
    Customer getById(Integer id);  
  
    List<Customer> search(String city, int minAge);  
  
    Customer update(Integer id, Customer customer);  
  
    void delete(Integer id);  
  
}
```

CustomerServiceImpl.java(SERVICE LAYER)

```
package com.eleventh.eleventh.service;  
  
import java.util.List;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;
```

```
import com.eleventh.eleventh.entity.Customer;
import com.eleventh.eleventh.repository.CustomerRepository;

@Service
public class CustomerServiceImpl implements CustomerService {

    @Autowired
    private CustomerRepository repository;

    @Override
    public Customer create(Customer customer) {

        return repository.save(customer);
    }

    @Override
    public Customer getById(Integer id) {

        return repository.findById(id).orElse(null);
    }

    @Override
    public List<Customer> search(String city, int minAge) {

        return repository.findByCityAndAgeGreaterThanOrEqualTo(city,
minAge);
    }

    @Override
    public Customer update(Integer id, Customer customer) {

        customer.setCustomerId(id);

        return repository.save(customer);
    }

    @Override
    public void delete(Integer id) {

        repository.deleteById(id);
    }
}
```

CustomerRestController.java(Controller File)

```
package com.eleventh.eleventh.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.eleventh.eleventh.entity.Customer;
import com.eleventh.eleventh.service.CustomerService;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    @Autowired
    private CustomerService service;

    // @RequestBody
    @PostMapping
    public Customer create(@RequestBody Customer customer) {

        return service.create(customer);
    }

    // @PathVariable
    @GetMapping("/{id}")
    public Customer getById(@PathVariable Integer id) {

        return service.getById(id);
    }

    // @RequestParam
    @GetMapping("/search")
    public List<Customer> search(

        @RequestParam String city,

        @RequestParam(defaultValue = "0") int minAge) {

        return service.search(city, minAge);
    }

    // PathVariable + RequestBody
    @PutMapping("/{id}")
    public Customer update(

        @PathVariable Integer id,

        @RequestBody Customer customer) {
```

```
        return service.update(id, customer);
    }

    // PathVariable
    @DeleteMapping("/{id}")
    public String delete(@PathVariable Integer id) {

        service.delete(id);

        return "Customer Deleted Successfully";
    }
}
```

Application.properties

```
spring.application.name=eleventh

spring.datasource.url=jdbc:mysql://localhost:3306/customer

spring.datasource.username=root

spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true
```

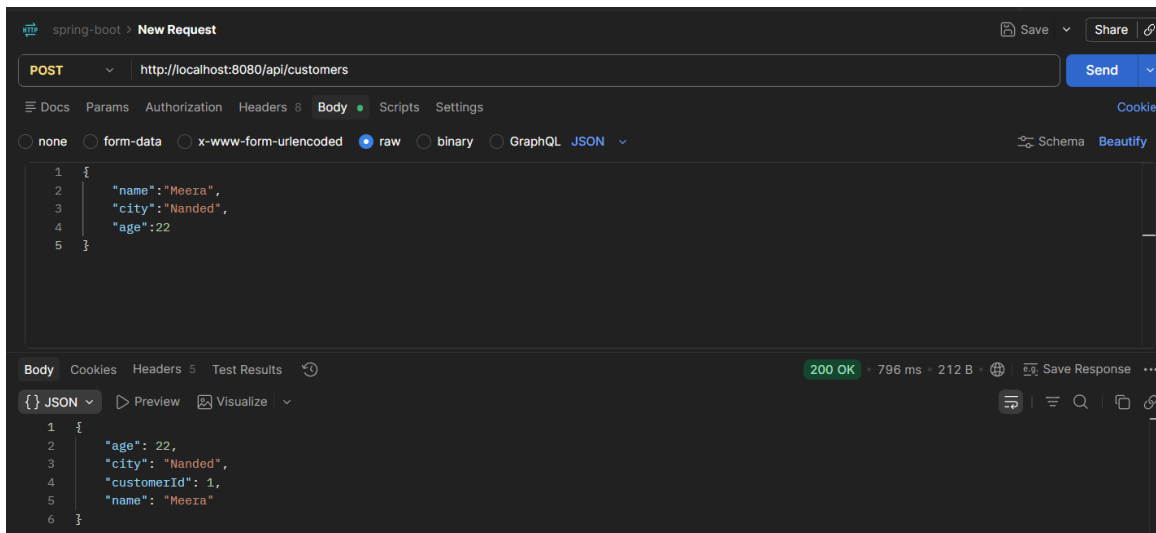
API'S TESTING USING POSTMAN

11.1 POST API – Add Customer (@RequestBody)

The POST API is used to create a new customer in the database. The customer details are sent in JSON format using the request body. Spring Boot converts the JSON into a Customer object using `@RequestBody`, and the service layer stores it in the database.

POST

<http://localhost:8080/api/customers>

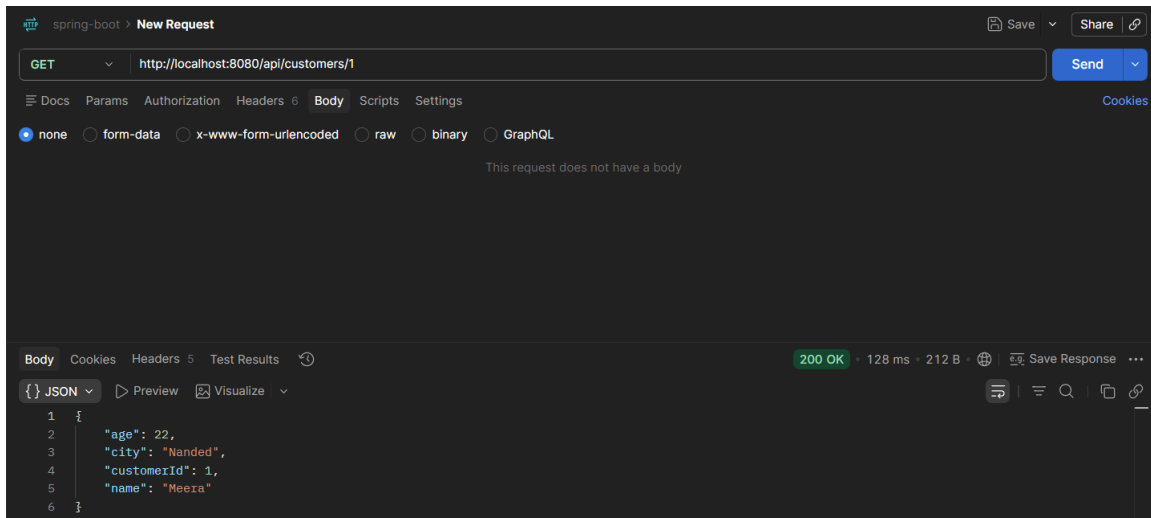


11.2 GET API – Display Customer by ID (@PathVariable)

The GET API is used to retrieve a specific customer using the customer ID. The ID is passed in the URL using `@PathVariable`. The controller receives the ID, forwards it to the service layer, and the customer details are returned.

GET

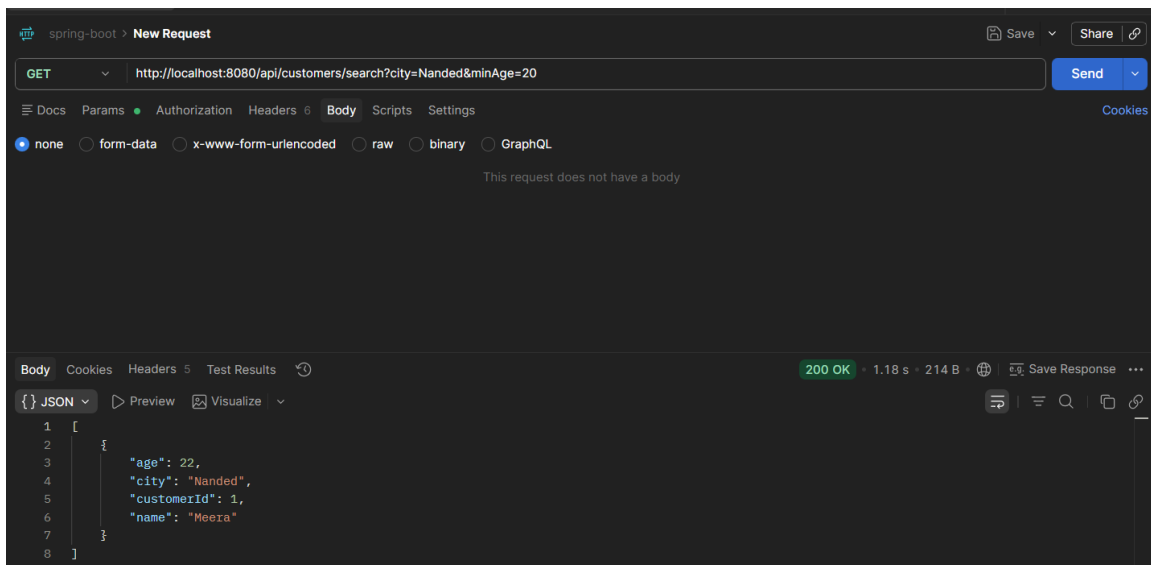
<http://localhost:8080/api/customers/1>



11.3 GET API – Search Customers (@RequestParam)

The GET API is used to search customers based on the city and minimum age. Both values are passed as query parameters using `@RequestParam`. The service layer filters the records and returns the matching customers. **GET**

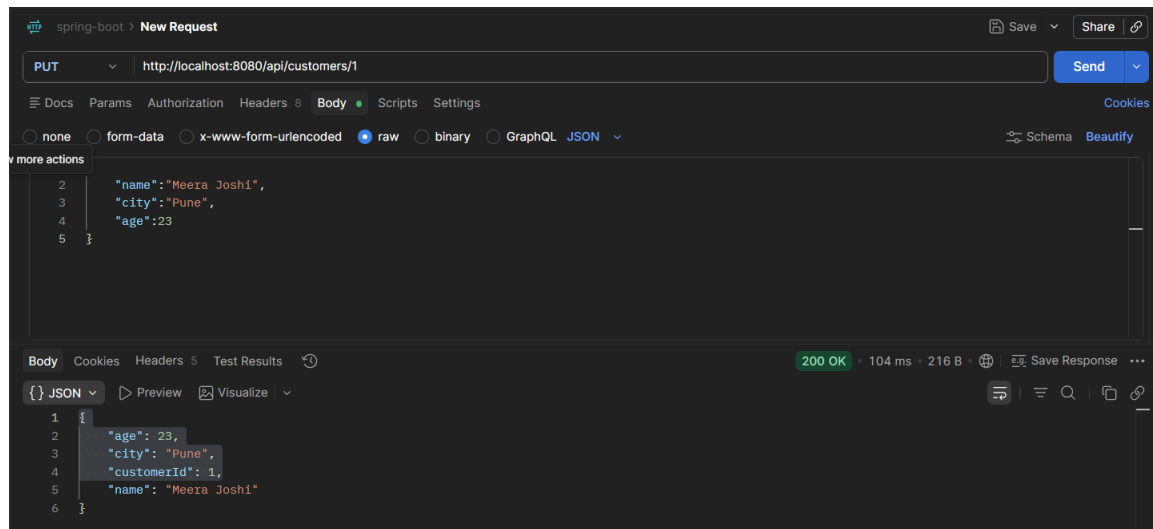
<http://localhost:8080/api/customers/search?city=Nanded&minAge=20>



11.4 PUT API – Update Customer (@PathVariable + @RequestBody)

The PUT API is used to update an existing customer's details. The customer ID is passed through @PathVariable, while the updated customer information is sent in the request body using @RequestBody. The service layer updates the record in the database.

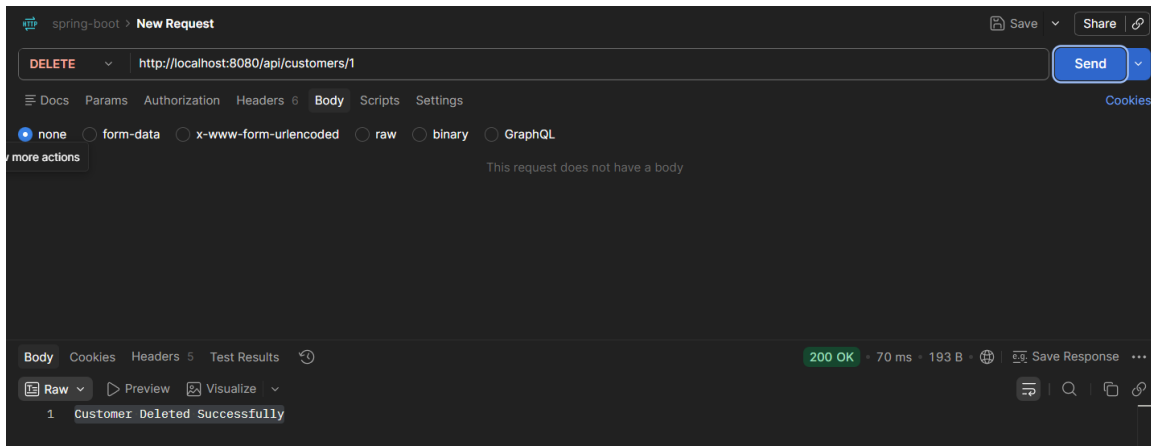
<http://localhost:8080/api/customers/1>



11.5 DELETE API – Delete Customer (@PathVariable)

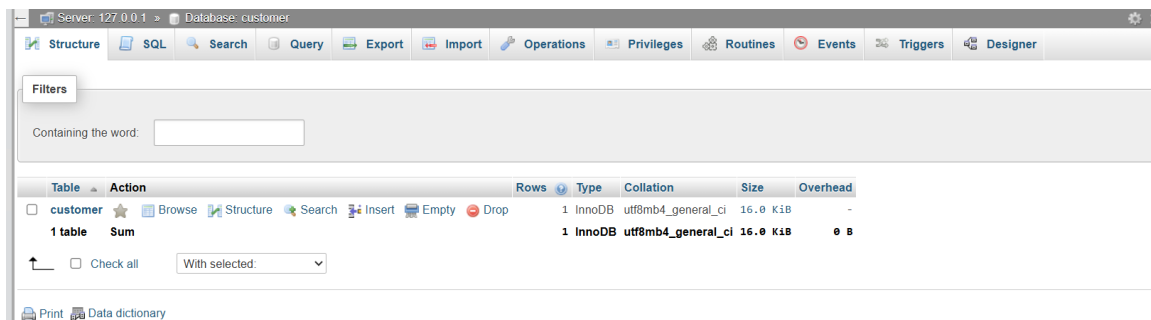
The DELETE API is used to remove a customer from the database using the customer ID. The ID is passed through the URL using @PathVariable. After successful deletion, a confirmation message is returned.

<http://localhost:8080/api/customers/1>

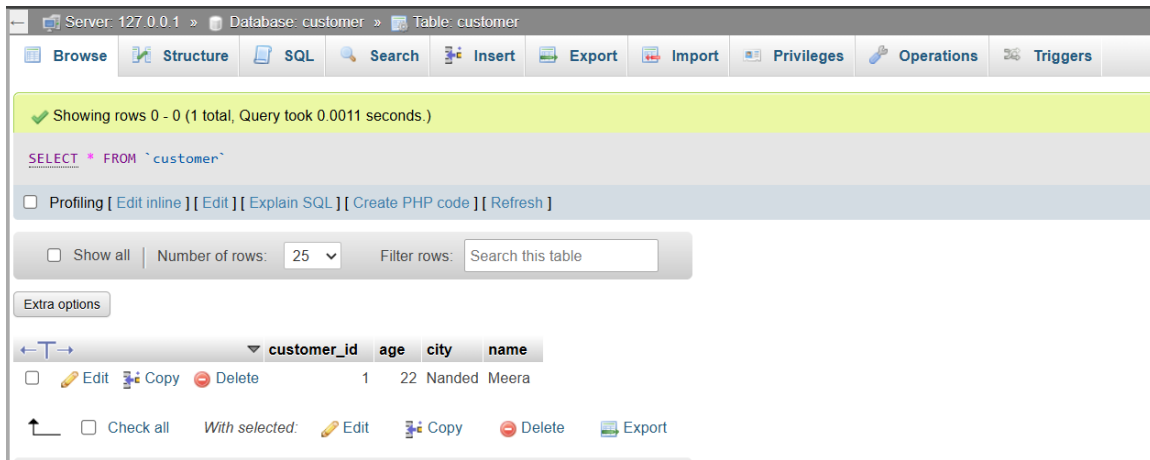


Database Output

TABLE'S AND DATA STORING SUCCEFULLY



WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS



Practical No. 12: DTO Pattern – Separating the Entity from the API Contract

Objective:

To decouple the persistent JPA entity from the data actually exposed by the REST API using the Data Transfer Object (DTO) pattern discussed in this chapter, hiding sensitive/internal fields and mapping between Entity and DTO.

Class Name(s) with Attributes:

Employee (@Entity — add sensitive field)

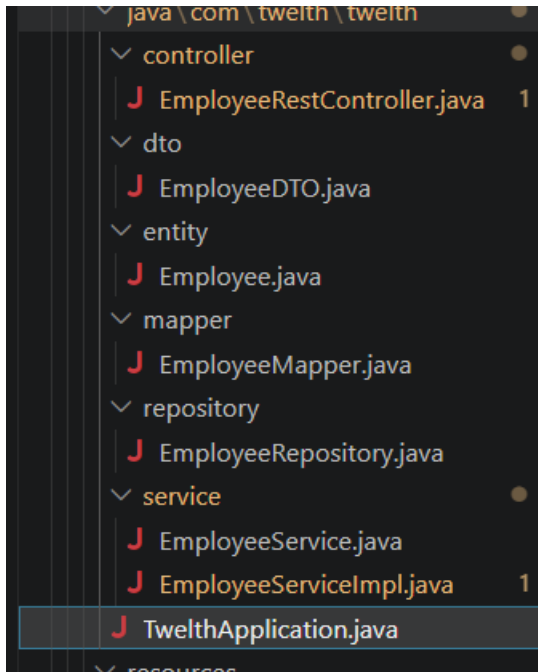
- empId, empName, department, salary
- password : String (sensitive — never exposed)

EmployeeDTO (POJO)

- empId : int • empName : String • department : String

EmployeeMapper (utility class)

- static EmployeeDTO toDto(Employee e)
- static Employee toEntity(EmployeeDTO d)



Dependencies

- Spring Web

Mysql driver

jpa

Important Files

Employee.java(MODEL ENTITY)

```
package com.twelfth.twelfth.entity;

import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;
```

```
private String empName;

private String department;

private double salary;

private String password;

public Employee() {
}

public Integer getEmpId() {
    return empId;
}

public void setEmpId(Integer empId) {
    this.empId = empId;
}

public String getEmpName() {
    return empName;
}

public void setEmpName(String empName) {
    this.empName = empName;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public double getSalary() {
    return salary;
}

public void setSalary(double salary) {
    this.salary = salary;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}
```

```
}
```

EmployeesRepository.java

```
package com.twelth.twelth.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.twelth.twelth.entity.Employee;  
  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Integer> {  
  
}
```

EmployeeService.java(SERVICE LAYER FOR BUSINESS)

```
package com.twelth.twelth.service;  
  
import java.util.List;  
  
import com.twelth.twelth.dto.EmployeeDTO;  
  
public interface EmployeeService {  
  
    EmployeeDTO save(EmployeeDTO dto);  
  
    List<EmployeeDTO> findAll();  
  
    EmployeeDTO update(Integer id, EmployeeDTO dto);  
  
    void delete(Integer id);  
  
}
```

EmployeeServiceImpl.java(SERVICE LAYER)

```
package com.twelth.twelth.service;

import java.util.ArrayList;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.twelth.twelth.dto.EmployeeDTO;
import com.twelth.twelth.entity.Employee;
import com.twelth.twelth.mapper.EmployeeMapper;
import com.twelth.twelth.repository.EmployeeRepository;

@Service
public class EmployeeServiceImpl implements EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    @Override
    public EmployeeDTO save(EmployeeDTO dto) {

        Employee employee = EmployeeMapper.toEntity(dto);

        employee.setSalary(50000);

        employee.setPassword("admin123");

        Employee saved = repository.save(employee);

        return EmployeeMapper.toDTO(saved);
    }

    @Override
    public List<EmployeeDTO> findAll() {

        List<Employee> employees = repository.findAll();

        List<EmployeeDTO> list = new ArrayList<>();

        for (Employee e : employees) {

            list.add(EmployeeMapper.toDTO(e));
        }

        return list;
    }
}
```

```
@Override
public EmployeeDTO update(Integer id,
    EmployeeDTO dto) {

    Employee employee = repository.findById(id).orElse(null);

    if (employee == null)
        return null;

    employee.setEmpName(dto.getEmpName());
    employee.setDepartment(dto.getDepartment());

    Employee updated = repository.save(employee);

    return EmployeeMapper.toDTO(updated);
}

@Override
public void delete(Integer id) {

    repository.deleteById(id);

}
}
```

EmployeeDTO.java(DTO class)

```
package com.twelfth.twelfth.dto;

public class EmployeeDTO {

    private Integer empId;

    private String empName;

    private String department;

    public EmployeeDTO() {
    }

    public Integer getEmpId() {
        return empId;
    }

    public void setEmpId(Integer empId) {
        this.empId = empId;
    }
}
```

```
public String getEmpName() {  
    return empName;  
}  
  
public void setEmpName(String empName) {  
    this.empName = empName;  
}  
  
public String getDepartment() {  
    return department;  
}  
  
public void setDepartment(String department) {  
    this.department = department;  
}  
}
```

EmployeeMapper.java

```
package com.twelth.twelth.mapper;  
  
import com.twelth.twelth.dto.EmployeeDTO;  
import com.twelth.twelth.entity.Employee;  
  
public class EmployeeMapper {  
  
    public static EmployeeDTO toDTO(Employee employee) {  
  
        EmployeeDTO dto = new EmployeeDTO();  
  
        dto.setEmpId(employee.getEmpId());  
        dto.setEmpName(employee.getEmpName());  
        dto.setDepartment(employee.getDepartment());  
  
        return dto;  
    }  
  
    public static Employee toEntity(EmployeeDTO dto) {
```

```
Employee employee = new Employee();

employee.setEmpId(dto.getEmpId());
employee.setEmpName(dto.getEmpName());
employee.setDepartment(dto.getDepartment());

return employee;
}
}
```

Application.properties

```
spring.application.name=twelfth

spring.datasource.url=jdbc:mysql://localhost:3306/twelth
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

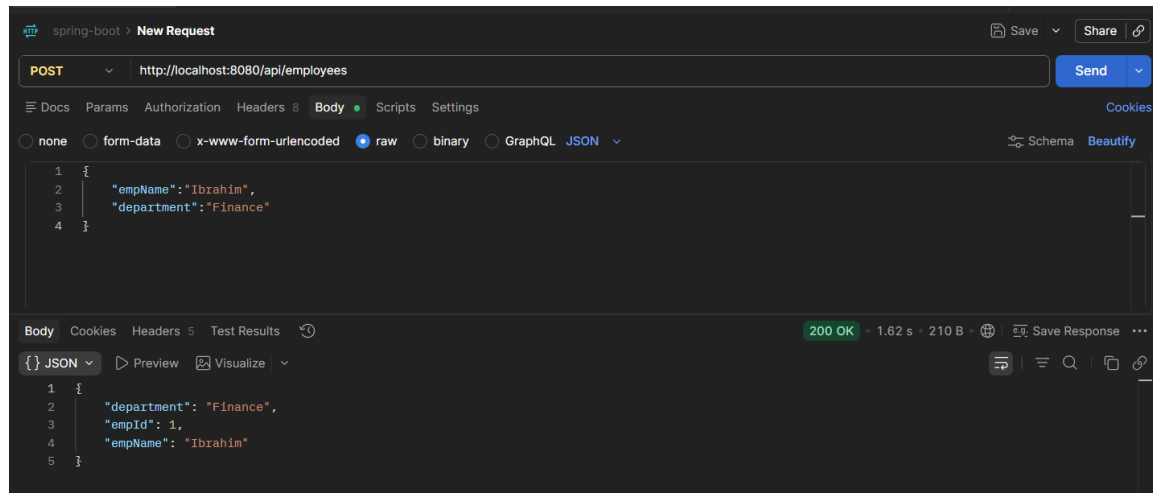
API'S TESTING USING POSTMAN

12.1 POST API – Add Employee (Using DTO)

The POST API is used to add a new employee to the database using the **EmployeeDTO** object. The employee details are received in JSON format through `@RequestBody`. The DTO is converted into an Entity using the `EmployeeMapper`, stored in the database, and the response is returned as a DTO without exposing sensitive

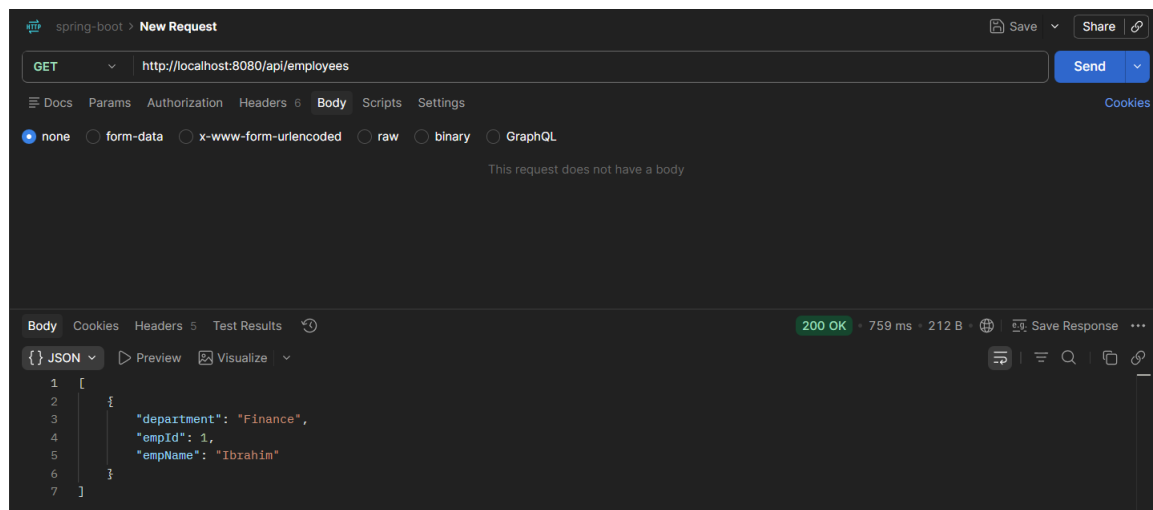
POST

<http://localhost:8080/api/employees>



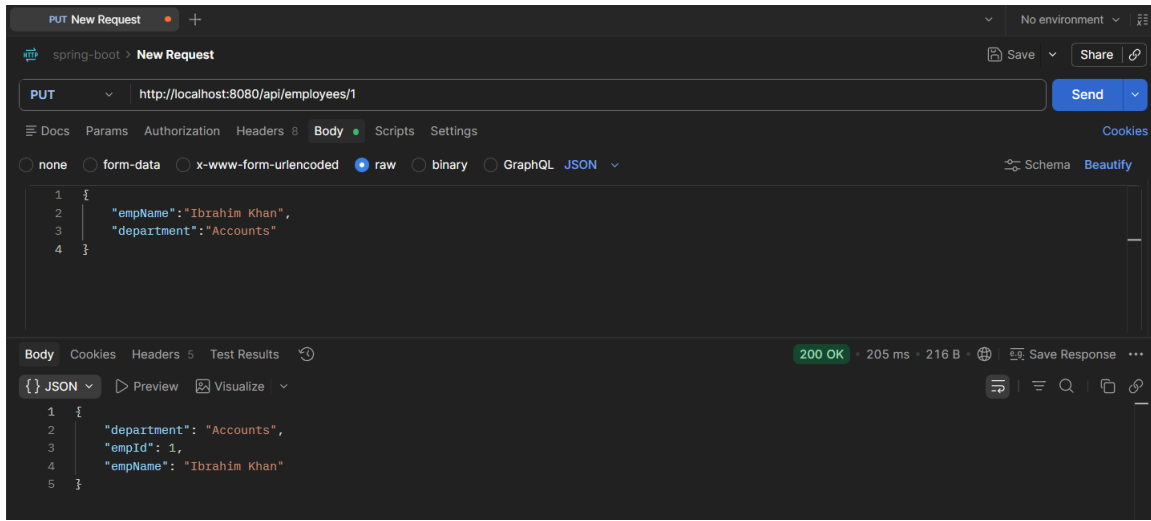
12.2 GET API – Display All Employees (Using DTO)

The GET API is used to retrieve all employee records from the database. The service layer fetches all Employee entities, converts them into EmployeeDTO objects using the EmployeeMapper, and returns only the required information. Sensitive fields such as **salary** and **password** are not included in the response.



12.3 PUT API – Update Employee (Using DTO)

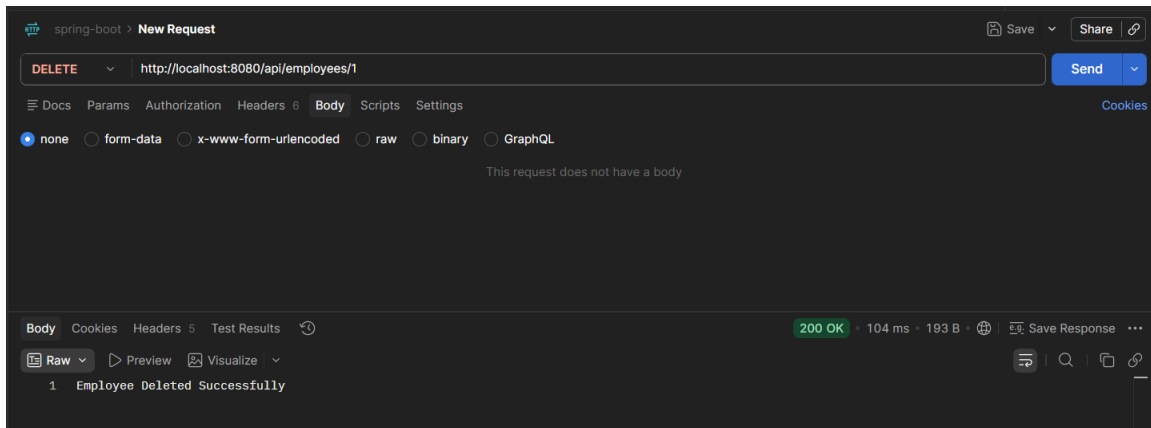
The PUT API is used to update an existing employee's information. The employee ID is passed using `@PathVariable`, while the updated details are received through `@RequestBody` as an `EmployeeDTO`. The service layer updates the record and returns the updated DTO.



12.4 DELETE API – Delete Employee

The DELETE API is used to remove an employee record from the database using the employee ID. The ID is received through `@PathVariable`, and the service layer deletes the corresponding record. A success message is returned after successful deletion.

DELETE

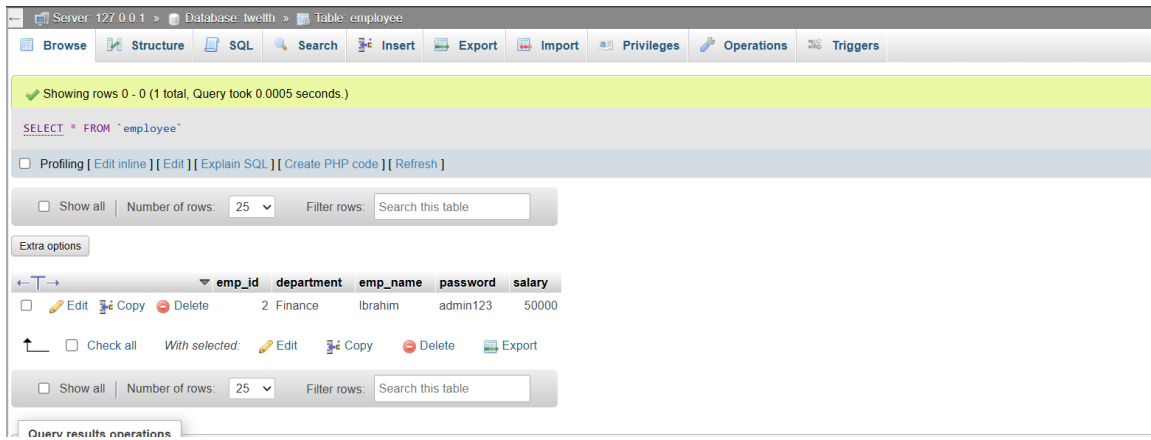


Database Output

TABLE'S AND DATA STORING SUCCEFULLY

A screenshot of a database management tool interface. The top menu bar includes 'Structure', 'SQL', 'Search', 'Query', 'Export', 'Import', 'Operations', 'Privileges', 'Routines', 'Events', 'Triggers', and 'Designer'. Below the menu is a 'Filters' section with a text input field. The main area displays a table structure for 'employee'. The table has one row with columns: 'Table', 'Action', 'Rows', 'Type', 'Collation', 'Size', and 'Overhead'. The table is named 'employee' and has 1 row. The table type is 'InnoDB', the collation is 'utf8mb4_general_ci', and the size is '16.0 KiB'. The overhead is '0 B'. There are also buttons for 'Browse', 'Structure', 'Search', 'Insert', 'Empty', and 'Drop'.

WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS



Server: 127.0.0.1 » Database: hwealth » Table: employee

Showing rows 0 - 0 (1 total, Query took 0.0005 seconds)

SELECT * FROM employee

Number of rows: 25 Filter rows: Search this table

emp_id	department	emp_name	password	salary
2	Finance	Ibrahim	admin123	50000

Number of rows: 25 Filter rows: Search this table

Practical No. 13: Request Validation using Bean Validation (@Valid) with RestController

Objective:

To validate incoming JSON payloads declaratively using Jakarta Bean Validation annotations (@NotBlank, @Email, @Min) combined with @Valid on the controller method, as covered in this chapter's discussion of building robust REST endpoints.

Class Name(s) with Attributes:

Customer (@Entity / DTO, reused)

- customerId : int (@Id @GeneratedValue)
- name : String (@NotBlank)
- email : String (@Email)
- age : int (@Min(18))

Project Structure

```
src
├── main
│   ├── java
│   │   ├── com.thirteenth.thirteenth
│   │   │   ├── ThirteenthApplication.java
│   │   │   ├── controller
│   │   │   │   ├── CustomerRestController.java
│   │   │   ├── entity
│   │   │   │   ├── Customer.java
│   │   │   ├── repository
│   │   │   │   ├── CustomerRepository.java
│   └── resources
│       └── application.properties
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver
- starter validation

Database

Database Name: thirteenth

Table: customer(created automatically by Hibernate)

Important Files

Customer.java(ENTITY model class)

```
package com.thirteenth.thirteenth.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer customerId;

    @NotBlank(message = "Name cannot be empty")
    private String name;

    @Email(message = "Enter a valid email")
    @NotBlank(message = "Email cannot be empty")
    private String email;

    @Min(value = 18, message = "Age must be at least 18")
    private int age;

    public Customer() {
    }

    public Integer getCustomerId() {
        return customerId;
    }

    public void setCustomerId(Integer customerId) {
        this.customerId = customerId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
```

```
        this.email = email;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

CustomerRepository.java(Repository file)

```
package com.thirteenth.thirteenth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.thirteenth.thirteenth.entity.Customer;

public interface CustomerRepository
    extends JpaRepository<Customer, Integer> {

}
```

EmployeeController.java(Controller file)

```
package com.thirteenth.thirteenth.controller;

import java.util.List;

import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.thirteenth.thirteenth.entity.Customer;
```

```
import
com.thirteenth.thirteenth.repository.CustomerRepository;

import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    @Autowired
    private CustomerRepository repository;

    // Add Customer
    @PostMapping
    public Customer addCustomer(@Valid @RequestBody
Customer customer) {

        return repository.save(customer);
    }

    // Display All Customers
    @GetMapping
    public List<Customer> getAllCustomers() {

        return repository.findAll();
    }

    // Display Customer By Id
    @GetMapping("/{id}")
    public Customer getCustomer(@PathVariable Integer id)
    {

        return repository.findById(id).orElse(null);
    }

    // Update Customer
    @PutMapping("/{id}")
    public Customer updateCustomer(@PathVariable Integer
id,
```

```
@Valid @RequestBody Customer customer) {  
  
    customer.setCustomerId(id);  
  
    return repository.save(customer);  
}  
  
// Delete Customer  
@DeleteMapping("/{id}")  
public String deleteCustomer(@PathVariable Integer id) {  
  
    repository.deleteById(id);  
  
    return "Customer Deleted Successfully";  
}  
}
```

Application.properties(all about url of databases and hibernate)

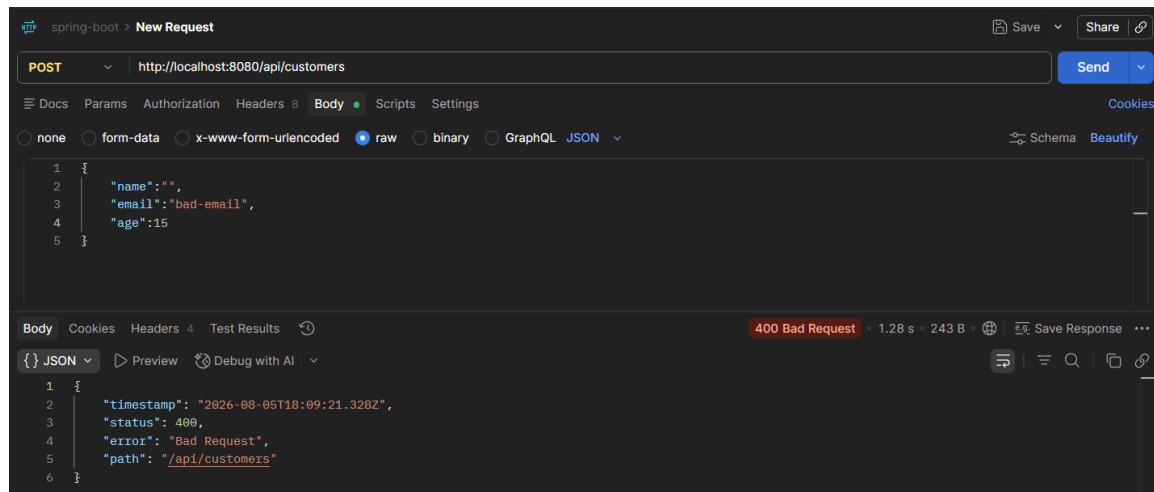
```
spring.application.name=thirteenth  
spring.datasource.url=jdbc:mysql://localhost:3306/thirteenth  
spring.datasource.username=root  
spring.datasource.password=  
  
spring.jpa.hibernate.ddl-auto=update  
  
spring.jpa.show-sql=true  
  
spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

13.1 POST API – Add Customer with Validation

The POST API is used to create a new customer in the database. Before saving the customer details, Spring Boot validates the input using the `@Valid` annotation along with validation constraints such as `@NotBlank`, `@Email`, and `@Min`. If all validations are satisfied, the customer record is stored successfully; otherwise, Spring Boot returns a validation error.

POST



Show's error cause it's not valid value's

Like

name : ""

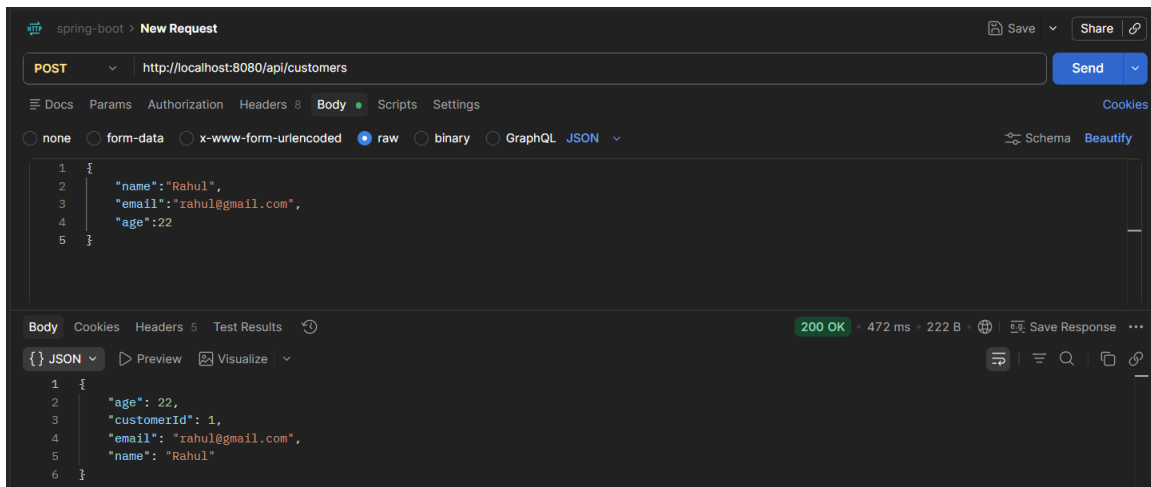
Email : "bad-email"

NAME: SOLANKI KARANSINH

ENROLLMENT: 25CI2110110

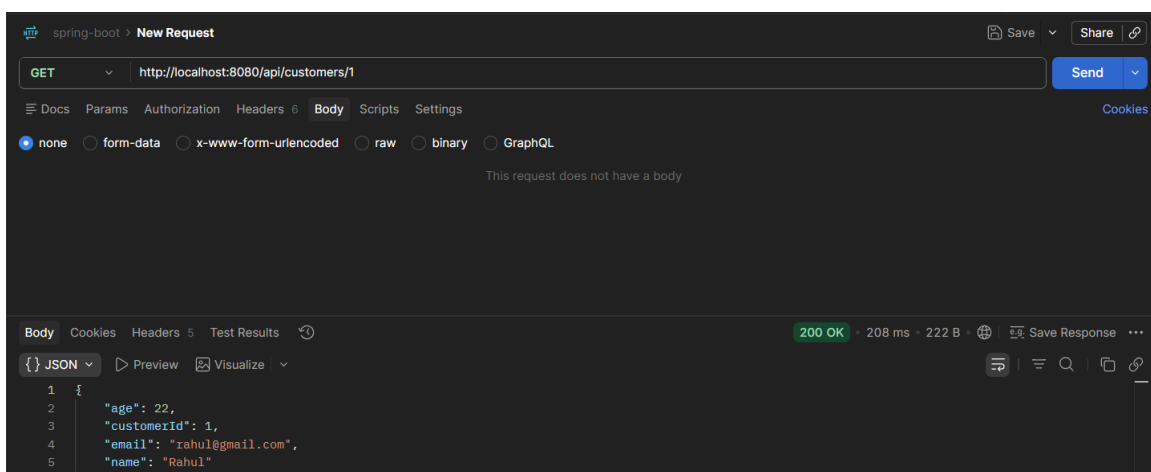
Age:15

Now we ll see correct data output



13.3 GET API – Display Customer by ID

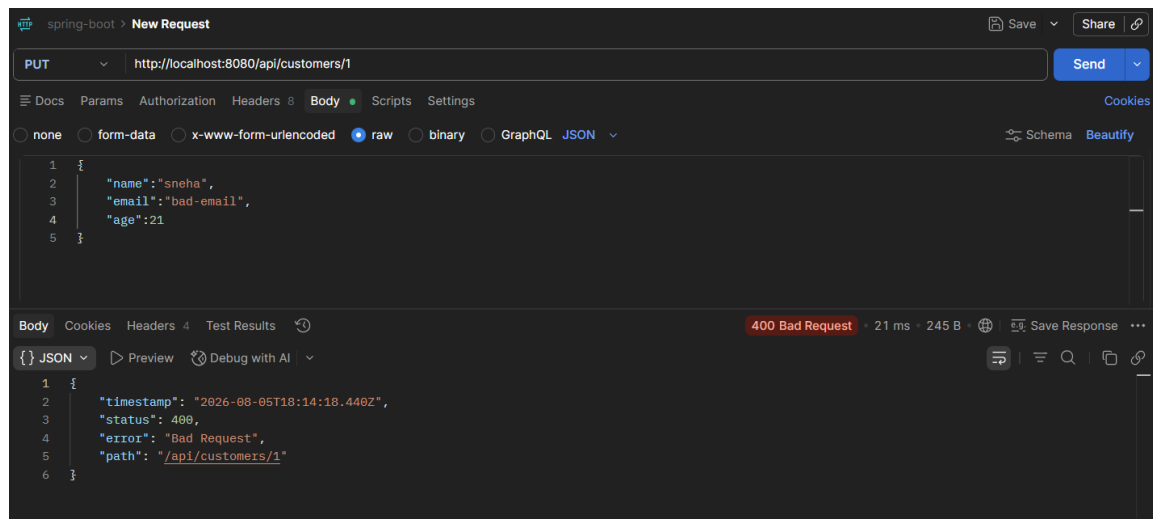
The GET API retrieves a single customer record using the customer ID. The ID is passed as a path variable, and Spring Boot returns the corresponding customer information if it exists.



SEE THE CUSTOMER DETAILS ARE DISPLAYED SUCCESSFULLY.

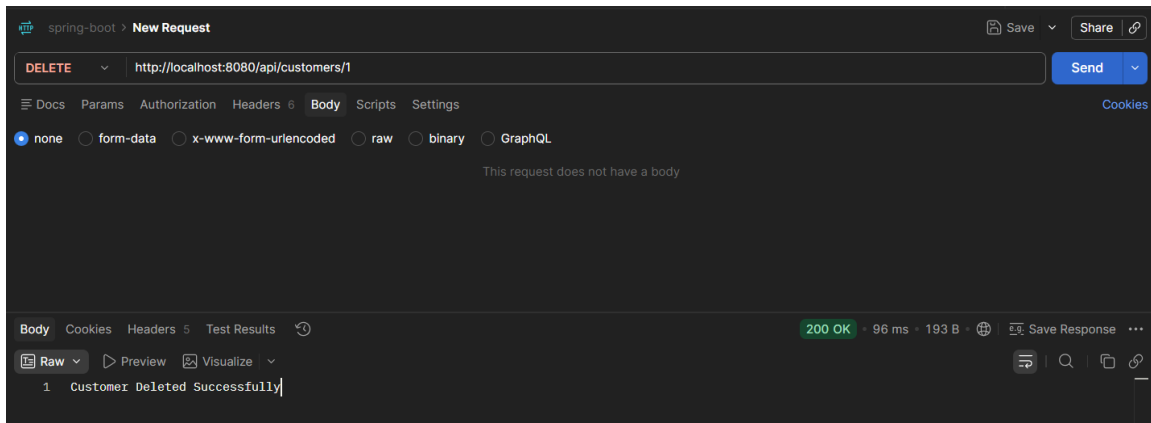
13.4 PUT API – Update Customer

The PUT API is used to update an existing customer's information. The customer ID is passed through the URL, while the updated customer details are provided in the request body. The input is validated before updating the record in the database.



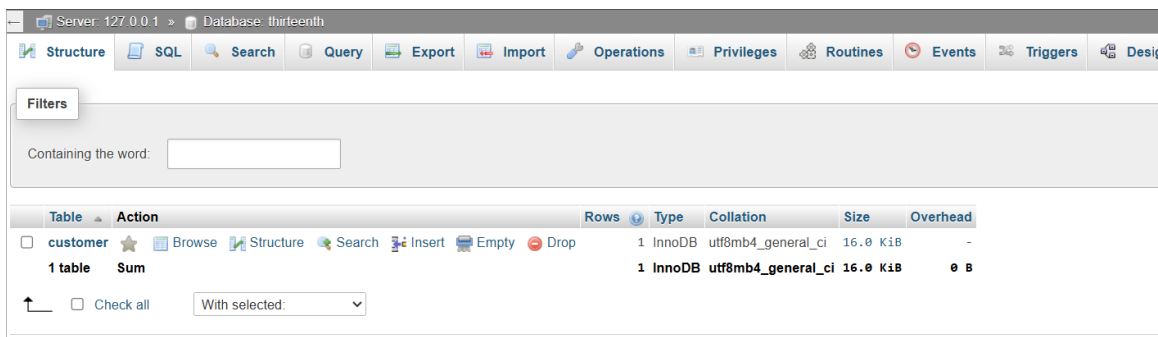
13.5 DELETE API – Delete Customer

The DELETE API is used to remove an existing customer record from the database. The customer ID is passed through the URL, and the corresponding record is deleted successfully.



Database Output

TABLE'S AND DATA STORING SUCCEFULLY



WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS

Browse	Structure	SQL	Search	Insert	Export	Import	Privileges	Operations	Triggers
--------	-----------	-----	--------	--------	--------	--------	------------	------------	----------

Showing rows 0 - 0 (1 total, Query took 0.0011 seconds.)

SELECT * FROM `customer`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	customer_id	age	email	name
☐ Edit Copy Delete	1	22	rahul@gmail.com	Rahul

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

Practical No. 14: Global Exception Handling using @ControllerAdvice and @ExceptionHandler

Objective:

To centralize exception handling across all REST controllers using `@ControllerAdvice`, returning a consistent, structured error response (timestamp, message, status, path) instead of default Spring stack-trace pages — the concluding topic of this chapter.

Class Name(s) with Attributes:

ResourceNotFoundException (custom RuntimeException)

ErrorResponse (POJO)

- timestamp : LocalDateTime
- message : String
- status : int
- path : String

GlobalExceptionHandler (@ControllerAdvice)

- Method: handleNotFound(ResourceNotFoundException)
- Method: handleValidation(MethodArgumentNotValidException)
- Method: handleGenericException(Exception)

**Dependencies**

- Spring Web
- Spring Data JPA
- MySQL Driver
- starter validation

Database

Database Name: thirteenth

Table: customer(created automatically by Hibernate)

Important Files

Employee.java(ENTITY model class)

```
package com.fourteenth.fourteenth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;
    private String department;
    private double salary;

    public Employee() {
    }

    public Integer getEmpId() {
        return empId;
    }

    public void setEmpId(Integer empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }
}
```

```
public String getDepartment() {  
    return department;  
}  
  
public void setDepartment(String department) {  
    this.department = department;  
}  
  
public double getSalary() {  
    return salary;  
}  
  
public void setSalary(double salary) {  
    this.salary = salary;  
}  
}
```

EmployeeRepository.java(Repository file)

```
package com.fourteenth.fourteenth.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.fourteenth.fourteenth.entity.Employee;  
  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Integer> {  
}
```

EmployeeController.java(Controller file)

```
package com.fourteenth.fourteenth.controller;  
  
import java.util.List;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.*;  
  
import com.fourteenth.fourteenth.entity.Employee;
```

```
import com.fourteenth.fourteenth.exception.ResourceNotFoundException;
import com.fourteenth.fourteenth.repository.EmployeeRepository;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository repository;

    // Add Employee
    @PostMapping
    public Employee addEmployee(@RequestBody Employee employee) {

        return repository.save(employee);
    }

    // Display All Employees
    @GetMapping
    public List<Employee> getAllEmployees() {

        return repository.findAll();
    }

    // Display Employee By Id
    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable Integer id) {

        return repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Employee
Not Found with ID : " + id));
    }

    // Update Employee
    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable Integer id,
        @RequestBody Employee employee) {

        Employee existing = repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Employee
Not Found with ID : " + id));

        existing.setEmpName(employee.getEmpName());
        existing.setDepartment(employee.getDepartment());
        existing.setSalary(employee.getSalary());

        return repository.save(existing);
    }

    // Delete Employee
```

```
@DeleteMapping("/{id}")
public String deleteEmployee(@PathVariable Integer id) {

    Employee employee = repository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Employee
Not Found with ID : " + id));

    repository.delete(employee);

    return "Employee Deleted Successfully";
}
}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=fourteenth
spring.datasource.url=jdbc:mysql://localhost:3306/fourteenth
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true
```

CUSTOM EXCEPTION FILE'S AND GLOBALEXCEPTION

ResponseNotFoundException.java(Custom Exception)

```
package com.fourteenth.fourteenth.exception;

public class ResourceNotFoundException
    extends RuntimeException {
```

```
public ResourceNotFoundException(String message) {  
    super(message);  
}  
}
```

ErrorResponse.java(Custom Exception)

```
package com.fourteenth.fourteenth.exception;  
  
import java.time.LocalDateTime;  
  
public class ErrorResponse {  
  
    private LocalDateTime timestamp;  
    private String message;  
    private int status;  
    private String path;  
  
    public ErrorResponse() {  
    }  
  
    public ErrorResponse(LocalDateTime timestamp,  
        String message,  
        int status,  
        String path) {  
  
        this.timestamp = timestamp;  
        this.message = message;  
        this.status = status;  
        this.path = path;  
    }  
  
    public LocalDateTime getTimestamp() {  
        return timestamp;  
    }  
  
    public void setTimestamp(LocalDateTime timestamp) {  
        this.timestamp = timestamp;  
    }  
  
    public String getMessage() {  
        return message;  
    }  
}
```

```
}

    public void setMessage(String message) {
        this.message = message;
    }

    public int getStatus() {
        return status;
    }

    public void setStatus(int status) {
        this.status = status;
    }

    public String getPath() {
        return path;
    }

    public void setPath(String path) {
        this.path = path;
    }
}
```

GlobalExceptionHandler.java(Exception Handler)

This will used to call the custom exception without getting 500 error

```
package com.fourteenth.fourteenth.exception;

import java.time.LocalDateTime;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

import jakarta.servlet.http.HttpServletRequest;

@ControllerAdvice
public class GlobalExceptionHandler {
```

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleResourceNotFound(
    ResourceNotFoundException ex,
    HttpServletRequest request) {

    ErrorResponse error = new ErrorResponse(
        LocalDateTime.now(),
        ex.getMessage(),
        HttpStatus.NOT_FOUND.value(),
        request.getRequestURI());

    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleException(
    Exception ex,
    HttpServletRequest request) {

    ErrorResponse error = new ErrorResponse(
        LocalDateTime.now(),
        ex.getMessage(),
        HttpStatus.INTERNAL_SERVER_ERROR.value(),
        request.getRequestURI());

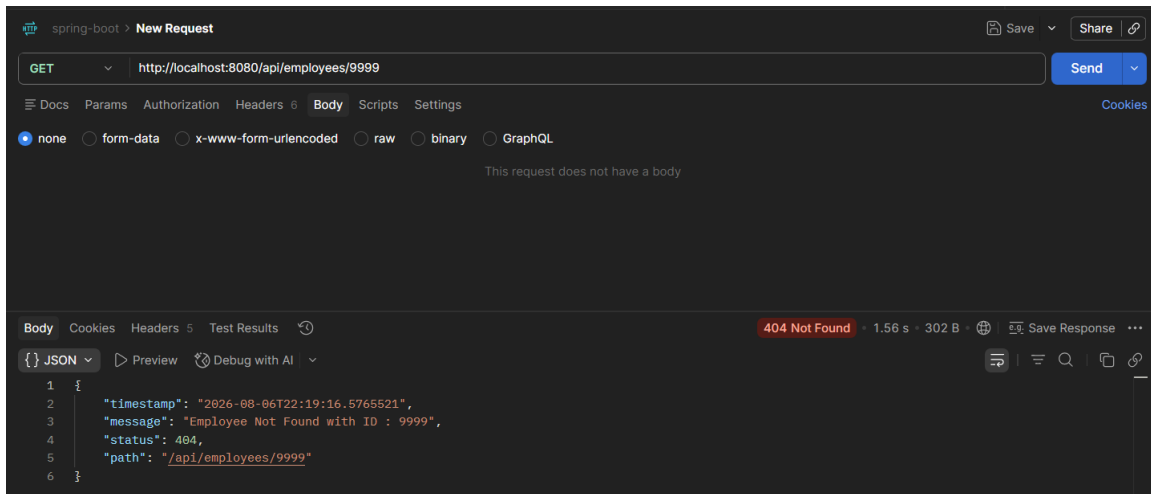
    return new ResponseEntity<>(error,
        HttpStatus.INTERNAL_SERVER_ERROR);
}
}
```

API'S TESTING USING POSTMAN

14.1 GET API – Resource Not Found Exception

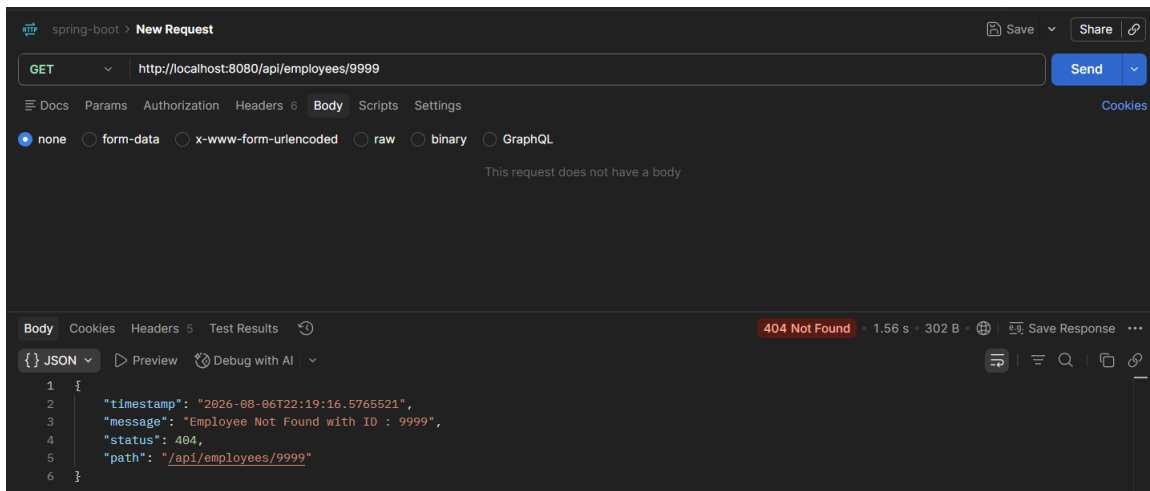
The GET API is used to retrieve an employee using the employee ID. When a non-existing employee ID is requested, the application throws a

ResourceNotFoundException. The GlobalExceptionHandler catches the exception and returns a custom JSON response containing the timestamp, message, status code, and request path instead of Spring Boot's default error page.



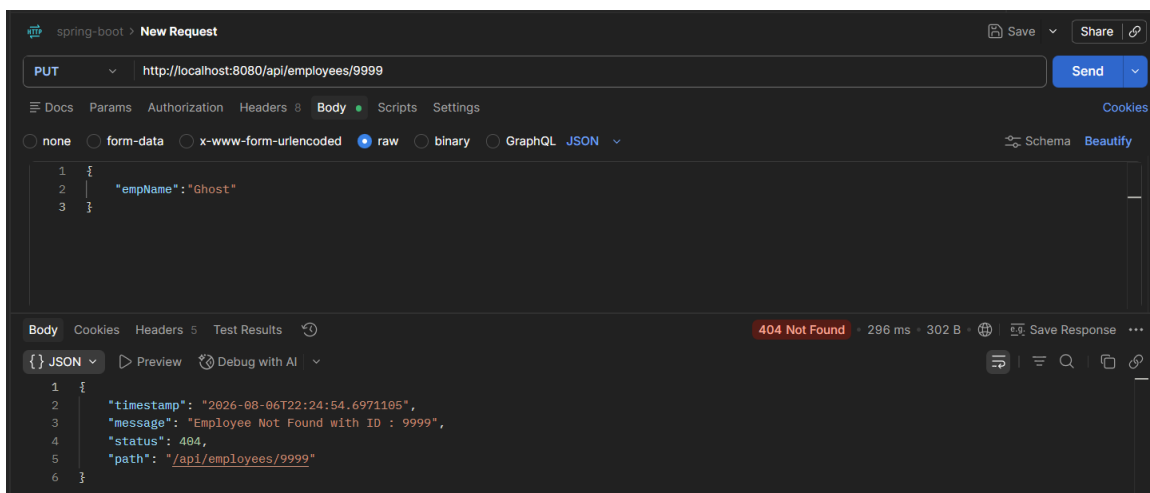
14.2 POST API – Validation Exception

The POST API is used to demonstrate Bean Validation together with Global Exception Handling. When invalid customer details such as an empty name, invalid email, or age below 18 are submitted, Spring Boot throws a validation exception. The GlobalExceptionHandler captures the exception and returns a structured JSON response containing field-level validation errors.



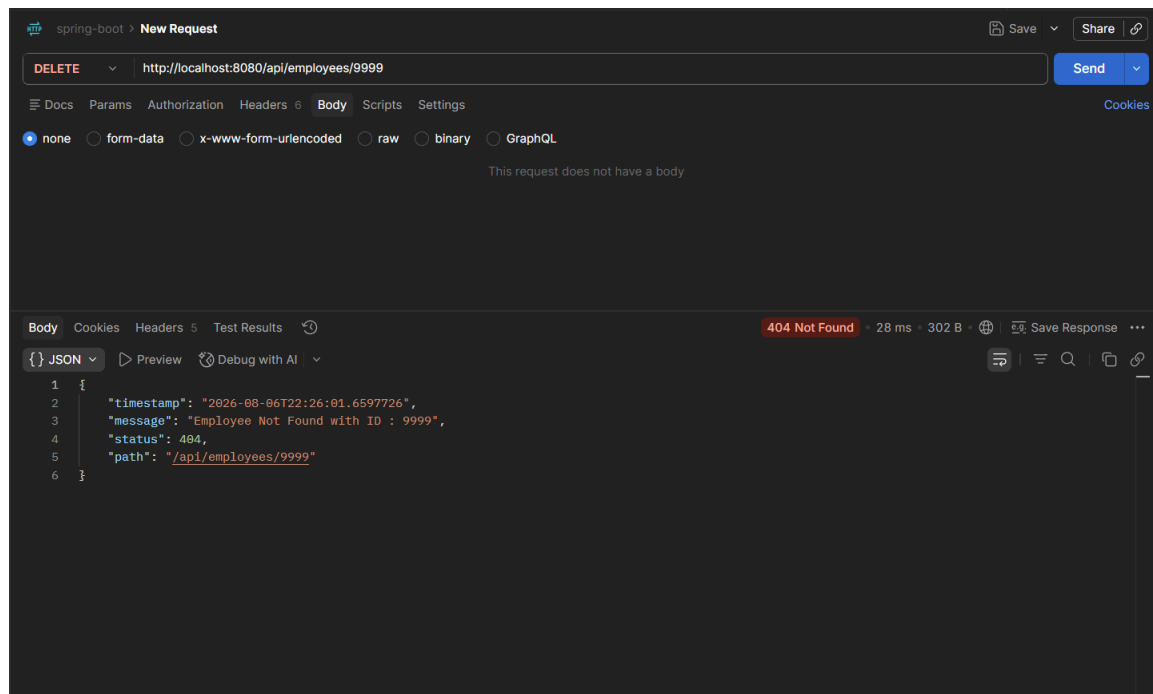
14.3 PUT API – Update Non-Existing Employee

The PUT API is used to update an employee record. If the specified employee ID does not exist, the application throws a ResourceNotFoundException. The Global Exception Handler catches the exception and returns a custom 404 Not Found response instead of displaying a stack trace



14.4 DELETE API – Delete Non-Existing Employee

The DELETE API is used to remove an employee record from the database. If the employee ID does not exist, a `ResourceNotFoundException` is thrown. The Global Exception Handler processes the exception and returns a structured JSON error response with the appropriate HTTP status code.



SEE THE CUSTOM ERROR RESPONSE IS RETURNED SUCCESSFULLY BY THE GLOBAL EXCEPTION HANDLER.

-----END-OF-PRACTICAL(14)-----

-----END-OF-CHAPTER(6)-----

Ch 7: Securing a Web Application

Practical No. 15: Securing REST API with HTTP Basic Authentication (Spring Security)

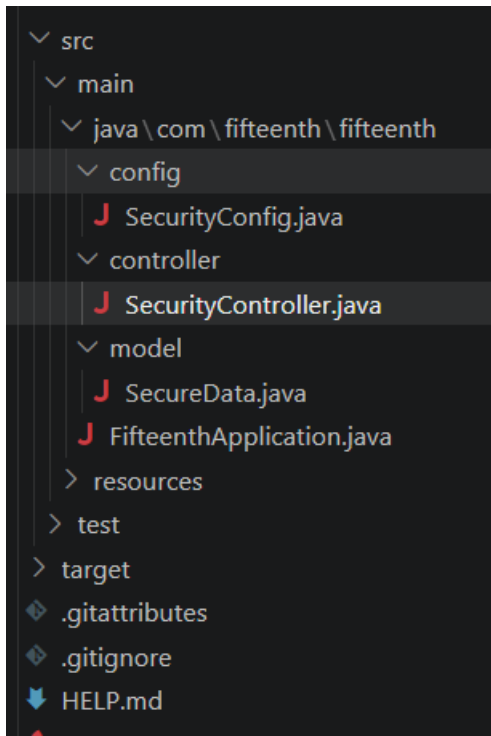
Objective:

To add Spring Security to a Spring Boot REST application, configure in-memory users with encoded passwords, and secure selected endpoints so that only authenticated requests can access them — the introductory security mechanism covered in this chapter.

Class Name(s) with Attributes:

SecurityConfig (@Configuration)

- Bean: SecurityFilterChain filterChain()
- Bean: PasswordEncoder passwordEncoder() — BCryptPasswordEncoder
- Bean: InMemoryUserDetailsManager userDetailsService()



Dependencies

-starter security

Important Files

SecureData.java(model class)

```
package com.fifteenth.fifteenth.model;

public class SecureData {

    private Integer id;
    private String text;

    public SecureData() {
    }

    public SecureData(Integer id, String text) {
        this.id = id;
        this.text = text;
    }
}
```

```
public Integer getId() {  
    return id;  
}  
  
public void setId(Integer id) {  
    this.id = id;  
}  
  
public String getText() {  
    return text;  
}  
  
public void setText(String text) {  
    this.text = text;  
}  
}
```

SecurityConfig.java(security file)

```
package com.fifteenth.fifteenth.config;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.security.config.Customizer;  
import  
org.springframework.security.config.annotation.web.builders.HttpSecurity;  
import org.springframework.security.core.userdetails.User;  
import org.springframework.security.core.userdetails.UserDetails;  
import  
org.springframework.security.provisioning.InMemoryUserDetailsManager;  
import org.springframework.security.core.userdetails.UserDetailsService;  
import  
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.security.web.SecurityFilterChain;  
  
@Configuration  
public class SecurityConfig {
```

```
@Bean
public PasswordEncoder passwordEncoder() {

    return new BCryptPasswordEncoder();
}

@Bean
public UserDetailsService userDetailsService() {

    System.out.println(passwordEncoder().encode("password"));

    UserDetails user = User.builder()

        .username("user")

        .password(passwordEncoder().encode("password"))

        .roles("USER")

        .build();

    return new InMemoryUserDetailsManager(user);
}

@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {

    http

        .csrf(csrf -> csrf.disable())

        .authorizeHttpRequests(auth -> auth

            .requestMatchers("/api/public/**").permitAll()

            .requestMatchers("/api/secure/**").authenticated()

            .anyRequest().authenticated())

        .httpBasic(Customizer.withDefaults());

    return http.build();
}
}
```

SecuritController.java(Controller file)

```
package com.fifteenth.fifteenth package com.fifteenth.fifteenth.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.web.bind.annotation.*;

import com.fifteenth.fifteenth.model.SecureData;

@RestController
@RequestMapping("/api")
public class SecurityController {

    private List<SecureData> dataList = new ArrayList<>();

    // Public API

    @GetMapping("/public/data")

    public String publicData() {

        return "This is Public Data";

    }

    // Secure GET

    @GetMapping("/secure/data")

    public List<SecureData> getData() {

        return dataList;

    }

    // Secure POST

    @PostMapping("/secure/data")

    public SecureData addData(

        @RequestBody SecureData data) {

        dataList.add(data);

    }

}
```

```
        return data;
    }

    // Secure PUT
    @PutMapping("/secure/data/{id}")
    public SecureData updateData(
        @PathVariable Integer id,
        @RequestBody SecureData data) {
        for (SecureData d : dataList) {
            if (d.getId().equals(id)) {
                d.setText(data.getText());
                return d;
            }
        }
        return null;
    }

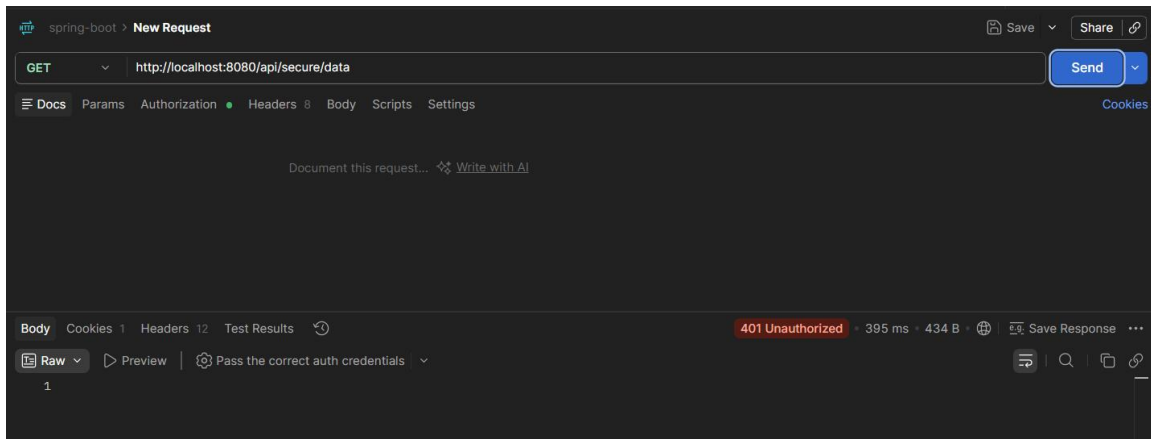
    // Secure DELETE
    @DeleteMapping("/secure/data/{id}")
    public String deleteData(
        @PathVariable Integer id) {
        dataList.removeIf(d -> d.getId().equals(id));
        return "Deleted Successfully";
    }
}
```

API'S TESTING USING POSTMAN

15.1 GET API – Access Secure Data Without Authentication

The GET API is used to access the secured REST endpoint without providing any authentication credentials. Since the endpoint is protected by Spring Security, the request is rejected and the server returns **401 Unauthorized**.

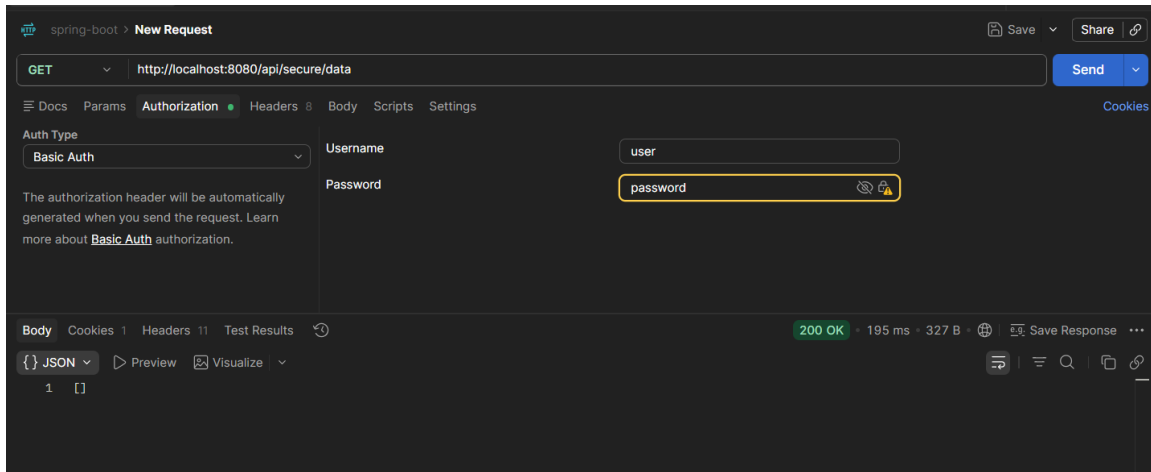
<http://localhost:8080/api/secure/data>



Shows' unauthorized

15.2 GET API – Access Secure Data With Basic Authentication

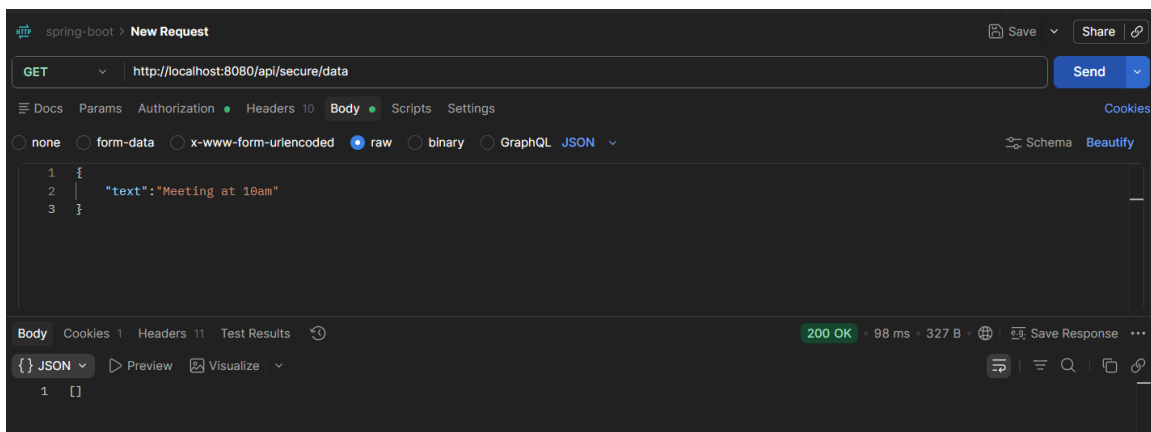
The GET API is used to access the secured REST endpoint using valid Basic Authentication credentials. Spring Security authenticates the user successfully and returns the protected data.



SEE THE SECURED DATA IS DISPLAYED SUCCESSFULLY AFTER SUCCESSFUL AUTHENTICATION.

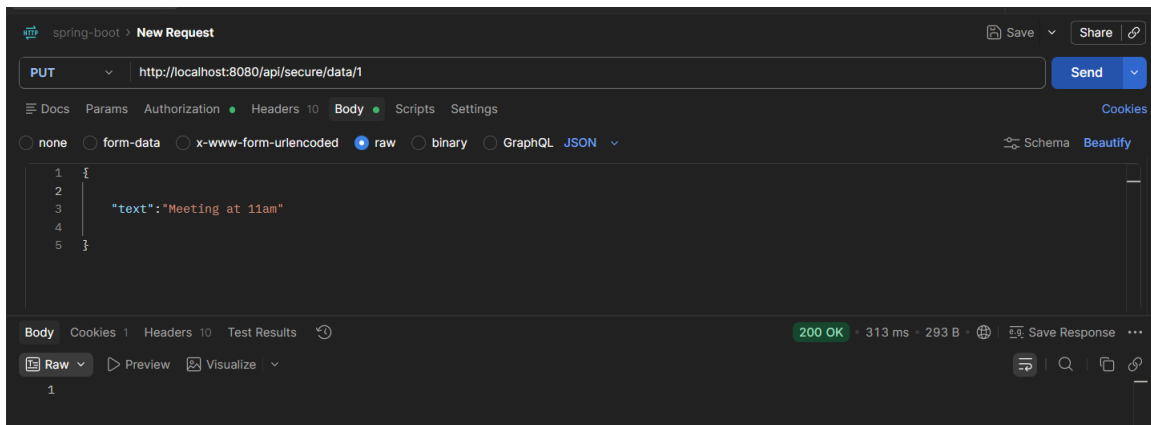
15.3 POST API – Add Secure Data

The POST API is used to create a new secure data record. The data is sent in JSON format through the request body. Only authenticated users can access this endpoint.



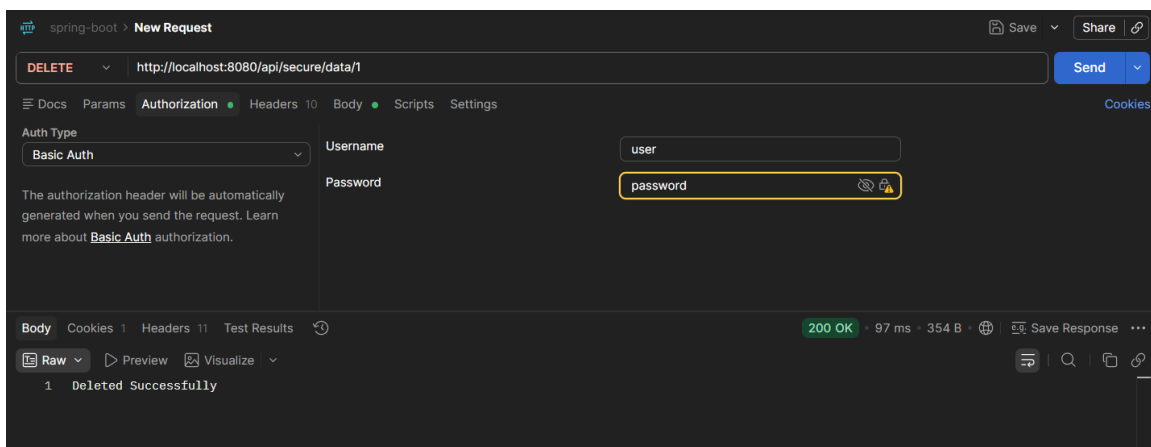
15.4 PUT API – Update Secure Data

The PUT API is used to update the existing secure data. The data ID is passed through the URL, while the updated information is sent in the request body. Authentication is required before updating the resource..



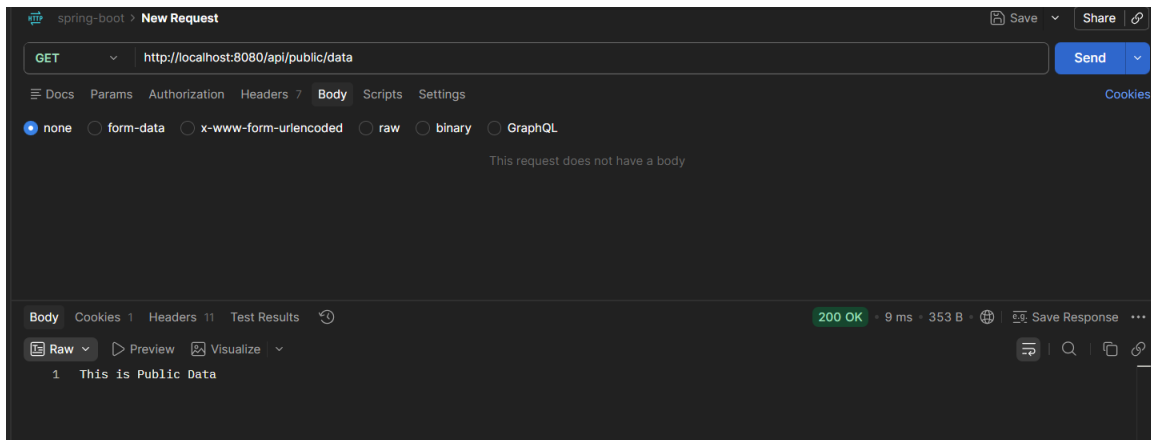
15.5 DELETE API – Delete Secure Data

The DELETE API is used to remove the secure data having ID **1**. Only authenticated users are allowed to delete the resource.



15.6 GET API – Access Public Data

The GET API is used to access the public REST endpoint. This endpoint is configured with `permitAll()` in Spring Security, so no authentication is required.



SEE THE PUBLIC DATA IS DISPLAYED SUCCESSFULLY WITHOUT LOGIN OR AUTHENTICATION.

Practical No. 16: User Registration and Login with BCrypt Password Encoding

Objective:

To implement a database-backed user registration and login mechanism, storing passwords securely using BCryptPasswordEncoder instead of plain text, and authenticating against the database via a custom UserDetailsService — extending the password-encoding concept from this chapter to a persistent user store

Class Name(s) with Attributes:

AppUser (@Entity)

- userId : int (@Id @GeneratedValue)
- username : String (unique)

- password : String (BCrypt-encoded)
- role : String

AppUserRepository (extends JpaRepository)

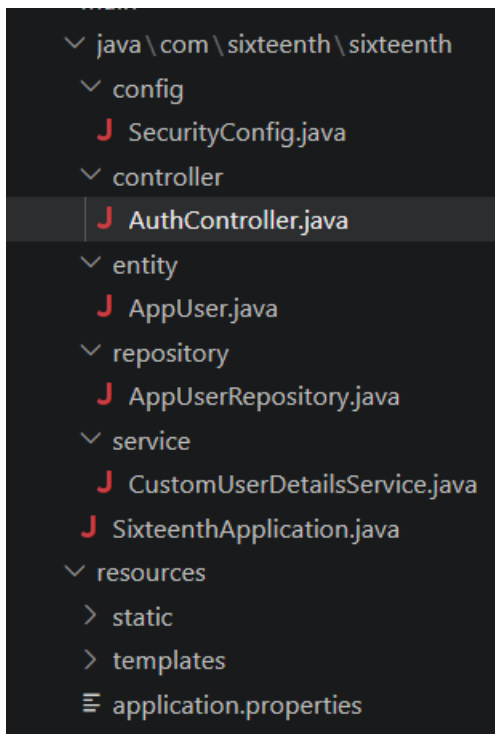
- Optional findByUsername(String username)

CustomUserDetailsService (implements UserDetailsService)

- appUserRepository : AppUserRepository

AuthController (@RestController)

- Method: register(@RequestBody AppUser) — encodes password before saving

**AppUser.java(entity)**

```
package com.sixteenth.sixteenth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer userId;

    private String username;

    private String password;

    private String role;

    public AppUser() {
    }

    public Integer getUserId() {
        return userId;
    }

    public void setUserId(Integer userId) {
        this.userId = userId;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }
}
```

```
public void setRole(String role) {  
    this.role = role;  
}  
}
```

SecurityConfig.java(security file)

```
package com.sixteenth.sixteenth.controller;  
  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.security.core.Authentication;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.web.bind.annotation.*;  
  
import com.sixteenth.sixteenth.entity.AppUser;  
import com.sixteenth.sixteenth.repository.AppUserRepository;  
  
@RestController  
@RequestMapping("/api/auth")  
public class AuthController {  
  
    private final AppUserRepository appUserRepository;  
    private final PasswordEncoder passwordEncoder;  
  
    public AuthController(AppUserRepository appUserRepository,  
        PasswordEncoder passwordEncoder) {  
  
        this.appUserRepository = appUserRepository;  
        this.passwordEncoder = passwordEncoder;  
    }  
  
    // Register new user  
    @PostMapping("/register")  
    public ResponseEntity<AppUser> register(  
        @RequestBody AppUser appUser) {  
  
        appUser.setPassword(  
            passwordEncoder.encode(appUser.getPassword()));  
  
        AppUser savedUser = appUserRepository.save(appUser);
```

```
        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(savedUser);
    }

    // Update authenticated user's profile
    @PutMapping("/profile")
    public ResponseEntity<AppUser> updateProfile(
        @RequestBody AppUser updatedUser,
        Authentication authentication) {

        String currentUsername = authentication.getName();

        AppUser existingUser = appUserRepository
            .findByUsername(currentUsername)
            .orElseThrow(() -> new RuntimeException("User not found"));

        if (updatedUser.getUsername() != null &&
            !updatedUser.getUsername().isBlank()) {

            existingUser.setUsername(updatedUser.getUsername());
        }

        if (updatedUser.getRole() != null &&
            !updatedUser.getRole().isBlank()) {

            existingUser.setRole(updatedUser.getRole());
        }

        AppUser savedUser = appUserRepository.save(existingUser);

        return ResponseEntity.ok(savedUser);
    }

    // Delete authenticated user's account
    @DeleteMapping("/register/{id}")
    public ResponseEntity<Void> deleteUser(
        @PathVariable Integer id,
        Authentication authentication) {

        AppUser user = appUserRepository
            .findById(id)
            .orElseThrow(() -> new RuntimeException("User not found"));

        if (!user.getUsername()
            .equals(authentication.getName())) {

            return ResponseEntity.status(HttpStatus.FORBIDDEN)
                .build();
        }
    }
```

```
        appUserRepository.deleteById(id);

        return ResponseEntity.noContent().build();
    }
}
```

AuthController.java(Controller file)

```
package com.sixteenth.sixteenth.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

import com.sixteenth.sixteenth.entity.AppUser;
import com.sixteenth.sixteenth.repository.AppUserRepository;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AppUserRepository appUserRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthController(AppUserRepository appUserRepository,
        PasswordEncoder passwordEncoder) {

        this.appUserRepository = appUserRepository;
        this.passwordEncoder = passwordEncoder;
    }

    // Register new user
    @PostMapping("/register")
    public ResponseEntity<AppUser> register(
        @RequestBody AppUser appUser) {

        appUser.setPassword(
            passwordEncoder.encode(appUser.getPassword()));

        AppUser savedUser = appUserRepository.save(appUser);

        return ResponseEntity
```

```
        .status(HttpStatus.CREATED)
        .body(savedUser);
    }

    // Update authenticated user's profile
    @PutMapping("/profile")
    public ResponseEntity<AppUser> updateProfile(
        @RequestBody AppUser updatedUser,
        Authentication authentication) {

        String currentUsername = authentication.getName();

        AppUser existingUser = appUserRepository
            .findByUsername(currentUsername)
            .orElseThrow(() -> new RuntimeException("User not found"));

        if (updatedUser.getUsername() != null &&
            !updatedUser.getUsername().isBlank()) {

            existingUser.setUsername(updatedUser.getUsername());
        }

        if (updatedUser.getRole() != null &&
            !updatedUser.getRole().isBlank()) {

            existingUser.setRole(updatedUser.getRole());
        }

        AppUser savedUser = appUserRepository.save(existingUser);

        return ResponseEntity.ok(savedUser);
    }

    // Delete authenticated user's account
    @DeleteMapping("/register/{id}")
    public ResponseEntity<Void> deleteUser(
        @PathVariable Integer id,
        Authentication authentication) {

        AppUser user = appUserRepository
            .findById(id)
            .orElseThrow(() -> new RuntimeException("User not found"));

        if (!user.getUsername()
            .equals(authentication.getName())) {

            return ResponseEntity.status(HttpStatus.FORBIDDEN)
                .build();
        }
    }
```

```
appUserRepository.deleteById(id);  
  
return ResponseEntity.noContent().build();  
}  
}
```

AppRepository.java

```
package com.sixteenth.sixteenth.repository;  
  
import java.util.Optional;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.sixteenth.sixteenth.entity.AppUser;  
  
public interface AppUserRepository extends JpaRepository<AppUser,  
Integer> {  
  
    Optional<AppUser> findByUsername(String username);  
  
}
```

CustomUserService.java

```
package com.sixteenth.sixteenth.service;  
  
import org.springframework.security.core.userdetails.User;  
import org.springframework.security.core.userdetails.UserDetails;  
import org.springframework.security.core.userdetails.UserDetailsService;  
import  
org.springframework.security.core.userdetails.UsernameNotFoundException;  
import org.springframework.stereotype.Service;  
  
import com.sixteenth.sixteenth.entity.AppUser;  
import com.sixteenth.sixteenth.repository.AppUserRepository;  
  
@Service  
public class CustomUserDetailsService implements UserDetailsService {
```

```
private final AppUserRepository appUserRepository;

public CustomUserDetailsService(AppUserRepository
appUserRepository) {
    this.appUserRepository = appUserRepository;
}

@Override
public UserDetails loadUserByUsername(String username)
throws UsernameNotFoundException {

    AppUser appUser = appUserRepository
        .findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException(
            "User not found: " + username));

    return User.builder()
        .username(appUser.getUsername())
        .password(appUser.getPassword())
        .roles(appUser.getRole())
        .build();
}
}
```

applicationproperties.java

```
spring.application.name=sixteenth

spring.datasource.url=jdbc:mysql://localhost:3306/sixteenth

spring.datasource.username=root
spring.datasource.password=

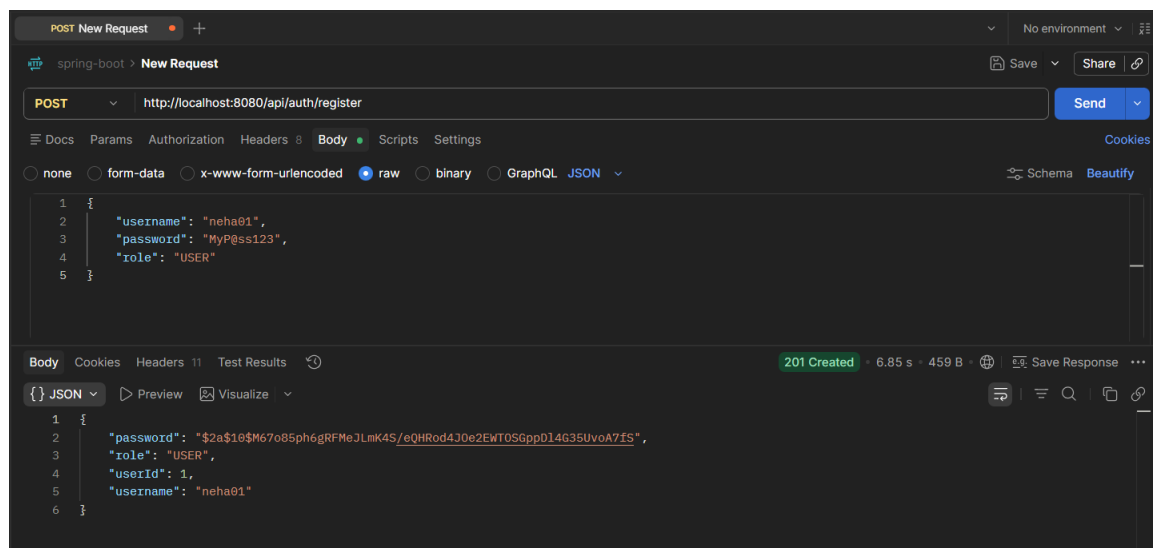
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

16.1 POST API – User Registration

The POST API is used to register a new user. The username, plain-text password, and role are sent in JSON format. Before saving, BCryptPasswordEncoder converts the password into a BCrypt hash and stores the hashed password in the database.

<http://localhost:8080/api/auth/register>

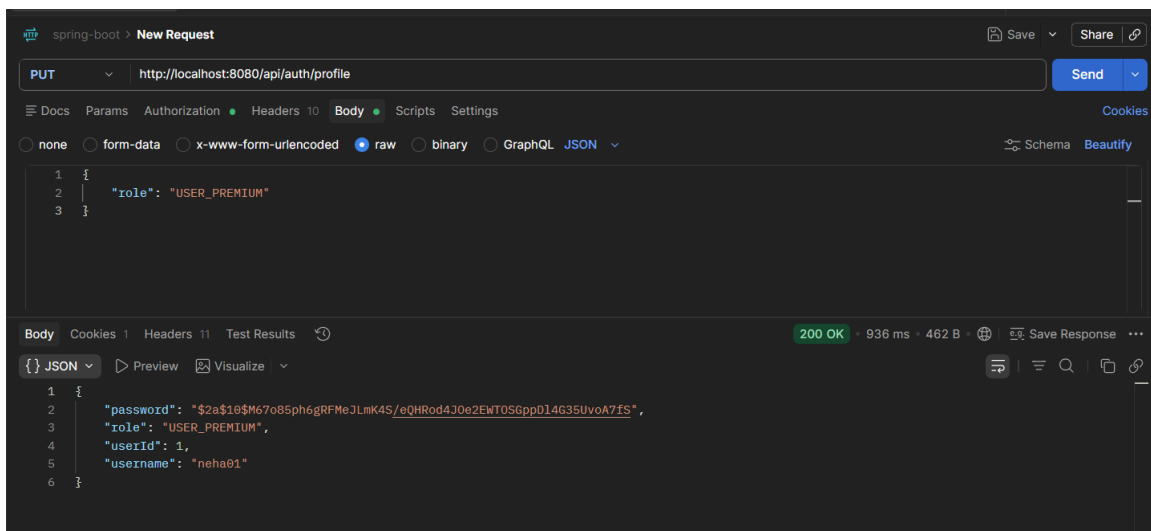


16.2 GET API – Access Secure Data

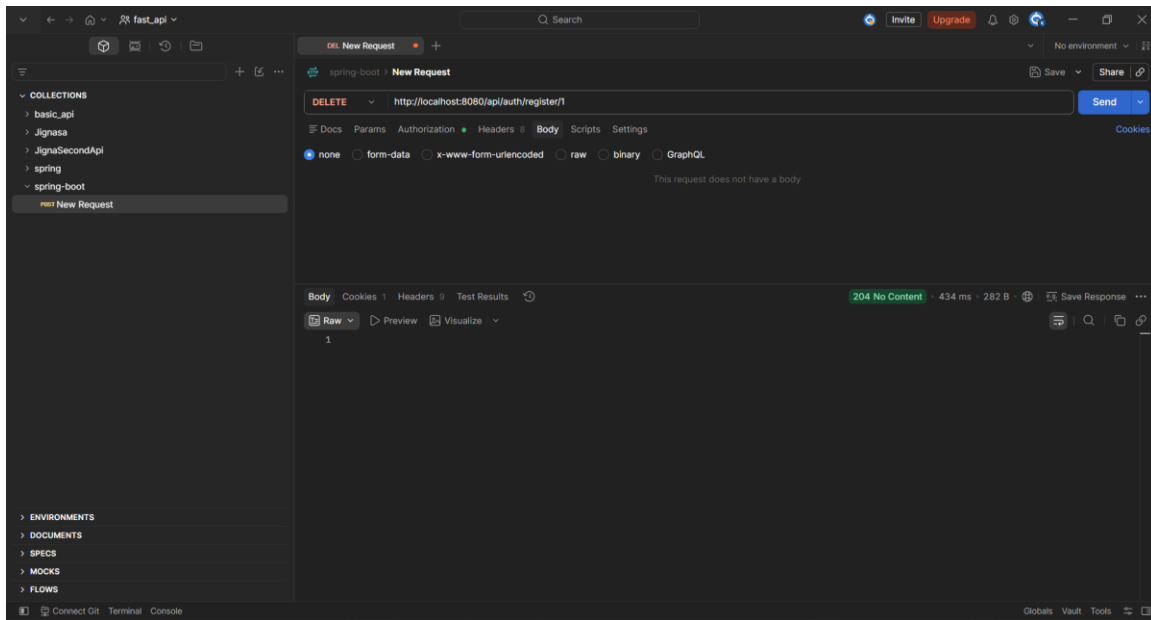
The GET API is used to access the secured data endpoint using the username and password of the newly registered user. Spring Security uses CustomUserDetailsService to find the username from the database and BCrypt to verify the password.

16.3 PUT API – Update User Profile

The PUT API is used to update the profile of the currently authenticated user. Basic Authentication is required, and the authenticated username is obtained from Spring Security.



16.4 to delete any user delete api key end point



Practical No. 17: Role-Based Access Control (RBAC) with Spring Security

Objective:

To extend Spring Security configuration for role-based authorization, defining users with different roles (ADMIN, USER) and restricting specific endpoints using `@PreAuthorize` or role-based matchers, as covered in this chapter's authorization section

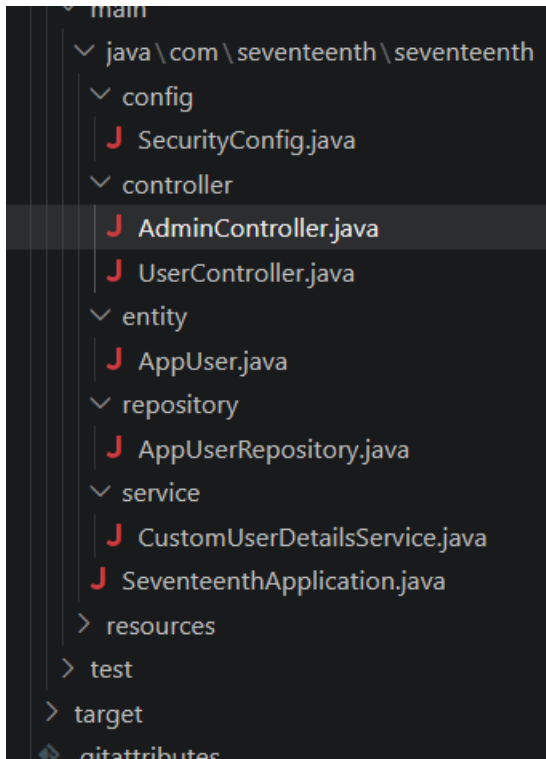
Class Name(s) with Attributes:

SecurityConfig (extended)

- Users carrying `ROLE_ADMIN` / `ROLE_USER`
- `@EnableMethodSecurity` enabled

AdminController (`@RestController`)

- Method: `getAdminData()` — `@PreAuthorize("hasRole('ADMIN')")`



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver
- Security

Database

Database Name: SevenTeenth

Table: User(created automatically by Hibernate)

Important Files

AppUser.java(entity)

```
package com.seventeenth.seventeenth.entity;

import jakarta.persistence.*;

@Entity
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer userId;

    @Column(unique = true, nullable = false)
    private String username;

    private String password;

    private String role;

    public AppUser() {
    }

    public Integer getUserId() {
        return userId;
    }

    public void setUserId(Integer userId) {
        this.userId = userId;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
```

```
        this.role = role;
    }
}
```

SecurityConfig.java(security file)

```
package com.seventeenth.seventeenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {

        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {

        http
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/auth/register")
                .permitAll()

                .requestMatchers("/api/public/**")
```

```
        .permitAll()

        .anyRequest()
        .authenticated()

        .httpBasic(Customizer.withDefaults());

    return http.build();
}
}
```

AdminController.java(Controller file)

```
package com.seventeenth.seventeenth.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    // GET - ADMIN ONLY
    @GetMapping("/data")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> getAdminData() {

        return ResponseEntity.ok(
            "Admin data accessed successfully.");
    }

    // POST - ADMIN ONLY
    @PostMapping("/data")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> createAdminData(
        @RequestBody String data) {

        return ResponseEntity.status(201)
            .body("Admin data created: " + data);
    }

    // PUT - ADMIN ONLY
    @PutMapping("/data/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> updateAdminData(
```

```
@PathVariable Integer id,  
@RequestBody String data) {  
  
    return ResponseEntity.ok(  
        "Admin data " + id +  
        " updated: " + data);  
  
    }  
  
    // DELETE - ADMIN ONLY  
@DeleteMapping("/data/{id}")  
@PreAuthorize("hasRole('ADMIN')")  
public ResponseEntity<Void> deleteAdminData(  
    @PathVariable Integer id) {  
  
    return ResponseEntity.noContent().build();  
  
    }  
}
```

AppUserRepository.java

```
package com.seventeenth.seventeenth.repository;  
  
import java.util.Optional;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.seventeenth.seventeenth.entity.AppUser;  
  
public interface AppUserRepository  
    extends JpaRepository<AppUser, Integer> {  
  
    Optional<AppUser> findByUsername(String username);  
  
    }
```

CustomUserDetailsService.java

```
package com.seventeenth.seventeenth.service;  
  
import org.springframework.security.core.userdetails.User;  
import org.springframework.security.core.userdetails.UserDetails;  
import org.springframework.security.core.userdetails.UserDetailsService;
```

```
import
org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import com.seventeenth.seventeenth.entity.AppUser;
import com.seventeenth.seventeenth.repository.AppUserRepository;

@Service
public class CustomUserDetailsService
    implements UserDetailsService {

    private final AppUserRepository appUserRepository;

    public CustomUserDetailsService(
        AppUserRepository appUserRepository) {

        this.appUserRepository = appUserRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        AppUser appUser = appUserRepository
            .findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found: " + username));

        return User.builder()
            .username(appUser.getUsername())
            .password(appUser.getPassword())
            .roles(appUser.getRole())
            .build();
    }
}
```

UserController.java

```
s package com.seventeenth.seventeenth.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
@RequestMapping("/api/user")
public class UserController {

    @GetMapping("/data")
    @PreAuthorize("hasAnyRole('ADMIN', 'USER')")
    public ResponseEntity<String> getUserData() {

        return ResponseEntity.ok(
            "User data accessed successfully.");
    }
}
```

applicationproperties.java

```
spring.application.name=seventeenth

spring.datasource.url=jdbc:mysql://localhost:3306/seventeenth

spring.datasource.username=root
spring.datasource.password=

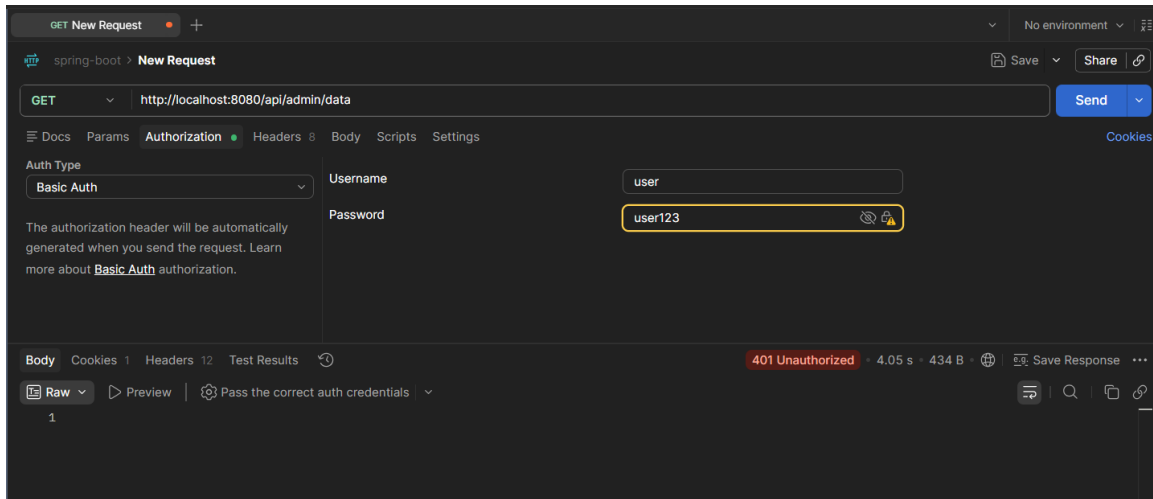
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

17.1 GET API – Admin Data with USER Credentials

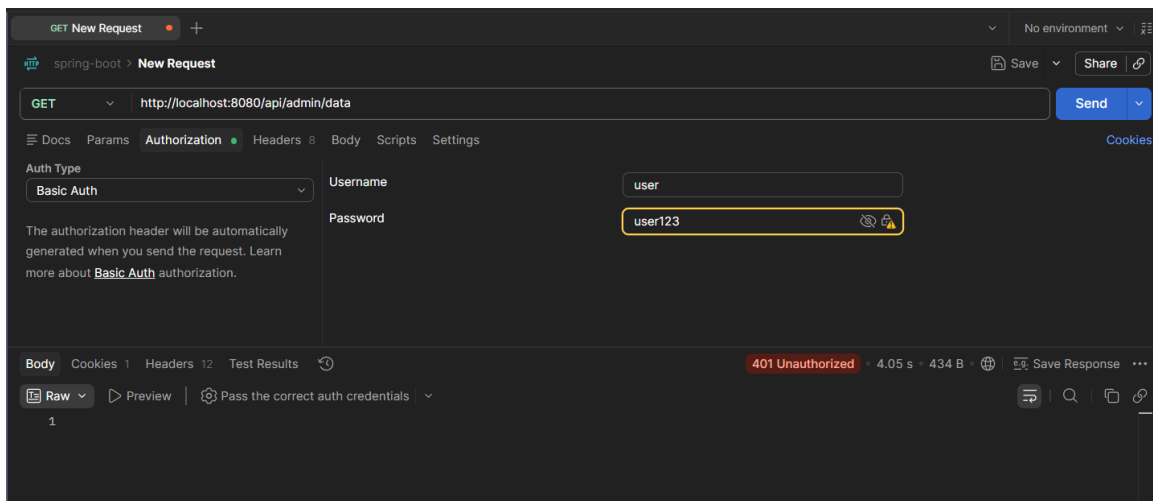
The GET API is used to access the admin-specific resource. This endpoint is protected using `@PreAuthorize("hasRole('ADMIN')")`, so only users having the ADMIN role can access it.

<http://localhost:8080/api/admin/data>



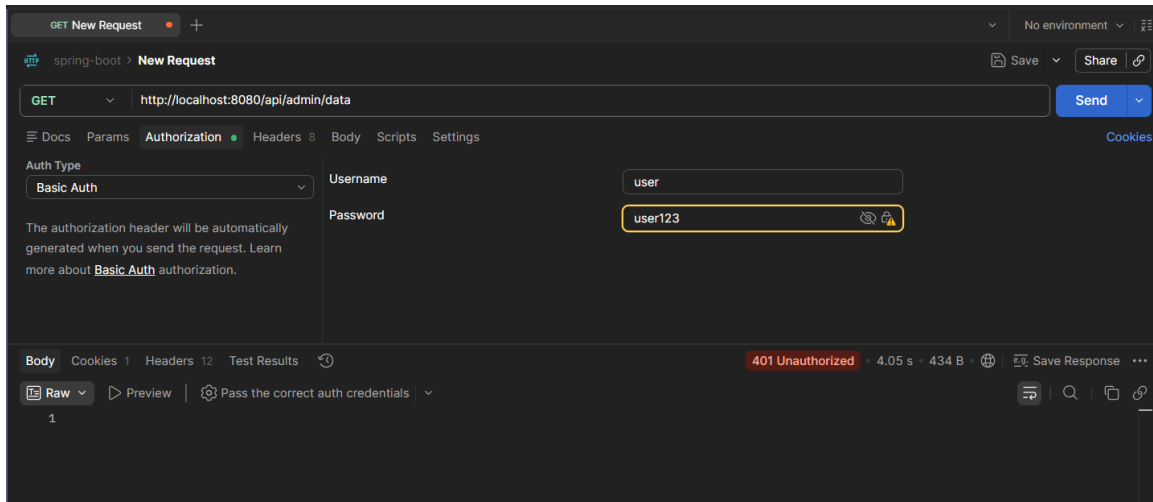
17.2 GET API – Admin Data with ADMIN Credentials

The same GET API is tested again using an ADMIN user's credentials. Since the authenticated user has the ADMIN role, Spring Security allows access to the endpoint.



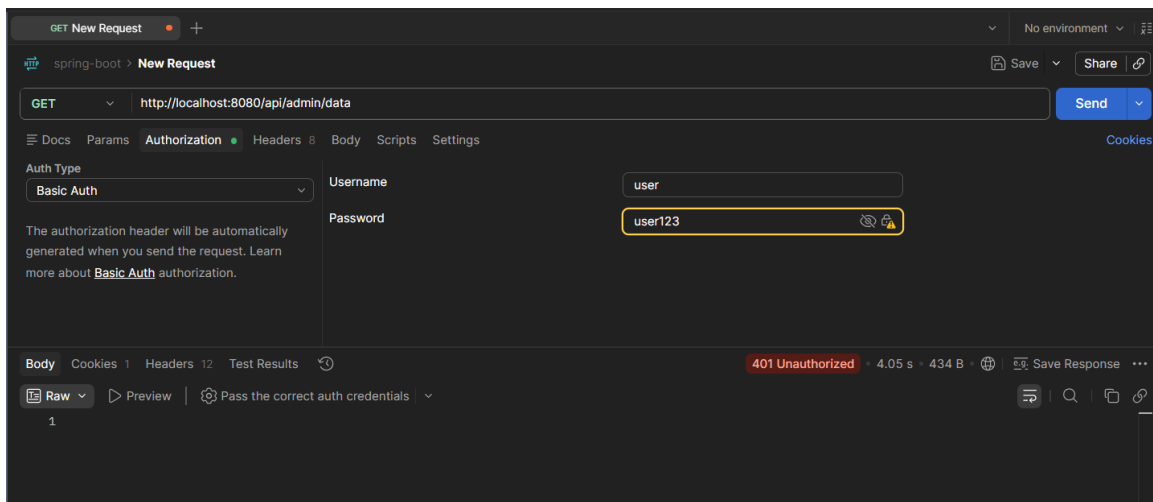
17.3 POST API – Create Admin Data

The POST API is used to create a new admin-only resource. The endpoint is protected with the ADMIN role, therefore only an ADMIN user can perform the operation.



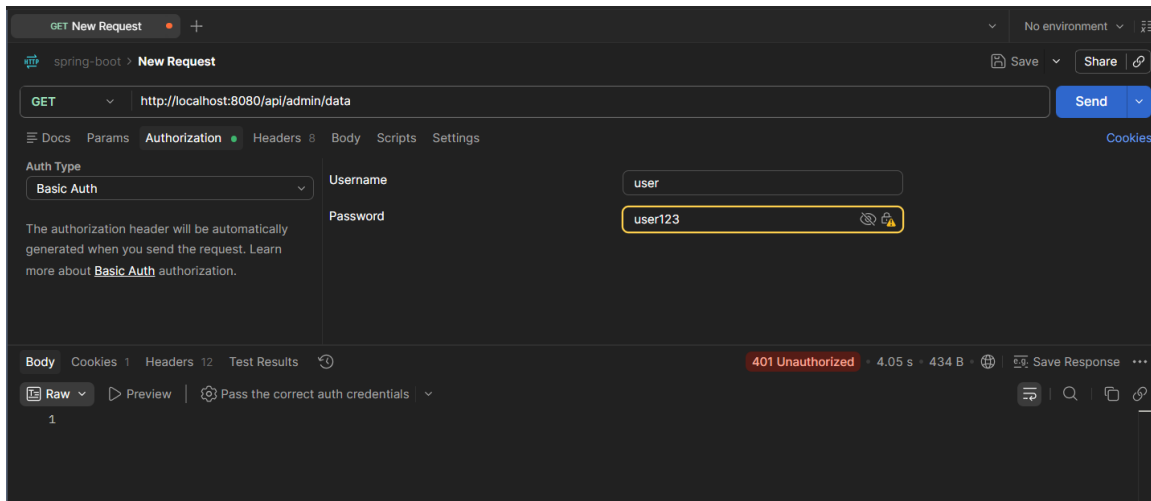
17.4 PUT API – Update Admin Data

The PUT API is used to update an existing admin resource. The resource ID is provided in the URL, while the updated data is sent in the request body. Only an ADMIN user can perform this operation.



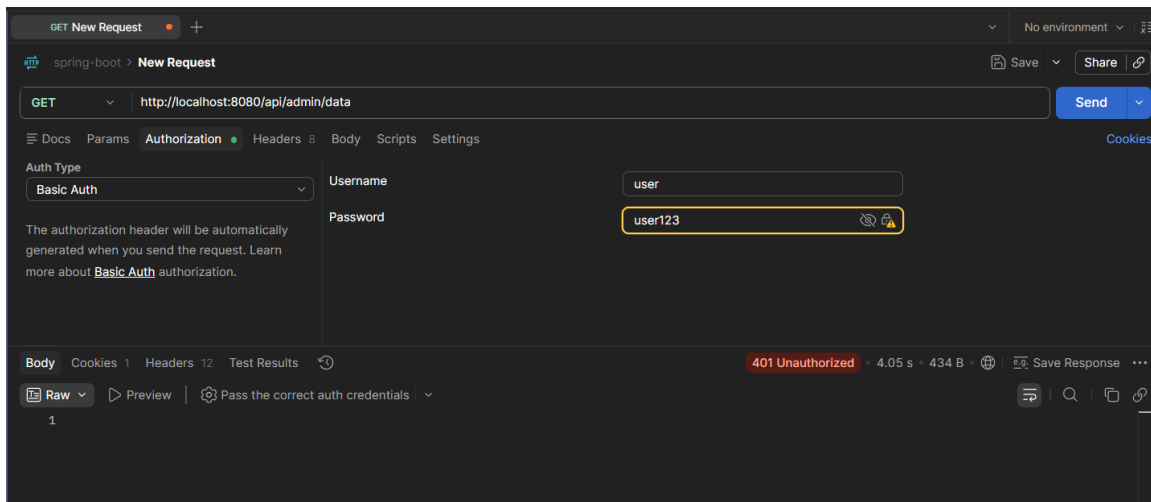
17.5 DELETE API – Delete Admin Data

The DELETE API is used to delete an admin resource using its ID. This endpoint is also restricted to users having the ADMIN role.



17.6 GET API – User Data

The GET API is used to access the user resource. It is configured using:



Practical No. 18: Securing REST API as an OAuth2 Resource Server with JWT

Objective:

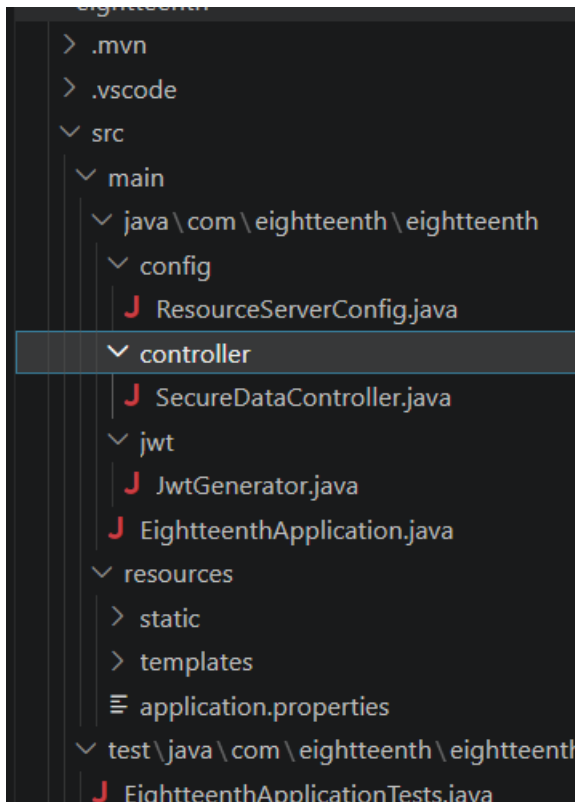
To configure the Spring Boot application as an OAuth2 Resource Server that validates JSON Web Tokens (JWT) issued by an Authorization Server, protecting REST endpoints so that only requests carrying a valid Bearer token succeed — the token-based security mechanism covered in this chapter

Class Name(s) with Attributes:**ResourceServerConfig (@Configuration — OAuth2 resource server)**

- Bean: SecurityFilterChain configured with oauth2ResourceServer().jwt()

SecureDataController (@RestController)

- Method: getSecureData() — requires valid JWT with configured scope

**Dependencies**

- oauth2 resource server for jwt

-Security

-jwt for getting token

Important Files

ResourceServerConfig.java(config)

```
package com.eightteenth.eightteenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/secure/**")
                .authenticated()

                .anyRequest()
                .permitAll()
            )

            .oauth2ResourceServer(
                oauth2 -> oauth2.jwt(Customizer.withDefaults())
            );

        return http.build();
    }
}
```

SecureDataController.java(Controller file)

```
package com.eightteenth.eightteenth.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/secure")
public class SecureDataController {

    @GetMapping("/data")
    public ResponseEntity<String> getSecureData() {

        return ResponseEntity.ok(
            "Protected resource data");
    }

    @PostMapping("/data")
    public ResponseEntity<String> createSecureData(
        @RequestBody String data) {

        return ResponseEntity
            .status(201)
            .body("Created: " + data);
    }

    @PutMapping("/data/{id}")
    public ResponseEntity<String> updateSecureData(
        @PathVariable Integer id,
        @RequestBody String data) {

        return ResponseEntity.ok(
            "Updated " + id + ": " + data);
    }

    @DeleteMapping("/data/{id}")
    public ResponseEntity<Void> deleteSecureData(
        @PathVariable Integer id) {

        return ResponseEntity.noContent().build();
    }
}
```

JwtGenrator.java(jwt)file

```
package com.eightteenth.eightteenth.jwt;

import java.util.Date;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;

@Component
public class JwtGenerator implements CommandLineRunner {

    private static final String SECRET =
        "01234567890123456789012345678901";

    @Override
    public void run(String... args) {

        String token = Jwts.builder()

            .subject("admin")

            .claim("scope", "read write")

            .issuedAt(new Date())

            .expiration(
                new Date(System.currentTimeMillis()
                    + 3600000)
            )

            .signWith(
                Keys.hmacShaKeyFor(
                    SECRET.getBytes()
                )
            )

            .compact();

        System.out.println();
        System.out.println("=====");
    };

    System.out.println("          JWT TOKEN");
    System.out.println("=====");
    };

    System.out.println(token);
```

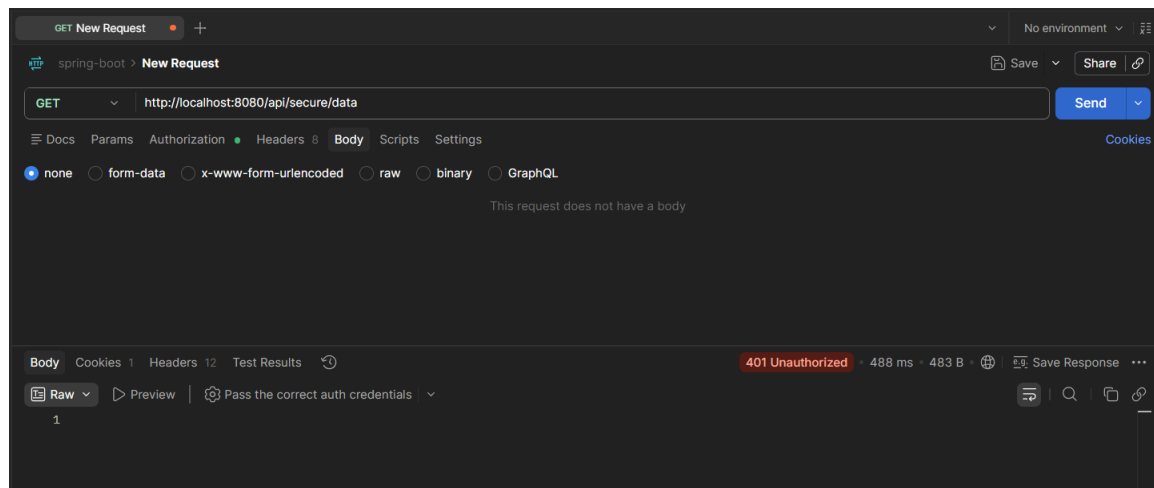
```
System.out.println("=====");
);
System.out.println();
}
}
```

API'S TESTING USING POSTMAN

18.1 GET API – Access Secure Data Without JWT

The GET API is used to access a protected resource. Since the /api/secure/** endpoints are secured using OAuth2 Resource Server, a valid JWT Bearer token is required to access the API. In this test, no authentication token is provided.

<http://localhost:8080/api/secure/data>



18.2 GET API – Access Secure Data with Valid JWT

NAME: SOLANKI KARANSINH

ENROLLMENT: 25CI2110110

The GET API is used to retrieve protected data. A valid JWT access token is sent through the Authorization header using the Bearer Token scheme. Spring Security validates the JWT before allowing the request to reach the controller.

<http://localhost:8080/api/secure/data>

```
PROBLEMS [94] OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

```
PS C:\Users\karansinh\Desktop\spring-practise> c:: cd 'c:\Users\karansinh\Desktop\spring-practise'; & 'C:\Users\karansinh\.vscode\extensions\r
dhat.java-1.55.0-win32-x64\jre\21.0.11-win32-x86_64\bin\java.exe' '-agentlib:jdpw=transport=dt_socket,server=n,suspend=y,address=localhost:6452
' '@C:\Users\KARANS~1\AppData\Local\Temp\cp_60mllicv1fky93eovue7xog4.argfile' 'com.eightteenth.eighteenth.EighteenthApplication'
ation in 3.932 seconds (process running for 4.502)

=====

JWT TOKEN

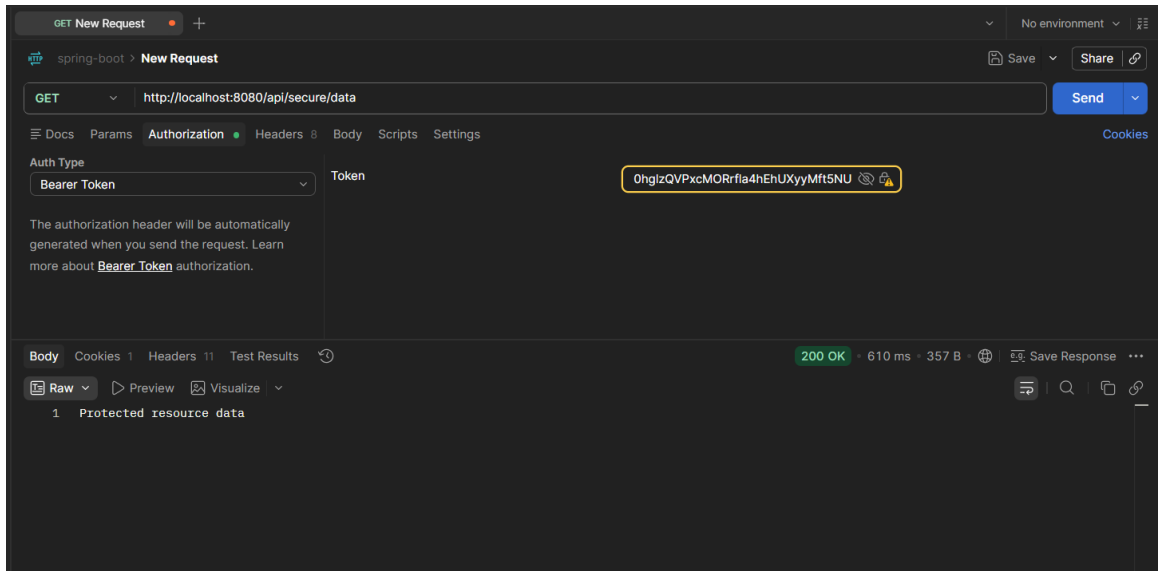
=====

eyJhbGciOiJIUzI1NiIsInR5cyJzdmUiOiJhZG1pbSIiInB1IjoicmVhZCB3cmMlZSIsImhhbmQ1MTYtNjMtNGEwOTQyOTg3NywiZXhwIjojODc2NDczNDc3fQ.CdM-gOficva20hg1zQPxcMORrfa
4hEHUXyyMftSNu

=====

2026-08-11T23:08:14.254+05:30 INFO 6816 --- [eightteenth] [nio-8080-exec-1] o.a.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring Dis
atcherServlet 'dispatcherServlet'
2026-08-11T23:08:14.254+05:30 INFO 6816 --- [eightteenth] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'd
spatcherServlet'
2026-08-11T23:08:14.263+05:30 INFO 6816 --- [eightteenth] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initializatio
in 8 ms
```

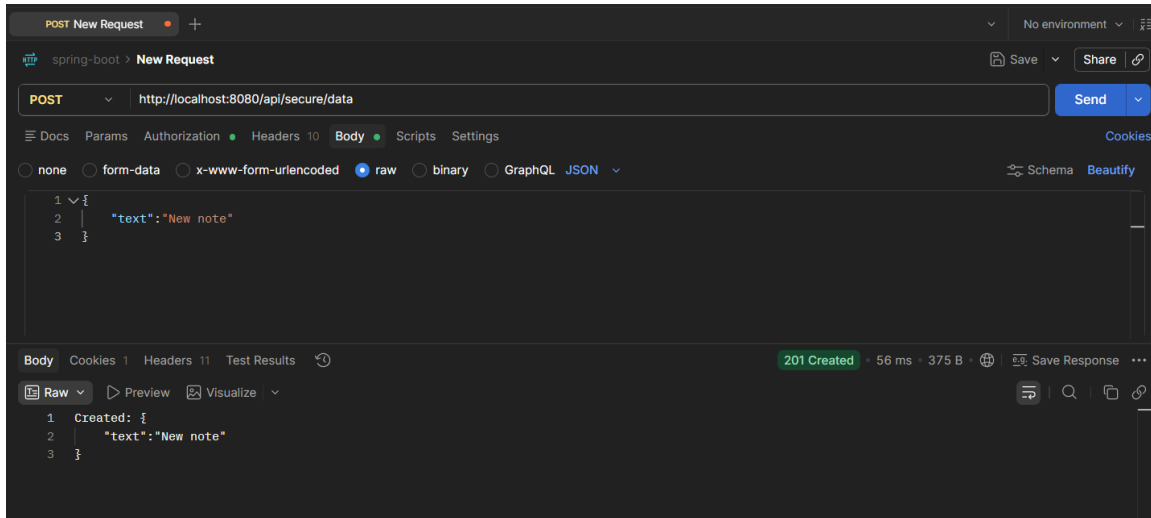
THEY GIVE TOKEN IN CONSOLE COPY AND PASTE IT IN THE POST MAN



18.3 POST API – Create Secure Data

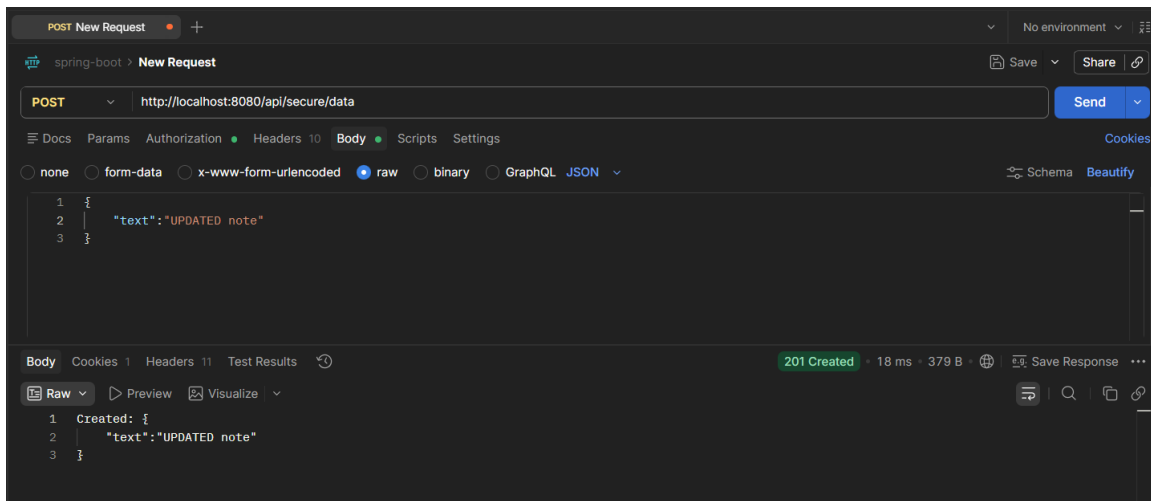
The POST API is used to create new protected data. The request requires a valid JWT access token with the required write scope. The data is sent in the request body.

<http://localhost:8080/api/secure/data>



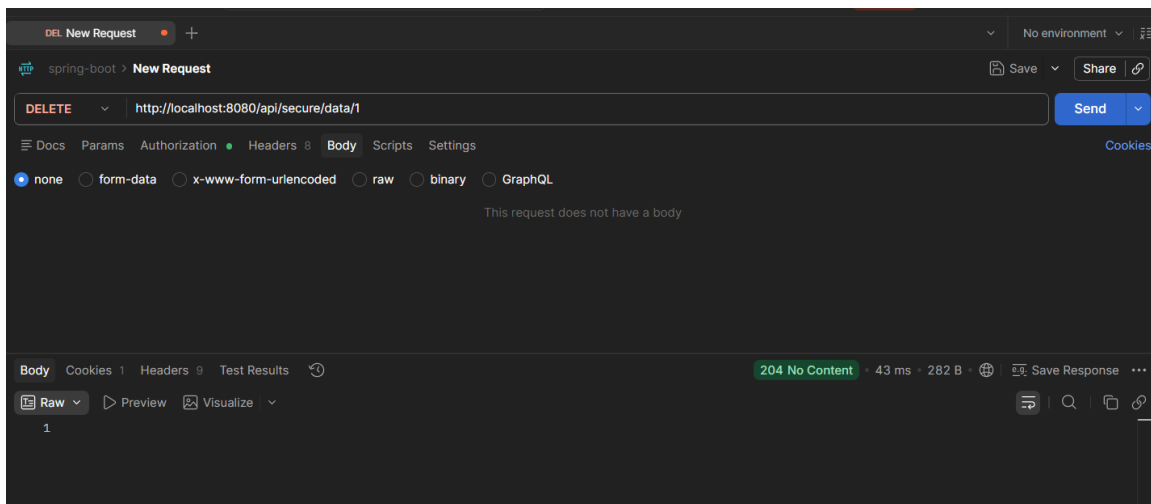
18.4 PUT API – Update Secure Data

The PUT API is used to update an existing protected resource. The resource ID is passed in the URL and the updated information is provided in the request body. A valid JWT with the required write scope is required.



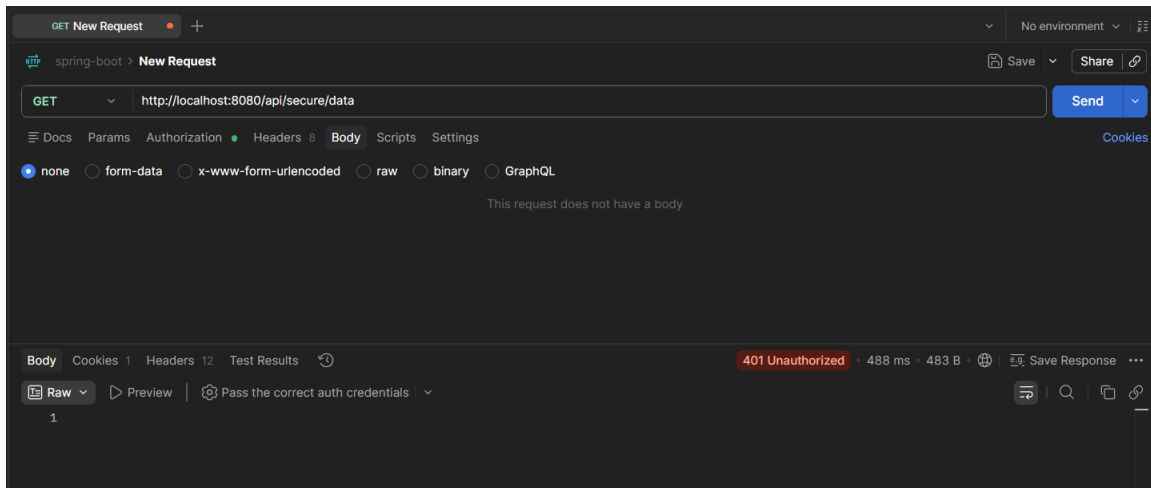
18.5 DELETE API – Delete Secure Data

The DELETE API is used to delete a protected resource using its ID. A valid JWT access token must be provided with the request.



18.6 GET API – Access Secure Data with Invalid JWT

The GET API is tested again using an invalid or expired JWT token. Spring Security validates the token before the request reaches the controller. Since the token is invalid, the request is rejected.



Practical No. 19: OAuth2 Login with a Third-Party Provider (Google/GitHub) and Accessing a Secured REST API

Objective:

To implement the OAuth2 Authorization Code grant using Spring Security's OAuth2 Client support with a real

provider (Google or GitHub), obtain an access token, and use that token to call a secured REST endpoint —

completing this chapter's end-to-end OAuth2 flow using Postman's built-in OAuth 2.0 authorization support.

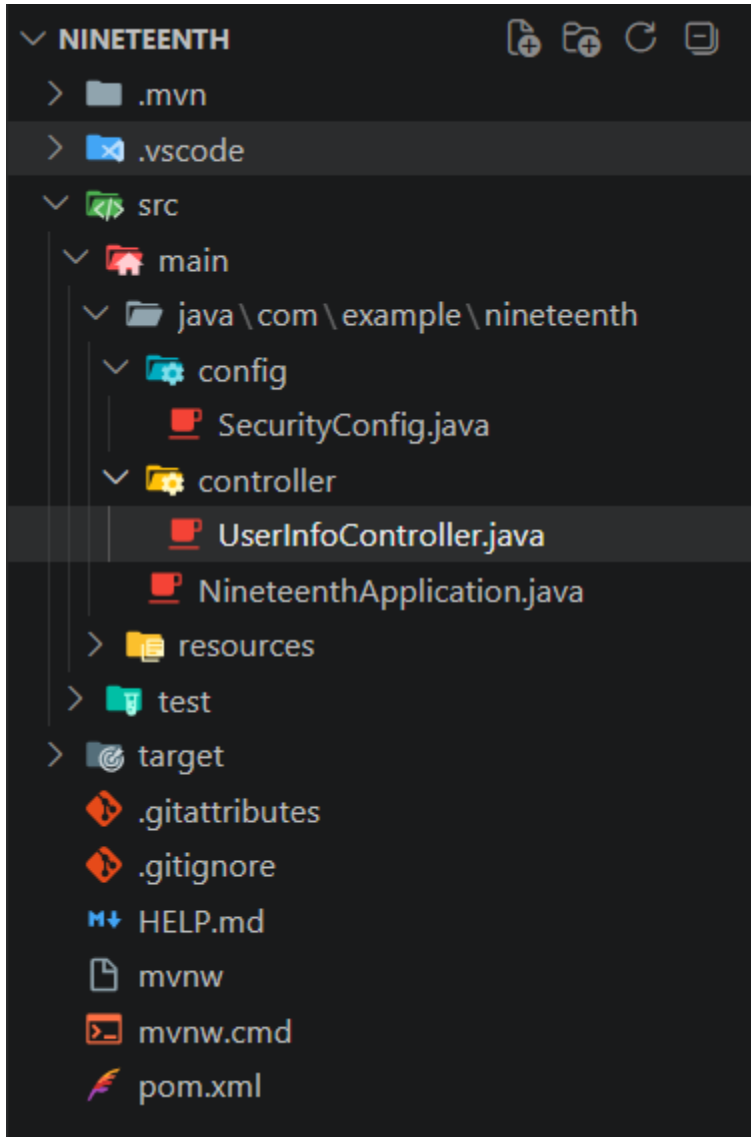
Class Name(s) with Attributes:

OAuth2ClientConfig (application.properties based client registration)

- spring.security.oauth2.client.registration.google.client-id
- spring.security.oauth2.client.registration.google.client-secret
- spring.security.oauth2.client.registration.google.scope

UserInfoController (@RestController)

- Method: getUserProfile(@AuthenticationPrincipal OAuth2User principal)
: Map<String,Object>



Dependencies

- oauth2 client
- Security
- jwt for getting token

Important Files

SecurityConfig.java(security)

```
package com.example.nineteenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http
            // Disable CSRF for REST API testing
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/oauth2/**")
                .permitAll()

                .requestMatchers("/login/**")
                .permitAll()

                .anyRequest()
                .authenticated()
            )

            .oauth2Login(oauth -> {});

        return http.build();
    }
}
```

SecurityConfig.java(security)

```
package com.example.nineteenth.controller;
```

```
import java.util.HashMap;
import java.util.Map;

import org.springframework.http.ResponseEntity;
import
org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/user")
public class UserInfoController {

    private final Map<Long, Map<String, Object>> notes = new HashMap<>();
    private long nextId = 1;

    // GET User Profile
    @GetMapping("/profile")
    public Map<String, Object> getUserProfile(
        @AuthenticationPrincipal OAuth2User principal) {

        return Map.of(
            "name", principal.getAttribute("name"),
            "email", principal.getAttribute("email")
        );
    }

    // POST Create Note
    @PostMapping("/notes")
    public ResponseEntity<Map<String, Object>> createNote(
        @RequestBody Map<String, String> request,
        @AuthenticationPrincipal OAuth2User principal) {

        Map<String, Object> note = new HashMap<>();

        note.put("id", nextId);
        note.put("text", request.get("text"));
        note.put("user", principal.getAttribute("email"));

        notes.put(nextId, note);

        nextId++;

        return ResponseEntity.status(201).body(note);
    }

    // PUT Update Note
    @PutMapping("/notes/{id}")
    public ResponseEntity<Map<String, Object>> updateNote(
```

```

    @PathVariable Long id,
    @RequestBody Map<String, String> request) {

    Map<String, Object> note = notes.get(id);

    if (note == null) {
        return ResponseEntity.notFound().build();
    }

    note.put("text", request.get("text"));

    return ResponseEntity.ok(note);
}

// DELETE Delete Note
@DeleteMapping("/notes/{id}")
public ResponseEntity<Void> deleteNote(
    @PathVariable Long id) {

    if (!notes.containsKey(id)) {
        return ResponseEntity.notFound().build();
    }

    notes.remove(id);

    return ResponseEntity.noContent().build();
}
}

```

Application.properties

```

spring.application.name=ninteenth

spring.security.oauth2.client.registration.google.client-id=196767098203-
leb4dmec933m49e7n189idvtlhpim2f2.apps.googleusercontent.com
spring.security.oauth2.client.registration.google.client-secret=GOCSPX-
ORwdQN_86F5wgqSHhSNWd5nGiwil

spring.security.oauth2.client.registration.google.scope=openid,profile,email
1

```

API'S TESTING USING POSTMAN

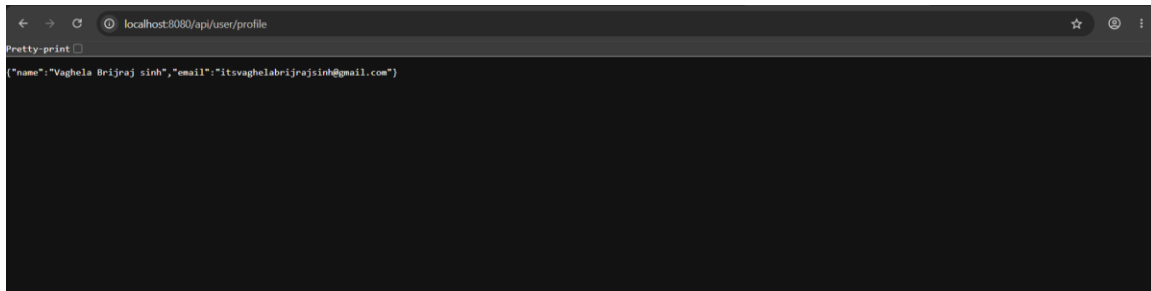
NAME: SOLANKI KARANSINH

ENROLLMENT: 25CI2110110

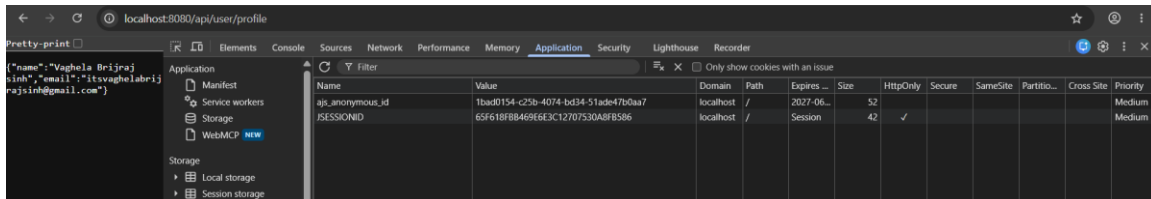
18.1 Open Below URL AND select the Email id to login

<http://localhost:8080/oauth2/authorization/google>

Then you will be redirect to this page where Show the name and email address when to use in login:



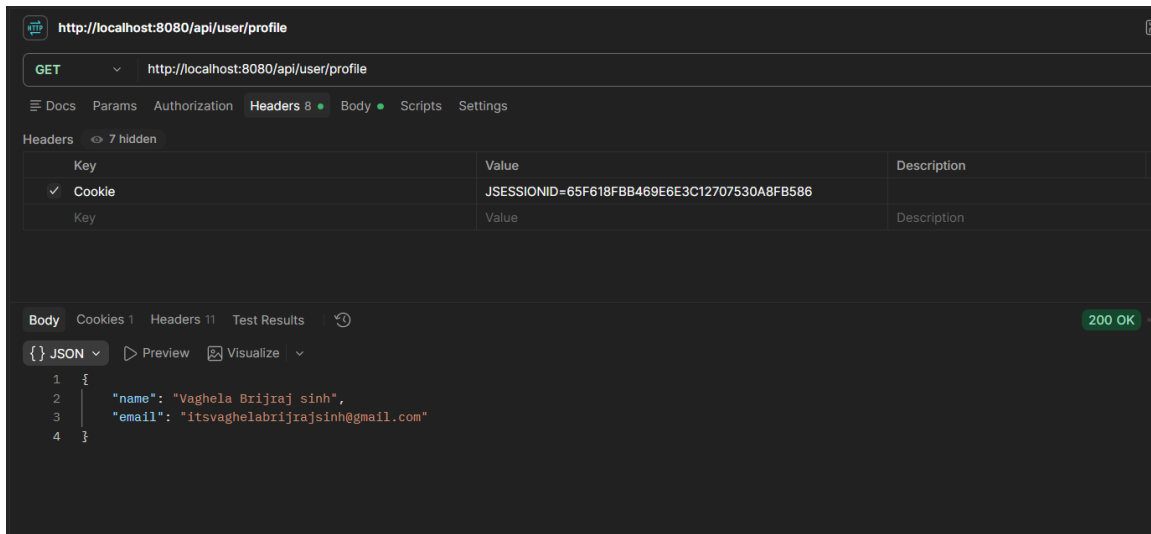
Then Press F12 and open in console Application under cookies show this type



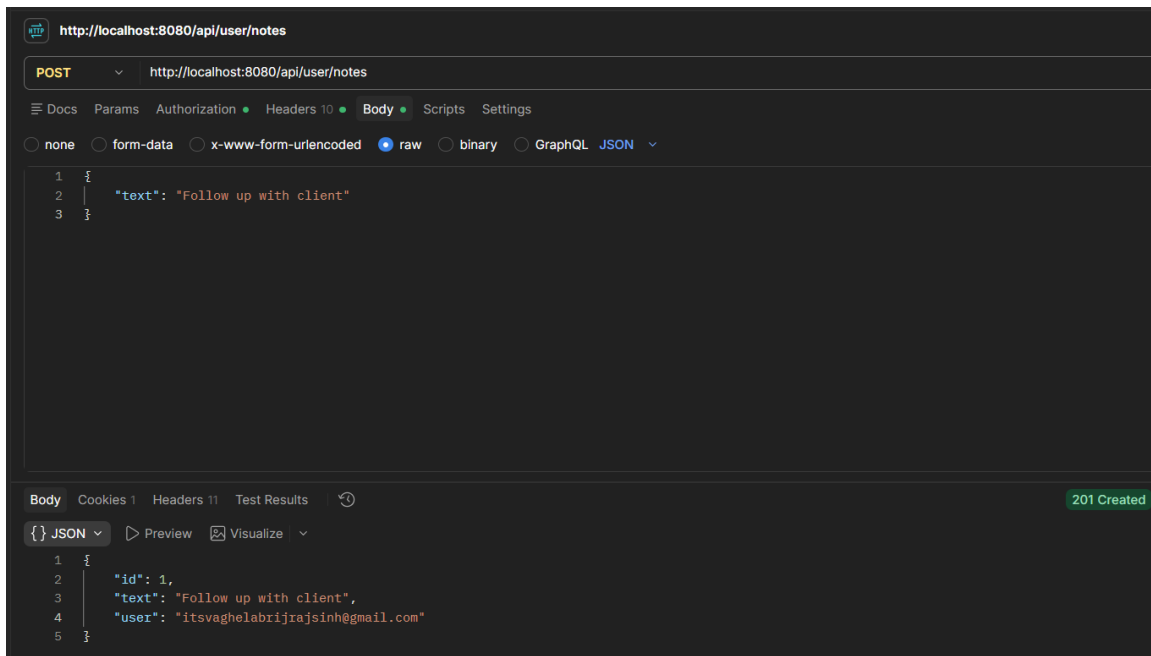
After fetch the data using cookies

Enter cookies and get the name and email:

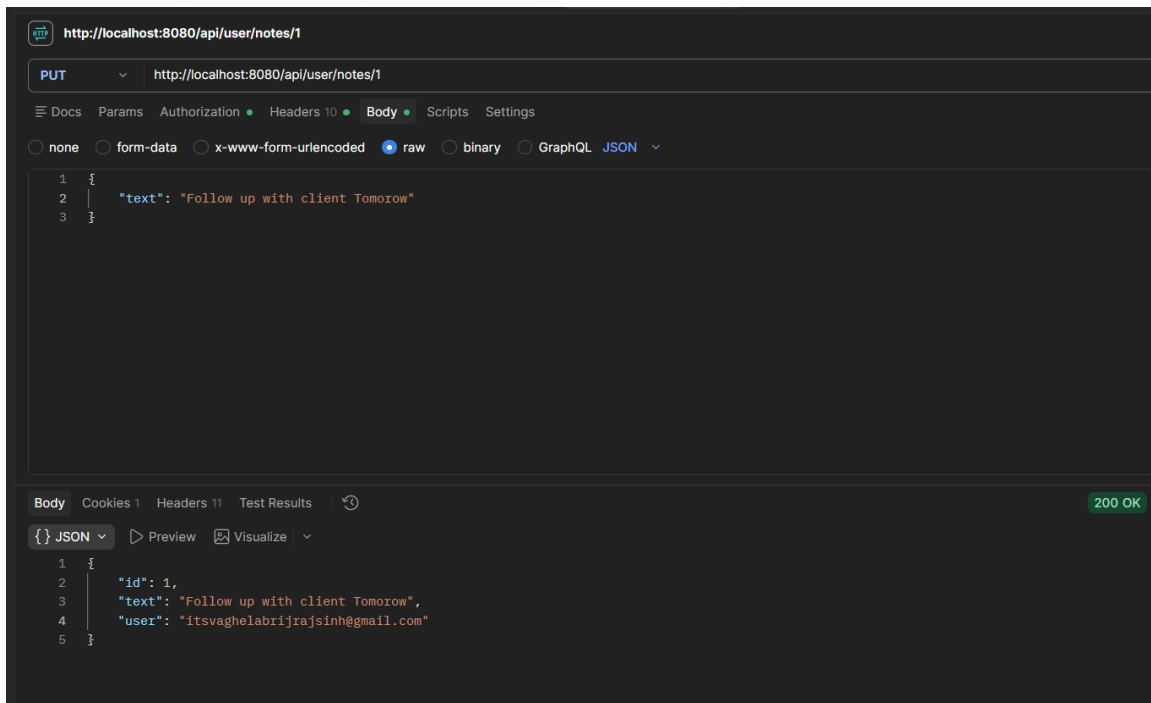
2. GET API:- RETRIVE EMAIL AND NAME



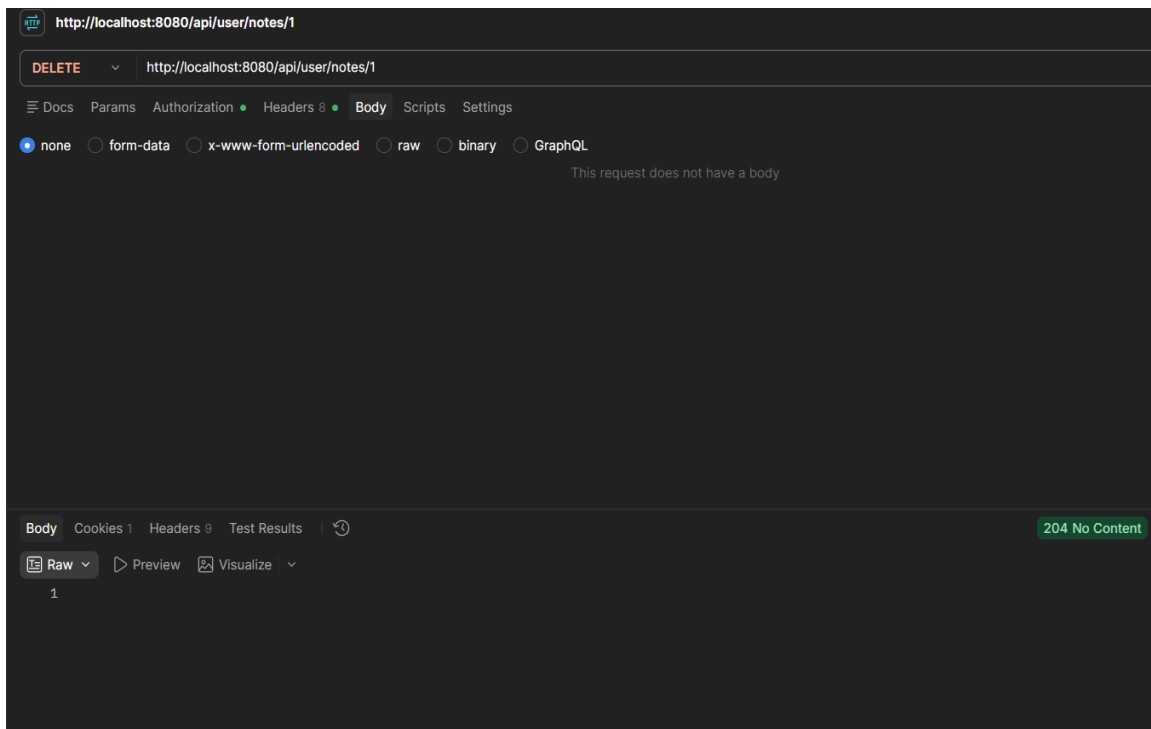
3.POST API:- ADDED NOTE



4.PUT API:- UPDATED NOTE



5.DELETE API:- NOTE DELETED



Practical No. 20: Cross-Origin Resource Sharing (CORS) Configuration for Secured REST APIs

Objective:

To configure CORS at the application level so that a secured REST API can be safely consumed by a browserbased frontend hosted on a different origin — a practical security configuration concern addressed alongside authentication/authorization in this chapter

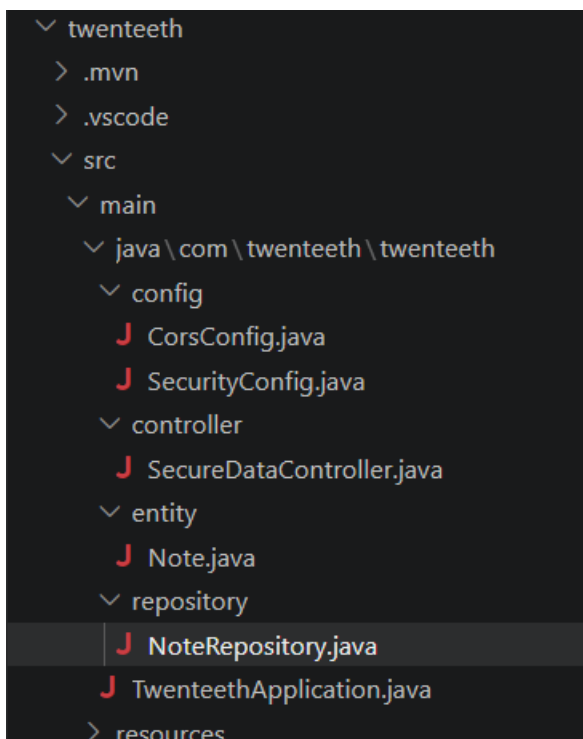
Class Name(s) with Attributes:

CorsConfig (@Configuration implements WebMvcConfigurer)

- Method: addCorsMappings(CorsRegistry registry)

SecurityConfig (extended)

- .cors(Customizer.withDefaults()) enabled in the SecurityFilterChain



Dependencies:-

- spring-boot-starter-web
- spring-boot-starter-security
- spring-boot-starter-data-jpa
- h2

Important Files

CorsConfig.java(config)

```
package com.twentieth.twentieth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
public class CorsConfig {

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {

        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(
            List.of("http://localhost:3000")
        );

        configuration.setAllowedMethods(
            List.of("GET", "POST", "PUT", "DELETE", "OPTIONS")
        );

        configuration.setAllowedHeaders(
            List.of("*")
        );

        UrlBasedCorsConfigurationSource source =
            new UrlBasedCorsConfigurationSource();

        source.registerCorsConfiguration(
            "/api/**",
```

```
        configuration
    );

    return source;
}
}
```

SecurityConfig.java(config)

```
package com.twenteeth.twenteeth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import
org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {

        http
            // Enable CORS
            .cors(Customizer.withDefaults())

            // Disable CSRF for REST API testing
            .csrf(csrf -> csrf.disable())

            // Authorization
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/secure/**")
                .authenticated()
                .anyRequest()
                .permitAll()
            )

            // Basic Authentication
```

```
        .httpBasic(Customizer.withDefaults());

    return http.build();
}

@Bean
public UserDetailsService userDetailsService() {

    UserDetails user = User
        .withUsername("admin")
        .password("{noop}1234")
        .roles("USER")
        .build();

    return new InMemoryUserDetailsManager(user);
}
}
```

SecureDataController.java(Controller)

```
package com.twenteeth.twenteeth.controller;

import com.twenteeth.twenteeth.entity.Note;
import com.twenteeth.twenteeth.repository.NoteRepository;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/secure/data")
public class SecureDataController {

    private final NoteRepository noteRepository;

    public SecureDataController(NoteRepository noteRepository) {
        this.noteRepository = noteRepository;
    }

    // GET
    @GetMapping
    public ResponseEntity<List<Note>> getAllNotes() {
```

```
        return ResponseEntity.ok(noteRepository.findAll());
    }

    // POST
    @PostMapping
    public ResponseEntity<Note> createNote(@RequestBody Note note) {

        Note savedNote = noteRepository.save(note);

        return
        ResponseEntity.status(HttpStatus.CREATED).body(savedNote);
    }

    // PUT
    @PutMapping("/{id}")
    public ResponseEntity<Note> updateNote(
        @PathVariable Long id,
        @RequestBody Note updatedNote) {

        return noteRepository.findById(id)
            .map(note -> {

                note.setText(updatedNote.getText());

                Note savedNote = noteRepository.save(note);

                return ResponseEntity.ok(savedNote);
            })
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteNote(@PathVariable Long id) {

        if (!noteRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }

        noteRepository.deleteById(id);

        return ResponseEntity.noContent().build();
    }
}
```

Note.java(Entity file)

```
package com.example.twentieth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Note {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String text;

    public Note() {
    }

    public Note(String text) {
        this.text = text;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getText() {
        return text;
    }

    public void setText(String text) {
        this.text = text;
    }
}
```

NoteRepository.java(Repository)

```
package com.twentieth.twentieth.repository;
```

```
import com.twenteeth.twenteeth.entity.Note;
import org.springframework.data.jpa.repository.JpaRepository;

public interface NoteRepository extends JpaRepository<Note, Long> {
}
```

Application.properties

```
spring.application.name=twenteeth

# H2 Database
spring.datasource.url=jdbc:h2:mem:corsdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

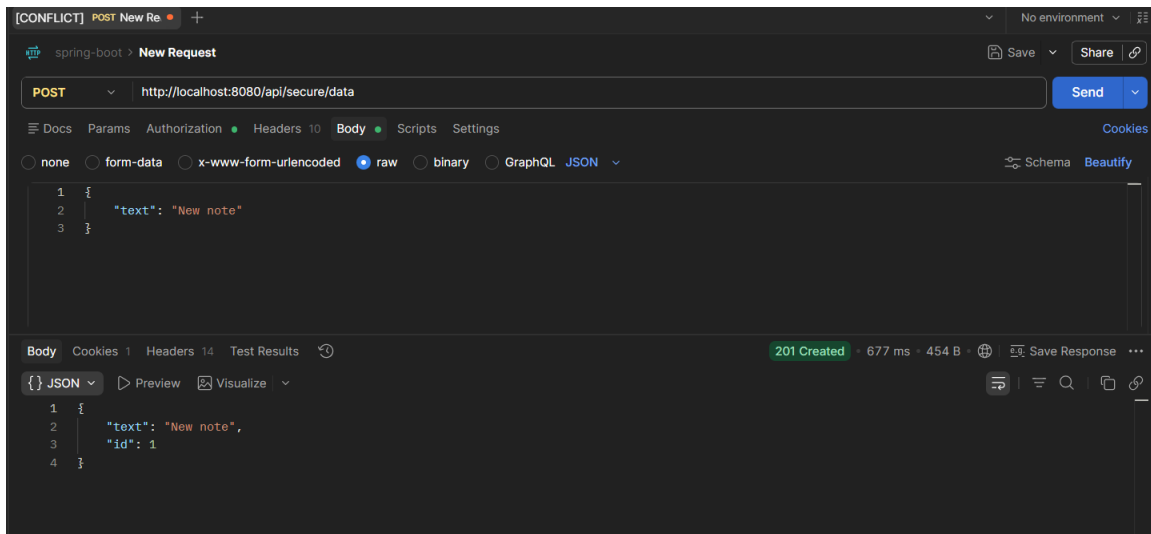
# JPA
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

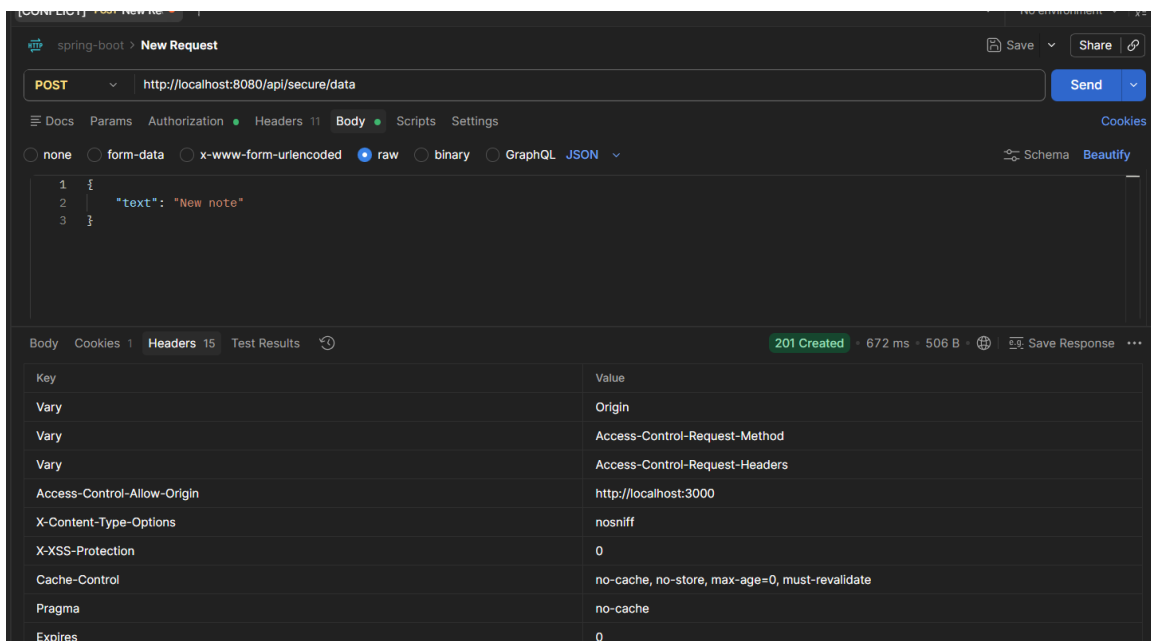
20.1 POST API

A POST request was sent to /api/secure/data with Basic Authentication and the Origin: http://localhost:3000 header. The note was successfully created with **201 Created**, and the CORS header was present in the response..

<http://localhost:8080/api/secure/data>



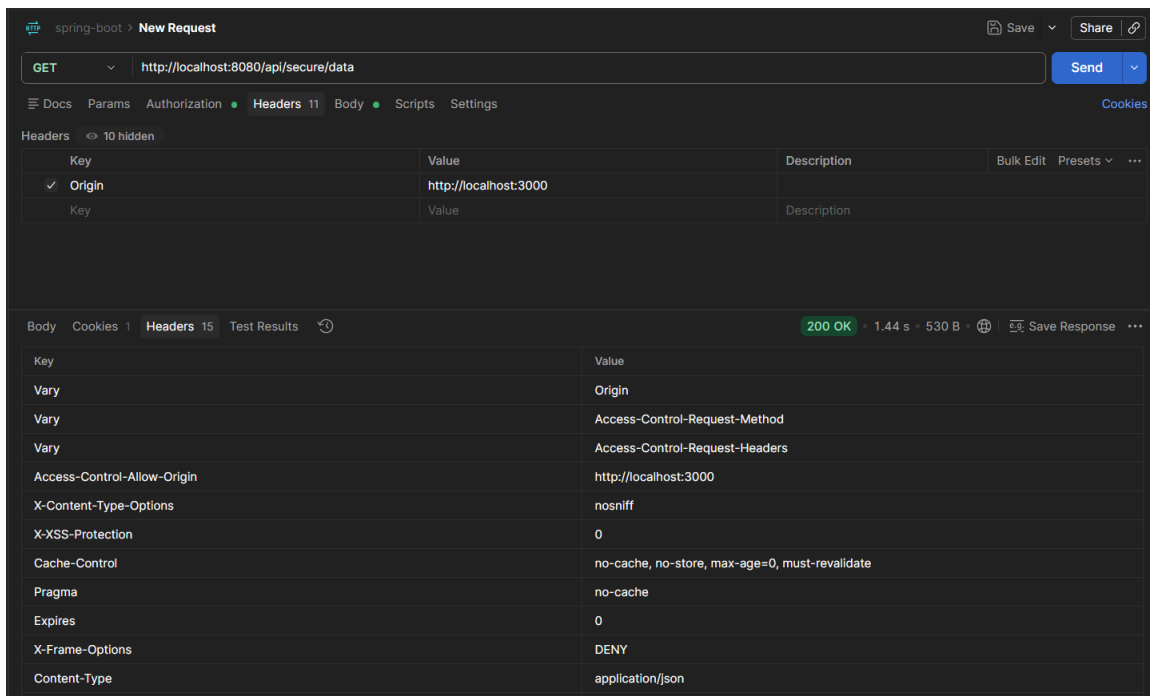
Verify in Header part in response show Access-control-Allow-origin.



20.2 GET – Valid Origin

A Retrive NOTE:- A GET request was sent to /api/secure/data with Basic Authentication and the allowed origin. The server successfully returned the stored notes with **200 OK**, along with the Access-Control-Allow-Origin header.

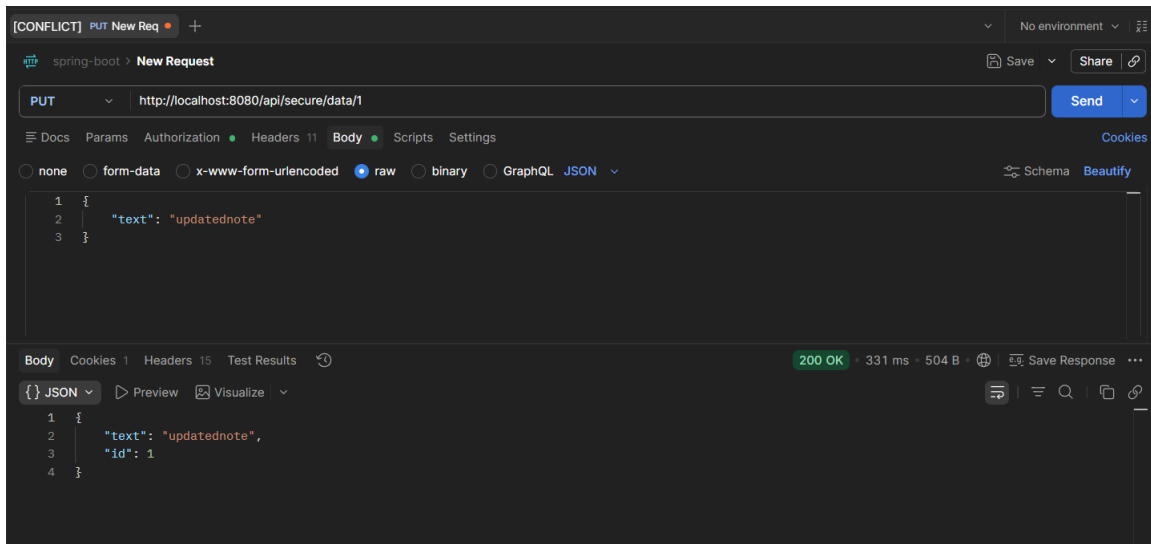
<http://localhost:8080/api/secure/data>



20.3 PUT – Update Note

A PUT request was sent to /api/secure/data/1 with the updated note data. The note was successfully updated with **200 OK**, and the CORS response header was present.

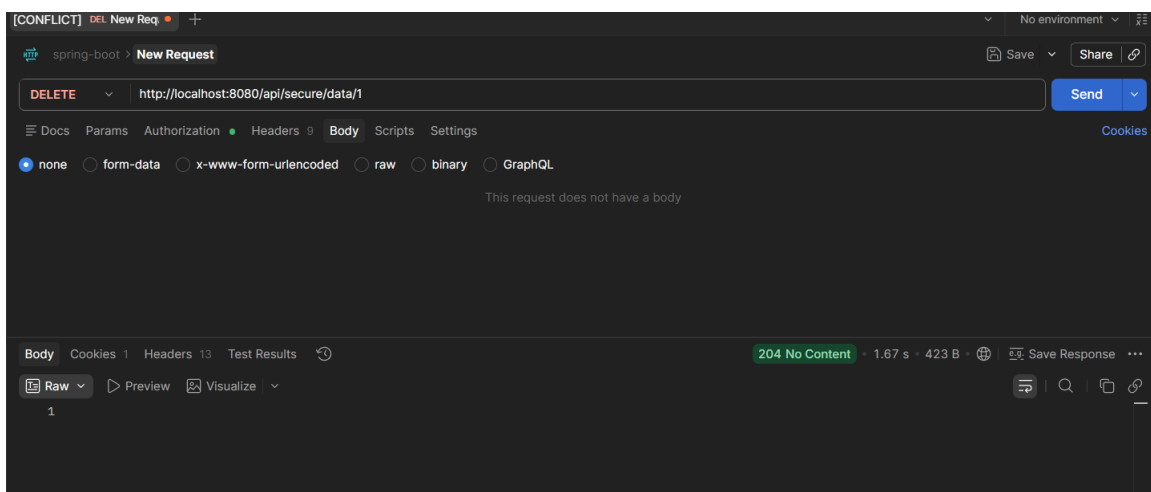
<http://localhost:8080/api/secure/data/1>



20.4 Delete -delete node

A DELETE request was sent to `/api/secure/data/1` with Basic Authentication and the allowed origin. The note was successfully deleted, returning **204 No Content** with the CORS header present.

<http://localhost:8080/api/secure/data/1>

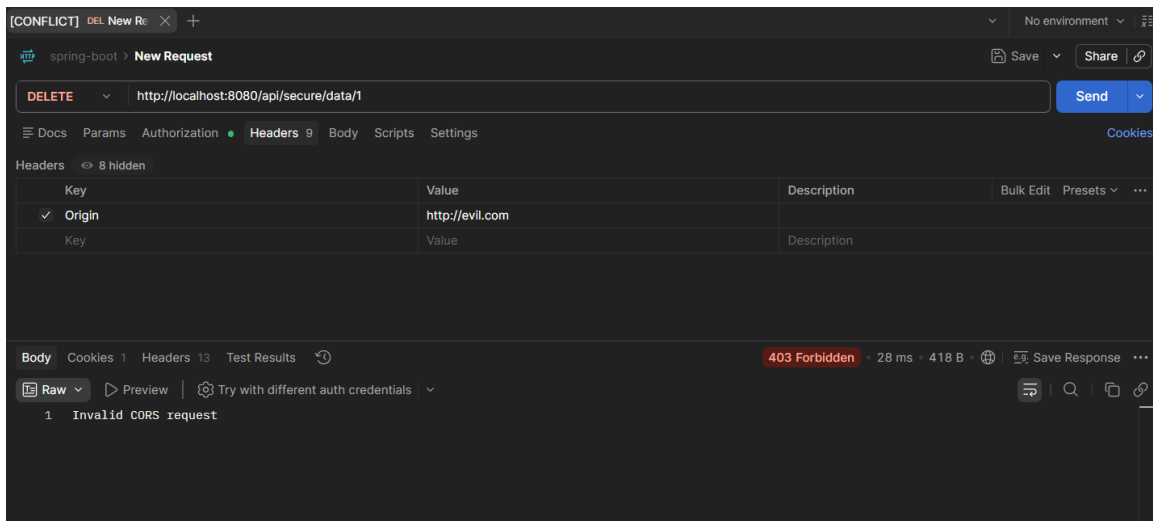


20.5. Disallowed Origin:-

First add url in origin under header <http://evil.com>

A GET request was tested using Origin: <http://evil.com> instead of the allowed origin. The request was rejected with **403 Forbidden**, and the Access-Control-Allow-Origin header for evil.com was not present.

<http://localhost:8080/api/secure/data>



END OF ALL PRACTICAL'S

THANK YOU..!!!!!!